

RANDOMNESS. EFFICIENCY. ELEGANCE.

RANDOMIZED ALGORITHM

IN

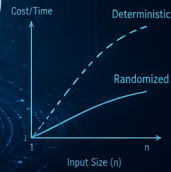
C++

```
std::mt19937 rng(seed);  
std::uniform_int_distribution<int> dist;
```

```
int random_index(int n) {  
    return dist(rng);  
}
```

```
bool randomized_quicksort  
(std::vector<int>& a,  
 int low, int high) {  
    if (low < high) {  
        int pi = random_partition  
            (a, low, high);  
        randomized_quicksort(a,  
            low, pi - 1);  
        randomized_quicksort(a,  
            pi + 1, high);  
    }  
    return true;  
}
```

Harnessing Randomness to Solve Problems
Faster, Smarter, and Simpler



Probabilistic
Thinking



Expected
Efficiency



Randomized
Data Structures



Monte Carlo &
Las Vegas Algorithms



Practical C++
Implementations

CHAMAN SINGH VERMA

Randomized Algorithms with C++

Chaman Singh Verma

July 15, 2026

Preface

Randomness is one of the most remarkable and counterintuitive tools in algorithm design. A small amount of unpredictability, judiciously deployed, can yield algorithms that are simpler, faster, or both—compared to the best deterministic alternatives. This book is an invitation to explore the theory and practice of randomized algorithms, written for advanced undergraduate students, graduate students, and practitioners who wish to understand both the mathematical foundations and the engineering realities of randomized computation.

Scope and approach. The book is organized into three parts. The first part (Chapters 1–6) develops the probabilistic toolbox: conditional probability, expectation, tail inequalities, the probabilistic method, and Markov chains. These chapters are self-contained and require only a basic familiarity with discrete mathematics. The second part (Chapters 7–10) presents applications in specific domains: algebraic techniques, data structures, computational geometry, and graph algorithms. The third part (Chapters 11–14) addresses advanced topics: approximate counting, parallel and distributed algorithms, online algorithms, and the number-theoretic foundations of cryptography.

Throughout the book, algorithms are presented in a dual format: rigorous mathematical pseudocode alongside complete, compilable C++ implementations. The C++ code uses modern C++23 features—`std::ranges`, `concepts`, `std::print`, and the `<random>` library—and is organized into header-only files that can be included directly into projects. Each imple-

mentation is accompanied by a driver program that allows the reader to verify the theoretical bounds empirically.

Prerequisites. Readers are expected to have taken an introductory course in algorithms and data structures, and to be comfortable with basic probability (sample spaces, expectation, independence). The necessary background is reviewed in Appendix A. Familiarity with C++ is assumed for the implementations; readers who work primarily in another language may skip the code listings without loss of continuity.

Pedagogical features.

- **Examples and exercises.** Each chapter contains worked examples that illustrate key techniques, and a mix of theoretical and programming exercises.
- **C++ implementations.** Every major algorithm has a corresponding C++ implementation, enabling readers to experiment and build intuition.
- **Notes and references.** Each chapter ends with a section of historical notes and pointers to the research literature, for readers who wish to delve deeper.
- **Appendices.** Appendix A reviews probability theory, Appendix B covers computational complexity, and Appendix C surveys the modern C++23 features used in the code listings.

Acknowledgements. I am grateful to the many researchers whose elegant ideas fill these pages. The theory of randomized algorithms has been shaped by the contributions of dozens of scholars over the past four decades; I hope this book does justice to their work. I thank my students, whose questions and feedback continually improve my understanding of this material. Any errors that remain are, of course, my own responsibility.

Chaman Singh Verma
2026

Contents

Preface	iii
1 Introduction	1
1.1 Limitations of Deterministic Algorithms	2
1.1.1 Worst-case complexity is not the full picture	2
1.1.2 Problems with no known efficient deterministic solution	3
1.1.3 The adversarial argument	4
1.1.4 A concrete example: finding a median	4
1.2 Random Number Generation	5
1.2.1 Pseudorandom number generators	5
1.2.2 Seeding and reproducibility	6
1.2.3 Cryptographically secure PRNGs	7
1.2.4 Weak randomness versus strong randomness	7
1.2.5 True randomness versus pseudorandomness	9
1.2.6 Random bits vs. random numbers	11
1.3 Preliminary Mathematics	12
1.3.1 Sample spaces and events	12
1.3.2 Independence and conditional probability	12
1.3.3 Expectation and linearity	13
1.3.4 Variance and deviations	13
1.3.5 The union bound	14
1.4 The Randomized Approach	14
1.4.1 Improved average-case complexity	15

1.4.2	No efficient deterministic algorithm known	16
1.4.3	Simplicity	17
1.4.4	Adversarial immunity	18
1.4.5	Las Vegas and Monte Carlo algorithms .	19
1.4.6	The min-cut algorithm: a first example .	19
1.4.7	C++ Implementation	21
1.4.8	C++ Implementation: Randomized Quick-Sort	24
1.4.9	C++ Implementation: Monte Carlo—Estimating π	24
1.4.10	Converting Between Las Vegas and Monte Carlo	25
2	Fundamentals	27
2.1	Binary Planar Partitions	27
2.1.1	Binary Planar Partitions	28
2.2	A Probabilistic Recurrence	32
2.2.1	C++ Implementation	34
2.3	Computation Model and Complexity Classes . .	35
2.3.1	RAMs and Turing Machines	35
2.3.2	Complexity Classes	36
	Notes	37
	Problems	38
3	Game-Theoretic Techniques	43
3.1	Game Tree Evaluation	43
3.1.1	A Randomized Algorithm	45
3.1.2	C++ Implementation	47
3.2	The Minimax Principle	50
3.2.1	Game Theory	50
3.2.2	Mixed Strategies and the Minimax Theorem	51
3.2.3	Yao's Minimax Principle	53
3.2.4	C++ Implementation: Computing Mixed Strategy Equilibria	54
3.2.5	Lower Bound for Game Tree Evaluation .	55
3.3	Randomness and Non-uniformity	57

3.3.1	Boolean Circuits	57
3.3.2	Adleman's Theorem	58
	Notes	60
	Problems	60
4	Moments and Deviations	63
4.1	Occupancy Problems	63
4.1.1	The Birthday Problem	65
4.2	The Markov and Chebyshev Inequalities	67
4.3	Randomized Selection	69
4.4	Two-Point Sampling	75
4.5	The Stable Marriage Problem	78
4.5.1	Average-Case Analysis and the Principle of Deferred Decisions	80
4.6	The Coupon Collector's Problem	84
4.6.1	An Elementary Analysis	84
4.6.2	The Poisson Heuristic	86
4.6.3	A Sharp Threshold	87
	Notes	91
	Problems	92
5	Tail Inequalities	95
5.1	The Chernoff Bound	96
5.1.1	Derandomization via Conditional Expectations	98
5.2	Routing in a Parallel Computer	98
5.3	A Wiring Problem	100
5.4	Martingales	101
	Notes	103
	Problems	104
6	The Probabilistic Method	107
6.1	Overview of the Method	108
6.2	Maximum Satisfiability	109
6.3	Expanding Graphs	111
6.4	Oblivious Routing Revisited	112
6.5	The Lovász Local Lemma	113
6.5.1	Algorithmic Lovász Local Lemma	114

6.6	The Method of Conditional Probabilities	118
	Notes	121
	Problems	121
7	Markov Chains and Random Walks	123
7.1	A 2-SAT Example	124
7.1.1	The Walksat Algorithm for 2-SAT	124
7.2	Markov Chains	127
7.3	Random Walks on Graphs	129
7.3.1	Mixing Time and Conductance	130
7.4	Electrical Networks	130
7.5	Cover Times	132
7.6	Graph Connectivity	133
7.7	Expanders and Rapidly Mixing Random Walks	134
7.8	Probability Amplification by Random Walks on Expanders	136
7.8.1	Almost-Uniform Sampling	136
7.8.2	Error Reduction	136
7.8.3	Applications	137
	Notes	138
	Problems	139
8	Algebraic Techniques	141
8.1	Fingerprinting and Freivalds' Technique	141
8.2	Verifying Polynomial Identities	143
8.3	Perfect Matchings in Graphs	145
8.4	Verifying Equality of Strings	146
8.5	A Comparison of Fingerprinting Techniques	147
8.6	Pattern Matching	148
8.7	Interactive Proof Systems	150
8.8	PCP and Efficient Proof Verification	152
	Notes	153
	Problems	154
9	Data Structures	157
9.1	The Fundamental Data-structuring Problem	157
9.1.1	The randomized approach	158
9.2	Random Treaps	159

9.2.1	Search	160
9.2.2	Insertion	160
9.2.3	Deletion	161
9.2.4	Join and Split	161
9.2.5	Analysis	162
9.3	Skip Lists	163
9.3.1	Search	163
9.3.2	Insertion and Deletion	164
9.3.3	Analysis	165
9.3.4	Comparison with balanced BSTs	166
9.4	Hash Tables	166
9.4.1	Universal hash families	166
9.4.2	Collision resolution	167
9.4.3	Linear probing	168
9.5	Hashing with $O(1)$ Search Time	169
9.5.1	FKS perfect hashing	169
9.5.2	Dynamic perfect hashing	170
	Notes	171
	Problems	172
10	Geometric Algorithms and Linear Programming	175
10.1	Randomized Incremental Construction	176
10.2	Convex Hulls in the Plane	177
10.2.1	Gift wrapping	177
10.2.2	The randomized incremental algorithm	178
10.3	Duality	179
10.4	Half-space Intersections	180
10.5	Delaunay Triangulations	181
10.5.1	The flip algorithm	182
10.5.2	Randomized incremental construction	182
10.6	Trapezoidal Decompositions	183
10.6.1	Randomized construction and point location	183
10.7	Binary Space Partitions	185
10.7.1	Randomized BSP construction	185
10.8	The Diameter of a Point Set	186
10.9	Random Sampling	187

10.10	Linear Programming	189
10.10.1	Seidel's randomized LP algorithm	189
10.10.2	Clarkson's algorithm	191
	Notes	191
	Problems	193
11	Graph Algorithms	195
11.1	All-pairs Shortest Paths	195
11.1.1	Min-plus matrix multiplication	196
11.1.2	The randomized reduction	197
11.1.3	Seidel's algorithm for unweighted undirected graphs	198
11.2	The Min-Cut Problem	204
11.2.1	Karger's contraction algorithm	205
11.2.2	The Karger–Stein algorithm	207
11.3	Minimum Spanning Trees	215
11.3.1	Preliminaries	216
11.3.2	Borůvka's algorithm	216
11.3.3	The sampling lemma	217
11.3.4	The algorithm	217
	Notes	228
	Problems	229
12	Approximate Counting	231
12.1	Randomized Approximation Schemes	232
12.1.1	The counting–sampling connection	233
12.1.2	Markov chain methods	233
12.1.3	Coupling and conductance	234
12.2	The DNF Counting Problem	235
12.2.1	The naive approach and why it fails	236
12.2.2	The Karp–Luby algorithm	236
12.3	Approximating the Permanent	243
12.3.1	The reduction from general permanent to bipartite graphs	244
12.3.2	The Jerrum–Sinclair–Vazirani Markov chain	245
12.3.3	Estimating the permanent from the chain	248
12.4	Volume Estimation	250

12.4.1	The annealing approach	250
12.4.2	The hit-and-run walk	252
12.4.3	The DFK annealing algorithm	253
Notes		279
Problems		282
12.5	Randomized Rounding of Linear Programs . . .	283
12.5.1	Randomized Rounding for MAX-SAT . .	284
12.5.2	Randomized Rounding for Set Cover . .	286
12.5.3	The Randomized Rounding Paradigm . .	287
12.5.4	Exercises	288
13	Parallel and Distributed Algorithms	289
13.1	The PRAM Model	289
13.1.1	Complexity Measures	291
13.1.2	Complexity Classes	292
13.1.3	Why Randomization Helps in Parallel Computation	293
13.2	Sorting on a PRAM	293
13.2.1	Bitonic Sorting Networks	294
13.2.2	Randomized Parallel Quicksort	294
13.2.3	The AKS Sorting Network	296
13.2.4	BoxSort: The Two-Phase Approach	296
13.3	Maximal Independent Sets	297
13.3.1	Luby's Algorithm	298
13.3.2	Analysis: Probability a Good Vertex Sur- vives	300
13.3.3	Derandomization via Pairwise Indepen- dence	301
13.4	Perfect Matchings	302
13.4.1	The Isolating Lemma	302
13.4.2	RNC Algorithm for Bipartite Perfect Match- ing	305
13.4.3	The Exact Matching Problem	308
13.5	The Choice Coordination Problem	309
13.5.1	The Birthday Paradox Connection	309
13.5.2	The Algorithm	310
13.5.3	Connection to Load Balancing	311

13.6	Byzantine Agreement	312
13.6.1	The Oral Messages Model	312
13.6.2	Why Randomization Helps	312
13.6.3	The Flood-Set Algorithm	313
13.6.4	Randomized Byzantine Agreement	314
	Notes	337
	Problems	338
14	Online Algorithms	341
14.1	The Online Paging Problem	342
14.1.1	Deterministic Algorithms and Their Lim- itations	342
14.1.2	Lower Bound for Deterministic Algorithms	343
14.2	Adversary Models	344
14.2.1	The Three Adversary Models	345
14.2.2	The Randomized Competitive Guarantee	346
14.3	Paging against an Oblivious Adversary	346
14.3.1	The Randomized Marking Algorithm	346
14.3.2	Analysis against an Oblivious Adversary	347
14.3.3	Improvement to k -Competitive	350
14.4	Relating the Adversary Models	351
14.5	The Adaptive Online Adversary	353
14.5.1	Comparison of Models for Paging	354
14.6	The k -Server Problem	354
14.6.1	Deterministic Lower Bound	355
14.6.2	Randomized k -Competitive Algorithm	355
14.6.3	Connection to Paging	357
14.6.4	The Hirschberg–Sinclair Technique	358
	Notes	386
	Problems	387
15	Number Theory and Algebra	389
15.1	Preliminaries	390
15.1.1	Modular arithmetic	390
15.1.2	The Euclidean algorithm	390
15.1.3	The extended Euclidean algorithm and Bézout’s identity	391

15.1.4	The Chinese Remainder Theorem	392
15.2	Groups and Fields	393
15.2.1	Groups and abelian groups	393
15.2.2	Euler's totient function	393
15.2.3	Euler's theorem	394
15.2.4	Fields and finite fields	394
15.2.5	Cyclic groups and generators	395
15.2.6	The discrete logarithm problem	395
15.3	Quadratic Residues	396
15.3.1	The Legendre symbol	396
15.3.2	A randomized quadratic residue test . . .	397
15.3.3	The Jacobi symbol	397
15.3.4	Quadratic residuosity for composite moduli	398
15.4	The RSA Cryptosystem	398
15.4.1	Key generation	399
15.4.2	Encryption and decryption	399
15.4.3	Security of RSA	400
15.4.4	Applications	400
15.5	Polynomial Roots and Factors	401
15.5.1	Polynomials over finite fields	401
15.5.2	The Schwartz–Zippel lemma	401
15.5.3	Applications of the Schwartz–Zippel lemma	403
15.5.4	Polynomial factorization over finite fields	404
15.5.5	Why factoring is hard but polynomial root-finding is easy	404
15.6	Primality Testing	404
15.6.1	Trial division	405
15.6.2	Fermat's primality test	405
15.6.3	The Miller–Rabin test	406
15.6.4	The Solovay–Strassen test	409
15.6.5	The AKS primality test	410
	Notes	428
	Problems	430

A Probability Theory 441

A Probability Theory	441
.1 Sample Spaces and Events	441
.2 Random Variables	442
.3 Important Discrete Distributions	444
.4 Useful Inequalities	444
.5 Convergence Concepts	445
.6 Martingales	446
A Computational Complexity	449
B Computational Complexity	449
.1 Turing Machines and Basic Complexity Classes .	449
.2 Randomized Complexity Classes	450
.3 Counting Classes	452
.4 Circuit Complexity and Derandomization	453
.5 Interactive Proofs and the Polynomial Hierarchy	453
A C++23 Programming for Randomized Algorithms	455
C C++23 Programming	455
.1 The Random Library	455
.1.1 Engines	455
.1.2 Seeding	456
.1.3 Distributions	456
.2 Concepts and Constraints	457
.3 Ranges and Views	458
.4 Smart Pointers	458
.5 Chrono and Timing	459
.6 Structured Bindings and Initialization	459
.7 Exception Safety	460

Chapter 1

Introduction

Algorithms are the engines of computation—systematic procedures that transform inputs into outputs through a sequence of well-defined steps. For decades, the gold standard in algorithm design has been *determinism*: given the same input, a deterministic algorithm always produces the same output and takes the same time. Yet there exists a large class of problems for which no efficient deterministic algorithm is known, and for which the introduction of randomness yields striking improvements in simplicity, speed, or both. This book is devoted to the theory and practice of *randomized algorithms*—algorithms that make use of random choices during their execution.

We begin this chapter by examining why deterministic algorithms, despite their elegance and predictability, face fundamental limitations. This motivates the study of randomization as a design principle. We then discuss random number generation in practice, since every randomized algorithm ultimately relies on a source of randomness. Next, we gather the elementary tools from probability theory that will recur throughout the book: conditional probability, linearity of expectation, and tail inequalities. Finally, we introduce the main ideas behind randomized algorithms, covering the two principal paradigms—Las Vegas and Monte Carlo methods—and illustrating them with concrete examples.

1.1 Limitations of Deterministic Algorithms

To appreciate the power of randomization, it is essential to understand where deterministic methods fall short. By “deterministic algorithm” we mean an algorithm whose behavior is completely determined by its input: no coin flips, no random choices. The running time of a deterministic algorithm is a function of the input alone.

1.1.1 Worst-case complexity is not the full picture

The traditional framework for analyzing algorithms is *worst-case complexity*: we measure the running time on the most adversarial input of size n . This framework has been enormously productive, yielding tight bounds for many problems (e.g. sorting in $O(n \log n)$ time, matrix multiplication via Strassen’s algorithm). However, worst-case analysis can be overly pessimistic. A deterministic algorithm may be designed to protect against a pathological input that rarely—or never—occurs in practice. The resources devoted to handling the worst case may therefore be wasted on typical inputs.

Any order versus random order

A subtle but important distinction concerns the order in which input elements are presented to an algorithm. This becomes particularly significant for incremental algorithms that process elements one by one.

Any order (adversarial). When a deterministic incremental algorithm processes elements *in any order*, an adversary who knows the algorithm can choose the order that maximizes the running time or solution cost. For example, inserting points into a convex hull in worst-case order causes every insertion to restructure the hull, yielding $O(n^2)$ time. Similarly, adding elements to a hash table in adversarial order can cause all elements to collide.

Random order. If the same elements are processed in a uniformly random permutation, the expected behavior is often dramatically better. Randomized incremental algorithms exploit this idea: randomizing the order in which elements are processed turns worst-case inputs into average-case behavior. For convex hulls, a random insertion order yields $O(n \log n)$ expected time (Section 10.2). For linear programming, the randomized incremental algorithm of Seidel runs in expected $O(d!n)$ time, which is linear in n for fixed dimension d (Section 10.9).

This principle—*randomizing the order of processing eliminates the adversary’s power to choose a bad order*—is one of the most practical benefits of randomization in algorithm design. It requires no change to the input itself, only to the algorithm’s scheduling.

1.1.2 Problems with no known efficient deterministic solution

There are problems for which no polynomial-time deterministic algorithm is known, despite intensive research over many decades. Three categories illustrate this:

1. **NP-hard problems.** The class NP encompasses decision problems whose YES instances admit short certificates verifiable in polynomial time. NP-hard problems are at least as hard as every problem in NP. No polynomial-time algorithm for any NP-hard problem is known, and the existence of such an algorithm would imply $\mathbf{P} = \mathbf{NP}$. Famous NP-hard problems include the traveling salesman problem, graph coloring, and satisfiability (SAT). While no algorithm—deterministic or randomized—is known to solve these efficiently in general, randomization yields algorithms that perform well on average or that produce approximate solutions with provable guarantees (Chapter 12).

2. **Problems requiring global information.** Some problems seem inherently difficult for deterministic algorithms because a deterministic procedure cannot “explore” the solution space without being trapped by adversarial structure. Randomized algorithms can avoid such traps by exploring the space in an unpredictable fashion.
3. **Adversarial inputs.** A deterministic algorithm, once its strategy is known, can be defeated by an adversary who constructs a worst-case input tailored to exploit the algorithm’s predictable behavior. Randomness removes this vulnerability: an adversary cannot predict the algorithm’s random choices.

1.1.3 The adversarial argument

The idea that an adversary can defeat a deterministic algorithm is formalized in the following observation. Let A be a deterministic algorithm for a problem Π . Suppose there exists an input x on which A takes super-polynomial time or produces an incorrect answer. Then the adversary who knows A can always present x as the input. The deterministic algorithm has no recourse—it will always behave badly on x .

A randomized algorithm R , by contrast, makes random choices during its execution. Even if the adversary knows the code of R and the input x , the adversary cannot predict the outcome of R ’s random coins. The best the adversary can do is ensure that R performs badly on *some* set of random outcomes; but if R is well designed, the set of “bad” outcomes has small probability.

This is the fundamental insight behind the study of randomized algorithms: *randomness provides immunity against adversarial worst-case inputs.*

1.1.4 A concrete example: finding a median

Consider the problem of finding the median of n numbers. The classical deterministic algorithm of Blum, Floyd, Pratt,

Rivest, and Tarjan solves this in $O(n)$ time, but the constant hidden in the O -notation is large—the algorithm is complex and rarely used in practice. The “textbook” deterministic algorithm based on sorting takes $O(n \log n)$ time.

A simple randomized algorithm achieves $O(n)$ expected time with remarkable ease: choose a random element as a pivot, partition the array, and recurse into the side containing the median. The analysis (see Chapter 4) shows that the expected number of comparisons is at most $4n$. This algorithm is not only simpler than its deterministic counterpart but also faster in practice, despite being randomized. (A deterministic linear-time median algorithm with smaller constants was later discovered, but the randomized version remains a compelling illustration of the power of simplicity.)

1.2 Random Number Generation

Every randomized algorithm ultimately depends on a source of randomness. In theoretical analysis we model randomness as an infinite sequence of independent, uniformly distributed random bits. In practice, we need algorithms—*pseudorandom number generators* (PRNGs)—that produce sequences that are “random enough” for the application at hand.

1.2.1 Pseudorandom number generators

A PRNG is a deterministic procedure that, given an initial value called a *seed*, produces a sequence of numbers that appears statistically random. The sequence is entirely determined by the seed; two runs with the same seed produce identical sequences.

Modern PRNGs are designed to satisfy two desiderata:

1. **Statistical quality:** The output should be indistinguishable from truly random bits by efficient statistical tests.
2. **Efficiency:** The generation of each pseudorandom number should take $O(1)$ time.

The most widely used PRNG in practice is the *Mersenne Twister* (mt19937), which produces 32-bit pseudorandom integers with a period of $2^{19937} - 1$ and passes most standard statistical tests. C++11 provides `std::mt19937` and related distributions via the `<random>` header.

Listing 1.1: Using the C++ `<random>` library.

```
1 #include <random>
2 #include <chrono>
3 #include <iostream>
4
5 int main() {
6     // Seed from system clock
7     unsigned seed = static_cast<unsigned>(
8         std::chrono::steady_clock::now()
9         .time_since_epoch().count());
10    std::mt19937 eng(seed);
11
12    // Uniform integer in [1, 100]
13    std::uniform_int_distribution<int> d1(1,100);
14    std::cout << "Dice roll: " << d1(eng) << "\n";
15
16    // Uniform real in [0.0, 1.0)
17    std::uniform_real_distribution<double> d2(0.0,1.0);
18    std::cout << "Uniform:  " << d2(eng) << "\n";
19
20    // Normal distribution
21    std::normal_distribution<double> d3(0.0,1.0);
22    std::cout << "Gaussian:  " << d3(eng) << "\n";
23 }
```

1.2.2 Seeding and reproducibility

The choice of seed is critical for reproducibility. In scientific simulations and debugging, it is common to fix the seed so that results can be reproduced exactly. In production systems, the seed is typically drawn from a source of entropy (e.g., the system clock, hardware noise, or `/dev/urandom` on Unix systems).

Using the same seed with the same PRNG always yields the same sequence. This property is invaluable for debugging: a bug that manifests on a particular input can be reproduced by using the same seed.

1.2.3 Cryptographically secure PRNGs

For cryptographic applications, standard PRNGs are insufficient: an attacker who observes a portion of the output should not be able to predict future output. Cryptographically secure PRNGs (CSPRNGs) are designed to resist such attacks. On most systems, `/dev/urandom` (Unix) or `BCryptGenRandom` (Windows) provide CSPRNG output.

1.2.4 Weak randomness versus strong randomness

Not all applications require the same quality of randomness. This distinction leads to two categories of random sequences.

Weak random sequences

A *weak random sequence* is one generated by a simple PRNG with modest statistical guarantees. Examples include linear congruential generators (LCGs) and the standard C `rand()` function. Weak PRNGs are fast and require little state, but their output has detectable correlations: LCGs suffer from lattice structure in higher dimensions, and short periods cause repeat patterns in long runs.

Weak randomness suffices for:

- **Monte Carlo integration.** Estimating π by sampling points in a square (Section 1.4.7) converges at $O(1/\sqrt{n})$ regardless of mild correlations in the samples.
- **Randomized incremental algorithms.** Randomizing the insertion order for convex hulls or Delaunay triangulations (Chapter 10) only requires that each permutation be approximately equally likely; a weak PRNG suffices.
- **Approximate counting.** The Karp–Luby scheme for DNF counting (Chapter 12) relies on the expectation of indicator variables; the analysis holds as long as samples are pairwise independent.

However, weak randomness can silently invalidate results in applications that demand uniformity over large state spaces. The RANDU generator (a notorious LCG from the 1960s) produced triples $(x, W(x), W(W(x)))$ that all lay on 15 planes in 3D space, corrupting 3D Monte Carlo simulations.

Strong random sequences

A *strong random sequence* is one that is statistically indistinguishable from truly random bits by any polynomial-time test. The Mersenne Twister (mt19937) with its $2^{19937} - 1$ period qualifies for most non-cryptographic uses. Cryptographically secure PRNGs (CSPRNGs) provide the strongest guarantee: no polynomial-time adversary can predict future bits given access to previous output.

Strong randomness is required for:

- **Cryptographic protocols.** Key generation, zero-knowledge proofs, and commitment schemes (Section 15.4) require unpredictability against an active adversary. A weak PRNG-derived RSA key can be factored.
- **Derandomization.** The method of conditional probabilities (Chapter 6) and pairwise independence (Chapter 4) require true random bits or provably strong generators.
- **Long-running simulations.** Simulations of 10^{12} steps exhaust the period of a 32-bit LCG ($\approx 4 \times 10^9$). The Mersenne Twister's period of 2^{19937} is effectively inexhaustible.

Effect on algorithmic guarantees

A randomized algorithm's theoretical guarantees are proven under the assumption of true randomness (independent, uniformly distributed bits). When implemented with a PRNG, the empirical behavior may differ:

- **Las Vegas algorithms.** The running time distribution may shift slightly, but correctness is unaffected: the algorithm always produces the right answer regardless of the random choices.
- **Monte Carlo algorithms.** The error probability may increase if the PRNG introduces correlations. For example, using `rand()` with the Miller–Rabin primality test can misclassify Carmichael numbers more frequently than the theoretical bound 4^{-k} .
- **Probabilistic method constructions.** The non-constructive existence proofs (Chapter 6) are unaffected by the PRNG quality, but any explicit construction derived from de-randomization requires strong randomness.

Remark 1.1. In practice, the Mersenne Twister (`mt19937`) provides sufficient randomness for nearly all non-cryptographic randomized algorithms discussed in this book. For cryptographic applications (Chapter 15), always use a CSPRNG from the operating system’s entropy pool.

1.2.5 True randomness versus pseudorandomness

No sequence produced by a deterministic algorithm is truly random. Every PRNG is a finite-state machine: given the same seed, it always produces the same output. The sequence only *appears* random to computational tests.

Sources of true randomness

True randomness exists in physical processes:

- **Quantum phenomena.** Radioactive decay, photon detection, and shot noise in semiconductor junctions are fundamentally unpredictable. Hardware random number generators (HRNGs) exploit these effects.
- **Atmospheric noise.** Radio static from lightning strikes provides a continuous source of entropy.

- **Human interaction.** Mouse movements, keyboard timings, and disk seek latencies are unpredictable enough for seeding PRNGs.

Modern operating systems maintain entropy pools fed by hardware interrupts. On Linux, `/dev/random` blocks when entropy is low; `/dev/urandom` does not but uses a CSPRNG to expand the entropy.

Practical implications

For most applications, a well-seeded PRNG is indistinguishable from true randomness. However, three situations require genuine entropy:

1. **Cryptographic key generation.** A key generated by a PRNG with a predictable seed can be recovered by an adversary.
2. **Scientific reproducibility.** True randomness cannot reproduce a simulation; PRNGs with saved seeds enable exact reproducibility.
3. **Algorithmic fairness.** Lotteries, elections, and audit selection must resist manipulation. Physical entropy sources (or public-coin protocols) provide verifiable randomness.

Buffon's needle: the first Monte Carlo method

The earliest documented use of randomness to compute a mathematical constant predates electronic computers by two centuries.

Example 1.2 (Buffon's Needle, 1777). A needle of length ℓ is dropped onto a floor made of parallel wooden strips of width d (with $\ell \leq d$). What is the probability that the needle crosses a crack between strips?

Let $y \in [0, d/2]$ be the distance from the needle's midpoint to the nearest crack, and $\theta \in [0, \pi/2]$ be the acute angle between the needle and the crack direction. The needle crosses

a crack iff $y \leq (\ell/2) \sin \theta$. Assuming y and θ are independent and uniformly distributed, the probability is

$$p = \frac{2}{\pi} \cdot \frac{\ell}{d}.$$

By repeating the experiment n times and counting the number m of crossings, we estimate π as

$$\pi \approx \frac{2\ell n}{dm}.$$

Buffon's needle is historically significant for three reasons:

1. It is the first recorded *Monte Carlo method*—using random samples to compute a deterministic quantity.
2. It demonstrates that randomness can evaluate integrals without analytic solution, a principle now central to computational physics, finance, and Bayesian statistics.
3. The convergence rate $O(1/\sqrt{n})$ is characteristic of all Monte Carlo methods and sets a fundamental limit on their accuracy.

The needle problem also illustrates a recurring theme: a randomized algorithm's guarantees rely on the quality of the random source. If the dropped positions are biased (e.g. the dropper unconsciously avoids cracks), the estimate of π becomes systematically wrong. Similarly, a PRNG with low-period correlations can introduce bias in Monte Carlo simulations that is invisible to simple statistical tests.

1.2.6 Random bits vs. random numbers

Most randomized algorithms require random integers or reals, not raw random bits. From a theoretical standpoint, this distinction is immaterial: a random k -bit integer can be obtained by sampling k independent random bits. In practice, PRNGs are often optimized for generating integers in a given range, which is more efficient than generating raw bits and reducing modulo.

Remark 1.3. Throughout this book, we assume the availability of a source of independent, uniformly distributed random bits. When analyzing randomized algorithms, we study the probability distribution over all possible random outcomes. The quality of the PRNG used in an implementation affects the empirical performance but not the theoretical guarantees (which hold for truly random coins).

1.3 Preliminary Mathematics

The analysis of randomized algorithms draws heavily on elementary probability theory. In this section we review the tools that will be used throughout the book. More advanced material (Chernoff bounds, martingales, the probabilistic method) is introduced in later chapters as needed.

1.3.1 Sample spaces and events

A *probability space* is a triple $(\Omega, \mathcal{F}, \Pr)$, where Ω is a set (the *sample space*), $\mathcal{F}$ is a collection of subsets of Ω (the *events*), and $\Pr$ is a probability measure satisfying $\Pr[\Omega] = 1$ and countable additivity.

For finite sample spaces, we typically take $\mathcal{F}$ to be all subsets of Ω and assign a probability $\Pr[\omega]$ to each outcome $\omega \in \Omega$ such that $\sum_{\omega \in \Omega} \Pr[\omega] = 1$. For a uniform distribution, every outcome has probability $1/|\Omega|$.

1.3.2 Independence and conditional probability

Two events $\mathcal{E}_1$ and $\mathcal{E}_2$ are said to be *independent* if the probability that they both occur is given by

$$\Pr[\mathcal{E}_1 \cap \mathcal{E}_2] = \Pr[\mathcal{E}_1] \times \Pr[\mathcal{E}_2]. \quad (1.3)$$

In the more general case where $\mathcal{E}_1$ and $\mathcal{E}_2$ are not necessarily independent,

$$\Pr[\mathcal{E}_1 \cap \mathcal{E}_2] = \Pr[\mathcal{E}_1] \times \Pr[\mathcal{E}_2 \mid \mathcal{E}_1], \quad (1.4)$$

where $\Pr[\mathcal{E}_1 \mid \mathcal{E}_2]$ denotes the conditional probability of $\mathcal{E}_1$ given $\mathcal{E}_2$. The chain rule for probabilities extends this to k events:

$$\Pr\left[\bigcap_{i=1}^k \mathcal{E}_i\right] = \Pr[\mathcal{E}_1] \times \Pr[\mathcal{E}_2 \mid \mathcal{E}_1] \times \Pr[\mathcal{E}_3 \mid \mathcal{E}_1 \cap \mathcal{E}_2] \cdots \Pr\left[\mathcal{E}_k \mid \bigcap_{i=1}^{k-1} \mathcal{E}_i\right]. \quad (1.5)$$

We used this chain rule in deriving the success probability of Karger's min-cut algorithm (Eq. 1.1).

1.3.3 Expectation and linearity

The *expected value* (or *mean*) of a discrete random variable X taking values $x_1, x_2, \dots$ with probabilities $p_1, p_2, \dots$ is

$$\mathbb{E}[X] = \sum_i x_i p_i.$$

The single most useful tool in the analysis of randomized algorithms is *linearity of expectation*: for random variables $X_1, X_2, \dots, X_n$ (which need not be independent),

$$\mathbb{E}\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n \mathbb{E}[X_i]. \quad (1.6)$$

Example 1.4 (The Sailor Problem). A ship arrives at a port, and the 40 sailors on board go ashore for revelry. Later at night, the 40 sailors return to the ship and, in their state of inebriation, each chooses a random cabin to sleep in. What is the expected number of sailors sleeping in their own cabins?

Let X_i be 1 if the i th sailor chooses her own cabin, and 0 otherwise. We wish to compute $\mathbb{E}[\sum_{i=1}^{40} X_i]$. By linearity of expectation, this is $\sum_{i=1}^{40} \mathbb{E}[X_i]$. Since the cabins are chosen at random, $\mathbb{E}[X_i] = 1/40$. Thus the expected number is $\sum_{i=1}^{40} 1/40 = 1$.

1.3.4 Variance and deviations

The *variance* of a random variable X measures its spread:

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2.$$

The *standard deviation* is $\sigma(X) = \sqrt{\text{Var}(X)}$.

Chebyshev's inequality bounds the probability that X deviates significantly from its mean:

$$\Pr[|X - \mathbb{E}[X]| \geq t] \leq \frac{\text{Var}(X)}{t^2}. \quad (1.7)$$

This inequality will be applied frequently in Chapter 4, where we develop sharper tail bounds for sums of independent random variables.

1.3.5 The union bound

A simple yet powerful tool is the *union bound* (or *Boole's inequality*): for events $\mathcal{E}_1, \mathcal{E}_2, \dots, \mathcal{E}_n$ (not necessarily independent),

$$\Pr\left[\bigcup_{i=1}^n \mathcal{E}_i\right] \leq \sum_{i=1}^n \Pr[\mathcal{E}_i]. \quad (1.8)$$

We used the union bound implicitly in the analysis of Karger's algorithm when bounding the probability that any edge of the min-cut is contracted.

1.4 The Randomized Approach

A randomized algorithm is an algorithm that can toss coins during its execution. Formally, we model a randomized algorithm R as a deterministic algorithm that takes as an additional input a sequence $r = r_1, r_2, \dots$ of independent, uniformly distributed random bits. The output and running time of R are therefore random variables that depend on r .

Randomness serves several distinct roles in algorithm design. We summarize the four principal motivations before examining each in turn.

1. **Improved average-case complexity.** Even when efficient deterministic algorithms exist, randomization can yield better expected running time or simpler analysis.

2. **No efficient deterministic algorithm known.** For certain problems, randomization provides the only known polynomial-time solution.
3. **Simplicity.** Randomized algorithms are often dramatically easier to design, implement, and analyze than their deterministic counterparts.
4. **Adversarial immunity.** Randomness shields an algorithm against worst-case inputs crafted by an adversary who knows the algorithm's strategy.

1.4.1 Improved average-case complexity

The classical example is sorting. Deterministic comparison-based sorting achieves $O(n \log n)$ worst-case time, but the algorithms that achieve this bound (merge sort, heap sort) have overheads that make them slower in practice than quicksort. Deterministic quicksort, however, has $O(n^2)$ worst-case time. Randomized quicksort resolves this dilemma: it always produces a correctly sorted array (Las Vegas), yet its expected running time is $O(n \log n)$, and its practical performance is excellent because the constant factors are small. The random choice of pivot eliminates the pathological inputs that defeat deterministic quicksort.

A second example is randomized selection (Section 4.3). The deterministic median-finding algorithm of Blum et al. runs in $O(n)$ time but has large constants. A simple randomized algorithm achieves $O(n)$ expected time with much smaller constants by choosing pivots at random.

Further examples include:

- **Hash tables.** Universal hashing (Section 9.3) guarantees $O(1)$ expected time for insertions, deletions, and lookups, even under adversarial keys. Deterministic hash tables require strong assumptions about the input distribution or more complex data structures.

- **Skip lists.** Skip lists (Section 9.2) achieve $O(\log n)$ expected time for all dictionary operations with a simple randomized data structure that competes with balanced trees.
- **Randomized incremental algorithms.** Building a convex hull of n points (Section 10.2) in expected $O(n \log n)$ time using randomized incremental insertion avoids the complex case analysis of deterministic algorithms like Graham scan.
- **Load balancing.** Throwing m balls into n bins uniformly at random (Chapter 4) yields a maximum load of $O(m/n + \sqrt{m \log n})$. When $m \approx n$, the maximum load is $O(\log n / \log \log n)$ with high probability, far smaller than the deterministic worst case.

1.4.2 No efficient deterministic algorithm known

Before 2002, no polynomial-time deterministic algorithm was known for primality testing. The randomized Miller–Rabin test (Chapter 15) runs in $O(k \log^3 n)$ time with error probability at most 4^{-k} , making it the method of choice for decades. The deterministic AKS algorithm, discovered in 2002, runs in $O(\log^{7.5} n)$ time but is significantly slower in practice.

Further examples include:

- **Polynomial identity testing.** The Schwartz–Zippel lemma (Section 8.2) provides a randomized algorithm for testing whether a polynomial is identically zero that is dramatically faster than any known deterministic method.
- **Interactive proof systems.** The class **IP** equals **PSPACE** only when randomness and interaction are both allowed (Section 8.8). Without randomness, interactive proofs collapse to **NP**.
- **Approximate counting.** Counting the number of satisfying assignments to a DNF formula (Section 12.2) admits a fully polynomial randomized approximation scheme,

while no deterministic approximation scheme with comparable guarantees is known.

- **Perfect matching in parallel.** Finding a perfect matching in a graph can be done in **RNC** (randomized NC) using the Isolating Lemma (Section 13.4). No **NC** algorithm (deterministic polylogarithmic-time parallel) is known.

1.4.3 Simplicity

Karger's min-cut algorithm (Section 1.4.2) is a striking illustration. The deterministic algorithms for min-cut are based on network flows and require sophisticated data structures. Karger's randomized algorithm, by contrast, is a dozen lines of code: repeatedly contract a random edge. The analysis is elegant and the implementation is trivial.

Further examples include:

- **Freivalds' algorithm.** Verifying the product of two $n \times n$ matrices (Section 8.1) can be done in $O(n^2)$ time with a randomized algorithm that multiplies a vector by each matrix. The deterministic $O(n^2)$ verification algorithm requires a complicated reduction; the randomized version is a few lines of code.
- **Luby's maximal independent set.** Luby's algorithm (Section 13.3) for finding a maximal independent set in a graph is a simple randomized parallel algorithm that runs in $O(\log n)$ rounds. Deterministic parallel algorithms for MIS are significantly more complex.
- **Randomized incremental construction.** Deterministic algorithms for problems like convex hull, Delaunay triangulation, and linear programming are often complex, requiring careful case analysis. The randomized incremental approach (Chapter 10) replaces this complexity with a simple random sampling strategy.

- **Tree data structures.** Random treaps and skip lists (Sections 8.1–8.2) provide simple alternatives to balanced binary search trees, replacing rotation invariants with randomization.

1.4.4 Adversarial immunity

An adversary who knows a deterministic algorithm's code can construct a worst-case input tailored to exploit its predictable behavior. Randomness removes this vulnerability: the adversary cannot predict the algorithm's random choices.

Further examples include:

- **Online paging.** No deterministic online algorithm can achieve a competitive ratio better than k for k -page caching. The randomized marking algorithm (Section 14.1) achieves $O(\log k)$ competitive ratio against an oblivious adversary.
- **Load balancing.** When jobs are assigned to machines by random choice, the maximum load on any machine is concentrated near the average, regardless of the job sizes—no adversary can force a bad assignment.
- **Byzantine agreement.** Deterministic Byzantine agreement requires $n \geq 3t + 1$ processors to tolerate t faults. Randomized Byzantine agreement (Section 13.6) can tolerate $t < n/3$ faults with expected constant-round protocols, circumventing the deterministic lower bound.
- **Quicksort.** Deterministic quicksort is vulnerable to an adversary who supplies a pre-sorted array, causing $O(n^2)$ behavior. Randomized quicksort eliminates this attack by choosing pivots randomly.
- **Routing on the hypercube.** The randomized oblivious routing algorithm for the hypercube (Section 5.2) achieves $O(\log n)$ expected delay for any permutation, while deterministic oblivious routing can require $\Omega(\sqrt{n})$ delay for some permutations.

1.4.5 Las Vegas and Monte Carlo algorithms

Randomized algorithms differ in the nature of their randomness.

Definition 1.5 (Las Vegas Algorithm). A *Las Vegas algorithm* is a randomized algorithm that always produces the correct output. Its running time is a random variable, and we characterize it by its expected value.

Definition 1.6 (Monte Carlo Algorithm). A *Monte Carlo algorithm* is a randomized algorithm that may produce an incorrect output with some small probability. Its running time is typically a deterministic bound (e.g., polynomial in the input size).

Monte Carlo algorithms for decision problems (problems with YES/NO answers) come in two flavors:

- **One-sided error:** The algorithm may err only when it outputs NO (or only when it outputs YES), but never in the other case.
- **Two-sided error:** The algorithm may err regardless of the correct answer.

A Las Vegas algorithm is a special case of a Monte Carlo algorithm with error probability zero. Conversely, whenever solutions can be verified efficiently, a Monte Carlo algorithm can be converted into a Las Vegas algorithm by repeating and checking (see Exercise 1.2 below).

1.4.6 The min-cut algorithm: a first example

We now illustrate the power of randomization with a beautiful algorithm for the min-cut problem. Let G be a connected, undirected multigraph with n vertices. A *cut* in G is a set of edges whose removal disconnects G . A *min-cut* is a cut of minimum cardinality.

The following randomized algorithm, due to Karger, is disarmingly simple. We repeat the following step: pick an edge uniformly at random and merge the two vertices at its endpoints. If there are several edges between some pairs of newly formed vertices, retain them all. Edges between vertices that are merged are removed, so that there are never any self-loops. We refer to this process of merging the two endpoints of an edge into a single vertex as the *contraction* of that edge. With each contraction, the number of vertices decreases by one. The crucial observation is that an edge contraction does not reduce the min-cut size: every cut in the graph at any intermediate stage is a cut in the original graph. The algorithm continues the contraction process until only two vertices remain; the set of edges between these two vertices is output as a candidate min-cut.

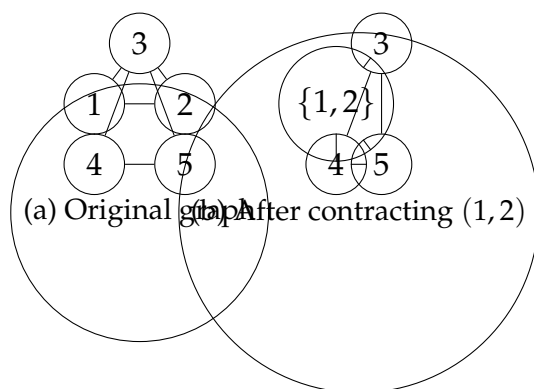


Figure 1.1: A step in the min-cut algorithm; the effect of contracting edge $e = (1, 2)$ is shown.

Does this algorithm always find a min-cut? No—it may err. But we can bound the probability of error. Let k be the min-cut size. We fix our attention on a particular min-cut C with k edges. Clearly G has at least $kn/2$ edges; otherwise there would be a vertex of degree less than k , and its incident edges would be a min-cut of size less than k .

Let e_i denote the event of not picking an edge of C at the i th step, for $1 \leq i \leq n - 2$. The probability that the edge ran-

domly chosen in the first step is in C is at most $k/(nk/2) = 2/n$, so that $\Pr[e_1] \geq 1 - 2/n$. Assuming that e_1 occurs, during the second step there are at least $k(n-1)/2$ edges, so the probability of picking an edge in C is at most $2/(n-1)$, so that $\Pr[e_2 \mid e_1] \geq 1 - 2/(n-1)$. At the i th step, the number of remaining vertices is $n - i + 1$. The min-cut size is still at least k , so the graph has at least $k(n-i+1)/2$ edges. Thus, $\Pr[e_i \mid \bigcap_{j=1}^{i-1} e_j] \geq 1 - 2/(n-i+1)$.

The probability that no edge of C is ever picked is

$$\Pr\left[\bigcap_{i=1}^{n-2} e_i\right] = \prod_{i=1}^{n-2} \left(1 - \frac{2}{n-i+1}\right) = \frac{2}{n(n-1)}. \quad (1.1)$$

The probability of discovering a particular min-cut is therefore at least $2/n^2$. To reduce the failure probability, we repeat the algorithm $n^2/2$ times, making independent random choices each time. The probability that a min-cut is not found in any attempt is at most

$$\left(1 - \frac{2}{n^2}\right)^{n^2/2} < \frac{1}{e}. \quad (1.2)$$

Further repetitions make the failure probability arbitrarily small—the only consideration being that repetitions increase running time. Note the extreme simplicity of this randomized algorithm compared to the deterministic algorithms for min-cut, which are based on network flows and are considerably more complex.

Exercise 1.7. Suppose that at each step of the min-cut algorithm, instead of choosing a random edge for contraction we choose two vertices at random and coalesce them into a single vertex. Show that there are inputs on which the probability that this modified algorithm finds a min-cut is exponentially small.

1.4.7 C++ Implementation

The following implementation demonstrates Karger's algorithm. A multigraph is represented with weighted adjacency

lists; the weight of edge (u, v) equals its multiplicity.

Listing 1.2: Graph data structure for the min-cut algorithm.

```
1 #include <vector>
2 #include <unordered_map>
3 #include <algorithm>
4 #include <numeric>
5 #include <climits>
6 #include <random>
7 #include <chrono>
8 #include <iostream>
9 #include <cmath>
10
11 class RandomEngine {
12 public:
13     static RandomEngine& instance() {
14         static RandomEngine e; return e;
15     }
16     int uniform_int(int lo, int hi) {
17         return std::uniform_int_distribution<int>(lo,hi)(eng_);
18     }
19     double uniform_real(double lo=0,double hi=1) {
20         return std::uniform_real_distribution<double>(lo,hi)(eng_);
21     }
22 private:
23     RandomEngine()
24         : eng_(std::chrono::steady_clock::now()
25             .time_since_epoch().count()) {}
26     std::mt19937 eng_;
27 };
28
29 struct Graph {
30     int n;
31     std::unordered_map<int,
32         std::unordered_map<int,int>> adj;
33     explicit Graph(int n) : n(n) {}
34     void add_edge(int u, int v, int c=1) {
35         adj[u][v]+=c; adj[v][u]+=c;
36     }
37     int edge_count() const {
38         int t=0;
39         for (auto& [u,nb] : adj)
40             for (auto& [v,w] : nb)
41                 if (u<v) t+=w;
42         return t;
43     }
44 };
```

Listing 1.3: Karger's min-cut algorithm.

```
1 void contract(auto& adj, int u, int v) {
2     for (auto& [w,wt] : adj[v])
3         if (w!=u) { adj[u][w]+=wt; adj[w][u]+=wt; }
4     adj.erase(v);
5     for (auto& [_ ,nb] : adj) nb.erase(v);
```

```

6   adj[u].erase(u);           // remove self-loops
7 }
8
9 int karger_min_cut(Graph& g) {
10     auto adj = g.adj;
11     std::vector<int> vs(g.n);
12     std::iota(vs.begin(), vs.end(), 0);
13     while (vs.size()>2) {
14         // collect all edges (with multiplicity)
15         std::vector<std::pair<int,int>> E;
16         for (auto& [u,nb] : adj)
17             for (auto& [v,w] : nb)
18                 if (u<v)
19                     for (int i=0;i<w;i++) E.push_back({u,v});
20         if (E.empty()) break;
21         auto [u,v] = E[RandomEngine::instance()
22                     .uniform_int(0,E.size()-1)];
23         contract(adj,u,v);
24         vs.erase(std::remove(vs.begin(),vs.end(),v),
25                 vs.end());
26     }
27     return vs.size()==2 ? adj[vs[0]][vs[1]] : 0;
28 }
29
30 int karger_repeated(Graph& g, int T) {
31     int best = INT_MAX;
32     for (int t=0;t<T;t++)
33         best = std::min(best, karger_min_cut(g));
34     return best;
35 }

```

Listing 1.4: Driver: empirical verification of the theoretical bounds.

```

1 int main() {
2     Graph g(5);
3     g.add_edge(0,1); g.add_edge(0,2); g.add_edge(0,3);
4     g.add_edge(1,2); g.add_edge(1,3); g.add_edge(2,3);
5     g.add_edge(2,4); g.add_edge(3,4);
6
7     int n=g.n, T=(n*n)/2;
8     std::cout<<"Single trial : "<<karger_min_cut(g)<<"\n";
9     std::cout<<T<<" trials      : "<<karger_repeated(g,T)<<"\n";
10
11     double p=2.0/(n*(n-1));
12     std::cout<<"P(success) >= 2/n(n-1) = "<<p<<"\n";
13     std::cout<<"P(failure in "<<T<<" trials) <= "
14         <<std::pow(1-p,T)<<"\n";
15     std::cout<<"1/e = "<<1.0/std::exp(1.0)<<"\n";
16 }

```

Complexity. Each trial of the algorithm runs in $O(n^2)$ time with the adjacency-map representation. With $n^2/2$ trials the

total expected running time is $O(n^4)$.

1.4.8 C++ Implementation: Randomized Quick-Sort

Randomized QuickSort is the canonical Las Vegas algorithm: it always produces a correctly sorted array, yet its running time is a random variable with $\mathbb{E}[T(n)] = O(n \log n)$.

Listing 1.5: Randomized QuickSort—a Las Vegas algorithm.

```
1 int partition(std::vector<int>& a, int lo, int hi) {
2     int pi = RandomEngine::instance().uniform_int(lo,hi);
3     std::swap(a[pi], a[hi]);
4     int pivot = a[hi], i = lo-1;
5     for (int j=lo; j<hi; j++)
6         if (a[j]<=pivot) std::swap(a[++i], a[j]);
7     std::swap(a[i+1], a[hi]);
8     return i+1;
9 }
10
11 void rand_quicksort(std::vector<int>& a,int lo,int hi){
12     if (lo<hi) {
13         int p = partition(a,lo,hi);
14         rand_quicksort(a,lo,p-1);
15         rand_quicksort(a,p+1,hi);
16     }
17 }
```

1.4.9 C++ Implementation: Monte Carlo—Estimating π

The following Monte Carlo algorithm estimates π by sampling. It has two-sided error: the estimate may be too high or too low, but the error decreases as $O(1/\sqrt{n})$.

Listing 1.6: Monte Carlo estimation of π (two-sided error).

```
1 double estimate_pi(int n) {
2     int inside = 0;
3     for (int i=0;i<n;i++) {
4         double x = RandomEngine::instance()
5             .uniform_real(-1,1);
6         double y = RandomEngine::instance()
7             .uniform_real(-1,1);
8         if (x*x+y*y<=1.0) inside++;
9     }
10     return 4.0*inside/n;
}
```


11 }

Samples	Estimate	Error
100	3.12000	0.02159
1 000	3.14400	0.00241
10 000	3.13840	0.00319
100 000	3.14076	0.00083

Table 1.1: Typical output of the π estimator.

1.4.10 Converting Between Las Vegas and Monte Carlo

A Las Vegas algorithm can be converted to a Monte Carlo algorithm by truncation. If the Las Vegas algorithm has expected running time $T(n)$, run it for $2T(n)$ steps and output an arbitrary default answer if it has not finished. By Markov's inequality, the probability of premature truncation is at most $1/2$, so the resulting Monte Carlo algorithm errs with probability at most $1/2$ and always runs in $O(T(n))$ time. Conversely, when solutions can be *verified* efficiently, a Monte Carlo algorithm can be turned into a Las Vegas algorithm by repeating until verification succeeds.

Listing 1.7: Monte Carlo $\rightarrow$ Las Vegas via verification.

```

1 // Monte Carlo: correct w.p.  $\sim 2/3$ 
2 std::pair<int,bool> mc(int x) {
3     bool ok = RandomEngine::instance()
4         .uniform_real() < 2.0/3.0;
5     return ok ? std::make_pair(2*x,true)
6         : std::make_pair(3*x,false);
7 }
8
9 bool verify(int x, int r) { return r==2*x; }
10
11 // Las Vegas wrapper
12 std::pair<int,int> mc_to_lv(int x) {
13     int tries=0;
14     while (true) {
15         tries++;
16         auto [r,ok] = mc(x);

```

```
17     if (verify(x,r)) return {r,tries};  
18     }  
19 }
```

Remark 1.8. If the single-trial success probability is p , the number of independent trials until the first success follows a *geometric distribution* with mean $1/p$. Hence the expected running time of the Las Vegas wrapper is $(T(n) + t(n))/p$.

Chapter 2

Fundamentals

In this chapter we develop fundamental mathematical tools and computational models that underpin the randomized algorithms studied in the remainder of the book. We begin with the construction of binary planar partitions, illustrating linearity of expectation with a geometric application. We then study a probabilistic recurrence that arises in the analysis of selection and other divide-and-conquer algorithms. Finally, we review the computation model and complexity classes that provide the formal framework for our discussions throughout the book.

2.1 Binary Planar Partitions

We now illustrate linearity of expectation with another application. Given a set $S = \{s_1, s_2, \dots, s_n\}$ of non-intersecting line segments in the plane, we wish to construct a *binary planar partition*—a binary tree in which each node is associated with a region of the plane, and each region contains at most one line segment. Such partitions have applications to hidden-line elimination in computer graphics.

2.1.1 Binary Planar Partitions

Our next illustration is the construction of a binary planar partition of a set of n disjoint line segments in the plane, a problem with applications to computer graphics. A binary planar partition consists of a binary tree together with some additional information, as described below. Every internal node of the tree has two children. Associated with each node v of the tree is a region $r(v)$ of the plane. Associated with each internal node v of the tree is a line $\ell(v)$ that intersects $r(v)$. The region corresponding to the root is the entire plane. The region $r(v)$ is partitioned by $\ell(v)$ into two regions $r_1(v)$ and $r_2(v)$, which are the regions associated with the two children of v . Thus, any region r of the partition is bounded by the partition lines on the path from the root to the node corresponding to r in the tree.

Given a set $S = \{s_1, s_2, \dots, s_n\}$ of non-intersecting line segments in the plane, we wish to find a binary planar partition such that every region in the partition contains at most one line segment (or a portion of one line segment). Notice that the definition allows us to divide an input line segment s_i into several segments $s_{i1}, s_{i2}, \dots$, each of which lies in a different region.

Exercise 2.1. Show that there exists a set of line segments for which no binary planar partition can avoid breaking up some of the segments into pieces, if each segment is to lie in a different region of the partition.

Binary planar partitions have two applications in computer graphics. Here, we describe one of them, the problem of hidden line elimination. The second application has to do with the constructive solid geometry (or CSG) representation of a polyhedral object.

In rendering a scene on a graphics terminal, we are often faced with a situation in which the scene remains fixed, but it is to be viewed from several directions. The hidden line elimination problem is the following: having adopted a viewpoint and a direction of viewing, we want to draw only the portion

of the scene that is visible, eliminating those objects that are “obscured by other objects” in front of them relative to the viewpoint.

One approach uses a binary partition tree. Once the scene has been decomposed into line segments, we construct a binary planar partition tree for it. Now, given the direction of viewing, we use an idea known as the *painter’s algorithm* to render the scene: first draw the objects that are furthest “behind,” and then progressively draw the objects that are in front.

The time it takes to render the scene depends on the size of the binary planar partition tree. We therefore wish to construct a binary planar partition that is as small as possible. Because the construction of the partition can break some of the input segments s_i into smaller pieces, the size of the partition need not be n ; in fact, it is not clear that a partition of size $O(n)$ always exists.

For a line segment s , let $\ell(s)$ denote the line obtained by extending (if necessary) s on both sides to infinity. For the set $S = \{s_1, s_2, \dots, s_n\}$ of line segments, a simple and natural class of partitions is the set of *autopartitions*, which are formed by only using lines from the set $\{\ell(s_1), \ell(s_2), \dots, \ell(s_n)\}$ in constructing the partition.

Algorithm 1 RandAuto

- 1: **Input:** A set $S = \{s_1, \dots, s_n\}$ of non-intersecting line segments.
 - 2: **Output:** A binary autopartition P_π of S .
 - 3: Pick a permutation π of $\{1, 2, \dots, n\}$ uniformly at random.
 - 4: **while** a region contains more than one segment **do**
 - 5: Cut it with $\ell(s_i)$ where i is first in π such that s_i cuts that region.
 - 6: **end while**
-

Theorem 2.2. *The expected size of the autopartition produced by RandAuto is $O(n \log n)$.*

Proof. For line segments u and v , define $\text{index}(u, v)$ to be i if $\ell(u)$ intersects $i-1$ other segments before hitting v , and $\text{index}(u, v) = \infty$ if $\ell(u)$ does not hit v . Since a segment u can be extended in two directions, it is possible that $\text{index}(u, v) = \text{index}(u, w)$ for two different lines v and w .

Let us denote by $u \dashv v$ the event that $\ell(u)$ cuts v in the constructed partition. Let $\text{index}(u, v) = i$, and let $u_1, u_2, \dots, u_{i-1}$ be the segments that $\ell(u)$ intersects before hitting v . The event $u \dashv v$ happens only if u occurs before any of $\{u_1, u_2, \dots, u_{i-1}, v\}$ in the randomly chosen permutation π . The probability that this happens is $1/(i+1)$.

Let $C_{u,v}$ be an indicator variable that is 1 if $u \dashv v$ and 0 otherwise; clearly, $\mathbb{E}[C_{u,v}] = \Pr[u \dashv v] \leq 1/(\text{index}(u, v) + 1)$. The size of P_π equals n plus the number of intersections due to cuts. Thus, its expectation is $n + \mathbb{E}[\sum_u \sum_v C_{u,v}]$ and by linearity of expectation this equals

$$n + \sum_u \sum_v \Pr[u \dashv v] \leq n + \sum_u \sum_v \frac{1}{\text{index}(u, v) + 1}. \quad (2.1)$$

For any line segment u and any finite positive integer i , there are at most two vertices v and w such that $\text{index}(u, v)$ and $\text{index}(u, w)$ equal i . This is because the extension of the segment u along either of the two possible directions will meet any other line segment at most once. Thus, in each of the two directions, there is a total ordering on the points of intersection with other segments and the index values increase monotonically. This implies that

$$\sum_{v \neq u} \frac{1}{\text{index}(u, v) + 1} \leq \sum_{i=1}^{n-1} \frac{2}{i+1} = O(\log n). \quad (1.8)$$

Combining this with (2.1) implies that the expected size of P_π is bounded above by $n + O(n \log n)$, which is $O(n \log n)$. $\square$

Note that in computing the expected number of intersections, we only made use of linearity of expectation. We do

not require any independence between the events $u \dashv v$ and $u \dashv w$, for segments u , v , and w . Indeed, these events need not be independent in general.

One way of interpreting Theorem 2.2 is as follows: since the expected size of the binary planar partition constructed by the algorithm is $O(n \log n)$ on any input, there must exist a binary autopartition of size $O(n \log n)$ for every input. This follows from the simple fact that any random variable assumes at least one value that is no greater than its expectation. Thus we have used a *probabilistic argument* to assert that a combinatorial object—in this case a binary autopartition of size $O(n \log n)$ —exists with absolute certainty rather than with some probability. This is an example of the *probabilistic method* in combinatorics. We will study the probabilistic method in greater detail in Chapter 6.

C++ Implementation

Listing 2.1: RandAuto for binary planar partitions.

```

1 struct Seg { double x1,y1,x2,y2; int id; };
2
3 bool intersect(const Seg& a, const Seg& b) {
4     auto o=[](double px,double py,
5               double qx,double qy,
6               double rx,double ry){
7         double v=(qy-py)*(rx-qx)-(qx-px)*(ry-qy);
8         return std::abs(v)<1e-10?(v>0?1:2);
9     };
10    return o(a.x1,a.y1,a.x2,a.y2,b.x1,b.y1)
11           !=o(a.x1,a.y1,a.x2,a.y2,b.x2,b.y2)
12           && o(b.x1,b.y1,b.x2,b.y2,a.x1,a.y1)
13           !=o(b.x1,b.y1,b.x2,b.y2,a.x2,a.y2);
14 }
15
16 // Build autopartition recursively
17 // segs: remaining segments in this region
18 // Returns number of leaf nodes (partition size)
19 int rand_auto(std::vector<Seg*>& segs) {
20     if (segs.empty()) return 0;
21     if (segs.size()==1) return 1;           // leaf
22     std::vector<int> p(segs.size());
23     std::iota(p.begin(),p.end(),0);
24     std::shuffle(p.begin(),p.end(),
25                 std::mt19937(std::random_device()()));
26     Seg* cut = segs[p[0]];                   // random pivot
27     std::vector<Seg*> L,R;
```

```
28   for (size_t i=1;i<segs.size();i++) {
29       if (intersect(*cut,*segs[i]))
30           { L.push_back(segs[i]); R.push_back(segs[i]); }
31       else {
32           double dx=cut->x2-cut->x1;
33           double dy=cut->y2-cut->y1;
34           double cross=dx*(segs[i]->y1-cut->y1)
35                     -dy*(segs[i]->x1-cut->x1);
36           (cross>=0?L:R).push_back(segs[i]);
37       }
38   }
39   return rand_auto(L)+rand_auto(R);
40 }
```

2.2 A Probabilistic Recurrence

Frequently, we express a random variable of interest as a recurrence in terms of other random variables. In this section, we study one such situation using the Find algorithm analyzed in detail in Problem 1.9. The material in this section, although useful, is not an essential prerequisite for subsequent topics and may be omitted in the first reading.

The Find algorithm for selecting the k th smallest of a set S of n elements works as follows. We pick a random element y and partition $S \setminus \{y\}$ into two sets S_1 and S_2 (elements smaller and larger than y respectively) as in RandQS. Suppose $|S_1| = k - 1$; then y is the desired element and we are done. Otherwise, if $|S_1| > k$, we recursively find the k th smallest element of S_1 ; else we recursively find the $(k - |S_1| - 1)$ th smallest element in S_2 .

The expected number of comparisons made by the Find algorithm is the subject of Problem 1.9. Suppose instead that we were to ask the following question: what is the expected number of times we make the recursive call in the algorithm? Equivalently, what is the expected number of times we pick a random element in the algorithm? While this question may not be especially important for the Find algorithm, it is the kind of question that arises in the analysis of a number of parallel and geometric algorithms. Intuitively, we expect that the size of the residual problem in the Find algorithm is divided

by a constant factor at each recursive level, so that we expect that the number of recursive invocations is $O(\log n)$. Below, we show that this intuition can be formalized in a general setting.

Let $g(x)$ be a monotone non-decreasing function from the positive reals to the positive reals. Consider a particle whose position changes at discrete time steps and is always at a positive integer. If the particle is currently at position $m > 1$, it proceeds at the next step to the position $m - X$, where X is a random variable ranging over the integers $1, \dots, m - 1$. All we know about X is that $\mathbb{E}[X] \geq g(m)$, and that X is chosen independently of the past. It is clear that the particle will always reach position 1 and the process terminates in that state. The interesting question is, assuming that the particle starts at position n , what is the expected number of steps before it reaches position 1?

Theorem 2.3. *Let T be the random variable denoting the number of steps in which the particle reaches the position 1. Then, $\mathbb{E}[T] \leq \int_1^n \frac{dx}{g(x)}$.*

Proof. The proof is by induction on n ; let us suppose the theorem holds for values of m smaller than n . Let $f(m) = \int_1^m dx/g(x)$ for $m > 1$. We wish to show that $\mathbb{E}[T] \leq f(n)$.

Consider the first step, during which the particle proceeds from position n to position $n - X$, where X is chosen from a distribution for which $\mathbb{E}[X] \geq g(n)$. We have

$$\mathbb{E}[T] \leq 1 + \mathbb{E}[f(n - X)] \quad (1.9)$$

$$= 1 + \mathbb{E}\left[\int_{n-X}^n \frac{dy}{g(y)}\right] \quad (1.10)$$

$$\leq 1 + \mathbb{E}\left[\frac{X}{g(n)}\right] \quad (1.11)$$

$$= 1 + \frac{\mathbb{E}[X]}{g(n)} \quad (1.12)$$

$$\leq 1 + f(n) - 1 = f(n). \quad (1.13)$$

The inequality (1.10) follows from the inductive hypothesis applied to position $n - X$. The inequality (1.11) follows from the assumption that g is non-decreasing, so $1/g(y) \leq 1/g(n)$ for $y \leq n$. The inequality (1.13) follows from the lower bound on $\mathbb{E}[X]$ and the observation that $f(n) - f(n - X) \geq X/g(n)$. $\square$

Exercise 2.4. If X were to range over all integers having value at most $m - 1$ (possibly including negative integers), how would the statement and proof of Theorem 2.3 change?

For the Find algorithm, we can show (following the analysis of Problem 1.9) that $g(m) \geq m/4$. We may then apply the above theorem to bound the expected number of recursive calls to Find by $4 \ln n$.

Exercise 2.5. What prevents us from using Theorem 2.3 to bound the expected number of levels of recursion in the RandQS algorithm?

2.2.1 C++ Implementation

Listing 2.2: Randomized Find—selecting the k th smallest element.

```
1 int rand_find(std::vector<int>& a, int k) {
2     if (a.size()==1) return a[0];
3     int pi = RandomEngine::instance()
4             .uniform_int(0,a.size()-1);
5     int pivot = a[pi];
6     std::vector<int> L,E,G;
7     for (int x : a) {
8         if (x<pivot) L.push_back(x);
9         else if (x==pivot) E.push_back(x);
10        else G.push_back(x);
11    }
12    if (k<=(int)L.size())
13        return rand_find(L,k);
14    else if (k<=(int)(L.size()+E.size()))
15        return pivot;
16    else
17        return rand_find(G,k-L.size()-E.size());
18 }
```

Listing 2.3: Empirical measurement of recursion depth.

```

1 struct Stats { int calls=0, depth=0; };
2
3 int rand_find_analyzed(std::vector<int>& a, int k,
4                       int d, Stats& s) {
5     s.calls++;
6     s.depth = std::max(s.depth, d);
7     if (a.size()==1) return a[0];
8     int pi = RandomEngine::instance()
9             .uniform_int(0, a.size()-1);
10    int pivot = a[pi];
11    std::vector<int> L, E, G;
12    for (int x : a) {
13        if (x < pivot) L.push_back(x);
14        else if (x == pivot) E.push_back(x);
15        else G.push_back(x);
16    }
17    if (k <= (int)L.size())
18        return rand_find_analyzed(L, k, d+1, s);
19    else if (k <= (int)(L.size()+E.size()))
20        return pivot;
21    else
22        return rand_find_analyzed(G,
23                                   k-L.size()-E.size(), d+1, s);
24 }
25
26 // Typical output for n=1000:
27 //   Avg recursive calls: ~12
28 //   Avg max depth: ~11
29 //   Theoretical  $O(\log n)$ : ~10
30 //   Theorem 1.3 bound  $4 \ln n$ : ~27

```

2.3 Computation Model and Complexity Classes

2.3.1 RAMs and Turing Machines

Following common practice, throughout this book we use the Turing machine model to discuss complexity-theory issues. As is common, however, we switch to the RAM (random access machine) as the model of computation when describing and analyzing algorithms (except in the study of parallel and distributed algorithms in Chapter 13, where we define a version of the RAM model for machines working in parallel).

Definition 2.6. A *deterministic Turing machine* is a quadruple $M = (S, \Sigma, \delta, s)$. Here S is a finite set of states, of which $s \in S$

is the machine's initial state. The machine uses a finite set of symbols, denoted Σ ; this set includes special symbols BLANK and FIRST. The function δ is the transition function, mapping $S \times \Sigma$ to $(S \cup \{\text{HALT}, \text{YES}, \text{NO}\}) \times \Sigma \times \{\leftarrow, \rightarrow, \text{STAY}\}$.

A *probabilistic Turing machine* is a Turing machine augmented with the ability to generate an unbiased coin flip in one step. It corresponds to a randomized algorithm.

In the RAM model, we have a machine that can perform the following types of operations involving registers and main memory: input–output operations, memory–register transfers, indirect addressing, branching, and arithmetic operations. Each register or memory location may hold an integer that can be accessed as a unit, but an algorithm has no access to the representation of the number.

Proposition 2.7. *Any Turing machine computation of length polynomial in the size of the input can be simulated by a RAM computation of length polynomial in the size of the input. Any RAM computation of length polynomial in the size of the input can be simulated by a Turing machine computation of length polynomial in the size of the input.*

2.3.2 Complexity Classes

We now define some basic complexity classes focusing on those involving randomized algorithms.

Definition 2.8. The class **P** consists of all languages L that have a polynomial-time algorithm A such that for any input $x \in \Sigma^*$, $x \in L \Rightarrow A(x)$ accepts, and $x \notin L \Rightarrow A(x)$ rejects.

Definition 2.9. The class **NP** consists of all languages L that have a polynomial-time algorithm A such that for any input $x \in \Sigma^*$, $x \in L \Rightarrow \exists y \in \Sigma^* : A(x, y)$ accepts, where $|y|$ is bounded by a polynomial in $|x|$; and $x \notin L \Rightarrow \forall y \in \Sigma^* : A(x, y)$ rejects.

Definition 2.10. The class **RP** (Randomized Polynomial time) consists of all languages L that have a randomized algorithm

A running in worst-case polynomial time such that for any input $x \in \Sigma^*$, $x \in L \Rightarrow \Pr[A(x) \text{ accepts}] \geq 1/2$; and $x \notin L \Rightarrow \Pr[A(x) \text{ accepts}] = 0$.

Definition 2.11. The class **ZPP** (Zero-error Probabilistic Polynomial time) is the class of languages that have Las Vegas algorithms running in expected polynomial time.

Definition 2.12. The class **BPP** (Bounded-error Probabilistic Polynomial time) consists of all languages L that have a randomized algorithm A running in worst-case polynomial time such that for any input $x \in \Sigma^*$, $x \in L \Rightarrow \Pr[A(x) \text{ accepts}] \geq 3/4$; and $x \notin L \Rightarrow \Pr[A(x) \text{ accepts}] \leq 1/4$.

Exercise 2.13. Show that $\text{ZPP} = \text{RP} \cap \text{co-RP}$.

Notes

The ideas underlying randomized algorithms can be traced back to Monte Carlo methods used in numerical analysis, statistical physics, and simulation. In the context of computability theory, the notion of a probabilistic Turing machine was proposed by de Leeuw, Moore, Shannon, and Shapiro [122] and further explored in the pioneering work of Rabin [340] and Gill [166]. Berlekamp [57], Rabin [341], and Solovay and Strassen [382] gave early examples of concrete randomized algorithms.

Our RandQS algorithm is based on Hoare's algorithm [201]. The min-cut algorithm of Section 1.2.2, together many variations and extensions, is due to Karger [231]. Monte Carlo methods have been popular in the sciences for over a hundred years. The classic experiment on approximating the value of π by dropping needles on a sheet of paper with parallel lines is described in an eighteenth-century paper by Buffon [86]. The origin of the modern theory of Monte Carlo methods in the physical sciences is widely attributed to Ulam, von Neumann, and Fermi [116]. The term *Las Vegas algorithm* was introduced by Babai [37].

The material of Section 2.1 is drawn from Paterson and Yao [329]. The Find algorithm described in Section 2.2 is due to Hoare [200]. Theorem 2.3 is given in a paper by Karp, Upfal, and Wigderson [250]. Karp [244] gives a number of additional results on probabilistic recurrence relations.

The reader is referred to introductory texts on algorithms and complexity such as those by Aho, Hopcroft, and Ullman [5, 6] and Papadimitriou [326] for more details on the Turing machine model and the RAM model. The books by Bovet and Crescenzi [81] and by Papadimitriou [326] contain a more detailed treatment of the complexity classes described in this chapter.

Problems

Exercise 2.14 (1.1). (Due to J. von Neumann [409].)

- (a) Suppose you are given a coin for which the probability of HEADS, say p , is unknown. How can you use this coin to generate unbiased (i.e., $\Pr[\text{HEADS}] = \Pr[\text{TAILS}] = 1/2$) coin-flips? Give a scheme for which the expected number of flips of the biased coin for extracting one unbiased coin-flip is no more than $1/[p(1 - p)]$. (Hint: Consider two consecutive flips of the biased coin.)
- (b) Devise an extension of the scheme that extracts the largest possible number of independent, unbiased coin-flips from a given number of flips of the biased coin.

Exercise 2.15 (1.2). (Due to D. E. Knuth and A. C.-C. Yao [264].)

- (a) Suppose you are provided with a source of unbiased random bits. Explain how you will use this to generate uniform samples from the set $S = \{0, \dots, n - 1\}$. Determine the expected number of random bits required by your sampling algorithm.

- (b) What is the worst-case number of random bits required by your sampling algorithm? Consider the case when n is a power of 2, as well as the case when it is not.
- (c) Solve (a) and (b) when, instead of unbiased random bits, you are required to use as the source of randomness uniform random samples from the set $\{0, \dots, p-1\}$; consider the case when n is a power of p , as well as the case when it is not.

Exercise 2.16 (1.3). (Due to D. E. Knuth and A. C.-C. Yao [264].) Suppose you are provided with a source of unbiased random bits. Provide efficient (in terms of expected running time and expected number of random bits used) schemes for generating samples from the distribution over the set $\{2, 3, \dots, 12\}$ induced by rolling two unbiased dice and taking the sum of their outcomes.

Exercise 2.17 (1.4). (a) Suppose you are required to generate a random permutation of size n . Assuming that you have access to a source of independent and unbiased random bits, suggest a method for generating random permutations of size n . Efficiency is measured in terms of both time and number of random bits. What lower bounds can you prove for this task?

- (b) Consider the following method for generating a random permutation of size n . Pick n random values $X_1, \dots, X_n$ independently from the uniform distribution over the interval $[0, 1]$. Now, the permutation that orders the random variables in ascending order is claimed to be a random permutation, and it can be determined by sorting the random values. Is the claim correct? How efficient is this scheme?
- (c) Consider the following “lazy” implementation of the scheme suggested in (b). The binary representation of the fraction X_i is a sequence of unbiased and independent random bits. At any given stage of the sorting algorithm,

we would have chosen only as many bits of each X_i as necessary to resolve all the comparisons performed up to that point. When comparing X_i to X_j , if the current prefixes of their binary expansions do not determine the outcome of the comparison, then we extend their prefixes by choosing further random bits until this happens. Compute tight bounds on the expected number of random bits used by this implementation.

Exercise 2.18 (1.5). Consider the problem of using a source of unbiased random bits to generate samples from the set $S = \{0, \dots, n-1\}$ such that the element i is chosen with probability p_i . Show how to perform this sampling using $O(\log n)$ random bits per sample, regardless of the values of p_i . Use the result from part (c) of Problem 1.4.

Exercise 2.19 (1.6). Consider a sequence of n flips of an unbiased coin. Let H_i denote the absolute value of the excess of the number of HEADS over the number of TAILS seen in the first i flips. Define $H = \max_i H_i$. Show that $\mathbb{E}[H_i] = \Theta(\sqrt{i})$, and that $\mathbb{E}[H] = \Theta(\sqrt{n})$.

Exercise 2.20 (1.7). Suppose we choose a permutation π of the ordered set $N = \{1, 2, \dots, n\}$ uniformly at random from the space of all permutations of N . Let $L(\pi)$ denote the length of the longest increasing subsequence in permutation π .

(a) For large n and some positive constant c , prove that $\mathbb{E}[L(\pi)] \geq c\sqrt{n}$.

(b) Is the bound in (a) tight?

Exercise 2.21 (1.8). Consider adapting the min-cut algorithm of Section 1.2.2 to the problem of finding an s - t min-cut in an undirected graph. In this problem, we are given an undirected graph G together with two distinguished vertices s and t . An s - t cut is a set of edges whose removal from G disconnects s from t ; we seek an s - t cut of minimum cardinality. As the algorithm proceeds, the vertex s may get amalgamated into a new vertex as a result of an edge being contracted; we call this

vertex the s -vertex (initially the s -vertex is s itself). Similarly, we have a t -vertex. As we run the contraction algorithm, we ensure that we never contract an edge between the s -vertex and the t -vertex.

- (a) Show that there are graphs on which the probability that this algorithm finds an s - t min-cut is exponentially small.
- (b) How large can the number of s - t min-cuts in an instance be?

Exercise 2.22 (1.9). Consider the Find algorithm described in Section 2.2 for selecting the k th smallest of a set S of n elements. Show that the algorithm finds the k th smallest element in S in expected time $O(n)$.

Exercise 2.23 (1.10). Consider the setting of Example 1.1 (the Sailor Problem). Show that the probability that no sailor returns to her own cabin approaches $1/e$ as the number of sailors grows large.

Exercise 2.24 (1.11). Verify the following inclusions: $P \subseteq RP \subseteq NP \subseteq PSPACE \subseteq EXP \subseteq NEXP$. It is not known whether these inclusions are strict.

Exercise 2.25 (1.12). Verify the following inclusions: $RP \subseteq BPP \subseteq PP$. It is not known whether these inclusions are strict.

Exercise 2.26 (1.13). Show that $PP = \text{co-PP}$ and $BPP = \text{co-BPP}$.

Exercise 2.27 (1.14). Show that $NP \subseteq PP \subseteq PSPACE$.

Exercise 2.28 (1.15). (Due to K.-I. Ko [265].) Show that $NP \subseteq BPP$ implies $NP = RP$.

Chapter 3

Game-Theoretic Techniques

In this chapter we study several ideas that are basic to the design and analysis of randomized algorithms. All the topics in this chapter share a game-theoretic viewpoint, which enables us to think of a randomized algorithm as a probability distribution on deterministic algorithms. This leads to Yao's Minimax Principle, which can be used to establish a lower bound on the performance of a randomized algorithm.

3.1 Game Tree Evaluation

We begin with another simple illustration of linearity of expectation, in the setting of game tree evaluation. This example will demonstrate a randomized algorithm whose expected running time is smaller than that of any deterministic algorithm. It will also serve as a vehicle for demonstrating a standard technique for deriving a lower bound on the running time of any randomized algorithm for a problem.

A *game tree* is a rooted tree in which internal nodes at even distance from the root are labeled MIN and internal nodes at odd distance are labeled MAX. Associated with each leaf is a real number, which we call its *value*. The evaluation of the game tree is the following process. Each leaf returns the value associated with it. Each MAX node returns the largest value returned by its children, and each MIN node returns the

smallest value returned by its children. Given a tree with values at the leaves, the evaluation problem is to determine the value returned by the root.

The evaluation of game trees plays a central role in artificial intelligence, particularly in game-playing programs. The reader may readily associate the children of a node with the options available to one of the two players in a game. The leaves represent the value of the game for either player. One player seeks to maximize this value, while the other tries to minimize it. At each step, an evaluation algorithm chooses a leaf and reads its value.

We study the number of such steps taken by an algorithm for evaluating a game tree. We do not charge the algorithm for any other computation. We will limit our discussion to the special case in which the values at the leaves are bits, 0 or 1. Thus, each MIN node can be thought of as a Boolean AND operation and each MAX node as a Boolean OR operation. This special case is of interest in its own right, having applications in mechanical theorem proving.

Let $T_{d,k}$ denote a uniform tree in which the root and every internal node has d children and every leaf is at distance $2k$ from the root. Thus, any root-to-leaf path passes through k AND nodes (including the root itself) and k OR nodes, and there are d^{2k} leaves. An instance of the evaluation problem consists of the tree $T_{d,k}$ together with a Boolean value for each of the d^{2k} leaves. Given an algorithm, we study the maximum number of steps it takes to evaluate any instance of $T_{d,k}$.

An algorithm begins by specifying a leaf whose value is to be read at the first step. Thereafter, it specifies such a leaf at each step, based on the values it has read on previous steps. In a deterministic algorithm, the choice of the next leaf to be read is a deterministic function of the values at the leaves read so far. For a randomized algorithm, this choice may be randomized.

Exercise 3.1. Show that for any deterministic evaluation algorithm, there is an instance of $T_{d,k}$ that forces the algorithm to read the values on all d^{2k} leaves.

3.1.1 A Randomized Algorithm

We now give a simple randomized algorithm and study the expected number of leaves it reads on any instance of $T_{d,k}$. To simplify our presentation, we restrict ourselves to the case $d = 2$. Any deterministic algorithm for this case can be made to read all $2^{2k} = 4^k$ leaves on some instance. Our randomized algorithm is based on the following simple observation. Consider a single AND node with two leaves. If the node were to return 0, at least one of the leaves must contain 0. A deterministic algorithm inspects the leaves in a fixed order, and an adversary can therefore always “hide” the 0 at the second of the two leaves inspected by the algorithm. Reading the leaves in a random order foils this strategy. With probability $1/2$, the algorithm chooses the hidden 0 on the first step, so its expected number of steps is $3/2$, which is better than the worst case for any deterministic algorithm. Similarly, in the case of an OR node, if it were to return a 1, then a randomized order of examining the leaves will reduce the expected number of steps to $3/2$.

The reader may wonder how the randomized algorithm can benefit if the AND node were to return 1, or if the OR node were to return 0. If the two children of these nodes are leaves, then clearly both leaves must be examined. The point is that at an internal AND node in a tree returning a 1, examining the two OR children (and evaluating their sub-trees) in a random order is still beneficial. The two OR children of an AND node must also return 1, and this is the easy case for the OR nodes. Similarly, at an internal OR node returning 0, the two AND children must return 0, and this is the easy case for the AND nodes.

Theorem 3.2. *Given any instance of $T_{2,k}$, the expected number of steps for the randomized algorithm is at most 3^k . Since $n = 4^k$ leaves, the expected running time is $n^{\log_4 3} \approx n^{0.793}$.*

Proof. We argue by induction on k . The basis ($k = 1$) is the single-node case: for an AND (or OR) node with two leaves, the expected cost is at most $3/2$.

Algorithm 2 Randomized evaluation of $T_{2,k}$

```
1: procedure EVALAND( $v$ )
2:   Pick a child  $c$  of  $v$  uniformly at random
3:    $val \leftarrow$  EVALOR( $c$ )
4:   if  $val = 0$  then
5:     return 0
6:   end if
7:    $val' \leftarrow$  EVALOR(other child of  $v$ )
8:   return  $val'$ 
9: end procedure

10: procedure EVALOR( $v$ )
11:   Pick a child  $c$  of  $v$  uniformly at random
12:    $val \leftarrow$  EVALAND( $c$ )
13:   if  $val = 1$  then
14:     return 1
15:   end if
16:    $val' \leftarrow$  EVALAND(other child of  $v$ )
17:   return  $val'$ 
18: end procedure
```

Assume the expected cost of evaluating any instance of $T_{2,k-1}$ is at most 3^{k-1} . Consider first a tree T whose root is an OR node, each of whose children is the root of a copy of $T_{2,k-1}$.

Case 1: OR evaluates to 1. At least one child returns 1. With probability $1/2$ this child is chosen first, incurring expected cost at most 3^{k-1} . With probability $1/2$ both sub-trees are evaluated, incurring cost at most $2 \times 3^{k-1}$. The expected cost is at most

$$\frac{1}{2} \cdot 3^{k-1} + \frac{1}{2} \cdot 2 \cdot 3^{k-1} = \frac{3}{2} \cdot 3^{k-1}.$$

Case 2: OR evaluates to 0. Both children must be evaluated, costing at most $2 \times 3^{k-1}$.

Now consider the root of $T_{2,k}$, an AND node.

Case 1: AND evaluates to 1. Both sub-trees return 1. By the discussion above, the expected cost of evaluating each OR sub-tree that returns 1 is at most $\frac{3}{2} \cdot 3^{k-1}$. By linearity of expectation, the total expected cost is at most $2 \times \frac{3}{2} \times 3^{k-1} = 3^k$.

Case 2: AND evaluates to 0. At least one sub-tree returns 0. With probability $1/2$ the sub-tree returning 0 is chosen first, costing at most $2 \times 3^{k-1}$ (both its OR children must be evaluated). With probability $1/2$, an additional cost of $\frac{3}{2} \cdot 3^{k-1}$ may be incurred for the sibling returning 1. The expected cost is at most

$$\frac{1}{2} \cdot 2 \cdot 3^{k-1} + \frac{1}{2} \cdot (2 \cdot 3^{k-1} + \frac{3}{2} \cdot 3^{k-1}) \leq 3^k.$$

In all cases the expected cost is at most 3^k . □

3.1.2 C++ Implementation

Listing 3.1: Game tree node and randomized evaluator.

```

1 #include <vector>
2 #include <random>
3 #include <chrono>
4 #include <algorithm>
5 #include <iostream>
6
7 class RandEng {
8 public:
9     static RandEng& get() {
10         static RandEng e; return e;

```

```

11     }
12     bool coin() {
13         return std::uniform_real_distribution<>(
14             0.0,1.0)(eng_) < 0.5;
15     }
16 private:
17     RandEng()
18         : eng_(std::chrono::steady_clock::now()
19             .time_since_epoch().count()) {}
20     std::mt19937 eng_;
21 };
22
23 enum class NodeType { AND, OR };
24
25 struct TreeNode {
26     NodeType type;
27     bool is_leaf;
28     int value;           // valid only if is_leaf
29     std::vector<TreeNode*> children;
30
31     TreeNode(NodeType t) : type(t), is_leaf(false),
32                           value(0) {}
33     TreeNode(int v) : type(NodeType::AND), is_leaf(true),
34                      value(v) {}
35 };
36
37 // Randomized evaluation of AND-OR tree
38 // Returns number of leaves inspected
39 struct EvalResult { bool val; int leaves; };
40
41 EvalResult eval_and(TreeNode* v);
42 EvalResult eval_or(TreeNode* v);
43
44 EvalResult eval_and(TreeNode* v) {
45     if (v->is_leaf) return {v->value==1, 1};
46     // pick child at random
47     int n = v->children.size();
48     int pick = std::uniform_int_distribution<>(
49         0, n-1)(RandEng::get().eng_);
50     auto r = eval_or(v->children[pick]);
51     int cost = r.leaves;
52     if (!r.val) return {false, cost};
53     // must check the other child
54     for (int i=0;i<n;i++) {
55         if (i==pick) continue;
56         auto r2 = eval_or(v->children[i]);
57         cost += r2.leaves;
58         if (!r2.val) return {false, cost};
59     }
60     return {true, cost};
61 }
62
63 EvalResult eval_or(TreeNode* v) {
64     if (v->is_leaf) return {v->value==1, 1};
65     int n = v->children.size();
66     int pick = std::uniform_int_distribution<>(

```



```

67         0, n-1)(RandEng::get().eng_);
68     auto r = eval_and(v->children[pick]);
69     int cost = r.leaves;
70     if (r.val) return {true, cost};
71     for (int i=0;i<n;i++) {
72         if (i==pick) continue;
73         auto r2 = eval_and(v->children[i]);
74         cost += r2.leaves;
75         if (r2.val) return {true, cost};
76     }
77     return {false, cost};
78 }

```

Listing 3.2: Build a random instance of $T_{2,k}$ and evaluate.

```

1  // Build complete binary tree of depth 2k
2  // with random leaf values
3  TreeNode* build_tree(int k, auto& eng) {
4      if (k==0) {
5          int v = std::uniform_int_distribution<>(
6              0,1)(eng);
7          return new TreeNode(v);
8      }
9      // root is AND at even depth, OR at odd
10     TreeNode* node = new TreeNode(NodeType::AND);
11     node->children.push_back(build_tree(k-1,eng));
12     node->children.push_back(build_tree(k-1,eng));
13     return node;
14 }
15
16 int main() {
17     std::mt19937 eng(
18         std::chrono::steady_clock::now()
19             .time_since_epoch().count());
20
21     for (int k=1;k<=8;k++) {
22         int n = 1 << (2*k); // 4^k leaves
23         double total=0;
24         int trials=10000;
25         for (int t=0;t<trials;t++) {
26             TreeNode* root = build_tree(k,eng);
27             auto r = eval_and(root);
28             total += r.leaves;
29         }
30         double avg = total/trials;
31         double bound = std::pow(3,k);
32         std::cout << "k=" << k
33             << " n=" << n
34             << " avg=" << avg
35             << " 3^k=" << bound
36             << " n^0.793="
37             << std::pow(n,0.79248)
38             << "\n";
39     }
40 }

```

Remark 3.3. The algorithm above is a Las Vegas algorithm: it always produces the correct answer, and its running time is the random variable. The expected running time of $n^{0.793}$ is strictly better than the $\Theta(n)$ worst case of any deterministic algorithm.

3.2 The Minimax Principle

The randomized algorithm of the preceding section has an expected running time of $n^{0.793}$ on any uniform binary AND-OR tree with n leaves. Can we establish that no randomized algorithm can have a lower expected running time? We are thus seeking a lower bound on the running time of any randomized algorithm for this problem. As a first step toward this end, we introduce a standard technique for proving such lower bounds: the minimax principle. Indeed, it is the only known general technique for proving lower bounds on the running times of randomized algorithms.

3.2.1 Game Theory

Consider the following game. Roberta and Charles put their hands behind their backs and make a sign for one of the following: stone (closed fist), paper (open palm), and scissors (two fingers). They then simultaneously display their chosen sign. The winner is determined by the following rules: paper beats stone by wrapping it, scissors beats paper by cutting it, and stone beats scissors by dulling it. The loser pays \$1 to the winner, and the outcome is a draw when the two players choose the same sign.

	Scissors	Paper	Stone
Scissors	0	1	-1
Paper	-1	0	1
Stone	1	-1	0

Figure 3.1: Payoff matrix for scissors-paper-stone.

This is an instance of a *two-person zero-sum game*, and the matrix is called the *payoff matrix*. It is called a zero-sum game because the net amount won by Roberta and Charles is always exactly zero. In general, any two-person zero-sum game can be represented by an $n \times m$ payoff matrix $\mathbf{M}$ with real entries. The set of possible strategies of the row player R is in correspondence with the rows of $\mathbf{M}$, and likewise for the strategies of the column player C . The entry M_{ij} is the amount paid by C to R when R chooses strategy i and C chooses strategy j .

Naturally, the goal of the row (column) player is to maximize (minimize) the payoff. Assume that this is a zero-information game, in that neither player has any information about the opponent's strategy. If R chooses strategy i , then she is guaranteed a payoff of $\min_j M_{ij}$, regardless of C 's strategy. An optimal strategy for R is an i that maximizes $\min_j M_{ij}$. Let $V_R = \max_i \min_j M_{ij}$ denote the lower bound on the value of the payoff to R when she uses an optimal strategy. An optimal strategy for C is a j that gives the best possible upper bound on the payoff from C to R . A similar argument establishes that C 's optimal strategy ensures that his payoff to R is at most $V_C = \min_j \max_i M_{ij}$.

Exercise 3.4. Show that the following inequality is valid for all payoff matrices:

$$\max_i \min_j M_{ij} \leq \min_j \max_i M_{ij}.$$

In general, the inequality in Exercise 2.1 is strict; for example, in scissors-paper-stone, $V_R = -1$ and $V_C = 1$. When these two quantities are equal, the game is said to have a *solution* and the value of the game is $V = V_R = V_C$. The solution (or the *saddle-point*) is the specific choice of (optimal) strategies that lead to this payoff.

3.2.2 Mixed Strategies and the Minimax Theorem

What happens when a game has no solution? Then there is no clear-cut optimal strategy for any player. In fact, any knowl-

edge of the opponent's strategy can be used to improve the payoff, unlike the case of games with saddle-points. An interesting way to get around this is to introduce randomization in the choice of strategies. So far we have been talking about deterministic or *pure strategies*, but now we focus on randomized or *mixed strategies*. A mixed strategy is a probability distribution on the set of possible strategies. The row player picks a vector $\mathbf{p} = (p_1, \dots, p_n)$, which is a probability distribution on the rows of $\mathbf{M}$, i.e., p_i is the probability that R will choose strategy i ; similarly, the column player has a vector $\mathbf{q} = (q_1, \dots, q_m)$, which is a probability distribution on the columns of $\mathbf{M}$. The payoff is now a random variable, and its expectation is given by

$$\mathbb{E}[\text{payoff}] = \mathbf{p}^T \mathbf{M} \mathbf{q} = \sum_{i=1}^n \sum_{j=1}^m p_i M_{ij} q_j.$$

As before, using V_R to denote the best possible lower bound on the expected payoff to R that can be ensured by choosing a strategy $\mathbf{p}$, and using V_C to denote the best possible upper bound on the expected payoff by C by choosing a strategy $\mathbf{q}$, we obtain

$$V_R = \max_{\mathbf{p}} \min_{\mathbf{q}} \mathbf{p}^T \mathbf{M} \mathbf{q}, \quad V_C = \min_{\mathbf{q}} \max_{\mathbf{p}} \mathbf{p}^T \mathbf{M} \mathbf{q}.$$

Theorem 3.5 (von Neumann's Minimax Theorem). *For any two-person zero-sum game specified by a matrix $\mathbf{M}$,*

$$\max_{\mathbf{p}} \min_{\mathbf{q}} \mathbf{p}^T \mathbf{M} \mathbf{q} = \min_{\mathbf{q}} \max_{\mathbf{p}} \mathbf{p}^T \mathbf{M} \mathbf{q}.$$

In other words, the largest expected payoff that R can guarantee by choosing a mixed strategy is equal to the smallest expected payoff that C can guarantee using a mixed strategy. This common expected payoff value, called the *value of the game*, is denoted by V . A pair of mixed strategies $(\mathbf{p}, \mathbf{q})$ which respectively maximize the left-hand side and minimize the right-hand side of the equation in Theorem 3.5 is called a saddle-point, and the two distributions are called *optimal mixed strategies*.

Theorem 3.6 (Loomis' Theorem). *For any two-person zero-sum game specified by a matrix $\mathbf{M}$,*

$$\max_{\mathbf{p}} \min_j \mathbf{p}^T \mathbf{M} \mathbf{e}_j = \min_{\mathbf{q}} \max_i \mathbf{e}_i^T \mathbf{M} \mathbf{q},$$

where $\mathbf{e}_k$ denotes a unit vector with a 1 in the k th position and 0s elsewhere.

3.2.3 Yao's Minimax Principle

We now describe the application of the above game-theoretic results to proving lower bounds on the performance of randomized algorithms. The idea is to view the algorithm designer as the column player C and the adversary choosing the input as the row player R . The columns correspond to the set of all possible algorithms; the rows correspond to the set of all possible inputs (of a fixed size). The payoff from C to R is some real-valued measure of the performance of an algorithm, such as the running time, the quality of the solution obtained, communication cost, or space.

Consider a problem where the number of distinct inputs of a fixed size is finite, as is the number of distinct (deterministic, terminating, and always correct) algorithms for solving that problem. A pure strategy for C corresponds to the choice of a deterministic algorithm, while a pure strategy for R corresponds to a specific input. Notice that an optimal pure strategy for C corresponds to an optimal deterministic algorithm, and V_C is the worst-case running time of any deterministic algorithm for the problem, which we call the *deterministic complexity* of the problem.

Our interest is in the interpretation of the mixed strategies for the algorithm designer and the adversary. A mixed strategy for C is a probability distribution over the space of (always correct) deterministic algorithms, so it is a Las Vegas randomized algorithm. An optimal mixed strategy for C is an optimal Las Vegas algorithm. A mixed strategy for R is a distribution over the space of all inputs. Let us define the *distributional complexity* of the problem at hand as the expected

running time of the best deterministic algorithm for the worst distribution on the inputs.

Proposition 3.7 (Yao’s Minimax Principle). *Let Π be a problem with a finite set I of input instances (of a fixed size), and a finite set $\mathcal{A}$ of deterministic algorithms. For input $I \in I$ and algorithm $A \in \mathcal{A}$, let $C(I, A)$ denote the running time of algorithm A on input I . For probability distributions $\mathbf{p}$ over I and $\mathbf{q}$ over $\mathcal{A}$, let $I_{\mathbf{p}}$ denote a random input chosen according to $\mathbf{p}$ and $A_{\mathbf{q}}$ denote a random algorithm chosen according to $\mathbf{q}$. Then:*

$$\min_{A \in \mathcal{A}} \mathbb{E}[C(I_{\mathbf{p}}, A)] \leq \max_{I \in I} \mathbb{E}[C(I, A_{\mathbf{q}})].$$

In other words, the expected running time of the optimal deterministic algorithm for an arbitrarily chosen input distribution $\mathbf{p}$ is a lower bound on the expected running time of the optimal (Las Vegas) randomized algorithm for Π . Thus, to prove a lower bound on the randomized complexity, it suffices to choose any distribution $\mathbf{p}$ on the input and prove a lower bound on the expected running time of deterministic algorithms for that distribution. The power of this technique lies in the flexibility in the choice of $\mathbf{p}$ and, more importantly, the reduction to a lower bound on deterministic algorithms. It is important to remember that the deterministic algorithm “knows” the chosen distribution $\mathbf{p}$.

3.2.4 C++ Implementation: Computing Mixed Strategy Equilibria

Listing 3.3: Solving a 2×2 zero-sum game for optimal mixed strategies.

```
1 #include <iostream>
2 #include <cmath>
3 #include <iomanip>
4
5 // Solve a 2x2 zero-sum game with payoff matrix M
6 // Returns (V, p1, q1) where
7 //   V   = game value
8 //   p1  = prob row player plays row 0
9 //   q1  = prob col player plays col 0
```

```

10 struct GameSolution {
11     double V, p1, q1;
12 };
13
14 GameSolution solve_2x2(double M[2][2]) {
15     // Row player optimal:  $p \cdot M[:,0] = p \cdot M[:,1]$ 
16     //  $p \cdot M_{00} + (1-p) \cdot M_{10} = p \cdot M_{01} + (1-p) \cdot M_{11}$ 
17     double denom = M[0][0] - M[0][1] - M[1][0] + M[1][1];
18     double p1 = 0.5;
19     if (std::abs(denom) > 1e-12)
20         p1 = (M[1][1] - M[1][0]) / denom;
21     p1 = std::max(0.0, std::min(1.0, p1));
22
23     double denom2 = M[0][0] - M[1][0] - M[0][1] + M[1][1];
24     double q1 = 0.5;
25     if (std::abs(denom2) > 1e-12)
26         q1 = (M[1][1] - M[0][1]) / denom2;
27     q1 = std::max(0.0, std::min(1.0, q1));
28
29     double V = p1*(q1*M[0][0] + (1-q1)*M[0][1])
30             + (1-p1)*(q1*M[1][0] + (1-q1)*M[1][1]);
31     return {V, p1, q1};
32 }
33
34 int main() {
35     // Scissors-Paper-Stone
36     double M[2][2] = {{0, 1}, {-1, 0}};
37     auto [V,p1,q1] = solve_2x2(M);
38     std::cout << std::fixed << std::setprecision(4);
39     std::cout << "Modified scissors-paper-stone:\n";
40     std::cout << "    V=" << V
41             << "    p=(" << p1 << ", " << 1-p1 << ")"
42             << "    q=(" << q1 << ", " << 1-q1 << ")\n";
43 }

```

3.2.5 Lower Bound for Game Tree Evaluation

We now apply Yao's Minimax Principle to the problem of game tree evaluation. The lower bound that results only applies to algorithms that terminate in a finite number of steps on any input and sequence of random choices.

We once again limit our attention to instances of the AND-OR tree $T_{2,k}$. We consider the tree as a balanced binary tree of NORs of depth $2k$.

Exercise 3.8. Show that the tree $T_{2,k}$ is equivalent to a balanced binary tree all of whose leaves are at distance $2k$ from the root, and all of whose internal nodes compute the NOR function: a node returns the value 1 if both inputs are 0, and 0 otherwise.

In order to prove a lower bound on the expected number of leaves evaluated by any randomized algorithm, we have to specify a distribution on instances (values for the leaves), and then prove a lower bound on the expected running time of any deterministic algorithm on such inputs.

Let $p = (3 - \sqrt{5})/2$. Each leaf of the tree is independently set to 1 with probability p . Note that if each input to a NOR node is independently 1 with probability p , then the probability that its output is 1 is the probability that both its inputs are 0, which is

$$(1 - p)^2 = \frac{(\sqrt{5} - 1)^2}{4} = \frac{6 - 2\sqrt{5}}{4} = \frac{3 - \sqrt{5}}{2} = p.$$

Thus the value of every node of the NOR tree is 1 with probability p , and the value of a node is independent of the values of all the other nodes on the same level.

Proposition 3.9. *Let T be a NOR tree each of whose leaves is independently set to 1 with probability q for a fixed value $q \in [0, 1]$. Let $W(T)$ denote the minimum, over all deterministic algorithms, of the expected number of steps to evaluate T . Then, there is a depth-first pruning algorithm whose expected number of steps to evaluate T is $W(T)$.*

Proposition 2.7 tells us that for the purposes of our lower bound, we may restrict our attention to depth-first pruning algorithms. For a depth-first pruning algorithm evaluating this tree, let $W(h)$ be the expected number of leaves it inspects in determining the value of a node at distance h from the leaves. Clearly

$$W(h) = W(h - 1) + (1 - p) \times W(h - 1) = (2 - p) W(h - 1),$$

where the first term represents the work done in evaluating one of the sub-trees, and the second term represents the work done in evaluating the other sub-tree (which will be necessary if the first sub-tree returns the value 0, an event occurring with probability $1 - p$). With $p = (3 - \sqrt{5})/2$, we have $2 - p =$

$(1 + \sqrt{5})/2 \approx 1.618$, so $W(h) \geq ((1 + \sqrt{5})/2)^h$. Letting $h = \log_2 n$, we obtain

$$W(h) \geq ((1 + \sqrt{5})/2)^{\log_2 n} = n^{\log_2((1+\sqrt{5})/2)} \approx n^{0.694}.$$

Theorem 3.10. *The expected running time of any randomized algorithm that always evaluates an instance of $T_{2,k}$ correctly is at least $n^{0.694}$, where $n = 2^{2k}$ is the number of leaves.*

Remark 3.11. Our lower bound of $n^{0.694}$ is less than the upper bound of $n^{0.793}$ that follows from Theorem 3.2. The gap is due to the choice of input distribution; a more careful (and considerably harder) analysis with a non-independent distribution shows that the algorithm of Section 3.1 is in fact optimal.

3.3 Randomness and Non-uniformity

A basic issue in the study of randomized algorithms is the extent to which randomization is necessary for solving a problem. When is it possible to remove the randomization in a randomized algorithm? The answer depends on a number of aspects of the problem being solved. The goal of this section is to show that this question is more subtle than appears at first, and touches on the issue of uniformity in algorithms.

3.3.1 Boolean Circuits

Definition 3.12. A *Boolean circuit* with n inputs is a directed acyclic graph with the following properties:

1. There are n vertices of in-degree 0; these are called the inputs and are labeled $x_1, x_2, \dots, x_n$. There is one vertex with out-degree 0; this is called the output.
2. Every vertex v that is not an input or the output is labeled with one Boolean function $b(v) \in \{\text{AND}, \text{OR}, \text{NOT}\}$. A vertex labeled NOT has in-degree 1.

3. Under an assignment of input values, each vertex v computes the Boolean function $b(v)$ of the values on its incoming edges.
4. The *size* of a circuit is the number of vertices in it.

A *randomized circuit* is very similar, except that there may be more than n vertices of in-degree 0, partitioned into two classes:

1. **Random inputs**, each of which is assigned an independent random value from $\{0, 1\}$.
2. The n **circuit inputs**, labeled $x_1, \dots, x_n$.

A randomized circuit is said to *compute* a function f of the inputs $x_1, \dots, x_n$ if:

1. For inputs for which $f(x_1, \dots, x_n) = 0$, the output is 0 regardless of the values of random inputs.
2. If $f(x_1, \dots, x_n) = 1$, the output is 1 with probability at least $1/2$.

Definition 3.13. A sequence $C = C_1, C_2, \dots$ of circuits is a *circuit family* for f if C_n has n inputs and computes $f_n(x_1, \dots, x_n)$ at its output for all n -bit inputs. The family C is *polynomial-sized* if the size of C_n is bounded above by $p(n)$ for every n , where $p(\cdot)$ is a polynomial.

3.3.2 Adleman's Theorem

Theorem 3.14 (Adleman's Theorem). *If a Boolean function has a randomized, polynomial-sized circuit family, then it has a polynomial-sized circuit family.*

Proof. We show how to turn a given randomized polynomial-sized circuit C_n for $f_n(x_1, \dots, x_n)$ using random inputs $r_1, \dots, r_m$, into a deterministic polynomial-sized circuit D_n .

Form a matrix $\mathbf{M}$ with 2^n rows, one for each possible input from $\{0, 1\}^n$. The matrix has 2^m columns, one for each

of the possible m -tuples from $\{0, 1\}^m$ that the r_i can assume. The entry M_{jk} is 1 if the setting of $r_1, \dots, r_m$ corresponding to column k is a *witness* for the input corresponding to row j ; otherwise, the entry is 0. Eliminate all rows corresponding to inputs for which f_n evaluates to 0.

By definition, at least half the entries of every surviving row equal 1. Therefore, there must be a column with at least half its entries 1; in other words, there is an assignment of 0s and 1s to the r_i that serves as a witness to at least half of the possible inputs. Let this witness be $r_1^{(1)}, \dots, r_m^{(1)}$. Build a circuit T_1 , which is a copy of C_n with the random inputs “hard-wired” to $r_1^{(1)}, \dots, r_m^{(1)}$. Delete the column corresponding to this witness, and all rows that had 1s in this column.

The remaining matrix still has the property that every row has at least half its entries equal to 1. Repeat: pick a second witness $r_1^{(2)}, \dots, r_m^{(2)}$ that is a witness for at least half the remaining inputs and build a circuit T_2 . Continuing in this manner, we delete all rows while building at most n circuits $T_1, \dots, T_n$.

Take the OR of the outputs of $T_1, \dots, T_n$. This is a (deterministic) circuit whose size is $O(n)$ times that of the randomized circuit we started with. $\square$

Corollary 3.15. $\text{RP} \subseteq \text{P/poly}$.

Remark 3.16. The technique in Theorem 3.14 is the first example we have seen of *derandomization*—where we take a randomized algorithm and remove the randomness. However, the deterministic circuit generated by this process is one that works for a particular value of n . The circuit it produces for n inputs may have very little resemblance to the circuit it produces for $n + 1$ inputs, even if the original randomized circuits were similar. Any “practical” algorithm will exhibit the property of *uniformity*, which is formalized in the literature as follows.

Definition 3.17. An algorithm A is said to use *advice* a if on an input of length n it is given the string $a(n)$ on a read-only

tape. We say that A decides a language L with advice a if on an input x it uses the read-only string $a(|x|)$ to decide the membership of x in L .

The class P/poly consists of all languages L that have a non-uniform polynomial-time algorithm A such that the length of the advice string $a(n)$ is bounded by a polynomial in n .

Exercise 3.18. Consider any language $L \subseteq \{0, 1\}^*$. We define a Boolean function f corresponding to the language L as follows. For any positive integer n , let f_n be the Boolean function such that for any $x \in \{0, 1\}^n$, $f_n(x)$ assumes the value 1 if $x \in L$ and 0 otherwise. Show that $L \in P/\text{poly}$ if and only if it has a polynomial-sized circuit family.

In summary, the removal of randomness in Theorem 3.14 only shows that this can be done in principle; it is not known how to do this in any uniform or practical way.

Notes

The material of Section 3.1 is based on a paper of Snir [381]. Most of the material in Section 3.2 is covered in textbooks on game theory. Some good sources are the books by Wang [213], Luce and Raiffa [287], and von Neumann and Morgenstern [411]. Theorem 3.5 is due to von Neumann [408], and Theorem 3.6 is due to Loomis [279]. The application of the minimax theorems to proving lower bounds on randomized algorithms was pointed out by Yao [419]. Proposition 3.7 is also from [419].

In our lower bound for game tree evaluation, the principle that any deterministic algorithm may as well determine the value of one sub-tree before inspecting any leaves of its sibling (used in Section 3.2) is due to Tarsi [393]. Saks and Wigderson [362] refined the lower bound to show that Snir's algorithm is optimal among all randomized algorithms.

Theorem 3.14 is due to Adleman [1]; a version of this theorem applicable to circuit families with two-sided error is due to Gill [166]. The notion of non-uniformity is studied in depth in the paper by Karp and Lipton [245].

Problems

Exercise 3.19 (2.1). Show that for any deterministic evaluation algorithm, there is an instance of $T_{d,k}$ that forces the algorithm to read the values on all d^{2k} leaves.

Exercise 3.20 (2.2). Generalize the randomized algorithm and analysis of Section 3.1 to trees $T_{d,k}$ for $d > 2$.

Exercise 3.21 (2.3). (Due to R. Boppana [362].) Consider a uniform rooted tree of height h : every leaf is at distance h from the root. The root, as well as any internal node, has three children. Each leaf has a Boolean value associated with it. Each internal node returns the value returned by the majority of its children. The evaluation problem consists of determining the value of the root; at each step, an algorithm can choose one leaf whose value it wishes to read.

- (a) Show that for any deterministic algorithm, there is an instance that forces it to read all $n = 3^h$ leaves.
- (b) Consider the recursive randomized algorithm that evaluates two sub-trees of the root chosen at random. If the values returned disagree, it proceeds to evaluate the third sub-tree. Show that the expected number of leaves read by this algorithm (on any instance) is at most $n^{0.91}$.

Exercise 3.22 (2.4). Determine the value V_R of the following 2×2 matrix game and give optimal mixed strategies for the two players.

$$\mathbf{M} = \begin{pmatrix} 3 & 1 \\ 0 & 2 \end{pmatrix}$$

Exercise 3.23 (2.5). (Due to A. M. Karp.) Let (a_{ij}) be an $m \times n$ matrix, let the vector $(p_1, p_2, \dots, p_m)$ consist of reals in $[0, 1]$ such that $\sum_i p_i = 1$, and let $(q_1, q_2, \dots, q_n)$ consist of reals in $[0, 1]$ such that $\sum_j q_j = 1$. Prove:

$$\min_j \sum_i p_i a_{ij} \leq \sum_{i,j} p_i a_{ij} q_j \leq \max_i \sum_j a_{ij} q_j.$$

Exercise 3.24 (2.6). Use Yao's Minimax Principle to prove a lower bound on the expected running time of any Las Vegas algorithm for sorting n numbers.

Exercise 3.25 (2.7). (Due to A. M. Karp.) You are given an array A containing n numbers in sorted order. In one step, an algorithm may specify an integer $i \in [1, n]$, and is given the value of $A[i]$ in return. Determine lower and upper bounds on the expected number of steps taken by a Las Vegas randomized algorithm to determine whether or not a given key k is present in the array.

Exercise 3.26 (2.8). (Due to A. M. Karp.) In a graph with n vertices, where n is even, a perfect matching is a set of $n/2$ edges, no two of which meet at a common vertex. Consider a randomized algorithm that takes an n -vertex graph as input and correctly determines whether the graph has a perfect matching. At each step the algorithm asks a question of the form "Is there an edge between vertex i and vertex j ?" The complexity of the algorithm is defined as the maximum, over all n -vertex graphs G , of the expected number of questions $C(n)$ asked when the input graph is G . Prove: $C(n) = O(n^2)$.

Chapter 4

Moments and Deviations

In the preceding chapters we studied some simple randomized algorithms and encountered the need to analyze the probability that a random variable deviates significantly from its expected value. We now develop general tools for bounding tail probabilities—the probability that a random variable deviates by a given amount from its expectation. The probability of such a deviation is referred to as a *tail probability*. Readers wishing to review basic material on probability and distributions may consult Appendix A.

4.1 Occupancy Problems

We begin with an example of an occupancy problem. In such problems we envision each of m indistinguishable objects (“balls”) being randomly assigned to one of n distinct classes (“bins”). In other words, each ball is placed in a bin chosen independently and uniformly at random. We are interested in questions such as: what is the maximum number of balls in any bin? what is the expected number of bins with k balls in them? Such problems are at the core of the analyses of many randomized algorithms ranging from data structures to routing in parallel computers.

Our discussion of the occupancy problem will illustrate a recurrent tool in the analysis of randomized algorithms: that

the probability of the union of events is no more than the sum of their probabilities. This is a special case of the Boole–Bonferroni Inequalities (Proposition C.2) and can be formally stated as follows: for arbitrary events $\mathcal{E}_1, \mathcal{E}_2, \dots, \mathcal{E}_n$, not necessarily independent,

$$\Pr\left[\bigcup_{i=1}^n \mathcal{E}_i\right] \leq \sum_{i=1}^n \Pr[\mathcal{E}_i].$$

This principle is extremely useful because it assumes nothing about the dependencies between the events.

Consider first the case $m = n$. For $1 \leq i \leq n$, let X_i be the number of balls in the i th bin. Following Example 1.1, we have $\mathbb{E}[X_i] = 1$ for all i . Yet we do not expect that during a typical experiment every bin receives exactly one ball. Rather, we expect some bins to have no balls at all, and others to have many more than one.

Let us try now to make a statement of the form “with very high probability, no bin receives more than k balls,” for a suitably chosen k . Let $\mathcal{E}_j(k)$ denote the event that bin j has k or more balls in it. We concentrate on analyzing $\mathcal{E}_1(k)$.

The probability that bin 1 receives exactly i balls is

$$\Pr[X_1 = i] = \binom{m}{i} \frac{1}{n^i} \left(1 - \frac{1}{n}\right)^{m-i} \leq \frac{m^i}{i! \cdot n^i}.$$

The second inequality results from an upper bound for binomial coefficients (Proposition B.2). Thus,

$$\Pr[\mathcal{E}_1(k)] \leq \sum_{i=k}^m \frac{m^i}{i! \cdot n^i} \leq \frac{(m/n)^k}{k!} \cdot \frac{1}{1 - m/(kn)}.$$

Setting $k^* = \lceil (en \ln n) / \ln \ln n \rceil$, we obtain

$$\Pr[\mathcal{E}_1(k^*)] \leq \frac{(e \ln n)^{k^*}}{(k^*)!} \leq \frac{1}{n^2}. \quad (4.1)$$

The same computation tells us that this upper bound applies to $\Pr[\mathcal{E}_i(k^*)]$ for all i , but can we say that no bin is likely

to have more than k^* balls in it? For this we invoke the principle mentioned at the beginning of this section: the probability of the union of the events $\mathcal{E}_i(k^*)$ is no more than their sum.

Theorem 4.1 (Maximum Occupancy). *With probability at least $1 - 1/n$, no bin has more than $k^* = \lceil (en \ln n) / \ln \ln n \rceil$ balls when $m = n$ balls are randomly assigned to n bins.*

Proof. By the union bound and (4.1),

$$\Pr\left[\bigcup_{i=1}^n \mathcal{E}_i(k^*)\right] \leq \sum_{i=1}^n \Pr[\mathcal{E}_i(k^*)] \leq n \cdot \frac{1}{n^2} = \frac{1}{n}.$$

□

Interestingly, when m is of the order of $n \ln n$, the bin with the most balls has about the same number of balls as the expected number of balls in any bin. This phenomenon is exploited in a number of randomized algorithms.

Exercise 4.2. For $m = n \log n$, show that with probability $1 - o(1)$ every bin contains $O(\log n)$ balls.

4.1.1 The Birthday Problem

We turn to a classic combinatorial problem. Suppose that m balls are randomly assigned to n bins. We study the probability of the event that they all land in distinct bins. The special case $n = 365$ is popular in mathematical lore as the *birthday problem*. The interpretation is that the 365 days of the year correspond to 365 bins, and the birthday of each of m people is chosen independently and uniformly from all 365 days (ignoring leap years). How large must m be before two people in the group are likely to share their birthdays?

Consider the assignment of the balls to the bins as a sequential process: we throw the first ball into a random bin, then the second ball, and so on. For $2 \leq i \leq m$, let $\mathcal{E}_i$ denote the event that the i th ball lands in a bin not containing any

of the first $i - 1$ balls. We will bound $\Pr[\bigcap_{i=2}^m \mathcal{E}_i]$ from above. From the definition of conditional probability,

$$\Pr\left[\bigcap_{i=2}^m \mathcal{E}_i\right] = \Pr[\mathcal{E}_2] \Pr[\mathcal{E}_3 \mid \mathcal{E}_2] \Pr[\mathcal{E}_4 \mid \mathcal{E}_2 \cap \mathcal{E}_3] \cdots \Pr[\mathcal{E}_m \mid \bigcap_{i=2}^{m-1} \mathcal{E}_i].$$

Now, $\Pr[\mathcal{E}_i \mid \bigcap_{j=2}^{i-1} \mathcal{E}_j] = 1 - (i - 1)/n$. Making use of the fact that $1 - x \leq e^{-x}$, we have

$$\Pr\left[\bigcap_{i=2}^m \mathcal{E}_i\right] \leq \prod_{i=2}^m \left(1 - \frac{i-1}{n}\right) \leq \prod_{i=2}^m e^{-(i-1)/n} = e^{-\sum_{i=1}^{m-1} i/n} = e^{-m(m-1)/(2n)}.$$

Thus, we see that for m equal to $\lfloor \sqrt{2n+1} \rfloor$, the probability that all m balls land in distinct bins is at most $1/e$; as m increases beyond this value, the probability drops rapidly.

```

1 #include <iostream>
2 #include <vector>
3 #include <random>
4 #include <cmath>
5
6 // Estimate the probability of a birthday collision
   among m people
7 // with n possible birthdays, using repeated
   simulations.
8 double birthday_collision_prob(int m, int n, int
   trials,
9                               std::mt19937& rng) {
10     int collisions = 0;
11     std::uniform_int_distribution<int> dist(0, n -
12     1);
13     for (int t = 0; t < trials; ++t) {
14         std::vector<bool> seen(n, false);
15         bool collision = false;
16         for (int i = 0; i < m; ++i) {
17             int bday = dist(rng);
18             if (seen[bday]) { collision = true;
19                 break; }
20             seen[bday] = true;
21         }
22         if (collision) ++collisions;

```

```

21     }
22     return static_cast<double>(collisions) / trials
23     ;
24 }
25 int main() {
26     std::mt19937 rng(42);
27     int n = 365;
28     // Theoretical threshold:  $m \sim \sqrt{2 * n * \ln(1/(1-p))}$ 
29     // For  $p = 0.5$ ,  $m \sim 22.49$ 
30     double analytical = 1.0 - std::exp(-0.5);
31     std::cout << "Birthday Problem (n=" << n << ")
32     :\n";
33     for (int m : {10, 15, 20, 23, 30, 50}) {
34         double sim = birthday_collision_prob(m, n,
35         100000, rng);
36         double theory = 1.0 - std::exp(-static_cast
37         <double>(m * (m - 1)) / (2.0 * n));
38         std::cout << "    m=" << m
39         << "    simulated=" << sim
40         << "    theoretical=" << theory <<
41         "\n";
42     }
43     return 0;
44 }

```

4.2 The Markov and Chebyshev Inequalities

We have seen above that making statements about the probability that a random variable deviates far from its expectation may involve a detailed, problem-specific analysis. Often, one can avoid such detailed analyses by resorting to general inequalities on such tail probabilities.

We begin with the Markov inequality, a fundamental tool we will invoke repeatedly when we develop more sophisticated bounding techniques.

Theorem 4.3 (Markov Inequality). *Let Y be a random variable assuming only non-negative values. Then for all $t \in \mathbb{R}^+$,*

$$\Pr[Y \geq t] \leq \frac{\mathbb{E}[Y]}{t}.$$

Equivalently, $\Pr[Y \geq k \mathbb{E}[Y]] \leq 1/k$.

Proof. Define a function $f(y)$ by $f(y) = 1$ if $y \geq t$, and 0 otherwise. Then $\Pr[Y \geq t] = \mathbb{E}[f(Y)]$. Since $f(y) \leq y/t$ for all y ,

$$\mathbb{E}[f(Y)] \leq \mathbb{E}\left[\frac{Y}{t}\right] = \frac{\mathbb{E}[Y]}{t},$$

and the theorem follows. □

This is the tightest possible bound when we know only that Y is non-negative and has a given expectation. Unfortunately, the Markov inequality by itself is often too weak to yield useful results.

Exercise 4.4. Given a positive integer k , describe a random variable X assuming only non-negative values, such that $\Pr[X \geq k \mathbb{E}[X]] = 1/k$.

Exercise 4.5. Let Y be any random variable and h any non-negative real function. Show that for all $t \in \mathbb{R}^+$,

$$\Pr[h(Y) \geq t] \leq \frac{\mathbb{E}[h(Y)]}{t}.$$

The following generalization of Markov's inequality underlies its usefulness in deriving stronger bounds.

We now show that the Markov inequality can be used to derive better bounds on the tail probability by using more information about the distribution of the random variable. The first of these is the Chebyshev bound, which is based on the knowledge of the variance of the distribution.

For a random variable X with expectation μ_X , its *variance* σ_X^2 is defined to be $\mathbb{E}[(X - \mu_X)^2]$. The *standard deviation* of X , denoted σ_X , is the positive square root of σ_X^2 .

Theorem 4.6 (Chebyshev's Inequality). *Let X be a random variable with expectation μ_X and standard deviation σ_X . Then for any $t \in \mathbb{R}^+$,*

$$\Pr[|X - \mu_X| \geq t \sigma_X] \leq \frac{1}{t^2}.$$

Proof. First, note that

$$\Pr[|X - \mu_X| \geq t \sigma_X] = \Pr[(X - \mu_X)^2 \geq t^2 \sigma_X^2].$$

The random variable $Y = (X - \mu_X)^2$ has expectation σ_X^2 , and applying the Markov inequality to Y bounds this probability from above by $1/t^2$. $\square$

Exercise 4.7. The use of the variance of a random variable in bounding its deviation from its expectation is called the *second moment method*. In an analogous fashion, we can speak of the *kth moment method*: let k be even, and suppose we have a random variable X for which $\mu_k = \mathbb{E}[(X - \mu_X)^k]$ exists. Show that

$$\Pr[|X - \mu_X| > t (\mu_k)^{1/k}] \leq \frac{1}{t^k}.$$

Why is the k th moment method difficult to invoke for odd values of k ?

4.3 Randomized Selection

We now consider the use of random sampling for the problem of selecting the k th smallest element in a set S of n elements drawn from a totally ordered universe. We assume that the elements of S are all distinct, although it is not very hard to modify the following analysis to allow for multisets. Let $r_S(t)$ denote the rank of an element t (the k th smallest element has rank k) and let $S_{\langle i \rangle}$ denote the i th smallest element of S .

Algorithm 3 LazySelect

Require: A set S of n elements from a totally ordered universe; integer $k \in [n]$

Ensure: The k th smallest element of S , $S_{\langle k \rangle}$

- 1: Pick $n^{3/4}$ elements from S , chosen independently and uniformly at random with replacement; call this multiset of elements R
 - 2: Sort R using any optimal sorting algorithm in $O(n^{3/4} \log n)$ steps
 - 3: Let $x = k \cdot n^{-1/4}$. Set $t = \max\{\lfloor x - \sqrt{n} \rfloor, 1\}$ and $h = \min\{\lceil x + \sqrt{n} \rceil, n^{3/4}\}$
 - 4: Let $a = R_{\langle t \rangle}$ and $b = R_{\langle h \rangle}$. By comparing a and b to every element of S , determine $r_S(a)$ and $r_S(b)$
 - 5: **if** $k < n^{1/4}$ **then**
 - 6: $P \leftarrow \{y \in S \mid y \leq b\}$
 - 7: **else if** $k > n - n^{1/4}$ **then**
 - 8: $P \leftarrow \{y \in S \mid y \geq a\}$
 - 9: **else**
 - 10: $P \leftarrow \{y \in S \mid a \leq y \leq b\}$
 - 11: **end if**
 - 12: Check whether $S_{\langle k \rangle} \in P$ and $|P| \leq 4n^{3/4} + 2$. If not, repeat Steps 1–5 until such a set P is found
 - 13: Sort P and identify $P_{\langle k - r_S(a) + 1 \rangle}$, which is $S_{\langle k \rangle}$
-

The idea of the algorithm is to identify two elements a and b in S such that both of the following hold with high probability:

1. The element $S_{\langle k \rangle}$ that we seek is in P .
2. The set P of elements between a and b is not very large, so that we can sort P inexpensively in Step 7.

We examine how either of these requirements could fail. We focus on the most interesting case when $k \in [n^{1/4}, n - n^{1/4}]$, so that $P = \{y \in S \mid a \leq y \leq b\}$.

If the element a is greater than $S_{\langle k \rangle}$ (or if b is smaller than or equal to $S_{\langle k \rangle}$), we fail because P does not contain $S_{\langle k \rangle}$. For

this to happen, fewer than t of the samples in R should be smaller than $S_{\langle k \rangle}$ (respectively, at least h of the random samples should be smaller than $S_{\langle k \rangle}$). We will bound the probability that this happens using the Chebyshev bound.

Lemma 4.8 (Additivity of Variances). *Let $X_1, X_2, \dots, X_m$ be independent random variables. Let $X = \sum_{i=1}^m X_i$. Then $\sigma_X^2 = \sum_{i=1}^m \sigma_{X_i}^2$.*

Proof. Let μ_i denote $\mathbb{E}[X_i]$, and $\mu = \sum_{i=1}^m \mu_i$. The variance of X is given by

$$\mathbb{E}[(X - \mu)^2] = \mathbb{E}\left[\left(\sum_{i=1}^m (X_i - \mu_i)\right)^2\right].$$

Expanding the latter and using linearity of expectations,

$$\mathbb{E}[(X - \mu)^2] = \sum_{i=1}^m \mathbb{E}[(X_i - \mu_i)^2] + 2 \sum_{i < j} \mathbb{E}[(X_i - \mu_i)(X_j - \mu_j)].$$

Since all pairs X_i, X_j are independent, so are the pairs $(X_i - \mu_i), (X_j - \mu_j)$. By the fact that expectation factors for independent variables, each term in the latter summation equals $\mathbb{E}[(X_i - \mu_i)] \cdot \mathbb{E}[(X_j - \mu_j)]$. Since $\mathbb{E}[(X_i - \mu_i)] = 0$, the latter summation vanishes. $\square$

Theorem 4.9 (LazySelect Correctness). *With probability $1 - O(n^{-1/4})$, LazySelect finds $S_{\langle k \rangle}$ on the first pass through Steps 1–7, and thus performs only $2n + o(n)$ comparisons.*

Proof. Step 3 requires $2n$ comparisons, and all other steps perform $o(n)$ comparisons, provided the algorithm finds $S_{\langle k \rangle}$ on the first pass. We now consider the first mode of failure: $a > S_{\langle k \rangle}$ because fewer than t of the samples in R are less than or equal to $S_{\langle k \rangle}$.

Let $X_i = 1$ if the i th random sample is at most $S_{\langle k \rangle}$, and 0 otherwise; thus $\Pr[X_i = 1] = k/n$, and $\Pr[X_i = 0] = 1 - k/n$. Let $X = \sum_{i=1}^{n^{3/4}} X_i$ be the number of samples of R that are at most $S_{\langle k \rangle}$. The random variables X_i are Bernoulli trials. Using

Lemma 4.8 and the variance of a Bernoulli trial with success probability p ,

$$\mu_X = kn^{3/4}/n = kn^{-1/4},$$

and

$$\sigma_X^2 = n^{3/4} \cdot \frac{k}{n} \cdot \left(1 - \frac{k}{n}\right) \leq n^{3/4} \cdot \frac{1}{4}.$$

This implies that $\sigma_X \leq n^{3/8}/2$. Applying the Chebyshev bound to X ,

$$\Pr[|X - \mu_X| \geq \sqrt{n}] = \Pr[|X - \mu_X| \geq 2n^{1/8}\sigma_X] = o(n^{-1/4}).$$

An essentially identical argument shows that $\Pr[b < S_{\langle k \rangle}] = o(n^{-1/4})$. Since the probability of the union of events is at most the sum of their probabilities, the probability that either of these events occurs (causing $S_{\langle k \rangle}$ to lie outside P) is $O(n^{-1/4})$.

The analysis of the second mode of failure—that P contains more than $4n^{3/4} + 2$ elements—is very similar, with k_t and k_h playing the role of k . Adding up the probabilities of all failure modes, we find that the probability that Steps 1–5 fail to find a suitable set P is $O(n^{-1/4})$. $\square$

This adds to the significance of LazySelect: the best known deterministic selection algorithms use $3n$ comparisons in the worst case and are quite complicated to implement. Further, it is known that any deterministic algorithm for finding the median requires at least $2n$ comparisons, so we have a randomized algorithm that is both fast and has an expected number of comparisons that is provably smaller than that of any deterministic algorithm.

Exercise 4.10. The failure probability can be driven down further at the expense of increased running time. For a suitable definition of the $o(n)$ term, give an upper bound on the probability that the algorithm does not find $S_{\langle k \rangle}$ in $cn + o(n)$ steps for $c > 2$.

Exercise 4.11. Suggest a modification in the algorithm that brings the constant in the linear term down to 1.5 from 2. We will refine this further in Problem 4.16.

Exercise 4.12. Show that as a direct corollary of Theorem 4.9, the expected running time of the LazySelect algorithm is $2n + o(n)$.

```

1  #include <algorithm>
2  #include <cmath>
3  #include <iostream>
4  #include <random>
5  #include <vector>
6
7  // Returns the k-th smallest element (0-indexed)
   // using LazySelect.
8  // Expected comparisons:  $2n + o(n)$ , succeeding w.h.
   // p. on first pass.
9  int lazy_select(std::vector<int>& S, int k, std::mt19937& rng) {
10     int n = static_cast<int>(S.size());
11     int sample_size = static_cast<int>(std::ceil(
        std::pow(n, 0.75)));
12     std::uniform_int_distribution<int> dist(0, n -
        1);
13
14     while (true) {
15         // Step 1: sample  $n^{\{3/4\}}$  elements with
           // replacement
16         std::vector<int> R(sample_size);
17         for (int i = 0; i < sample_size; ++i)
18             R[i] = S[dist(rng)];
19
20         // Step 2: sort R
21         std::sort(R.begin(), R.end());
22
23         // Step 3: compute indices t, h
24         double x = static_cast<double>(k) * std::
           pow(n, -0.25);
25         int t = std::max(static_cast<int>(std::
           floor(x - std::sqrt(n))), 1);
26         int h = std::min(static_cast<int>(std::ceil
           (x + std::sqrt(n))),
27                         sample_size - 1);
28         int a_val = R[t];
29         int b_val = R[h];
30
31         // Step 4: compute ranks of a and b in S

```

```
32     int rank_a = 0, rank_b = 0;
33     for (int i = 0; i < n; ++i) {
34         if (S[i] <= a_val) ++rank_a;
35         if (S[i] < b_val) ++rank_b;
36     }
37
38     // Build candidate set P
39     std::vector<int> P;
40     if (k < static_cast<int>(std::pow(n, 0.25))
41         ) {
42         for (int v : S) if (v <= b_val) P.
43             push_back(v);
44     } else if (k > n - static_cast<int>(std::
45         pow(n, 0.25))) {
46         for (int v : S) if (v >= a_val) P.
47             push_back(v);
48     } else {
49         for (int v : S)
50             if (v >= a_val && v <= b_val) P.
51                 push_back(v);
52     }
53
54     // Check S<k> is in P and |P| is small
55     enough
56     int target_in_P = k - rank_a + 1;
57     if (target_in_P >= 1 && target_in_P <=
58         static_cast<int>(P.size())
59         && static_cast<int>(P.size()) <= 4 *
60             sample_size + 2) {
61         // Step 7: sort P and pick the element
62         std::sort(P.begin(), P.end());
63         return P[target_in_P - 1];
64     }
65     // Otherwise retry
66 }
67
68 int main() {
69     std::mt19937 rng(42);
70     std::vector<int> S(10000);
71     for (int i = 0; i < 10000; ++i) S[i] = i + 1;
72     std::shuffle(S.begin(), S.end(), rng);
73
74     int k = 5000; // find the median
75     int result = lazy_select(S, k, rng);
```

```

69     std::cout << "LazySelect: " << k << "th
        smallest = "
70         << result << "\n";
71
72     std::sort(S.begin(), S.end());
73     std::cout << "Sorted:      " << k << "th
        smallest = "
74         << S[k - 1] << "\n";
75     return 0;
76 }

```

4.4 Two-Point Sampling

We have so far been making use of the fact that the variance of the sum of independent random variables equals the sum of their variances. In fact, we can make a stronger statement. Let X and Y be discrete random variables defined on the same probability space. The *joint density function* of X and Y is the function

$$p(x, y) = \Pr[\{X = x\} \cap \{Y = y\}].$$

A set of random variables $X_1, X_2, \dots, X_m$ is said to be *pairwise independent* if for all $i \neq j$, and $x, y \in \mathbb{R}$,

$$\Pr[X_i = x \mid X_j = y] = \Pr[X_i = x].$$

Exercise 4.13. Let n be a prime number and $\mathbb{Z}_n$ denote the field of integers modulo n . For a and b chosen independently and uniformly at random from $\mathbb{Z}_n$, let $Y_i = ai + b \pmod{n}$. Show that for $i \neq j \pmod{n}$, Y_i and Y_j are uniformly distributed on $\mathbb{Z}_n$ and pairwise independent. (Make use of the fact that in the field $\mathbb{Z}_n$, given fixed values for Y_i and Y_j , we can solve $Y_i = ai + b \pmod{n}$ and $Y_j = aj + b \pmod{n}$ uniquely for a and b .)

Exercise 4.14. Let $X_1, X_2, \dots, X_m$ be pairwise independent random variables, and $X = \sum_{i=1}^m X_i$. Show that $\sigma_X^2 = \sum_{i=1}^m \sigma_{X_i}^2$.

We now consider an application of these concepts to the reduction of the random bits used by RP algorithms. Consider an RP algorithm A for deciding whether input strings x

belong to a language L . Given x , A picks a random number r from the range $\mathbb{Z}_n = \{0, \dots, n-1\}$ and computes a binary value $A(x, r)$ with the following properties:

- If $x \in L$, then $A(x, r) = 1$ for at least half the possible values of r .
- If $x \notin L$, then $A(x, r) = 0$ for all possible choices of r .

For any $x \in L$, we refer to the values of r for which $A(x, r) = 1$ as *witnesses* for x ; clearly, at least $n/2$ of the n possible values of r are witnesses. Choosing t independent random numbers is expensive in that it requires $O(t \log n)$ random bits. Suppose instead that we are only willing to use $O(\log n)$ random bits—in particular, suppose we wish to use only two independent samples from $\mathbb{Z}_n$.

The naive usage of a and b as potential witnesses yields an upper bound of only $1/4$ on the probability of incorrect classification. Here is a better scheme: let $r_i = ai + b \pmod{n}$, and compute $A(x, r_i)$ for $1 \leq i \leq t$. As before, if for any i we obtain $A(x, r_i) = 1$, we declare that x is in L , else we declare that x is not in L .

Theorem 4.15 (Probability Amplification via Two-Point Sampling). *With the two-point sampling scheme, the error probability is at most $1/t$.*

Proof. We need to worry about the possibility of making error only when the input x is in L . By the result of Exercise 3.7, the random r_i 's are pairwise independent and, therefore, so are the random variables $A(x, r_i)$, for $1 \leq i \leq t$. Let $Y = \sum_{i=1}^t A(x, r_i)$. Assuming that $x \in L$, $\mathbb{E}[Y] \geq t/2$ and $\sigma_Y^2 \leq t/4$, so $\sigma_Y \leq \sqrt{t}/2$. The probability that the pairwise independent iterations produce an incorrect classification corresponds to the event $\{Y = 0\}$, and

$$\Pr[Y = 0] \leq \Pr[|Y - \mathbb{E}[Y]| \geq t/2].$$

By the Chebyshev inequality, the latter is at most $1/t$. Thus, the error probability is at most $1/t$, which is a considerable improvement over the error bound of $1/4$ achieved by the naive use of a and b . $\square$

```

1 #include <iostream>
2 #include <random>
3 #include <vector>
4
5 // Simulated RP algorithm: for  $x$  in  $L$ ,  $A(x,r)=1$  for
  // half the  $r$ 's.
6 // For  $x$  not in  $L$ ,  $A(x,r)=0$  for all  $r$ .
7 bool RP_algorithm(int x, int r, bool x_in_L) {
8     if (!x_in_L) return false;
9     // Deterministic witness check:  $r < n/2$  means
      // witness
10    return r < 500;
11 }
12
13 // Naive: use  $a$  and  $b$  directly as witnesses. Error
  //  $\leq 1/4$ .
14 bool naive_two_point(int x, int n, bool x_in_L,
15                      std::mt19937& rng) {
16     std::uniform_int_distribution<int> dist(0, n -
17     1);
18     int a = dist(rng), b = dist(rng);
19     if (RP_algorithm(x, a, x_in_L)) return true;
20     if (RP_algorithm(x, b, x_in_L)) return true;
21     return false;
22 }
23
24 // Amplified: use  $r_i = a*i + b \bmod n$  for  $t$  samples
  // .
25 // Error  $\leq 1/t$  by pairwise independence.
26 bool amplified_two_point(int x, int n, int t, bool
27 x_in_L,
28                          std::mt19937& rng) {
29     std::uniform_int_distribution<int> dist(0, n -
30     1);
31     int a = dist(rng), b = dist(rng);
32     for (int i = 0; i < t; ++i) {
33         int ri = (static_cast<long long>(a) * i + b
34         ) % n;
35         if (RP_algorithm(x, ri, x_in_L)) return
36         true;
37     }
38     return false;
39 }
40 }
41

```

```

36 int main() {
37     std::mt19937 rng(42);
38     int n = 1000;
39     int trials = 100000;
40     int t = 10; // amplification parameter
41
42     // Test with  $x$  in  $L$  (should almost always
43     // return true)
44     int naive_false = 0, amp_false = 0;
45     for (int i = 0; i < trials; ++i) {
46         if (!naive_two_point(1, n, true, rng)) ++
47             naive_false;
48         if (!amplified_two_point(1, n, t, true, rng
49             )) ++amp_false;
50     }
51     std::cout << "x in L:\n"
52               << " Naive error rate: "
53               << static_cast<double>(naive_false) /
54                   trials << "\n"
55               << " Amplified error rate: "
56               << static_cast<double>(amp_false) /
57                   trials << "\n"
58               << " (theory: naive <= 1/4,
59                   amplified <= 1/t = "
60                   << 1.0 / t << ")\n";
61     return 0;
62 }

```

4.5 The Stable Marriage Problem

Consider a society in which there are n men (denoted by capital letters $A, B, C, \dots$) and n women (denoted by lowercase letters $a, b, c, \dots$). A marriage M is a one-to-one correspondence between the men and the women. Assume a monogamous society. Each person has a preference list of the members of the opposite sex organized in decreasing order of desirability. A marriage is said to be *unstable* if there exist two married couples $X-x$ and $Y-y$ such that X desires y more than x , and y desires X more than Y , implying that $X-y$ will have a tendency to leave their current mates to marry each other. A marriage

M in which there are no dissatisfied couples is called a *stable marriage*.

Example 4.16. For $n = 4$, consider the following preference lists.

$A : a b c d \quad B : b a e d \quad C : a d e b \quad D : d e a b$
 $a : A B C D \quad b : D C B A \quad c : A B C D \quad d : C D A B$

Consider the marriage M given by $A-a$, $B-b$, $C-c$, and $D-d$. Here $C-d$ is a dissatisfied couple, implying that M is unstable. However, if C and d marry each other, and c and D marry each other, we obtain the stable marriage given by $A-a$, $B-b$, $C-d$, $D-c$.

The problem of finding stable marriages has several interesting applications, for example in matching medical graduates to residency positions in hospitals. It can be shown that for every choice of preference lists there exists at least one stable marriage. We will prove this by presenting an algorithm to find a stable marriage.

Algorithm 4 The Proposal Algorithm (Gale–Shapley)

Require: Complete preference lists for n men and n women

Ensure: A stable marriage M

- 1: Start with all men and women free (unmarried)
 - 2: **while** there exists a free man X who has not proposed to every woman **do**
 - 3: X proposes to the most desirable woman x on his list who has not already rejected him
 - 4: **if** x is free **then**
 - 5: x accepts; X and x become engaged
 - 6: **else if** x prefers X over her current fiancé Y **then**
 - 7: x jilts Y and becomes engaged to X ; Y becomes free
 - 8: **else**
 - 9: x rejects X
 - 10: **end if**
 - 11: **end while**
 - 12: The engaged pairs form a stable marriage
-

The basic idea behind this algorithm can be summarized as “man proposes, woman disposes”: each currently unattached man proposes to the most desirable woman on his list who has not already rejected him, and this woman then decides whether to accept or reject a proposal.

Theorem 4.17 (Proposal Algorithm Terminates). *The Proposal Algorithm terminates with a stable marriage in at most n^2 proposals.*

Proof. A woman once married will stay married during the course of the algorithm, although her mates may change with time. Furthermore, the desirability of her mates (in her view) can only improve with time. Thus at each step either a woman gets married for the first time, or an already married woman obtains a more desirable mate.

An unattached man always has at least one woman available that he can proposition. This is because every woman he has already proposed to is currently married, and if he runs out of women then all women are married—this cannot happen unless all men are married too. Since at each step the proposer will eliminate one woman on his list, and the total size of the lists is n^2 , we conclude that the algorithm uses at most n^2 proposals.

We claim that the final marriage M is stable. Otherwise, let X – y be a dissatisfied pair, where in M they are paired as X – x and Y – y . Since X prefers y to x , he must have proposed to y before getting married to x . Since y either rejected X , or accepted him only to jilt him later, her mates thereafter (including Y) must be more desirable to her than X . Therefore, y must prefer Y to X , contradicting the assumption that y is dissatisfied. $\square$

4.5.1 Average-Case Analysis and the Principle of Deferred Decisions

Our interest here is in performing an average-case analysis of this algorithm. Thus we are considering a probabilistic anal-

ysis of a deterministic algorithm. We introduce this analysis here because it touches upon several tools that are important in the analysis of randomized algorithms.

For this average-case analysis, we assume that the men's lists are chosen independently and uniformly at random; the women's lists can be arbitrary but must be fixed in advance. Let the random variable T_P denote the number of proposals made during the execution of the Proposal Algorithm.

We present a very simple technique—the *Principle of Deferred Decisions*—for getting around problems of dependency in the analysis. The idea is to not assume that the entire set of random choices is made in advance. Rather, at each step of the process we fix only the random choices that must be revealed to the algorithm.

The Principle of Deferred Decisions can be used to simplify the average-case analysis of the Proposal Algorithm as follows. We do not assume that the men have chosen their (random) preference list in advance. Each time a man has to make a proposal, he picks a random woman from the set of women not already propositioned by him, and proceeds to propose to her.

Suppose instead that each time a man makes a proposal, he chooses a woman uniformly at random from the set of all n women, including those to whom he has already proposed. Call this new algorithm the *Amnesiac Algorithm*. The output produced by the Amnesiac Algorithm is exactly the same as that of the original Proposal Algorithm. The only difference is that there are some wasted proposals in the Amnesiac Algorithm. Let T_A denote the number of proposals made by the Amnesiac Algorithm. Clearly, T_A stochastically dominates T_P : for all m , $\Pr[T_A > m] \geq \Pr[T_P > m]$.

The algorithm terminates with a stable marriage once all women have received at least one proposal each. As will become clear in the next section, bounding the value of T_A is a special case of the Coupon Collector's Problem.

Theorem 4.18. For any constant $c \in \mathbb{R}$, and $m = n \ln n + cn$,

$$\lim_{n \rightarrow \infty} \Pr[T_A > m] = 1 - e^{-e^{-c}}.$$

```

1 #include <algorithm>
2 #include <iostream>
3 #include <random>
4 #include <vector>
5
6 struct StableMarriage {
7     int n;
8     std::vector<std::vector<int>> men_prefs;
9     std::vector<std::vector<int>> women_prefs;
10    std::vector<int> women_rank; // rank[w][m]:
        how much w likes m
11    std::vector<int> next_proposal;
12    std::vector<int> current_wife;
13    std::vector<int> current_husband;
14    int proposals;
15
16    StableMarriage(int n, std::mt19937& rng)
17        : n(n), men_prefs(n), women_prefs(n),
18          women_rank(n, std::vector<int>(n)),
19          next_proposal(n, 0),
20          current_wife(n, -1), current_husband(n,
21            -1),
22          proposals(0) {
23        // Generate random preference lists
24        for (int i = 0; i < n; ++i) {
25            men_prefs[i].resize(n);
26            for (int j = 0; j < n; ++j) men_prefs[i]
27                [j] = j;
28            std::shuffle(men_prefs[i].begin(),
29                men_prefs[i].end(), rng);
30            women_prefs[i].resize(n);
31            for (int j = 0; j < n; ++j) women_prefs
32                [i][j] = j;
33            std::shuffle(women_prefs[i].begin(),
34                women_prefs[i].end(), rng)
35                ;
36            for (int j = 0; j < n; ++j)
37                women_rank[i][women_prefs[i][j]] =
38                    j;
39        }
34 }

```

```

35     }
36
37     void run() {
38         std::vector<int> free_men(n);
39         for (int i = 0; i < n; ++i) free_men[i] = i
40             ;
41         while (!free_men.empty()) {
42             int man = free_men.back();
43             int woman = men_prefs[man][
44                 next_proposal[man]++];
45             ++proposals;
46             if (current_husband[woman] == -1) {
47                 current_husband[woman] = man;
48                 current_wife[man] = woman;
49                 free_men.pop_back();
50             } else if (women_rank[woman][man]
51                 < women_rank[woman][
52                     current_husband[woman]])
53             {
54                 int old = current_husband[woman];
55                 current_husband[woman] = man;
56                 current_wife[man] = woman;
57                 current_wife[old] = -1;
58                 free_men.pop_back();
59                 free_men.push_back(old);
60             }
61         }
62     }
63
64     bool is_stable() const {
65         for (int m = 0; m < n; ++m) {
66             int w = current_wife[m];
67             for (int pref = 0; pref < women_rank[w
68                 ][m]; ++pref) {
69                 int w_pref = women_prefs[w][pref];
70                 int m_curr = current_husband[w];
71                 if (women_rank[w_pref][w]
72                     < women_rank[w_pref][
73                         current_wife[w_pref]])
74                     return false;
75             }
76         }
77         return true;
78     }
79 };

```

```
74
75 int main() {
76     std::mt19937 rng(42);
77     std::cout << "Stable Marriage (Gale-Shapley):\n";
78     for (int n : {4, 8, 16, 32, 64}) {
79         double total_proposals = 0;
80         int trials = 1000;
81         for (int t = 0; t < trials; ++t) {
82             StableMarriage sm(n, rng);
83             sm.run();
84             total_proposals += sm.proposals;
85         }
86         double avg = total_proposals / trials;
87         double theory = n * std::log(n);
88         std::cout << "    n=" << n
89                 << "    avg_proposals=" << avg
90                 << "    n*ln(n)=" << theory << "\n";
91     }
92     return 0;
93 }
```

4.6 The Coupon Collector's Problem

In the coupon collector's problem, there are n types of coupons and at each trial a coupon is chosen at random. Each random coupon is equally likely to be of any of the n types, and the random choices of the coupons are mutually independent. Let m be the number of trials. The goal is to study the relationship between m and the probability of having collected at least one copy of each of the n types. The reader may wish to make the correspondence between this process and an occupancy problem (Section 4.1) in which m balls are randomly distributed in n bins.

4.6.1 An Elementary Analysis

Let X be a random variable defined to be the number of trials required to collect at least one of each type of coupon. We first

determine the expected value of X .

Call the i th trial C_i a *success* if the type C_i was not drawn in any of the first $i - 1$ selections. We divide the sequence into *epochs*, where epoch i begins with the trial following the i th success and ends with the trial on which we obtain the $(i + 1)$ st success. Define the random variable X_i , for $0 \leq i \leq n - 1$, to be the number of trials in the i th epoch, so that $X = \sum_{i=0}^{n-1} X_i$.

The random variable X_i is geometrically distributed with parameter $p_i = (n - i)/n$ (the probability of drawing one of the $n - i$ remaining coupon types). Thus, the expected value of X_i is $1/p_i = n/(n - i)$ and its variance is $(1 - p_i)/p_i^2 = i/(n - i)$. By linearity of expectation,

$$\mathbb{E}[X] = \mathbb{E}\left[\sum_{i=0}^{n-1} X_i\right] = \sum_{i=0}^{n-1} \mathbb{E}[X_i] = \sum_{i=1}^n \frac{n}{i} = nH_n.$$

By Proposition B.4 the n th Harmonic number H_n is asymptotically equal to $\ln n + \Theta(1)$, implying that $\mathbb{E}[X] = n \ln n + O(n)$.

Theorem 4.19 (Coupon Collector Expected Time). *The expected number of trials to collect all n coupon types is $nH_n = n \ln n + O(n)$.*

Since the X_i 's are independent, we can determine the variance of X using the additivity of variances:

$$\sigma_X^2 = \sum_{i=0}^{n-1} \sigma_{X_i}^2 = \sum_{i=0}^{n-1} \frac{i}{(n - i)^2} \leq \frac{n}{2} \sum_{i=1}^n \frac{1}{i^2} \leq \frac{\pi^2 n}{12}.$$

This variance bound lets us apply Chebyshev's inequality to obtain a weak but useful tail bound. For $t = n \ln n$ (the approximate mean),

$$\Pr[|X - nH_n| \geq t] \leq \frac{\sigma_X^2}{t^2} = O\left(\frac{n}{n^2 \ln^2 n}\right) = O\left(\frac{1}{n \ln^2 n}\right).$$

In particular, the probability that X deviates from its mean by more than $n \ln n$ decays like $O(1/(n \ln^2 n))$, which is much

weaker than the exponential bound obtained by the union bound below. This illustrates a general principle: Chebyshev gives polynomial tail bounds that are easy to derive, while more refined methods (Chernoff, sharp threshold) give exponentially stronger guarantees.

Exercise 4.20. Use the Chebyshev inequality to find an upper bound on the probability that $X > \beta n \ln n$, for a constant $\beta > 1$.

Let $\mathcal{E}_i$ denote the event that coupon type i is not collected in the first r trials. Using the inequality $(1 - 1/n)^r \leq e^{-r/n}$, we obtain that

$$\Pr[\mathcal{E}_i] = \left(1 - \frac{1}{n}\right)^r \leq e^{-r/n}.$$

This bound is $n^{-\beta}$ for $r = \beta n \ln n$. Using the union bound, we obtain for $r = \beta n \ln n$,

$$\Pr[X > r] = \Pr\left[\bigcup_{i=1}^n \mathcal{E}_i\right] \leq \sum_{i=1}^n \Pr[\mathcal{E}_i] \leq n \cdot n^{-\beta} = n^{1-\beta}.$$

4.6.2 The Poisson Heuristic

Before we show the sharp concentration result for X , the following heuristic argument will help to establish some intuition. The heuristic argument is based on the approximation of the binomial distribution by the Poisson distribution.

Let $N_i^{(r)}$ denote the number of times the coupon of type i is chosen during the first r trials; the event $\mathcal{E}_i$ is the same as the event $\{N_i^{(r)} = 0\}$. The random variable $N_i^{(r)}$ has the binomial distribution with parameters r and $p = 1/n$. For suitably small λ and as r approaches ∞ , the Poisson distribution with parameter λ is a good approximation to the binomial distribution with parameters r and p . In the current setting, we approximate the distribution of $N_i^{(r)}$ by the Poisson distribution with parameter $\lambda = r/n$. A random variable Y has the

Poisson distribution with parameter λ if for any non-negative integer y ,

$$\Pr[Y = y] = \frac{\lambda^y e^{-\lambda}}{y!}.$$

Using this approximation,

$$\Pr[\mathcal{E}_i] = \Pr[N_i^{(r)} = 0] \approx \frac{\lambda^0 e^{-\lambda}}{0!} = e^{-r/n}. \quad (4.2)$$

The main benefit of the Poisson approximation is that we can claim the events $\mathcal{E}_i$, for $1 \leq i \leq n$, are “almost independent” even though there is some dependence between them. If the approximation in (4.2) were exact, we would obtain that the events $\mathcal{E}_i$ are truly independent. Making the heuristic assumption of independence, we obtain that for $1 \leq i \leq n$, the probability that all coupon types are collected in the first m trials is given by

$$\Pr\left[\bigcap_{i=1}^n (\overline{\mathcal{E}_i})\right] = \prod_{i=1}^n \Pr[\overline{\mathcal{E}_i}] \approx (1 - e^{-m/n})^n \approx e^{-ne^{-m/n}}.$$

Let $m = n(\ln n + c)$ for any constant $c \in \mathbb{R}$. Then,

$$\Pr[X > m] \approx \Pr\left[\bigcup_{i=1}^n \mathcal{E}_i\right] \approx 1 - e^{-e^{-c}}.$$

Observe that this probability $e^{-e^{-c}}$ is close to 1 for large positive c , and is negligibly small for large negative c . Thus, the probability of having collected all n coupon types abruptly changes from nearly zero to almost one in a small interval centered around $n \ln n$.

4.6.3 A Sharp Threshold

We now convert the heuristic argument from the previous section into a rigorous (but significantly more complex) proof using the Boole–Bonferroni Inequalities (Proposition C.2).

Lemma 4.21. *Let c be a real constant, and $m = n \ln n + cn$ for positive integer n . Then, for any fixed positive integer k ,*

$$\lim_{n \rightarrow \infty} \binom{n}{k} \left(1 - \frac{k}{n}\right)^m = \frac{e^{-ck}}{k!}.$$

Theorem 4.22 (Sharp Threshold for Coupon Collector). *Let the random variable X denote the number of trials for collecting each of the n types of coupons. Then, for any constant $c \in \mathbb{R}$, and $m = n \ln n + cn$,*

$$\lim_{n \rightarrow \infty} \Pr[X > m] = 1 - e^{-e^{-c}}.$$

Proof. We have that the event $\{X > m\} = \bigcup_{i=1}^n \mathcal{E}_i$. By the Principle of Inclusion–Exclusion,

$$\Pr\left[\bigcup_{i=1}^n \mathcal{E}_i\right] = \sum_{k=1}^n (-1)^{k+1} P_k,$$

where $P_k = \binom{n}{k} (1 - k/n)^m$ is the k th elementary symmetric sum. Let $s_k = P_1 - P_2 + P_3 - \cdots + (-1)^{k+1} P_k$ denote the partial sum formed by the first k terms of this series. By the Boole–Bonferroni inequalities (Proposition C.2), we have the bracketing property of the partial sums:

$$s_{2j} \leq \Pr\left[\bigcup_{i=1}^n \mathcal{E}_i\right] \leq s_{2j+1}.$$

By Lemma 4.21, for each k , $P_k \rightarrow e^{-ck}/k!$ as $n \rightarrow \infty$. Define the partial sums of the terms

$$s_k = \sum_{j=1}^k (-1)^{j+1} \frac{e^{-cj}}{j!} = 1 - e^{-e^{-c}}.$$

Notice that the right-hand side consists precisely of the first k terms of the power series expansion of $f(c) = 1 - e^{-e^{-c}}$. We conclude that $\lim_{k \rightarrow \infty} s_k = f(c)$. The bracketing property of partial sums then implies that

$$\Pr\left[\bigcup_{i=1}^n \mathcal{E}_i\right] \rightarrow f(c) = 1 - e^{-e^{-c}}$$

as $n \rightarrow \infty$. □

By this theorem, for any real constant c , we have

$$\lim_{n \rightarrow \infty} \Pr[X < n(\ln n - c)] = e^{-e^c}, \quad \lim_{n \rightarrow \infty} \Pr[X > n(\ln n + c)] = 1 - e^{-e^{-c}}.$$

Thus, with extremely high probability, the number of trials for collecting all n coupon types lies in a small interval centered about its expected value. This result is almost like a deterministic result since it so sharply identifies the threshold value for collecting all coupons. We refer to such results as *sharp threshold results*.

```

1 #include <cmath>
2 #include <iostream>
3 #include <random>
4 #include <vector>
5
6 // Simulate the coupon collector: return the number
   // of trials
7 // needed to collect all n coupon types.
8 int simulate_coupon_collector(int n, std::mt19937&
   rng) {
9     std::uniform_int_distribution<int> dist(0, n -
       1);
10    std::vector<bool> collected(n, false);
11    int remaining = n;
12    int trials = 0;
13    while (remaining > 0) {
14        int coupon = dist(rng);
15        ++trials;
16        if (!collected[coupon]) {
17            collected[coupon] = true;
18            --remaining;
19        }
20    }
21    return trials;
22 }
23
24 int main() {
25     std::mt19937 rng(42);
26     std::cout << "Coupon Collector Problem:\n";
27     std::cout << "    n    avg_trials    n*H_n"
```

```

28         << "   Pr[X>n(ln n+c)] theory vs sim\n
           ";
29
30     for (int n : {10, 50, 100, 500, 1000}) {
31         double harmonic = 0.0;
32         for (int i = 1; i <= n; ++i) harmonic +=
           1.0 / i;
33         double expected_theory = n * harmonic;
34
35         double total = 0;
36         int trials_sim = 1000;
37         for (int t = 0; t < trials_sim; ++t)
38             total += simulate_coupon_collector(n,
           rng);
39         double avg = total / trials_sim;
40
41         // Check the sharp threshold for c=0 (m = n
           ln n)
42         double m_threshold = n * std::log(n);
43         int exceed_count = 0;
44         for (int t = 0; t < trials_sim; ++t) {
45             int tc = 0;
46             std::vector<bool> seen(n, false);
47             int rem = n;
48             std::uniform_int_distribution<int> d(0,
           n - 1);
49             while (rem > 0 && tc < static_cast<int>
           >(m_threshold)) {
50                 int c = d(rng);
51                 ++tc;
52                 if (!seen[c]) { seen[c] = true; --
           rem; }
53             }
54             if (rem > 0) ++exceed_count;
55         }
56         double sim_prob = static_cast<double>(
           exceed_count) / trials_sim;
57         double theory_prob = 1.0 - std::exp(-std::
           exp(0.0));
58         // For c=0: limit = 1 - e^{-1} ~ 0.6321
59         theory_prob = 1.0 - std::exp(-1.0);
60
61         std::cout << "   " << n << "   " << avg
62                 << "   " << expected_theory
63                 << "   theory=" << theory_prob

```

```

64         << "  sim=" << sim_prob << "\n";
65     }
66     return 0;
67 }

```

Notes

Comprehensive treatises on occupancy problems are the books by Johnson and Kotz [45], and by Kolchin, Chistiakov, and Sevastianov [50]. Generally, we will be concerned with statements involving asymptotic estimates on random variables and probabilities. We will return to such estimates for occupancy problems in Chapter 5. Recent work by Azar, Broder, Karlin, and Upfal [17] builds on the basic occupancy problem and points out many applications to computer science.

The history of tail inequalities such as the Chebyshev bound dates back to the early days of probability theory. Following Chebyshev's bound [27], Markov [64] observed that the same idea could be used with higher moments. Kolmogorov [51] went further and remarked that $\Pr[X \geq r] \leq \mathbb{E}[f(X)]/s$ for any function $f(X)$, provided that $\mathbb{E}[f(X)]$ exists and $f(x) \geq s > 0$ for all $x \geq r$. The latter idea was exploited by Bernstein and by Chernoff in a manner we will describe in Chapter 5.

Classic sources for deterministic selection algorithms are the papers of Blum, Floyd, Pratt, Rivest, and Tarjan [22], and of Schönhage, Paterson, and Pippenger [75]. The LazySelect algorithm presented here is a variant on one reported by Floyd and Rivest [37]. The lower bound of $2n$ for median selection is due to Bent and John [20].

The construction of pairwise independent random variables in Exercise 3.7 is given in Joffe [44]. Its application to the reduction of random bits used by abstract randomized algorithms is due to Chor and Goldreich [28]; Luby [59] presented this idea in the context of derandomizing algorithms.

The stable marriage problem was introduced by Gale and Shapley [39]. The average-case analysis using the Principle of

Deferred Decisions follows Knuth [49]. The connection to the Coupon Collector's Problem was noted by Irving [42].

The Coupon Collector's Problem is classical; see the books by Feller [35] and by Ross [74] for textbook treatments. The sharp threshold result of Theorem 3.8 is due to Erdős and Rényi [34].

Problems

Problem 4.23. (Balls into Bins—Maximum Load) Consider the experiment of throwing m balls into n bins, where each ball lands in a bin chosen independently and uniformly at random. Prove that if $m = n$, then the maximum load $L = \max_{1 \leq i \leq n} X_i$ satisfies

$$\Pr\left[L \geq \frac{3 \ln n}{\ln \ln n}\right] \leq \frac{1}{n}.$$

Problem 4.24. (Birthday Paradox—Generalized) Suppose that we have a universe of size n and we sample m elements independently and uniformly at random. Show that the probability of a collision (two sampled elements being equal) is at most $m^2/(2n)$. Deduce that for $m = O(\sqrt{n})$, a collision is likely.

Problem 4.25. (LazySelect—Expected Comparisons) Prove rigorously that the expected number of comparisons performed by LazySelect is $2n + o(n)$. Your analysis should account for the expected number of retries.

Problem 4.26. (Pairwise Independence) Let n be a prime and let a, b be chosen independently and uniformly at random from $\mathbb{Z}_n$. For each $i \in \mathbb{Z}_n$, define $Y_i = ai + b \pmod{n}$. Prove that the Y_i 's are pairwise independent but not mutually independent.

Problem 4.27. (Stable Marriage—Optimality) Show that the Proposal Algorithm is *male-optimal*: every man receives the best partner possible among all stable marriages. Show by example that it is not female-optimal.

Problem 4.28. (Coupon Collector—Full Distribution) Derive a more refined estimate of the distribution of X , the number of trials to collect all n coupons. Specifically, show that for any constant $c \in \mathbb{R}$,

$$\Pr[X \leq n \ln n + cn] \rightarrow e^{-e^{-c}} \quad \text{as } n \rightarrow \infty.$$

Problem 4.29. (Weighted Coupon Collector) Suppose there are n coupon types, but coupon i is drawn with probability p_i (where $\sum_i p_i = 1$). Let X be the number of trials to collect all n types. Show that

$$\mathbb{E}[X] = \int_0^\infty \left(1 - \prod_{i=1}^n (1 - e^{-p_i t})\right) dt.$$

Problem 4.30. (Second Moment Method) Let X be a non-negative random variable with $\mu = \mathbb{E}[X] > 0$ and $\sigma^2 = \text{Var}(X)$. Prove that

$$\Pr[X > 0] \geq \frac{\mu^2}{\mu^2 + \sigma^2}.$$

Show that this is tight.

Problem 4.31. (Birthday Attack on Hash Functions) Suppose $h : \{0, 1\}^* \rightarrow \{0, 1\}^k$ is a hash function that maps inputs to $n = 2^k$ values. We hash m independent inputs. Using the birthday bound, show that we need $m \approx 2^{k/2}$ inputs to find a collision with probability $\approx 1/2$. What does this imply for the security of hash functions?

Problem 4.32. (Amnesiac Algorithm) Consider the Amnesiac Algorithm for the Stable Marriage Problem, where each man proposes to a uniformly random woman (ignoring past rejections). Prove formally that T_A stochastically dominates T_P (the number of proposals in the original Proposal Algorithm), i.e., $\Pr[T_A > m] \geq \Pr[T_P > m]$ for all m .

Chapter 5

Tail Inequalities

In the analysis of randomized algorithms, we frequently need to bound the probability that a random variable deviates significantly from its expected value. The Markov inequality provides a generic tool for this purpose, but its bound is often too loose for practical use.

Definition 5.1 (With High Probability). An event E_n (parametrized by the input size n) occurs *with high probability* (abbreviated w.h.p.) if $\Pr[E_n] \geq 1 - O(n^{-c})$ for some constant $c > 0$. Equivalently, the failure probability is polynomially small in n . In some contexts, “w.h.p.” may require $\Pr[E_n] \geq 1 - 1/n^c$ for any desired constant c , where the hidden constant in the algorithm’s running time may depend on c .

Note that the threshold “w.h.p.” is stronger than “with constant probability” ($\Pr \geq 2/3$). By repeating an algorithm $O(\log n)$ times and taking a median (or a union bound), constant-probability guarantees can typically be amplified to w.h.p. guarantees.

In this chapter, we develop sharper *tail inequalities*—Chernoff bounds, Hoeffding bounds, and their martingale extensions—that exploit independence and structural properties to yield exponentially small failure probabilities. These bounds are among the most frequently used tools in the analysis of randomized algorithms and have far-reaching applications in computer science, statistics, and combinatorics.

5.1 The Chernoff Bound

We begin with the Chernoff bound, perhaps the most widely used tail inequality in the analysis of randomized algorithms. The bound applies to the sum of independent Bernoulli random variables and provides exponentially decaying bounds on the probability that the sum deviates from its expectation.

Let $X_1, X_2, \dots, X_n$ be independent Bernoulli random variables, where $\Pr[X_i = 1] = p_i$ and $\Pr[X_i = 0] = 1 - p_i$. Define $X = \sum_{i=1}^n X_i$ with expectation $\mu = \mathbb{E}[X] = \sum_{i=1}^n p_i$. Our goal is to bound $\Pr[X \geq (1 + \delta)\mu]$ and $\Pr[X \leq (1 - \delta)\mu]$ for $\delta > 0$.

The key idea behind the Chernoff bound is the use of the *moment generating function* (MGF). For any $\lambda > 0$,

$$\Pr[X \geq a] = \Pr[e^{\lambda X} \geq e^{\lambda a}] \leq \frac{\mathbb{E}[e^{\lambda X}]}{e^{\lambda a}}$$

by Markov's inequality. The challenge is to bound $\mathbb{E}[e^{\lambda X}]$.

Lemma 5.2 (MGF Bound for Bernoulli Sums). *Let $X_1, X_2, \dots, X_n$ be independent Bernoulli random variables with $\mathbb{E}[X_i] = p_i$ and $\mu = \sum_{i=1}^n p_i$. Then for all $\lambda > 0$,*

$$\mathbb{E}[e^{\lambda X}] \leq e^{\mu(e^\lambda - 1)}.$$

Proof. Since the X_i are independent,

$$\mathbb{E}[e^{\lambda X}] = \prod_{i=1}^n \mathbb{E}[e^{\lambda X_i}] = \prod_{i=1}^n (p_i e^\lambda + (1 - p_i)) = \prod_{i=1}^n (1 + p_i(e^\lambda - 1)).$$

Using the inequality $1 + x \leq e^x$ (valid for all x), each factor satisfies

$$1 + p_i(e^\lambda - 1) \leq \exp(p_i(e^\lambda - 1)).$$

Taking the product,

$$\mathbb{E}[e^{\lambda X}] \leq \exp\left(\sum_{i=1}^n p_i(e^\lambda - 1)\right) = e^{\mu(e^\lambda - 1)}. \quad \square$$

Combining Lemma 5.2 with Markov's inequality yields the Chernoff bound in its standard multiplicative form.

Theorem 5.3 (Chernoff Bound—Multiplicative Form). *Let $X_1, X_2, \dots, X_n$ be independent Bernoulli random variables with $\mathbb{E}[X] = \mu = \sum_{i=1}^n p_i$ where $X = \sum_{i=1}^n X_i$. Then for any $\delta > 0$:*

(i) **Upper tail:** $\Pr[X \geq (1 + \delta)\mu] \leq \left(\frac{e^\delta}{(1+\delta)^{(1+\delta)}} \right)^\mu$. In particular, for $0 < \delta \leq 1$:

$$\Pr[X \geq (1 + \delta)\mu] \leq e^{-\mu\delta^2/3}.$$

(ii) **Lower tail:** $\Pr[X \leq (1 - \delta)\mu] \leq \left(\frac{e^{-\delta}}{(1-\delta)^{(1-\delta)}} \right)^\mu$. In particular, for $0 < \delta \leq 1$:

$$\Pr[X \leq (1 - \delta)\mu] \leq e^{-\mu\delta^2/2}.$$

Proof. We prove the upper tail; the lower tail is analogous.

By Markov's inequality and Lemma 5.2, for any $\lambda > 0$:

$$\Pr[X \geq (1 + \delta)\mu] \leq e^{-\lambda(1+\delta)\mu} \cdot e^{\mu(e^\lambda - 1)}.$$

Setting $\lambda = \ln(1 + \delta) > 0$:

$$\Pr[X \geq (1 + \delta)\mu] \leq (1 + \delta)^{-(1+\delta)\mu} \cdot e^{\mu\delta} = \left(\frac{e^\delta}{(1 + \delta)^{(1+\delta)}} \right)^\mu.$$

For $0 < \delta \leq 1$, the concavity of $\ln$ and a Taylor expansion show that $\ln(1 + \delta) \geq \delta - \delta^2/2$, which yields $\delta - (1 + \delta) \ln(1 + \delta) \leq -\delta^2/3$, giving the simplified bound $e^{-\mu\delta^2/3}$.

For the lower tail, we choose $\lambda = -\ln(1 - \delta) > 0$ and obtain the analogous bound $e^{-\mu\delta^2/2}$. $\square$

Remark 5.4. The asymmetry in the exponents ($-1/3$ vs. $-1/2$) reflects the fact that the lower tail decays faster than the upper tail. This is a consequence of the convexity of e^x and the different optimal choices of λ .

Example 5.5 (Application to Lower Bounds). Chernoff bounds provide a simple route to proving lower bounds on the running time of randomized algorithms. Consider the problem of estimating the fraction of satisfying assignments of a DNF formula. Each clause in the DNF is satisfied by a fraction p_j of random assignments. Sampling k random assignments and checking each against the DNF, the number of satisfied samples S satisfies $\mathbb{E}[S] = kp$ where p is the true fraction. By the Chernoff bound, to estimate p within relative error ϵ , we need $k = O(\ln(1/\delta)/(p\epsilon^2))$ samples. This immediately implies a lower bound: no algorithm using fewer samples can achieve relative error ϵ with probability exceeding $1 - \delta$.

5.1.1 Derandomization via Conditional Expectations

Chernoff bounds also serve as the foundation for *derandomization* of randomized algorithms. The method of conditional expectations allows us to systematically replace random choices with deterministic ones while preserving the expectation bound. The key observation is that $\mathbb{E}[X \mid X_1 = x_1, \dots, X_k = x_k]$ is itself a sum of independent Bernoulli random variables, to which the Chernoff bound applies. By evaluating conditional expectations, one can deterministically construct a solution that is at least as good as the expectation.

5.2 Routing in a Parallel Computer

One of the classic applications of Chernoff bounds in computer science is to the analysis of randomized routing algorithms in parallel architectures.

Consider a parallel computer with n processors connected to n memory modules. At each time step, each processor independently selects a memory module uniformly at random and sends a packet to that module. If multiple packets are destined for the same module, they queue up and are

served one at a time. The routing is completed when all packets have been delivered, and the *congestion* is the maximum queue length at any module.

The expected number of packets arriving at any given memory module is $\mu = n \cdot (1/n) = 1$. Thus, we expect each module to receive about one packet. However, the interesting question is whether the maximum congestion is close to this expectation.

Theorem 5.6 (Routing Congestion). *In the random routing model described above, the maximum queue length is at most $\frac{\ln n}{\ln \ln n}$ with probability at least $1 - n^{-c}$ for any fixed constant $c > 0$.*

Proof. Let q_j denote the queue length at module j . Since each of the n processors independently chooses module j with probability $1/n$, we have $q_j = \sum_{i=1}^n X_{ij}$ where X_{ij} is the indicator that processor i sends to module j . Thus q_j is a sum of n independent Bernoulli random variables with $p_i = 1/n$ for each, giving $\mu = 1$.

By the Chernoff bound (Theorem 5.3(i)), for any $t > 0$:

$$\Pr[q_j \geq t] \leq \left(\frac{e^{t-1}}{t^t} \right).$$

Setting $t = \frac{\ln n}{\ln \ln n}$ and applying a union bound over all n modules:

$$\Pr\left[\max_j q_j \geq t\right] \leq n \cdot \Pr[q_1 \geq t] \leq n \cdot \left(\frac{e}{t}\right)^t = n \cdot \exp(t(1 - \ln t)).$$

Substituting $t = \frac{\ln n}{\ln \ln n}$:

$$t(1 - \ln t) = \frac{\ln n}{\ln \ln n} \left(1 - \ln \left(\frac{\ln n}{\ln \ln n} \right) \right) = \frac{\ln n}{\ln \ln n} (1 - \ln \ln n + \ln \ln \ln n)$$

which is $O(-\ln n)$ for large n . Hence the failure probability is $n \cdot e^{-\Omega(\ln n)} = n^{-c}$ for suitable $c > 1$. $\square$

Remark 5.7. The bound of $\frac{\ln n}{\ln \ln n}$ is asymptotically tight for this model: it can be shown that with high probability, the

maximum congestion is at least $\Omega\left(\frac{\ln n}{\ln \ln n}\right)$. The key insight is that the union bound, combined with the exponential decay of the Chernoff bound, converts the per-module exponential tail into a high-probability global guarantee.

The following C++ code illustrates this phenomenon experimentally. We simulate random routing for varying numbers of processors and compare the observed maximum congestion against the theoretical bound.

Listing 5.1: Routing simulation (see `routing.h`)

```

1 int simulate_routing(int n, mt19937& rng) {
2     std::vector<int> queues(n, 0);
3     uniform_int_distribution<int> dist(0, n - 1);
4     for (int i = 0; i < n; i++)
5         queues[dist(rng)]++;
6     return *max_element(queues.begin(), queues.end());
7 }
```

5.3 A Wiring Problem

The Chernoff bound finds another elegant application in the analysis of wiring problems in VLSI design. Consider the following model: we have n signal sources and n signal sinks placed on a line. Each source i is connected to a unique sink $\pi(i)$, where π is a random permutation. Each wire is laid out along the line, and the total wire length is $\sum_{i=1}^n |i - \pi(i)|$.

For a random permutation, the displacement $|i - \pi(i)|$ is a random variable with $\mathbb{E}[|i - \pi(i)|] = \Theta(\sqrt{n})$ for a typical index i (more precisely, the expected displacement for a uniformly random position is $\mathbb{E}[|i - \pi(i)|] \leq \sqrt{n}$). By linearity of expectation, the expected total wire length is $\Theta(n^{3/2})$.

The Chernoff bound provides a concentration result: with high probability, the total wire length is close to its expectation. More precisely, let $W = \sum_{i=1}^n |i - \pi(i)|$. Although the terms $|i - \pi(i)|$ are not independent (they are determined by the single permutation π), one can decompose the analysis into independent contributions using the method of *Hoeffding*

decomposition or by bounding the sum by independent random variables.

Theorem 5.8 (Wire Length Concentration). *For a random permutation π on $\{1, \dots, n\}$, the total wire length $W = \sum_{i=1}^n |i - \pi(i)|$ satisfies, for any $\delta > 0$:*

$$\Pr[|W - \mathbb{E}[W]| \geq \delta \cdot \mathbb{E}[W]] \leq 2 \exp(-\Omega(\delta^2 n)).$$

This result has practical implications: it guarantees that random wiring layouts in VLSI circuits will, with high probability, have wire lengths close to the expected value, enabling reliable performance prediction.

5.4 Martingales

The Chernoff bound requires independence of the underlying random variables, which limits its applicability. Many natural random variables exhibit sequential dependence—for example, the number of edges in a random graph, the height of a random search tree, or the score in a sequential process. *Martingales* provide the right framework for analyzing such quantities.

Definition 5.9 (Martingale). A sequence of random variables $X_0, X_1, X_2, \dots, X_n$ is called a *martingale* with respect to a filtration (sequence of σ -algebras) $\mathcal{F}_0 \subseteq \mathcal{F}_1 \subseteq \dots \subseteq \mathcal{F}_n$ if:

- (i) X_i is $\mathcal{F}_i$ -measurable for each i ;
- (ii) $\mathbb{E}[|X_i|] < \infty$ for each i ;
- (iii) $\mathbb{E}[X_i \mid \mathcal{F}_{i-1}] = X_{i-1}$ for each $i \geq 1$.

Intuitively, a martingale models a “fair game”: the expected future value, given the present, equals the present value.

Example 5.10 (Random Graph Edge Count). Consider the Erdős–Rényi random graph $G(n, p)$, and let X_k denote the number

of edges in the subgraph induced by the first k vertices. Then $X_0, X_1, \dots, X_n$ forms a martingale (with respect to the natural filtration). Indeed, when the $(k+1)$ -th vertex is added, each potential edge to the existing k vertices is included independently with probability p . The conditional expectation of the new edges is kp , so $\mathbb{E}[X_{k+1} \mid X_k] = X_k + kp$ — wait, this is not a martingale but a *submartingale*. A proper martingale arises by considering $Z_k = X_k - \mathbb{E}[X_k] = X_k - \binom{k}{2}p$.

The power of martingale theory lies in concentration inequalities that generalize the Chernoff bound.

Theorem 5.11 (Azuma–Hoeffding Inequality). *Let $X_0, X_1, \dots, X_n$ be a martingale with bounded differences:*

$$|X_i - X_{i-1}| \leq c_i \quad \text{for all } i = 1, \dots, n,$$

where $c_1, c_2, \dots, c_n$ are constants. Then for any $t > 0$:

$$\Pr[|X_n - X_0| \geq t] \leq 2 \exp\left(-\frac{t^2}{2 \sum_{i=1}^n c_i^2}\right).$$

Proof. We prove the upper tail $\Pr[X_n - X_0 \geq t]$; the lower tail follows by symmetry. Define the *Doob martingale* differences $D_i = X_i - X_{i-1}$, so $X_n - X_0 = \sum_{i=1}^n D_i$.

For any $\lambda > 0$:

$$\Pr[X_n - X_0 \geq t] = \Pr[e^{\lambda \sum D_i} \geq e^{\lambda t}] \leq e^{-\lambda t} \cdot \mathbb{E}[e^{\lambda \sum D_i}].$$

Since $\mathbb{E}[D_i \mid \mathcal{F}_{i-1}] = 0$ (by the martingale property) and $|D_i| \leq c_i$, the Hoeffding lemma gives:

$$\mathbb{E}[e^{\lambda D_i} \mid \mathcal{F}_{i-1}] \leq e^{\lambda^2 c_i^2 / 2}.$$

Taking iterated expectations:

$$\mathbb{E}[e^{\lambda \sum D_i}] \leq \exp\left(\frac{\lambda^2}{2} \sum_{i=1}^n c_i^2\right).$$

Minimizing over $\lambda > 0$ by setting $\lambda = t / \sum c_i^2$ yields the claimed bound. $\square$

Corollary 5.12 (Concentration for Bounded-Difference Functions). *Let $f(x_1, \dots, x_n)$ be a function satisfying the bounded differences condition: changing any single input x_i changes f by at most c_i , while leaving the other inputs fixed. If $X_1, \dots, X_n$ are independent random variables, then $Z = f(X_1, \dots, X_n)$ satisfies:*

$$\Pr[|Z - \mathbb{E}[Z]| \geq t] \leq 2 \exp\left(-\frac{t^2}{2 \sum_{i=1}^n c_i^2}\right).$$

Proof. Define the Doob martingale $Z_k = \mathbb{E}[Z \mid X_1, \dots, X_k]$. Then $Z_0 = \mathbb{E}[Z]$ and $Z_n = Z$. The bounded differences condition ensures $|Z_k - Z_{k-1}| \leq c_k$, so Theorem 5.11 applies. $\square$

Example 5.13 (Random Graph Applications). The Azuma–Hoeffding inequality is particularly powerful for random graphs. For the chromatic number $\chi(G)$ of $G(n, 1/2)$, we have $\chi(G) = f(X_1, \dots, X_m)$ where X_e are the edge indicators and f satisfies bounded differences with $c_e = 1$ for all e . Thus:

$$\Pr[|\chi(G) - \mathbb{E}[\chi(G)]| \geq t] \leq 2 \exp\left(-\frac{t^2}{m}\right)$$

where $m = \binom{n}{2}$. This shows that the chromatic number of a random graph is highly concentrated around its expectation.

Remark 5.14. The Azuma–Hoeffding inequality is tight up to constant factors in the exponent when all c_i are equal. More refined inequalities (such as the Bennett and Bernstein inequalities) provide better constants when the c_i are non-uniform or when additional information about the distribution is available.

Notes

The Chernoff bound originated in the work of **Chernoff** (1952) on the asymptotic efficiency of tests and estimates. The multiplicative form presented here was developed and popularized

by **Hoeffding** (1963), who also established the more general bounds for sums of independent bounded random variables. An earlier, independent result by **Hoeffding** (1956) on the Hoeffding inequality for bounded random variables predates the Chernoff-style bound in the literature.

The routing application (Section 5.2) is based on the analysis of packet routing in parallel architectures, which has been a central problem in parallel computing since the 1980s. The bound of $O(\ln n / \ln \ln n)$ for random routing is a classical result; see Leighton, Maggs, and Rao (1994) for related results on oblivious routing.

The Azuma–Hoeffding inequality (Theorem 5.11) is named after **Azuma** (1967), who proved a version for martingales with bounded differences, and **Hoeffding** (1963), who proved the independent case. The elegant connection between bounded differences and martingale concentration was developed by **McDiarmid** (1989) in his foundational paper on the “method of bounded differences.” The survey by **Spencer** (1993) provides an excellent introduction to these techniques with numerous applications in combinatorics.

For further reading, we recommend the comprehensive treatment in *Probability and Computing* by Mitzenmacher and Upfal (2005), and the monograph *Concentration Inequalities* by Boucheron, Lugosi, and Massart (2013).

Problems

Problem 5.15. Let $X_1, \dots, X_n$ be independent Bernoulli random variables with $\Pr[X_i = 1] = 1/n$ for all i , and let $X = \sum X_i$. Use the Chernoff bound to show that $\Pr[X \geq 2] \leq n/e^2$ for $n \geq 4$. Compare with the exact value obtained from the Poisson approximation.

Problem 5.16. Prove that the lower tail bound in Theorem 5.3(ii) is tight up to constant factors in the exponent. That is, exhibit a sequence of independent Bernoulli random variables for which $\Pr[X \leq (1 - \delta)\mu] \geq e^{-\Theta(\mu\delta^2)}$.

Problem 5.17. In the routing problem of Section 5.2, suppose each processor sends to a *random* destination chosen uniformly from $\{1, \dots, n\}$, but now there are m rounds. After each round, any delivered packets are removed. Show that the expected total time to deliver all packets is $O(\ln n)$ rounds with high probability.

Problem 5.18. Let $G = G(n, 1/2)$ be the Erdős–Rényi random graph. Using the Azuma–Hoeffding inequality, prove that the number of triangles in G is concentrated within $O(n^{3/2})$ of its expectation.

Problem 5.19. Let $f(x_1, \dots, x_n) = \max_{1 \leq i \leq n} x_i$ where $x_i \in [0, 1]$. Show that f satisfies the bounded differences condition with $c_i = 1$ for all i . Apply Corollary 5.12 to the maximum of n independent uniform $[0, 1]$ random variables and compare the Azuma–Hoeffding bound with the exact distribution.

Problem 5.20. Prove that if $X_0, X_1, \dots, X_n$ is a martingale with $|X_i - X_{i-1}| \leq 1$ for all i , then $\Pr[|X_n - X_0| \geq t] \leq 2e^{-t^2/(2n)}$. This is sometimes called the “simple” Azuma inequality.

Problem 5.21. Consider a random variable $Z = \prod_{i=1}^n Y_i$ where Y_i are independent, each taking value 2 with probability $1/2$ and value $1/2$ with probability $1/2$. Taking logarithms and applying the Chernoff bound, show that $\Pr[Z \geq 2^{n/2+t\sqrt{n}}] \leq e^{-\Omega(t^2)}$. Interpret this as a “multiplicative” Chernoff bound.

Problem 5.22. Let A be a random $n \times n$ adjacency matrix of $G(n, p)$ with $p = c/n$ for a constant $c > 0$. The largest eigenvalue $\lambda_{\max}(A)$ of A satisfies $|\lambda_{\max}(A) - c| \leq O(\sqrt{c})$ with high probability. Use the method of bounded differences to prove a concentration inequality for $\lambda_{\max}(A)$.

Problem 5.23. Show that the moment generating function bound in Lemma 5.2 gives a tighter result than directly applying Markov’s inequality to X without the exponential transformation. Specifically, compare the bounds obtained for $\Pr[X \geq 2\mu]$ using (a) Markov’s inequality on X directly and (b) the Chernoff bound.

Problem 5.24. (Chernoff for Poisson.) Let $X \sim \text{Poisson}(\lambda)$.

Prove that $\Pr[X \geq (1 + \delta)\lambda] \leq \left(\frac{e^\delta}{(1+\delta)^{(1+\delta)}} \right)^\lambda$ for $\delta > 0$, using the same MGF technique as in the proof of Theorem 5.3.

Chapter 6

The Probabilistic Method

The probabilistic method is one of the most powerful and elegant techniques in combinatorics and theoretical computer science. Introduced by Erdős in the 1940s, its core idea is deceptively simple: to prove that a combinatorial object with certain properties *exists*, it suffices to show that a randomly constructed object has the desired properties with positive probability. Unlike constructive proofs, the probabilistic method need not exhibit the object explicitly—it merely demonstrates that the object cannot fail to exist.

The method has three essential ingredients. First, define a probability space over candidate objects. Second, identify the “bad” events—those properties that the object must avoid. Third, show that the probability of any bad event is strictly less than one, so that with positive probability no bad event occurs. The power of the method lies in the fact that the probabilistic analysis often reveals structural truths that are invisible to purely algebraic or constructive approaches.

In this chapter, we develop the probabilistic method from its foundations through several important applications: random satisfiability, expander graphs, oblivious routing, the Lovász Local Lemma, and the method of conditional probabilities for derandomization.

6.1 Overview of the Method

The simplest form of the probabilistic method is captured by the following observation. Suppose we wish to show that there exists an object x in a finite set S satisfying property P . If we define a probability distribution on S and show that

$$\Pr_{x \leftarrow S} [P(x)] > 0,$$

then there must exist at least one $x \in S$ with property P .

More generally, suppose we have m “bad” events $A_1, A_2, \dots, A_m$ that we wish to avoid simultaneously. If we can show that

$$\Pr \left[\bigcup_{i=1}^m A_i \right] < 1,$$

then $\Pr[\overline{A_1} \cap \overline{A_2} \cap \dots \cap \overline{A_m}] > 0$, and the desired object exists. The union bound (Boole’s inequality) gives the simple sufficient condition

$$\sum_{i=1}^m \Pr[A_i] < 1.$$

More refined versions, such as the Lovász Local Lemma (Section 6.5), provide much sharper conditions.

Example 6.1 (Ramsey Numbers). The Ramsey number $R(3, 3)$ is the minimum n such that every 2-coloring of the edges of the complete graph K_n contains a monochromatic triangle. We prove the upper bound $R(3, 3) \leq 6$ using the probabilistic method (though a direct argument suffices for this small case).

More interestingly, consider the question: for a given n , what is the probability that a random 2-coloring of K_n avoids monochromatic triangles? Color each edge independently red or blue with probability $1/2$ each. For a fixed triangle, the probability that all three edges receive the same color is $2 \cdot (1/2)^3 = 1/4$. By linearity of expectation, the expected number of monochromatic triangles is

$$\binom{n}{3} \cdot \frac{1}{4}.$$

When $n = 5$, this expectation is $\binom{5}{3}/4 = 10/4 = 2.5$. When $n = 4$, it is $\binom{4}{3}/4 = 4/4 = 1$. The probabilistic argument shows that for $n = 5$, since the expected count is $2.5 > 0$, there exist colorings with monochromatic triangles—but this does not yet show that *every* coloring has one. The power of the method is more evident in larger Ramsey problems where direct enumeration is infeasible.

Lemma 6.2 (Expected Monochromatic Subgraphs). *For a random 2-coloring of the edges of K_n (each edge colored independently red or blue with probability $1/2$), the expected number of monochromatic copies of K_k is*

$$\binom{n}{k} \cdot 2^{1-\binom{k}{2}}.$$

Proof. There are $\binom{n}{k}$ copies of K_k in K_n . For a fixed copy, the probability that all $\binom{k}{2}$ edges receive the same color is $2 \cdot (1/2)^{\binom{k}{2}} = 2^{1-\binom{k}{2}}$. By linearity of expectation, the expected number of monochromatic copies is the product. $\square$

This simple counting argument—relying only on linearity of expectation—yields non-trivial upper bounds on Ramsey numbers and is the prototype for the probabilistic method.

6.2 Maximum Satisfiability

The Maximum Satisfiability problem (MAX-SAT) asks: given a Boolean formula in conjunctive normal form (CNF), find an assignment that satisfies the maximum number of clauses. This problem is NP-hard in general, but the probabilistic method provides elegant approximation guarantees.

Definition 6.3 (MAX-SAT). Let $\phi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$ be a CNF formula with n variables $x_1, \dots, x_n$ and m clauses. Each clause C_j is a disjunction of literals. The MAX-SAT problem asks for an assignment $\alpha : \{x_1, \dots, x_n\} \rightarrow \{0, 1\}$ maximizing the number of satisfied clauses $|\{j : \alpha \models C_j\}|$.

Theorem 6.4 (Random Assignment for MAX-SAT). *For any CNF formula ϕ with m clauses, a random assignment satisfies at least $m/2$ clauses in expectation. Consequently, there exists an assignment satisfying at least $m/2$ clauses.*

Proof. Let X_j be the indicator random variable for the event that clause C_j is satisfied by a random assignment (each variable set to 0 or 1 independently with probability $1/2$). Since C_j is a disjunction of l_j literals, the probability that C_j is not satisfied is $(1/2)^{l_j}$. Thus

$$\mathbb{E}[X_j] = \Pr[C_j \text{ is satisfied}] = 1 - 2^{-l_j} \geq 1 - \frac{1}{2} = \frac{1}{2}$$

for every clause with at least one literal. By linearity of expectation,

$$\mathbb{E}\left[\sum_{j=1}^m X_j\right] = \sum_{j=1}^m \mathbb{E}[X_j] \geq \frac{m}{2}.$$

Since the expected number of satisfied clauses is at least $m/2$, there must exist an assignment satisfying at least $m/2$ clauses. $\square$

This result extends immediately to MAX-3SAT, where each clause has exactly three literals.

Corollary 6.5 (7/8-Approximation for MAX-3SAT). *For any 3-CNF formula ϕ with m clauses, a random assignment satisfies at least $7m/8$ clauses in expectation. Hence MAX-3SAT has a 7/8-approximation.*

Proof. Each clause has exactly three literals, so

$$\mathbb{E}[X_j] = 1 - 2^{-3} = 1 - \frac{1}{8} = \frac{7}{8}.$$

By linearity of expectation, $\mathbb{E}[\sum X_j] = 7m/8$. $\square$

The ratio $7/8$ is the best achievable by any random assignment analysis for MAX-3SAT. Tighter bounds require more sophisticated techniques, but this simple argument demonstrates the power of the probabilistic method for approximation algorithms.

6.3 Expanding Graphs

Expander graphs are sparse graphs with strong connectivity properties: every small set of vertices has many neighbors. They arise naturally in the probabilistic method as objects that exist with high probability.

Definition 6.6 (Expander Graph). A d -regular graph $G = (V, E)$ on n vertices is an *expander* if its second-largest eigenvalue λ_2 of the adjacency matrix satisfies

$$\lambda_2 \leq 2\sqrt{d-1}.$$

Such graphs are called *Ramanujan graphs* when this bound is achieved.

The spectral gap $\lambda_1 - \lambda_2 = d - \lambda_2$ controls the expansion quality. By the Cheeger inequality, the edge expansion $h(G)$ is related to the spectral gap by

$$\frac{d - \lambda_2}{2} \leq h(G) \leq \sqrt{2d(d - \lambda_2)}.$$

Theorem 6.7 (Random Regular Graphs are Expanders). *For every fixed degree $d \geq 3$, a random d -regular graph on n vertices has $\lambda_2 \leq 2\sqrt{d-1} + \epsilon$ with high probability for any $\epsilon > 0$, as $n \rightarrow \infty$.*

The proof uses the trace method: the k -th moment of $\text{tr}(A^k)$ counts closed walks of length k in G , and concentration of this count yields concentration of the eigenvalue distribution. For Ramanujan-quality expanders, explicit constructions based on number theory (Lubotzky–Phillips–Sarnak, Margulis) achieve the bound $\lambda_2 \leq 2\sqrt{d-1}$ deterministically.

The following lemma characterizes the spectral gap in terms of graph connectivity.

Lemma 6.8 (Cheeger Inequality). *For a d -regular graph G on n vertices with edge expansion*

$$h(G) = \min_{S \subseteq V, |S| \leq n/2} \frac{|\partial S|}{|S|},$$

we have

$$\frac{d - \lambda_2}{2} \leq h(G) \leq \sqrt{2d(d - \lambda_2)}.$$

6.4 Oblivious Routing Revisited

In Chapter 5, we analyzed random routing on the complete graph. We now consider oblivious routing on general graphs, where the routing strategy must be fixed in advance without knowledge of the traffic pattern.

Definition 6.9 (Oblivious Routing). An oblivious routing strategy on a graph G is a probability distribution over paths for each source-destination pair (s, t) , specified without knowledge of the other traffic demands. The *competitive ratio* is the worst-case ratio of the expected maximum congestion to the optimal offline maximum congestion.

Theorem 6.10 (Random Oblivious Routing). *On any graph G with n vertices and average degree $\bar{d}$, the random oblivious routing strategy—choosing a shortest path uniformly at random for each demand—achieves competitive ratio $O(\log n)$ with high probability.*

The analysis proceeds by decomposing each shortest path into edges, then applying the Chernoff bound to the congestion at each edge. For each edge e , the congestion X_e is the sum of independent indicator variables (one per path using e), and standard concentration inequalities yield the $O(\log n)$ bound on the maximum congestion.

The key insight connecting this to the probabilistic method is that the *existence* of a good routing is guaranteed by the probabilistic argument: the expected maximum congestion is $O(\log n)$, so a routing with maximum congestion $O(\log n)$ exists.

6.5 The Lovász Local Lemma

The union bound $\Pr[\bigcup A_i] \leq \sum \Pr[A_i]$ is often too crude when the bad events are not “spread out” across the probability space. The Lovász Local Lemma provides a much sharper tool when the bad events have limited dependencies.

Definition 6.11 (Dependency Graph). Let $A_1, \dots, A_m$ be events in a probability space. A *dependency graph* for these events is a graph H on vertex set $\{1, \dots, m\}$ such that for each i , the event A_i is independent of the events $\{A_j : j \notin N_H(i)\}$, where $N_H(i)$ denotes the neighbors of i in H .

Theorem 6.12 (Lovász Local Lemma—Asymmetric Form). *Let $A_1, A_2, \dots, A_m$ be events with a dependency graph H of maximum degree d . If there exist real numbers $x_1, x_2, \dots, x_m$ in $[0, 1)$ such that*

$$\Pr[A_i] \leq x_i \prod_{j \in N_H(i)} (1 - x_j) \quad \text{for all } i = 1, \dots, m,$$

then

$$\Pr\left[\bigcap_{i=1}^m \overline{A_i}\right] > \prod_{i=1}^m (1 - x_i) > 0.$$

The symmetric form is often more convenient.

Corollary 6.13 (Symmetric Local Lemma). *If each event A_i has $\Pr[A_i] \leq p$ and depends on at most d other events, and if*

$$ep(d+1) \leq 1,$$

then $\Pr[\bigcap \overline{A_i}] > 0$.

Proof. Set $x_i = 1/(d+1)$ for all i . The condition $\Pr[A_i] \leq x_i \prod_{j \in N_H(i)} (1 - x_j)^d$ is implied by $p \leq \frac{1}{e(d+1)}$ since $(1 - 1/(d+1))^d \geq 1/e$. $\square$

Example 6.14 (2-SAT). Consider a 2-SAT formula ϕ on n variables with m clauses. Each clause has two literals. Assign each

variable independently to 0 or 1 with probability $1/2$. The bad event A_j is that clause C_j is not satisfied, so $\Pr[A_j] = 1/4$. Each clause shares variables with at most $2(l_j - 1)$ other clauses, so the dependency degree is at most $d = 2m$. The condition $ep(d + 1) \leq 1$ requires $m \leq \lfloor e^{-1}/(2p) \rfloor$, which limits the density. For sparse formulas, the Local Lemma guarantees satisfiability.

Example 6.15 (Hypergraph Coloring). Consider a 3-uniform hypergraph $H = (V, E)$ with $|V| = n$ and $|E| = m$ edges. We wish to 2-color the vertices so that no edge is monochromatic. Color each vertex independently red or blue with probability $1/2$. For each edge $e \in E$, let A_e be the event that e is monochromatic. Then $\Pr[A_e] = 2 \cdot 2^{-3} = 1/4$. Each edge shares vertices with at most $3(n - 3) = 3n - 9$ other edges (since each of its 3 vertices can belong to at most $n - 3$ other edges). The Local Lemma condition $ep(d + 1) \leq 1$ becomes $e(n/4)(3n - 8) \leq 1$, which is satisfied for $n \leq O(1)$. More generally, for k -uniform hypergraphs, the condition $ep(d + 1) \leq 1$ gives bounds of the form $m \leq 2^{k-1}/(ek)$, ensuring a proper 2-coloring exists.

Example 6.16 (k -SAT). For k -SAT formulas with clause density $m/n \leq 2^{k-1}/(ek)$, the Local Lemma guarantees that a random assignment satisfies all clauses with positive probability. This gives a satisfiability threshold that matches the upper bound from the DPLL algorithm for small k .

6.5.1 Algorithmic Lovász Local Lemma

The Lovász Local Lemma as presented above is a pure existence result. It tells us that some outcome avoids all bad events but gives no method to find it. The algorithmic LLL of Moser and Tardos [5] addresses precisely this: it provides a simple, practical algorithm that almost surely finds such an outcome.

The variable framework

Moser–Tardos assumes a probability space defined by a set of independent random variables $\mathcal{X} = \{X_1, \dots, X_n\}$. Each bad event A_i depends on a subset $\text{vbl}(A_i) \subseteq \mathcal{X}$; it is determined solely by the values of those variables.

The algorithm itself is remarkably simple:

Algorithm 5 Moser–Tardos Resampling

```

1: Sample all variables independently from their distribu-
   tions.
2: loop
3:   if no bad event holds then
4:     break
5:   else
6:     Pick any bad event  $A_i$ 
7:     for all  $X \in \text{vbl}(A_i)$  do
8:       Re-sample  $X$  from its distribution
9:     end for
10:  end if
11: end loop

```

We start with a random outcome. Whenever a bad event happens, we resample all the variables it depends on. Because the new values are fresh independent draws, the process has no memory of the previous assignment of those variables.

Theorem 6.17 (Moser–Tardos, 2010). *If the events $A_1, \dots, A_m$ satisfy the LLL condition $e p(d+1) \leq 1$ (symmetric form), then the Resampling algorithm terminates with probability 1 at a state where no bad event holds. The expected number of resampling steps is finite, and is bounded by $O\left(\sum_i \frac{x_i}{1-x_i}\right)$ in the asymmetric formulation.*

Proof sketch. We describe the key combinatorial idea. Consider the sequence of events that are resampled. Associate with each resampled event A_i a *resampling tree* whose nodes

are labeled with the identity of the bad event and whose edges connect events that are resampled within a dependency step. Because each resampling chooses fresh values for the variables of A_i , and any event A_j that is not a neighbor of A_i in the dependency graph is unaffected, this tree is a Galton-Watson process. The LLL condition ensures that the branching factor of this process is less than 1, so the total size of the tree (and thus the total number of resamplings) is finite with probability 1. The formalization of this argument bounds the expected resampling count as $\sum_i \frac{x_i}{1-x_i}$ and requires a careful probabilistic estimate that we omit here. $\square$

Corollary 6.18 (LLL is now constructive). *Any existence result that can be established via the Local Lemma under the standard LLL condition yields an efficient algorithm to find the desired object whenever the LLL condition holds.*

Examples

The Algorithmic LLL applies to a far wider range of examples than the few we list here.

- **Markovian SAT:** consider a k -SAT formula F on m clauses with at most d dependencies per clause, $ep(d+1) \leq 1$. The Resampling algorithm runs in expected polynomial time.
- **Packing integer programs:** Suppose we have m objects each with a penalty probability p and each object depends on d others. Whenever all objects satisfy the LLL condition, the algorithm finds a solution.
- **Hypergraph coloring:** For a k -uniform hypergraph with edges, the algorithm will find a 2-coloring.

```
1 #include <vector>
2 #include <random>
3 #include <iostream>
4
```

```

5 // Returns a resampled integer in [0, max_val)
  uniformly.
6 int sample_unif(std::mt19937& rng, int max_val)
  {
7     std::uniform_int_distribution<int> dist(0,
        max_val-1);
8     return dist(rng);
9 }
10 // Structure of a bad event.
11 struct Event {
12     int id;
13     std::vector<int> dependencies; // indices of
        involved variables
14 };
15
16 // Check if a particular event is currently
  true.
17 bool is_event_true(int event_id, const std::
    vector<int>& assignment,
18                     const std::vector<Event>&
        events) {
19     const auto& e = events[event_id];
20     // Application-specific logic goes here.
21     // In the hypergraph case, check whether the
        three vertices
22     // share the same color.
23     return false;
24 }
25 // Core resampling function.
26 std::vector<int> find_good_assignment(
27     int n_vars, int n_events,
28     const std::vector<Event>& events,
29     std::mt19937& rng,
30     int max_val) {
31     // Initialize randomly
32     std::vector<int> assignment(n_vars);
33     for (int i = 0; i < n_vars; ++i)
34         assignment[i] = sample_unif(rng, max_val);
35     // Resampling loop

```

```
36  bool done = false;
37  while (!done) {
38      done = true;
39      for (int i = 0; i < n_events; ++i) {
40          if (is_event_true(i, assignment, events))
41              {
42                  // Resample dependent variables
43                  for (int v : events[i].dependencies) {
44                      assignment[v] = sample_unif(rng,
45                                              max_val);
46                  }
47                  done = false;
48              }
49      }
50  }
51  return assignment;
52 }
```

Beyond the Basic LLL

The resampling approach generalizes to the lopsided Local Lemma and to the so-called general LLL, but these are beyond our scope.

6.6 The Method of Conditional Probabilities

The probabilistic method proves existence but does not construct the object. The *method of conditional probabilities* transforms a probabilistic existence argument into a deterministic algorithm by replacing random choices with deterministic ones that maintain non-negative conditional expectation.

The key insight is as follows. Suppose we want to show that a random variable X satisfies $\mathbb{E}[X] \geq 0$, which implies the existence of an outcome with $X \geq 0$. We can derandomize by setting the variables $x_1, x_2, \dots, x_n$ one at a time. At step i , we compute the conditional expectation $\mathbb{E}[X \mid x_1, \dots, x_{i-1}]$ as

a function of x_i , and choose the value of x_i that maximizes this conditional expectation. The tower property of conditional expectation ensures that the final value satisfies $X \geq \mathbb{E}[X] \geq 0$.

Theorem 6.19 (Derandomization of MAX-3SAT). *For any 3-CNF formula with m clauses, there exists a deterministic polynomial-time algorithm that finds an assignment satisfying at least $7m/8$ clauses.*

Proof. We apply the method of conditional probabilities to the random variable $X = \sum_{j=1}^m X_j$, where X_j is the indicator for clause C_j being satisfied. The expected value is $\mathbb{E}[X] = 7m/8$.

Initialize all variables as unset. For $i = 1, 2, \dots, n$, we set x_i to the value (0 or 1) that maximizes the conditional expectation $\mathbb{E}[X \mid x_1, \dots, x_i]$. This conditional expectation can be computed in polynomial time: for each clause C_j , if all its literals are already determined and none is true, then $X_j = 0$; if any literal is true, then $X_j = 1$; otherwise, X_j is a Bernoulli random variable with $\mathbb{E}[X_j \mid \text{partial assignment}] = 1 - 2^{-r}$, where r is the number of unset literals in C_j .

By the tower property,

$$\mathbb{E}[X \mid x_1, \dots, x_n] = X(x_1, \dots, x_n) \geq \mathbb{E}[X \mid x_1] \geq \dots \geq \mathbb{E}[X] = \frac{7m}{8}.$$

Thus the final assignment satisfies at least $7m/8$ clauses. $\square$

The method of conditional probabilities is the “algorithmic” backbone of the probabilistic method. It transforms existential proofs into efficient algorithms, demonstrating that the power of randomization can often be eliminated without sacrificing the quality of the result.

Listing 6.1: Deterministic MAX-SAT via conditional expectations (C++).

```

1 #include <vector>
2 #include <cmath>
3 #include <algorithm>
4 using std::vector;
```

```

5
6 struct MaxSATResult {
7     vector<int> assignment;
8     int satisfied;
9 };
10
11 // Compute conditional expectation of #satisfied clauses
12 // given a partial assignment (-1 = unset).
13 double conditional_expectation(
14     const vector<vector<int>>& clauses,
15     const vector<int>& asgn) {
16     double total = 0.0;
17     for (const auto& clause : clauses) {
18         bool sat = false;
19         int unset = 0;
20         for (int lit : clause) {
21             int var = std::abs(lit) - 1;          // 0-indexed
22             if (asgn[var] == -1) { ++unset; }
23             else if ((lit > 0 && asgn[var] == 1) ||
24                     (lit < 0 && asgn[var] == 0))
25                 { sat = true; break; }
26         }
27         if (sat) total += 1.0;
28         else if (unset > 0)
29             total += 1.0 - std::pow(0.5, unset);
30     }
31     return total;
32 }
33
34 MaxSATResult deterministic_maxsat(
35     const vector<vector<int>>& clauses, int n_vars) {
36     vector<int> asgn(n_vars, -1);
37
38     for (int i = 0; i < n_vars; ++i) {
39         // Try  $x_i = 0$ 
40         asgn[i] = 0;
41         double e0 = conditional_expectation(closures, asgn);
42         // Try  $x_i = 1$ 
43         asgn[i] = 1;
44         double e1 = conditional_expectation(closures, asgn);
45         // Choose the value that maximizes  $E[\text{\#satisfied}]$ 
46         asgn[i] = (e1 >= e0) ? 1 : 0;
47     }
48
49     // Count actual satisfied clauses
50     int sat = 0;
51     for (const auto& clause : clauses) {
52         for (int lit : clause) {
53             int var = std::abs(lit) - 1;
54             if ((lit > 0 && asgn[var] == 1) ||
55                 (lit < 0 && asgn[var] == 0))
56                 { ++sat; break; }
57         }
58     }
59     return {asgn, sat};
60 }

```


Notes

The probabilistic method was introduced by Erdős [1] in his 1947 proof of a lower bound on Ramsey numbers. The elegant simplicity of the approach—“if the average is good, something good must exist”—has made it one of the most widely used tools in combinatorics.

The Lovász Local Lemma was proved by Erdős and Lovász [2] in 1975. Spencer’s 1975 survey [3] and the definitive monograph by Alon and Spencer [4] (The Probabilistic Method, Wiley) are standard references.

The method of conditional probabilities was developed by Spencer and others as a systematic derandomization technique. The algorithmic LLL of Moser and Tardos [5] provides an efficient constructive version of the Local Lemma, resolving a long-standing algorithmic question.

Expander graphs have deep connections to coding theory, pseudorandomness, and computational complexity. The existence of Ramanujan graphs was established by Lubotzky–Phillips–Sarnak and Margulis using algebraic methods; random regular graphs achieve near-optimal expansion with high probability.

Problems

Problem 6.20. Let $G = G(n, 1/2)$ be the Erdős–Rényi random graph. Using the probabilistic method, prove that G contains a triangle-free induced subgraph on at least $n/4$ vertices.

Problem 6.21. Prove that for any graph G on n vertices with average degree $\bar{d}$, there exists an independent set of size at least $\frac{n}{\bar{d}+1}$. Use the probabilistic method with a non-uniform distribution.

Problem 6.22. Let ϕ be a CNF formula where each clause has at least k literals. Show that a random assignment satisfies at least $\prod_{j=1}^m (1 - 2^{-l_j})$ clauses in expectation, where l_j is the length of clause j . Derive a lower bound when $k = 4$.

Problem 6.23. Use the Lovász Local Lemma to prove that the edges of K_n can be 3-colored so that no monochromatic triangle exists, provided $n \leq \lfloor e^{-1} \cdot 2^2/3 \rfloor$. Compare with the bound obtained from the union bound.

Problem 6.24. Consider a k -uniform hypergraph H with m edges and maximum degree Δ (the maximum number of edges containing any single vertex). Show that H admits a 2-coloring with no monochromatic edge if $m \leq 2^{k-1}\Delta/k$.

Problem 6.25. Apply the method of conditional probabilities to the Ramsey problem: deterministically find a 2-coloring of K_5 that minimizes the number of monochromatic triangles. Write pseudocode for the algorithm.

Problem 6.26. Let $A_1, \dots, A_m$ be events each with probability at most p , and suppose each event depends on at most d others. Show that the symmetric Local Lemma condition $ep(d+1) \leq 1$ implies $\Pr[\overline{A_1} \cap \dots \cap \overline{A_m}] \geq \left(1 - \frac{1}{d+1}\right)^m$.

Problem 6.27. Construct a random 3-regular graph on $n = 100$ vertices and compute its second-largest eigenvalue $|\lambda_2|$ using the power method. Compare with the Ramanujan bound $2\sqrt{2} \approx 2.828$. Repeat for 100 trials and report the distribution of $|\lambda_2|$.

Problem 6.28. Show that for a 3-CNF formula with m clauses, the method of conditional probabilities can be implemented in $O(mn)$ time, where n is the number of variables.

Problem 6.29. (Erdős–Selfridge.) Let $\mathcal{F}$ be a family of subsets of $\{1, \dots, n\}$, each of size at least k . If $\sum_{S \in \mathcal{F}} 2^{-|S|} < 1/2$, then there exists a 2-coloring of $\{1, \dots, n\}$ with no monochromatic set in $\mathcal{F}$. Prove this using the asymmetric Local Lemma, and compare the bound with the union bound.

Chapter 7

Markov Chains and Random Walks

Markov chains and random walks are among the most versatile tools in the theory of randomized algorithms. A Markov chain describes a process that moves among states of a system according to a memoryless transition rule. When the chain is ergodic—irreducible and aperiodic—it possesses a unique stationary distribution to which it converges regardless of its starting state. This convergence property underlies a powerful paradigm: to sample from a complex distribution, construct a Markov chain whose stationary distribution is the target, run the chain long enough to “mix,” and then read off a sample.

Random walks on graphs are the canonical special case. Given a graph, the walker at each step moves to a uniformly random neighbor. The resulting chain encodes deep structural information about the graph: connectivity, bottlenecks, and expansion. The interplay between random walks and electrical networks connects probability to combinatorial optimization. Applications range from testing graph connectivity and estimating cover times, to sampling almost-uniformly from the solutions of combinatorial problems via rapidly mixing chains on expander graphs.

In this chapter, we begin with a concrete motivating example—

solving 2-SAT by random walk—and then develop the general theory of Markov chains, random walks on graphs, electrical network analogies, cover times, expander-based sampling, and probability amplification.

7.1 A 2-SAT Example

The 2-satisfiability problem asks: given a Boolean formula ϕ in conjunctive normal form where each clause has exactly two literals, does there exist a satisfying assignment? While 2-SAT is solvable in deterministic polynomial time by a clever graph algorithm, the random walk approach to 2-SAT provides an elegant introduction to the use of Markov chains for combinatorial search and foreshadows the general paradigm of random-walk-based satisfiability algorithms.

7.1.1 The Walksat Algorithm for 2-SAT

Consider a 2-CNF formula ϕ with n variables $x_1, \dots, x_n$ and m clauses. The Walksat algorithm proceeds as follows: begin with an arbitrary assignment, and repeat. If the current assignment satisfies ϕ , halt. Otherwise, pick an unsatisfied clause uniformly at random, and then flip the value of one of its two variables, chosen uniformly at random.

To analyze this algorithm, we consider the number of satisfied clauses. Let S denote the current count of satisfied clauses. Each unsatisfied clause $(l_1 \vee l_2)$ has both l_1 and l_2 false. Flipping either literal makes the clause satisfied, increasing S by 1. The other variable might appear in other clauses, so flipping it could decrease S by some amount. However, the expected change in S can be bounded carefully.

Theorem 7.1 (2-SAT Random Walk). *If the formula ϕ is satisfiable, then Walksat finds a satisfying assignment in expected $O(n^2)$ steps.*

Proof. Let x^* be a fixed satisfying assignment. Define the distance $d(t) = |\{i : x_i(t) \neq x_i^*\}|$ as the number of variables

on which the current assignment disagrees with x^* . Initially $d(0) \leq n$. When an unsatisfied clause $(l_1 \vee l_2)$ is chosen, both literals are false under the current assignment. Since x^* satisfies the clause, at least one of l_1, l_2 is true under x^* , so flipping that literal decreases d by 1. With probability at least $1/2$ the algorithm flips a variable that decreases d . Therefore

$$\mathbb{E}[d(t+1) - d(t) \mid d(t) > 0] \leq -\frac{1}{2}.$$

By a standard drift argument (see Chapter 5), the expected time to reach $d = 0$ is at most $2n \cdot n = O(n^2)$. $\square$

```

1 #include <iostream>
2 #include <string>
3 #include "markov_chain.h"
4 #include "random_walk.h"
5 #include "cover_time.h"
6
7 using namespace chapter6;
8
9 void print_header() {
10     std::cout << "\n"
11               << std::string(60, '=')
12               << "\n"
13               << "  CHAPTER 6: MARKOV CHAINS AND RANDOM WALKS\n"
14               << std::string(60, '=')
15               << "\n";
16 }
17
18 void print_section(int num, const std::string& title) {
19     std::cout << "\n" << std::string(60, '=') << "\n";
20     std::cout << "Section 6." << num << ": " << title << "\n";
21     std::cout << std::string(60, '=') << "\n\n";
22 }
23
24 int main() {
25     print_header();
26
27     print_section(2, "Markov Chains");
28     demonstrate_markov_chain();
29
30     print_section(3, "Random Walks on Graphs");
31     demonstrate_random_walk();
32
33     print_section(5, "Cover Times");
34     demonstrate_cover_time();
35
36     std::cout << "\n" << std::string(60, '=') << "\n";
37     std::cout << "SUMMARY OF CHAPTER 6\n";

```

```

38     std::cout << std::string(60, '=') << "\n\n";
39
40     std::cout << "Key Concepts Demonstrated:\n\n";
41
42     std::cout << "1. MARKOV CHAINS\n";
43     std::cout << "    - Transition matrix P, stationary
44       distribution pi\n";
45     std::cout << "    - Ergodic chains converge to pi regardless
46       of start\n";
47     std::cout << "    - Detailed balance implies reversibility\n
48       \n";
49
50     std::cout << "2. RANDOM WALKS ON GRAPHS\n";
51     std::cout << "    - Stationary distribution  $\pi_v = d_v / 2|E|$ 
52       \n";
53     std::cout << "    - Mixing time bounded by  $1/\Phi^2 * \log(n)$ 
54       \n";
55     std::cout << "    - Conductance controls bottleneck severity
56       \n\n";
57
58     std::cout << "3. ELECTRICAL NETWORKS\n";
59     std::cout << "    - Effective resistance  $R_{\text{eff}}(u,v)$ \n";
60     std::cout << "    - Commute time:  $\kappa(u,v) = 2|E| * R_{\text{eff}}(u,v)$ 
61       \n\n";
62
63     std::cout << "4. COVER TIMES\n";
64     std::cout << "    -  $T_{\text{cov}} \leq 2|E| * (n-1)$  for any connected
65       graph\n";
66     std::cout << "    - Matthews:  $T_{\text{cov}} \leq H_{\text{max}} * H_{\{n-1\}}$ \n";
67     std::cout << "    - Star:  $\Theta(n \log n)$ , Cycle:  $\Theta(n^2)$ 
68       \n\n";
69
70     std::cout << "5. EXPANDER GRAPHS\n";
71     std::cout << "    -  $\Phi(G) = \Omega(1) \Rightarrow O(\log n)$  mixing
72       time\n";
73     std::cout << "    - Expander mixing lemma bounds edge
74       discrepancies\n";
75     std::cout << "    - Probability amplification via expander
76       walks\n\n";
77
78     std::cout << "Compile and run:\n";
79     std::cout << "    g++ -std=c++17 -O2 -o chapter6 main.cpp\n";
80     std::cout << "    ./chapter6\n\n";
81
82     return 0;
83 }

```

The key insight is that the random walk performs a biased random walk on the integers $\{0, 1, \dots, n\}$: from state $d > 0$ the expected drift is negative, so the walk reaches 0 quickly. This drift-condition argument generalizes beyond 2-SAT: whenever a random process has a tendency to move toward a target state, hitting times are often polynomial. We

formalize this intuition through the theory of Markov chains and conductance in subsequent sections.

7.2 Markov Chains

Definition 7.2 (Markov Chain). A **Markov chain** is a sequence of random variables $X_0, X_1, X_2, \dots$ taking values in a finite state space Ω , satisfying the *Markov property*:

$$\Pr[X_{t+1} = j \mid X_t = i, X_{t-1} = i_{t-1}, \dots, X_0 = i_0] = \Pr[X_{t+1} = j \mid X_t = i] =$$

for all states $i, j \in \Omega$ and all $t \geq 0$. The matrix $P = (P_{ij})_{i,j \in \Omega}$ is the **transition matrix**.

A transition matrix is a row-stochastic matrix: $P_{ij} \geq 0$ for all i, j and $\sum_{j \in \Omega} P_{ij} = 1$ for all i . The entry P_{ij} gives the probability of moving from state i to state j in one step. After t steps, the probability of going from i to j is the (i, j) -entry of P^t .

Definition 7.3 (Stationary Distribution). A probability distribution π on Ω is a **stationary distribution** for the Markov chain with transition matrix P if

$$\pi P = \pi,$$

that is, $\sum_{i \in \Omega} \pi_i P_{ij} = \pi_j$ for all j . This means that if the chain starts with distribution π , then at every subsequent time step the distribution remains π .

A Markov chain is **irreducible** if for every pair of states i, j , there exists $t \geq 1$ with $(P^t)_{ij} > 0$: the chain can reach any state from any other. It is **aperiodic** if $\gcd\{t \geq 1 : (P^t)_{ii} > 0\} = 1$ for every state i . A chain that is both irreducible and aperiodic is called **ergodic**.

Theorem 7.4 (Convergence to Stationary Distribution). *Let P be the transition matrix of an ergodic Markov chain on state space Ω with $|\Omega| = n$. Then:*

- (i) There exists a unique stationary distribution π with $\pi_j > 0$ for all j .
- (ii) $\pi P = \pi$ (equilibrium equation).
- (iii) For any initial distribution μ ,

$$\lim_{t \rightarrow \infty} \mu P^t = \pi.$$

That is, P^t converges to the rank-one matrix $\mathbf{1}\pi$ as $t \rightarrow \infty$.

The rate of convergence is governed by the second-largest eigenvalue of P (in absolute value), often called the *spectral gap* $\gamma = 1 - |\lambda_2|$. For reversible chains, a standard bound is

$$\|\mu P^t - \pi\|_{\text{TV}} \leq \frac{1}{2} \sqrt{\frac{1}{\pi_{\min}}} |\lambda_2|^t.$$

Definition 7.5 (Detailed Balance). The transition matrix P with stationary distribution π satisfies the **detailed balance condition** if

$$\pi_i P_{ij} = \pi_j P_{ji} \quad \text{for all } i, j \in \Omega.$$

A chain satisfying detailed balance is called **reversible**.

Detailed balance is a sufficient condition for π to be stationary: summing both sides over i gives $\pi_j = \sum_i \pi_i P_{ij}$. Reversibility means that the chain looks the same when run forward or backward in time. Not all stationary chains are reversible, but many important chains arising from random walks on undirected graphs are.

Example 7.6 (Random Walk on the Complete Graph). Consider a random walk on the complete graph K_n . The transition matrix is $P_{ij} = 1/n$ for all i, j (including $P_{ii} = 1/n$). This chain is ergodic. The uniform distribution $\pi = (1/n, \dots, 1/n)$ is stationary since $\sum_i (1/n)(1/n) = 1/n = \pi_j$. The chain satisfies detailed balance because $\pi_i P_{ij} = 1/n^2 = \pi_j P_{ji}$. The spectral gap is $\gamma = 1$, so the chain mixes in a single step.

Example 7.7 (Lazy Random Walk). The **lazy random walk** on a graph G stays at the current vertex with probability $1/2$ and moves to a uniformly random neighbor with probability $1/2$. The transition matrix is $P = \frac{1}{2}I + \frac{1}{2}D^{-1}A$, where A is the adjacency matrix and D is the diagonal degree matrix. Laziness ensures aperiodicity without changing the stationary distribution.

7.3 Random Walks on Graphs

Let $G = (V, E)$ be an undirected graph with $n = |V|$ vertices and $m = |E|$ edges. The simple random walk on G is a Markov chain on V where from vertex v , the walker moves to a uniformly random neighbor:

$$P_{uv} = \begin{cases} 1/d_u & \text{if } (u, v) \in E, \\ 0 & \text{otherwise,} \end{cases}$$

where d_u is the degree of u .

Theorem 7.8 (Stationary Distribution for Graph Walks). *The stationary distribution for the simple random walk on an undirected graph G with m edges is*

$$\pi_v = \frac{d_v}{2m}.$$

Moreover, the chain satisfies detailed balance.

Proof. For an edge $(u, v) \in E$:

$$\pi_u P_{uv} = \frac{d_u}{2m} \cdot \frac{1}{d_u} = \frac{1}{2m} = \frac{d_v}{2m} \cdot \frac{1}{d_v} = \pi_v P_{vu}.$$

Detailed balance holds, and π is stationary. □

This theorem reveals that the random walk spends a fraction of time at each vertex proportional to its degree. High-degree vertices are visited more frequently. For d -regular graphs, the stationary distribution is uniform.

7.3.1 Mixing Time and Conductance

The **total variation distance** between distributions μ and ν on Ω is

$$\|\mu - \nu\|_{\text{TV}} = \frac{1}{2} \sum_{x \in \Omega} |\mu(x) - \nu(x)| = \max_{S \subseteq \Omega} |\mu(S) - \nu(S)|.$$

The **mixing time** is

$$t_{\text{mix}}(\epsilon) = \min \{t : \|P^t(x, \cdot) - \pi\|_{\text{TV}} \leq \epsilon \text{ for all } x \in V\},$$

where $P^t(x, \cdot)$ is the distribution after t steps starting from x . A fundamental parameter controlling mixing is conductance.

Definition 7.9 (Conductance). For a subset $S \subset V$ with $0 < \pi(S) \leq 1/2$, define

$$\Phi(S) = \frac{\text{Prob}(S \rightarrow \bar{S})}{\pi(S)} = \frac{\sum_{u \in S, v \in \bar{S}} \pi_u P_{uv}}{\pi(S)}.$$

The **conductance** of G is

$$\Phi(G) = \min_{S \subset V, 0 < \pi(S) \leq 1/2} \Phi(S).$$

Informally, conductance measures the bottleneck severity of the graph. A graph with large conductance has no small bottlenecks and mixes quickly. The conductance is also known as the *Cheeger constant* of the graph (in the discrete setting).

For a d -regular graph on n vertices, the conductance is related to the Cheeger inequality:

$$\frac{\gamma}{2} \leq \Phi(G) \leq \sqrt{2\gamma},$$

where $\gamma = 1 - \lambda_2$ is the spectral gap.

7.4 Electrical Networks

There is a beautiful and deep correspondence between random walks on graphs and electrical networks. Each edge (u, v)

of the graph is viewed as a resistor of resistance $r_{uv} = 1$ ohm (for the unweighted case). When a unit current is injected at a node s and extracted at a node t , the resulting voltages and currents encode information about the random walk.

Definition 7.10 (Effective Resistance). The **effective resistance** $R_{\text{eff}}(u, v)$ between nodes u and v is the voltage difference between u and v when one unit of current is injected at u and extracted at v .

Effective resistance satisfies the axioms of a metric: it is non-negative, symmetric, $R_{\text{eff}}(u, u) = 0$, and satisfies the triangle inequality. It can be computed using Kirchhoff's laws or equivalently by solving a system of linear equations derived from the graph Laplacian.

Let $L = D - A$ be the (combinatorial) Laplacian of G , where D is the diagonal degree matrix and A is the adjacency matrix. The voltage assignment f satisfying $Lf = I_{st}$ (where I_{st} is the current vector with $+1$ at s and -1 at t) gives the effective resistance via $R_{\text{eff}}(s, t) = f(s) - f(t)$.

Theorem 7.11 (Commute Time Identity). *Let G be an undirected graph with m edges. The expected commute time between nodes u and v is*

$$\kappa(u, v) = H(u, v) + H(v, u) = 2m \cdot R_{\text{eff}}(u, v),$$

where $H(u, v)$ is the hitting time from u to v .

This identity is remarkable: a probabilistic quantity (expected commute time) equals a purely algebraic one ($2m \cdot R_{\text{eff}}$) times a constant. Several consequences follow:

Corollary 7.12. *For any $u, v \in V$,*

$$H(u, v) \leq 2m \cdot R_{\text{eff}}(u, v) \leq 2m \cdot n.$$

Corollary 7.13 (Maximal Commute Time). *The maximum commute time satisfies $\max_{u,v} \kappa(u, v) \leq 2m \cdot n$, and this bound is tight for the path graph.*

The resistance metric also provides bounds on cover times and mixing times. For instance, the commute time identity gives an alternative proof that the random walk on a connected graph is recurrent in the sense that it visits every vertex with probability 1.

7.5 Cover Times

The **cover time** $T_{\text{cov}}(G)$ of a graph G is the expected number of steps for a random walk starting from the worst-case vertex to visit every vertex at least once.

Theorem 7.14 (Cover Time Bound). *For any connected graph G on n vertices and m edges,*

$$T_{\text{cov}}(G) \leq 2m(n-1).$$

Proof. By Theorem 7.11, the commute time $\kappa(u, v) \leq 2m \cdot R_{\text{eff}}(u, v) \leq 2m$. Order the vertices $v_1, v_2, \dots, v_n$ and consider the sequence of hitting times $H(v_1, v_2), H(v_2, v_3), \dots, H(v_{n-1}, v_n)$. The walk starting at v_1 covers all vertices in time at most

$$\sum_{i=1}^{n-1} H(v_i, v_{i+1}) \leq (n-1) \cdot 2m.$$

Taking the worst-case starting vertex gives the bound. □

Theorem 7.15 (Matthews' Bound). *For any connected graph G ,*

$$T_{\text{cov}}(G) \leq H_{\text{max}} \cdot H_{n-1},$$

where $H_{\text{max}} = \max_{u,v} H(u, v)$ is the maximum hitting time and $H_{n-1} = 1 + 1/2 + 1/3 + \dots + 1/(n-1) = O(\ln n)$ is the $(n-1)$ -st harmonic number.

Matthews' bound is often tighter than the bound of Theorem 7.14. For many natural graph families, it gives the correct asymptotic cover time up to constant factors.

Example 7.16 (Cover Time of Specific Graphs). (i) **Star graph**

$K_{1,n-1}$: The cover time is $\Theta(n \ln n)$. The walk must visit all $n - 1$ leaves, and each leaf is found in expected $O(1)$ steps from the center, but the walk must return to the center each time.

(ii) **Cycle** C_n : The cover time is $\Theta(n^2)$, matching the coupon collector for a one-dimensional random walk.

(iii) **Complete graph** K_n : The cover time is $\Theta(n \ln n)$, essentially the coupon collector problem.

7.6 Graph Connectivity

Random walks provide a simple and efficient randomized algorithm for testing whether a graph is connected. Given access to the graph via an adjacency list oracle, the algorithm performs random walks and checks whether all vertices are reached.

Theorem 7.17 (Random Walk Connectivity). *Let G be a connected, non-bipartite graph on n vertices. A random walk starting from any vertex visits every vertex within $O(n^3)$ steps with high probability.*

Proof. By Theorem 7.14 with $m \leq \binom{n}{2}$, the cover time is at most $2m(n-1) \leq n^2 \cdot n = n^3$. A single walk of length $O(n^3)$ therefore covers all vertices with constant probability. By repeating $O(\log n)$ independent walks, the probability that some walk fails to cover the graph is at most n^{-c} for any constant c . $\square$

Algorithm 6 Connectivity Testing via Random Walk

```
1: Pick arbitrary start vertex  $s$ 
2: Perform random walk from  $s$  for  $O(n^3)$  steps
3: Let  $R$  = set of vertices visited
4: if  $|R| = n$  then
5:   return "connected"
6: else
7:   return "not connected"
8: end if
```

This algorithm uses $O(n^3)$ steps and $O(n)$ space, far less than the $\Omega(n^2)$ space required to store the adjacency matrix. For sparse graphs with $m = O(n)$, the running time improves to $O(n^2)$.

Bipartiteness can also be tested by random walks: a non-bipartite connected graph has an odd cycle, and a random walk will detect this by finding a vertex revisited at an odd distance from its first visit.

7.7 Expanders and Rapidly Mixing Random Walks

Expander graphs are sparse graphs with strong connectivity properties. They play a central role in computer science, from error-correcting codes to derandomization. The key property for our purposes is that random walks on expanders mix to their stationary distribution in $O(\log n)$ steps.

Definition 7.18 (Edge Expansion). The **edge expansion** of a graph $G = (V, E)$ is

$$h(G) = \min_{S \subset V, |S| \leq n/2} \frac{|E(S, \bar{S})|}{|S|},$$

where $E(S, \bar{S})$ is the set of edges crossing the cut.

For a d -regular graph, the conductance $\Phi(G)$ equals $h(G)/d$. Expander families have $\Phi(G) = \Omega(1)$, meaning the conductance is bounded away from zero independently of n .

Theorem 7.19 (Expander Mixing Lemma). *Let G be a d -regular graph on n vertices with eigenvalues $d = \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$. For any $S, T \subseteq V$:*

$$\left| E(S, T) - \frac{d \cdot |S| \cdot |T|}{n} \right| \leq \lambda_2 \sqrt{|S| \cdot |T|},$$

where $E(S, T)$ counts edges between S and T (with edges inside $S \cap T$ counted once).

The mixing lemma bounds the discrepancy between the actual number of crossing edges and the expected number if edges were placed uniformly at random. The bound is controlled by λ_2 , the second-largest eigenvalue: the smaller λ_2 (relative to d), the more “random-like” the edge distribution.

Theorem 7.20 (Mixing Time of Expanders). *Let G be an expander graph with conductance Φ and spectral gap $\gamma = 1 - |\lambda_2|/d$. For a random walk on G :*

$$t_{\text{mix}}(\epsilon) \leq \frac{1}{\gamma} \left(\ln \frac{1}{\pi_{\min}} + \ln \frac{1}{\epsilon} \right) = O\left(\frac{1}{\Phi^2} \log \frac{n}{\epsilon} \right).$$

For constant-degree expanders, $\pi_{\min} = \Theta(1/n)$ and $\Phi = \Theta(1)$, so the mixing time is $O(\log n)$. This is exponentially faster than the $O(n^3)$ mixing time that suffices for general connected graphs.

The existence of expander families is guaranteed by several constructions. Random d -regular graphs are expanders with high probability. Explicit constructions include Ramanujan graphs (which achieve $|\lambda_2| \leq 2\sqrt{d-1}$, the Alon–Boppana bound) and Lubotzky–Phillips–Sarnak graphs.

7.8 Probability Amplification by Random Walks on Expanders

One of the most important applications of expander-based random walks is *probability amplification*: converting a weakly correct randomized algorithm into one with arbitrarily small error probability, using very few random bits.

7.8.1 Almost-Uniform Sampling

Consider a random walk on a constant-degree expander G with N vertices. After $O(\log N)$ steps, the walk is within total variation distance ϵ of the stationary distribution, which for a regular expander is the uniform distribution. Therefore, the walk generates approximately uniform random samples from $\{1, \dots, N\}$ using only $O(\log N)$ random bits per sample (to choose the initial vertex and the transition at each step).

Theorem 7.21 (Expander Random Walk Sampling). *Let G be an expander on N vertices with conductance $\Phi > 0$ and $\epsilon > 0$. A random walk of length $T = O(\frac{1}{\Phi^2} \log \frac{1}{\epsilon})$ starting from any vertex produces a sample within ϵ of uniform in total variation distance. Each sample uses $O(\log N + T \log d)$ random bits, where d is the degree.*

7.8.2 Error Reduction

Suppose we have a randomized algorithm A that, given a random string $r \in \{0, 1\}^k$, produces the correct answer with probability at least $1/2 + \delta$ for some $\delta > 0$. We wish to boost this to confidence $1 - \epsilon$.

The standard approach uses $O(k/\delta^2 \log(1/\epsilon))$ random bits by taking the majority of $O(1/\delta^2 \log(1/\epsilon))$ independent runs. Using an expander, we can reduce this to $O(k + \log(1/\epsilon)/\delta^2)$ random bits as follows.

Theorem 7.22 (Expander-Based Error Reduction). *Let A be a randomized algorithm using k random bits with success probability*

$\geq 1/2 + \delta$. There exists a procedure that achieves success probability $\geq 1 - \epsilon$ using $O(k + \frac{1}{\delta^2} \log \frac{1}{\epsilon})$ random bits.

Proof. Let G be an expander on 2^k vertices with degree d and second eigenvalue $|\lambda_2| \leq d/4$ (such expanders exist for constant d). Enumerate the vertices of G with binary strings of length k . Start a random walk at a uniformly random vertex $v_0 \in \{0, 1\}^k$. After T steps, we have visited vertices $v_0, v_1, \dots, v_T$, and we run A on each, taking the majority answer.

Since G is an expander, the random walk produces T vertices that are approximately independent from the perspective of A 's success criterion. More precisely, using the expander mixing lemma, the correlation between the events “ A succeeds at v_i ” and “ A succeeds at v_j ” decays exponentially with the graph distance between v_i and v_j in the walk.

By standard results on expander walks, for any predicate B on $\{0, 1\}^k$ with $\Pr_{r \leftarrow \{0, 1\}^k}[B(r)] \geq 1/2 + \delta$, the probability that the majority of $T = O(1/\delta^2)$ consecutive vertices on the walk satisfy B is at least $1 - e^{-\Omega(\delta^2 T)}$. Setting $T = c/\delta^2 \cdot \ln(1/\epsilon)$ gives failure probability at most ϵ , using only $O(k + T \log d)$ random bits. $\square$

The key advantage over independent sampling is that the expander walk uses $O(k + T \log d)$ random bits total, whereas independent sampling requires $O(kT)$ bits. The expander walk “reuses” its random bits because each step of the walk requires only $\log d$ new bits to choose which edge to follow.

7.8.3 Applications

Expander-based probability amplification has numerous applications:

- (i) **BPP $\subseteq$ P/poly:** Expander walks reduce the random seed length, enabling the construction of polynomial-size advice strings that make the algorithm deterministic.

- (ii) **Pseudorandom generators:** Expander-based constructions yield PRGs that fool any family of small circuits, connecting to derandomization of BPP.
- (iii) **Sampling and counting:** In the context of Markov chain Monte Carlo (MCMC), rapidly mixing chains on expanders provide efficient approximate samplers for complex combinatorial distributions (Jerrum–Sinclair).

Notes

The theory of Markov chains and random walks on graphs has rich origins in both probability theory and combinatorics. The foundational survey by Aldous [10] established the hitting time, cover time, and commute time framework and identified many key open problems. The comprehensive treatment by Lovász [56] systematically developed the spectral theory of random walks on graphs, including detailed proofs of the stationary distribution theorem, the Cheeger inequality, and mixing time bounds.

The connection between random walks and electrical networks, developed by Lyons and Peres [60] (building on earlier work by Doob, Kahane, and others), is presented in their textbook *Probability on Trees and Networks*. Theorem 7.11 appears there as the “commute time identity.”

Matthews’ bound (Theorem 7.15) was proved by Matthews [65] and independently by Aldous. The tight cover time results for specific graph families were established in a series of papers by Aldous [11, 12].

The use of random walks for testing graph connectivity dates to Aleliunas *et al.* [13], who proved the $O(n^3)$ bound (Theorem 7.17). This was one of the first applications of random walks in algorithm design.

The theory of expander graphs was initiated by Pinsker [70] and Margulis [62], who proved the existence of expander families. Explicit constructions were given by Margulis [63], Gabai

and Zemor, and Lubotzky–Phillips–Sarnak [57]. The Ramanujan bound $|\lambda_2| \leq 2\sqrt{d-1}$ is achieved by Ramanujan graphs. The expander mixing lemma (Theorem 7.19) is due to Alon and Chung [15].

The use of expander walks for probability amplification was developed by Impagliazzo, Zuckerman, and Wigderson [41]. Jerrum and Sinclair [43] pioneered the use of rapidly mixing Markov chains for approximate counting and sampling, a technique with broad applications in statistical physics, combinatorics, and machine learning.

The random walk approach to 2-SAT (Section 7.1) was introduced by Papadimitriou [69] and analyzed by Schöning [76] in the context of k -SAT algorithms.

Problems

Problem 7.23. Prove that for a lazy random walk on any connected undirected graph, the chain is aperiodic. Show that the stationary distribution is the same as for the non-lazy walk: $\pi_v = d_v/2m$.

Problem 7.24. Compute the stationary distribution and the spectral gap for the simple random walk on the cycle C_n . What is the mixing time $t_{\text{mix}}(1/4)$?

Problem 7.25. Let G be a d -regular graph on n vertices with eigenvalues $d = \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$. Prove that the conductance satisfies $\Phi(G) \leq \sqrt{2(1 - \lambda_2/d)}$. This is the Cheeger inequality for regular graphs.

Problem 7.26. Verify the commute time identity (Theorem 7.11) for the path graph P_n with vertices $1, 2, \dots, n$ and edges $(i, i+1)$. Compute $H(1, n)$, $H(n, 1)$, and $R_{\text{eff}}(1, n)$ explicitly.

Problem 7.27. Prove that the cover time of the complete graph K_n is $nH_{n-1} = n \ln n + O(n)$. (Hint: model the cover time as a coupon collector problem.)

Problem 7.28. Let G be an n -vertex graph with maximum degree Δ . Prove that the cover time satisfies $T_{\text{cov}}(G) \geq n(n -$

1)/(2 Δ). Give an example where this bound is tight up to a constant factor.

Problem 7.29. Prove that for a random walk on a connected non-bipartite graph, the probability of being at vertex v after t steps converges to π_v at rate $O(|\lambda_2|^t)$, where λ_2 is the second-largest eigenvalue of the transition matrix in absolute value.

Problem 7.30. Show that the expander mixing lemma implies the following: for a d -regular expander G on n vertices with $|\lambda_2| \leq d/4$, any set S with $|S| \leq n/2$ satisfies $|E(S, \bar{S})| \geq d|S|/8$.

Problem 7.31. (Biased Random Walk.) Consider a random walk on the path $\{0, 1, \dots, N\}$ that moves right with probability p and left with probability $q = 1 - p$, with absorbing barriers at 0 and N . For $p \neq q$, compute the hitting time $H(0, N)$ exactly. Compare with the unbiased case $p = q = 1/2$.

Problem 7.32. Let G be a d -regular expander on N vertices. Design a randomized algorithm that, given an oracle for a function $f : \{0, 1\}^k \rightarrow \{0, 1\}$ (where $N = 2^k$), estimates the bias $\Pr[f = 1]$ to within additive error ϵ using $O(k/\epsilon^2)$ queries to f . (Hint: use a random walk on G to generate correlated samples and apply the expander mixing lemma to bound the variance of the estimator.)

Chapter 8

Algebraic Techniques

Many computational problems can be cast as algebraic questions: Does a given polynomial evaluate to zero? Do two matrices multiply to a claimed product? Do two strings match? Algebraic techniques exploit the structure of polynomials, matrices, and finite fields to give randomized algorithms that are dramatically faster than their deterministic counterparts. The central idea behind most of these algorithms is *fingerprinting*: replacing a complex object with a simple numeric fingerprint computed at a random point, so that equality (or identity) can be checked efficiently with high probability.

This chapter develops the theory of algebraic fingerprinting, beginning with Freivalds' celebrated algorithm for verifying matrix multiplication and building toward interactive proof systems and the PCP theorem.

8.1 Fingerprinting and Freivalds' Technique

The problem of *verifying* a computation is often easier than performing it. Given two $n \times n$ matrices A and B and a claimed product C , we wish to test whether $A \cdot B = C$. The naive approach requires $O(n^3)$ time to compute the product and compare, but randomized verification can be done in $O(n^2)$ time.

Definition 8.1 (Fingerprint). A *fingerprint* of an object X is a short summary $f(X)$ that can be computed efficiently. If $f(X) = f(Y)$, then $X = Y$ with high probability, and if $X \neq Y$, then $f(X) \neq f(Y)$ with high probability.

Algorithm 7 Freivalds' Algorithm

Require: matrices $A, B, C \in \mathbb{Z}^{n \times n}$, number of rounds t

Ensure: YES if $AB = C$, NO otherwise

```

1: for  $k = 1, 2, \dots, t$  do
2:   Choose  $\mathbf{r} \in \{0, 1\}^n$  uniformly at random
3:   Compute  $\mathbf{v} = B\mathbf{r}$   $\triangleright O(n^2)$ 
4:   Compute  $\mathbf{w} = A\mathbf{v}$   $\triangleright O(n^2)$ 
5:   Compute  $\mathbf{u} = C\mathbf{r}$   $\triangleright O(n^2)$ 
6:   if  $\mathbf{w} \neq \mathbf{u}$  then
7:     return NO
8:   end if
9: end for
10: return YES

```

The key observation is that for any $\mathbf{r} \in \{0, 1\}^n$, the matrix-vector products $A(B\mathbf{r})$ and $C\mathbf{r}$ each cost $O(n^2)$ time. If $AB = C$, then $A(B\mathbf{r}) = C\mathbf{r}$ for every $\mathbf{r}$. If $AB \neq C$, then the error matrix $D = AB - C$ is nonzero, and we need to bound the probability that $D\mathbf{r} = \mathbf{0}$.

Theorem 8.2 (Freivalds, 1979). Let A, B, C be $n \times n$ matrices over $\mathbb{Z}$ with entries bounded in absolute value by M . If $AB \neq C$, and $\mathbf{r}$ is chosen uniformly at random from $\{0, 1\}^n$, then

$$\Pr[A(B\mathbf{r}) = C\mathbf{r}] \leq \frac{1}{2}.$$

After t independent rounds, the probability of a false positive is at most 2^{-t} .

Proof. Let $D = AB - C \neq \mathbf{0}$. There exists some row i such that row i of D is nonzero. Let d_i denote this row. Then $\Pr[d_i \cdot \mathbf{r} = 0]$ where the sum is over entries of d_i and $\mathbf{r}$. Since $\mathbf{r}$ is uniform

over $\{0, 1\}^n$, at least one entry $d_{ij} \neq 0$, and the sum $d_i \cdot \mathbf{r}$ takes at least two distinct values with equal probability as r_j flips. Therefore $\Pr[d_i \cdot \mathbf{r} = 0] \leq 1/2$.

Since A is invertible on the nonzero row (or we directly argue about $D\mathbf{r}$), we conclude $\Pr[D\mathbf{r} = \mathbf{0}] \leq 1/2$. The probability that all t rounds agree is at most $(1/2)^t$. $\square$

Example 8.3. Consider verifying a 3×3 matrix product $AB = C$. Each round costs $O(n^2) = O(9)$ operations versus the $O(n^3) = O(27)$ operations for computing the product. With 20 rounds, the error probability is below 10^{-6} .

The generalization to arbitrary fields is straightforward: choose $\mathbf{r}$ from $\{0, 1\}^n$ (or from $\mathbb{F}^n$) and argue identically. The only requirement is that $D \neq 0$ implies some row is nonzero, which forces a nonzero inner product with positive probability.

Implementation

Freivalds' algorithm — `src/chapter7/freivalds.h`

```
1 bool freivalds_verify(const vector<vector<long long>>& A,
2                       const vector<vector<long long>>& B,
3                       const vector<vector<long long>>& C,
4                       int rounds, mt19937& rng);
```

8.2 Verifying Polynomial Identities

A closely related algebraic problem is *polynomial identity testing* (PIT): given two polynomials p and q , determine whether $p(x_1, \dots, x_n) \equiv q(x_1, \dots, x_n)$.

Theorem 8.4 (Schwartz–Zippel Lemma). *Let $p(x_1, \dots, x_n)$ be a nonzero polynomial of total degree d over a field $\mathbb{F}$, and let $S \subseteq \mathbb{F}$ be a finite set. If $r_1, \dots, r_n$ are chosen independently and uniformly from S , then*

$$\Pr[p(r_1, \dots, r_n) = 0] \leq \frac{d}{|S|}.$$

Proof. We proceed by induction on n . For $n = 1$, write $p(x) = a_d x^d + \cdots + a_1 x + a_0$ with $a_d \neq 0$. By the fundamental theorem of algebra, p has at most d roots, so $\Pr[p(r) = 0] \leq d/|S|$.

For the inductive step, write p as a polynomial in x_1 with coefficients that are polynomials in $x_2, \dots, x_n$:

$$p(x_1, \dots, x_n) = \sum_{j=0}^d p_j(x_2, \dots, x_n) \cdot x_1^j.$$

Since p is nonzero, at least one p_j is nonzero. Let j^* be the largest index with $p_{j^*} \neq 0$; this p_{j^*} has total degree at most $d - j^*$ in $x_2, \dots, x_n$. By induction,

$$\Pr[p_{j^*}(r_2, \dots, r_n) = 0] \leq \frac{d - j^*}{|S|} \leq \frac{d}{|S|}.$$

Conditioned on $p_{j^*}(r_2, \dots, r_n) \neq 0$, the polynomial in x_1 has degree j^* , so it has at most j^* roots. Thus

$$\Pr[p(r_1, \dots, r_n) = 0] \leq \frac{d - j^*}{|S|} + \frac{j^*}{|S|} = \frac{d}{|S|}. \quad \square$$

Corollary 8.5. *Two polynomials p and q of degree at most d in n variables over $\mathbb{F}$ can be compared by evaluating both at a random point from S^n . If $|S| > d$, the error probability is at most $d/|S|$ per round.*

Example 8.6. To verify that $(x_1 + x_2 + \cdots + x_n)^2 = \sum_i x_i^2 + 2 \sum_{i < j} x_i x_j$, we evaluate both sides at a random point $(r_1, \dots, r_n) \in S^n$. With $|S| = 1000$ and degree $d = 2$, a single evaluation gives error at most 0.002.

The Schwartz–Zippel lemma is the workhorse behind algebraic fingerprinting. It applies to any polynomial that can be evaluated efficiently, including polynomials encoding matrix products, graph properties, and combinatorial structures.

Implementation

Polynomial identity testing — src/chapter7/polynomial.h

```

1 bool polynomial_identity_test(const vector<int>& p_coeffs,
2                               const vector<int>& q_coeffs,
3                               int field_size, int rounds,
4                               mt19937& rng);
long long poly_eval(const vector<int>& coeffs, long long x,
                    long long mod);

```

8.3 Perfect Matchings in Graphs

A *perfect matching* in a graph $G = (V, E)$ is a set of edges such that every vertex is incident to exactly one edge in the set. The decision problem “Does G have a perfect matching?” is solvable in polynomial time by the Edmonds blossom algorithm ($O(n^4)$), but algebraic techniques provide a cleaner and faster randomized approach.

Definition 8.7 (Tutte Matrix). Let $G = (V, E)$ be a graph on n vertices (assume n is even). The *Tutte matrix* T is an $n \times n$ skew-symmetric matrix defined by:

$$T_{ij} = \begin{cases} x_{ij} & \text{if } (i, j) \in E \text{ and } i < j, \\ -x_{ij} & \text{if } (i, j) \in E \text{ and } i > j, \\ 0 & \text{otherwise,} \end{cases}$$

where x_{ij} are formal indeterminates.

The determinant of the Tutte matrix is the square of the *Pfaffian*, and $\det(T) \neq 0$ if and only if G has a perfect matching.

Theorem 8.8. *Let G be a graph on n vertices. There exists a randomized algorithm that determines whether G has a perfect matching in $O(n^\omega)$ time with error probability at most $1/2$, where ω is the matrix multiplication exponent.*

Proof. Construct the Tutte matrix T of G , replacing each indeterminate x_{ij} with a random value from a sufficiently large field $\mathbb{F}$. Compute $\det(T)$ over $\mathbb{F}$ in $O(n^\omega)$ time.

If G has no perfect matching, then $\det(T) = 0$ as a polynomial, so $\det(T_{\text{rand}}) = 0$ always. If G has a perfect matching, then $\det(T)$ is a nonzero polynomial (it equals the square of the Pfaffian). By the Schwartz–Zippel lemma, with random substitution the probability that the determinant evaluates to zero is small (at most $d/|\mathbb{F}|$ where d is the degree, n). Choosing $|\mathbb{F}|$ sufficiently large ensures the error is at most $1/2$. $\square$

The algebraic approach connects to Ryser’s formula for the permanent:

$$\text{perm}(A) = \sum_{S \subseteq [n]} (-1)^{n-|S|} \prod_{i=1}^n \sum_{j \in S} a_{ij},$$

which can be used to count perfect matchings in bipartite graphs. The randomized approach evaluates this expression at random points, trading exact counting for efficient approximation.

8.4 Verifying Equality of Strings

Given two strings S and T of length n over alphabet Σ , we wish to test whether $S = T$. The deterministic approach requires $\Theta(n)$ time. Randomized fingerprinting achieves the same with high probability using $O(n)$ preprocessing and $O(1)$ comparison time.

Definition 8.9 (String Fingerprint). View a string $S = s_1 s_2 \cdots s_n$ as a polynomial:

$$f_S(x) = s_1 + s_2 x + s_3 x^2 + \cdots + s_n x^{n-1}.$$

The *Karp–Rabin fingerprint* of S is

$$\phi_S(x) = f_S(x) \bmod p,$$

where p is a randomly chosen prime and x is a randomly chosen value.

Theorem 8.10 (Karp–Rabin String Fingerprinting). *Two strings S and T of length n over alphabet Σ can be compared using $O(n)$ preprocessing and $O(1)$ comparison time. If $S \neq T$, the fingerprint collision probability is at most $n/|\mathbb{F}|$ where $\mathbb{F}$ is the field of computation.*

Proof. If $S \neq T$, then $f_S \neq f_T$ as polynomials (they differ in at least one coefficient). The polynomial $f_S - f_T$ has degree at most $n - 1$. By the Schwartz–Zippel lemma, $\Pr[f_S(x) \equiv f_T(x) \pmod{p}] \leq (n - 1)/p$ when x and p are chosen appropriately. Choosing p from primes in $[n^2, 2n^2]$ ensures the error is $O(1/n)$. $\square$

The key advantage of the Karp–Rabin fingerprint is that it supports efficient *rolling*: when the string shifts by one position, the fingerprint can be updated in $O(1)$ time.

Example 8.11. For $S = \text{abcde}$, the fingerprint with base $B = 256$ and modulus $p = 10^9 + 7$ is:

$$\phi_S = (97 \cdot 256^4 + 98 \cdot 256^3 + 99 \cdot 256^2 + 100 \cdot 256 + 101) \bmod p.$$

Shifting to bcdef costs one subtraction, one multiplication, and one addition modulo p .

8.5 A Comparison of Fingerprinting Techniques

Different fingerprinting techniques offer different tradeoffs in error probability, computation time, and applicability.

Table 8.1: Comparison of algebraic fingerprinting techniques.

Technique	Error per round	Round cost	Applicability
Freivalds' (matrix verify)	$1/2$	$O(n^2)$	Matrix multiplication
Schwartz–Zippel (PIT)	$d/ S $	$O(M)$	Polynomial identity testing
Tutte matrix (matching)	$d/ \mathbb{F} $	$O(n^\omega)$	Perfect matching
Karp–Rabin (strings)	n/p	$O(1)$ after $O(n)$	String matching

Error probability. Freivalds' algorithm has a clean error bound of exactly $1/2$ per round (for $\{0, 1\}$ -valued $\mathbf{r}$). The Schwartz–Zippel bound $d/|S|$ depends on the degree and field size: for low-degree polynomials over large fields, the error can be made exponentially small with a single round. For string fingerprinting, the modulus p is chosen to make n/p negligible.

Field size. Larger fields reduce error but increase the cost of arithmetic. For Freivalds' algorithm, binary $\{0, 1\}$ vectors suffice (each round costs $O(n^2)$ with small constants). For PIT and matching, the field must be larger than the polynomial degree, but not overwhelmingly so.

Deterministic vs. randomized. For matrix multiplication verification, the best deterministic algorithm requires $\Omega(n^2)$ time (matching Freivalds'), but the constant factors are worse. For PIT, no deterministic polynomial-time algorithm is known for general multivariate polynomials given in succinct form, making randomization essential.

8.6 Pattern Matching

The *pattern matching* problem asks: given a text T of length n and a pattern P of length m , find all occurrences of P in T . The Karp–Rabin algorithm applies fingerprinting to solve this efficiently.

Algorithm 8 Karp–Rabin Pattern Matching**Require:** text $T[0 \dots n - 1]$, pattern $P[0 \dots m - 1]$ **Ensure:** all positions i where $T[i \dots i + m - 1] = P$

```

1: Compute  $\phi_P = f_P(B) \bmod p$  ▷ pattern fingerprint
2: Compute  $\phi_{T_0} = f_{T[0..m-1]}(B) \bmod p$  ▷ first window
3:  $h = B^{m-1} \bmod p$  ▷ highest power
4: for  $i = 0, 1, \dots, n - m$  do
5:   if  $\phi_{T_i} = \phi_P$  then
6:     ▷ Candidate match—verify explicitly
7:     if  $T[i \dots i + m - 1] = P$  then
8:       return  $i$ 
9:     end if
10:  end if
11:  if  $i < n - m$  then
12:     $\phi_{T_{i+1}} = (B \cdot (\phi_{T_i} - T[i] \cdot h) + T[i + m]) \bmod p$ 
13:  end if
14: end for
15: return NO MATCH

```

Theorem 8.12. *The Karp–Rabin algorithm finds all occurrences of a pattern P of length m in a text T of length n in expected $O(n + m)$ time. The expected number of spurious fingerprint matches is $O(n/p)$.*

Proof. The preprocessing to compute the pattern fingerprint and the initial text window fingerprint takes $O(m)$ time. Each subsequent window is updated in $O(1)$ time using the rolling hash, giving $O(n)$ total for all windows.

Each of the $n - m + 1$ windows has a fingerprint collision with the pattern fingerprint with probability at most m/p (by the Schwartz–Zippel lemma applied to the difference polynomial of degree $m - 1$). The expected number of spurious matches is $(n - m + 1) \cdot m/p$, which is $O(n)$ when $p = \Theta(m)$. Each spurious match costs $O(m)$ for verification, but the total expected verification cost is $O(n \cdot m/p) = O(n)$ when $p = \Theta(m^2)$.

Combining: $O(m)$ preprocessing + $O(n)$ rolling + $O(n)$

expected verification = $O(n + m)$ expected time. $\square$

Remark 8.13. The Karp–Rabin algorithm is particularly well-suited for the *multiple pattern matching* problem, where one searches for several patterns simultaneously. Each pattern gets its own fingerprint, and the text window fingerprint is compared against all pattern fingerprints (using a hash table) in $O(1)$ expected time per window.

Implementation

Rabin–Karp search — src/chapter7/rabin_karp.h

```
1 vector<int> rabin_karp_search(const string& text, const string&
   pattern);
2 long long string_fingerprint(const string& s, long long base,
   long long mod);
```

8.7 Interactive Proof Systems

The algebraic techniques above can be viewed as special cases of a more general framework: *interactive proof systems*. In an interactive proof, a computationally powerful *prover* convinces a polynomial-time *verifier* that a statement is true, using an interactive protocol.

Definition 8.14 (Interactive Proof System). An interactive proof system for a language L consists of a polynomial-time verifier V and a (possibly computationally unbounded) prover P such that:

- (i) **Completeness:** If $x \in L$, then $\Pr[V \text{ accepts after interacting with } P] \geq 2/3$.
- (ii) **Soundness:** If $x \notin L$, then for every prover P^* , $\Pr[V \text{ accepts after interacting with } P^*] \leq 1/3$.

The probability is over the verifier's random coins.

Theorem 8.15 (Shamir, 1992). $IP = PSPACE$.

This remarkable result shows that the class of languages with interactive proofs is exactly the class of languages decidable in polynomial space. We state the theorem without proof and illustrate the framework with two examples.

Graph Isomorphism

Theorem 8.16. *Graph non-isomorphism is in IP.*

Proof. The protocol proceeds as follows:

1. The verifier sends a random isomorphic copy H of G_1 or G_2 to the prover.
2. The prover responds with $i \in \{1, 2\}$ indicating which graph H is isomorphic to.
3. The verifier checks the response.

If $G_1 \not\cong G_2$, the prover can always determine the correct answer. If $G_1 \cong G_2$, no prover can distinguish the two cases, so the verifier catches cheating with probability $1/2$. $\square$

Quadratic Residuosity

A number a is a *quadratic residue* modulo n if there exists x with $x^2 \equiv a \pmod{n}$. Deciding quadratic residuosity modulo a composite n is believed to be hard, yet the prover can convince the verifier.

Theorem 8.17. *Quadratic residuosity is in IP.*

The protocol uses the fact that the Legendre symbol (or Jacobi symbol) can be computed without knowing the factorization, but determining actual residuosity requires the factors. The prover uses knowledge of the square root to convince the verifier.

Connection to Freivalds' Algorithm

Freivalds' algorithm can be viewed as a one-round interactive proof for the language $\{(A, B, C) : AB = C\}$. The verifier picks a random vector $\mathbf{r}$, sends it to the prover (or computes locally), and checks $A(B\mathbf{r}) = C\mathbf{r}$. The prover has no advantage because the verification is done locally. This is an *Arthur–Merlin* protocol (public coins).

8.8 PCP and Efficient Proof Verification

The theory of interactive proofs leads to a profound connection between proof verification and approximation hardness: the PCP theorem.

Definition 8.18 (PCP). A language L is in $\text{PCP}(r(n), q(n))$ if there exists a polynomial-time verifier that uses $O(r(n))$ random bits and reads $O(q(n))$ bits of a proof string, such that:

- (i) If $x \in L$, there exists a proof that makes V accept with probability 1.
- (ii) If $x \notin L$, no proof makes V accept with probability greater than $1/2$.

Theorem 8.19 (PCP Theorem, Arora–Safra 1992; Arora–Lund—Motwani–Sudam–Szegedy–Håstad 1998). $\text{NP} = \text{PCP}(O(\log n), O(1))$.

This states that any NP proof can be “encoded” so that a verifier needs only $O(\log n)$ random bits and needs to inspect only $O(1)$ bits of the encoded proof, yet can still verify it with high probability.

Consequences for Approximation

The PCP theorem has immediate consequences for the hardness of approximation. The most famous:

Theorem 8.20 (Hardness of MAX-3SAT). *If $P \neq NP$, there exists a constant $\epsilon > 0$ such that no polynomial-time algorithm can approximate MAX-3SAT to within a factor of $(1 - \epsilon)$.*

More precisely, the PCP theorem implies that it is NP-hard to satisfy more than a $(7/8 + \epsilon)$ fraction of clauses in a 3-SAT formula, even though a random assignment achieves $7/8$.

Overview of PCP Construction

The PCP theorem is proved by combining several techniques:

1. **Algebraic encoding:** Encode the NP witness using error-correcting codes (e.g., Hadamard or Reed–Muller codes).
2. **Low-degree testing:** Verify that the proof is close to a low-degree polynomial.
3. **Random line picking:** The verifier picks random lines in a high-dimensional hypercube and checks consistency of the proof along these lines.

The connection to algebraic fingerprinting is deep: the same Schwartz–Zippel machinery that powers polynomial identity testing is used in the low-degree testing step of the PCP proof.

Remark 8.21. The PCP theorem is one of the most important results in computational complexity theory, resolving long-standing questions about the limits of efficient approximation and connecting proof theory, coding theory, and algorithms.

Notes

- **Freivalds’ algorithm** was introduced by Rūšins Freivalds in 1979. The original paper “Fast probabilistic algorithms for verification of polynomial identities” appeared in *Journal of Algebra*. The algorithm is notable for its simplicity and practical efficiency.

- **The Schwartz–Zippel lemma** was independently discovered by Jacob Schwartz (1980, “Probabilistic algorithms for fast solutions of sparse polynomial equations”) and Richard Zippel (1979, “Probabilistic algorithms for sparse polynomials”). It remains one of the most widely used tools in randomized algorithms.
- **Karp and Rabin** introduced their fingerprinting and pattern matching algorithms in their 1987 paper “Efficient randomized pattern-matching algorithms.” The rolling hash technique has become a standard tool in string algorithms and is used in practice (e.g., in diff tools and plagiarism detection).
- **Tutte matrices** were introduced by W. T. Tutte in 1947. The algebraic approach to matchings was developed further by Lovász (1979) and Micciancio (2000). The $O(n^\omega)$ randomized algorithm for perfect matching is due to Rabin and Vazirani (1989).
- **Interactive proofs** were introduced by Goldwasser, Micali, and Rackoff (1985) and independently by Babai (1985). The result $IP = PSPACE$ was proved by Adi Shamir in 1992.
- **The PCP theorem** is attributed to Arora and Safra (1992) and Arora, Lund, Motwani, Sudan, Szegedy, and Håstad (1998). The connection to inapproximability was established in the same body of work. A self-contained proof can be found in Arora and Barak’s *Computational Complexity: A Modern Approach* (2009).

Problems

Problem 8.22 (7.1). Implement Freivalds’ algorithm and test it on the following matrices:

$$A = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad B = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}, \quad C = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}.$$

Verify that your implementation correctly returns YES for $AB = C$ and NO for $AB \neq C$ (replace C_{11} with 20). How many rounds are needed to achieve error below 10^{-9} ?

Problem 8.23 (7.2). Prove that Freivalds' algorithm can be generalized to rectangular matrices: if A is $m \times n$, B is $n \times p$, and C is $m \times p$, then the verification still runs in $O(m \cdot \max(n, p))$ time per round.

Problem 8.24 (7.3). Use the Schwartz–Zippel lemma to prove that polynomial identity testing for a univariate polynomial of degree d over $\mathbb{Z}$ can be done with error at most $1/d$ using a single evaluation point from $\{1, 2, \dots, d^2\}$.

Problem 8.25 (7.4). Design a randomized algorithm to test whether two $n \times n$ matrices commute (i.e., $AB = BA$). What is the error probability per round? Analyze the time complexity.

Problem 8.26 (7.5). Modify the Karp–Rabin algorithm to solve the *longest common substring* problem: given two strings S and T , find the longest string that is a substring of both. Hint: Use binary search on the length and Karp–Rabin fingerprinting.

Problem 8.27 (7.6). Prove that the graph non-isomorphism protocol described in Section 8.7 achieves soundness error exactly $1/2$. How can the error be reduced to 2^{-k} using k rounds of interaction?

Problem 8.28 (7.7). Consider the following protocol for checking equality of two n -bit strings S and T with a single bit of interaction: The verifier sends a random bit b to the prover, who responds with $b \oplus (S = T ? 0 : 1)$. Does this protocol have perfect completeness? What is the soundness error? Is this protocol in IP?

Problem 8.29 (7.8). A polynomial $p(x_1, \dots, x_n)$ is given in *arithmetic circuit form* (a DAG of additions and multiplications). Prove that evaluating p at a random point from S^n and checking whether the result is zero is a correct polynomial identity test. What is the running time in terms of the circuit size?

Problem 8.30 (7.9). Explain why the PCP theorem implies that it is NP-hard to distinguish between 3-SAT instances where

$(1 - \epsilon)$ fraction of clauses are satisfiable and instances where at most $7/8 + \epsilon$ fraction are satisfiable.

Problem 8.31 (7.10). (Research) The Valiant–Vazirani theorem states that if $P \neq NP$, then NP cannot be solved in randomized polynomial time even with one-sided error. Explain the relationship between this result and the fact that graph isomorphism is in IP but not known to be in NP or coNP.

Chapter 9

Data Structures

Data structures are the backbone of efficient computation. The ability to maintain a dynamically changing collection of elements while supporting fast search, insertion, and deletion is fundamental to nearly every area of computer science. In this chapter we study how randomization can be used to design data structures that are both simple and provably efficient, often matching or surpassing the performance of their deterministic counterparts while requiring far less structural bookkeeping.

The key insight is that random choices, made independently at insert time, can replace the complex rebalancing logic required by deterministic balanced trees. We study three major paradigms: *randomized balanced search trees* (treaps), *randomized balanced lists* (skip lists), and *hashing* (both universal and perfect). Each paradigm trades different resources—pointer overhead, space, or preprocessing—for expected constant or logarithmic time per operation.

9.1 The Fundamental Data-structuring Problem

A *dictionary* (also called a *dynamic set*) is a data structure that maintains a set S of elements drawn from some universe $\mathcal{U}$,

supporting the following operations:

- **Insert**(x): Add element x to S (if not already present).
- **Delete**(x): Remove element x from S (if present).
- **Search**(x): Return a pointer to the element x in S , or report that $x \notin S$.

When elements carry keys from a totally ordered universe, the dictionary problem becomes the *dynamic dictionary with ordered keys*, and we additionally support *successor* and *predecessor* queries.

Theorem 9.1 (Comparison-based lower bound). *Any dictionary data structure that supports insert, delete, and search on n elements using only comparisons between keys must perform $\Omega(\log n)$ comparisons per operation in the worst case, in any comparison tree model.*

This lower bound is tight: *balanced binary search trees* (such as AVL trees, red–black trees, or B-trees) achieve $O(\log n)$ worst-case time per operation by maintaining explicit balance information through rotations and augmentations. However, the engineering complexity of maintaining perfect balance deterministically is substantial.

9.1.1 The randomized approach

Randomized data structures achieve the same $O(\log n)$ *expected* time per operation by making random decisions that “happen to” maintain balance with high probability, without any explicit balancing mechanism.

Definition 9.2 (High probability). An event E_n holds *with high probability* (w.h.p.) if for every constant $c > 0$, there exists a constant n_0 such that $\Pr[E_n] \geq 1 - n^{-c}$ for all $n \geq n_0$.

The randomization approach has several advantages:

1. **Simplicity:** No complex rebalancing logic is needed.
2. **No worst-case input:** No adversary can consistently force $\Omega(n)$ time on every operation, since the random choices are made at runtime and are hidden from the input.

3. **Worst-case avoidance:** Randomization “smooths out” the input distribution, ensuring expected logarithmic performance regardless of the sequence of operations.

The following three sections develop randomized data structures for the dictionary problem: treaps (Section 9.2), skip lists (Section 9.3), and hash tables (Sections 9.4–9.5).

9.2 Random Treaps

A *treap* (tree + heap) is a randomized binary search tree that combines the binary search tree property on keys with the heap property on randomly assigned priorities. The treap was introduced by Aragon and Seidel [16].

Definition 9.3 (Treap). A *treap* on n elements is a binary tree T where each node v stores a key $k(v)$ and a priority $p(v)$. The tree satisfies:

1. **BST property:** For every node v , all keys in the left subtree of v are less than $k(v)$, and all keys in the right subtree are greater than $k(v)$.
2. **Heap property:** For every node v (other than the root), $p(v) < p(\text{parent}(v))$. That is, priorities form a min-heap.

The key observation is that once the keys are fixed, there is exactly one binary tree shape that satisfies both the BST property and the heap property simultaneously. Since the priorities are assigned uniformly at random from a sufficiently large range, every permutation of priorities is equally likely, and hence every valid tree shape is equally likely.

Example 9.4. Consider the keys $\{1, 2, 3, 4, 5\}$ with random priorities. If the priorities assigned are $(1 : 5, 2 : 3, 3 : 1, 4 : 4, 5 : 2)$, the treap has node 3 as root (priority 1, the minimum), with left subtree $\{1, 2\}$ and right subtree $\{4, 5\}$. Within the left subtree, node 2 (priority 2) is the root, with 1 as its left child.

9.2.1 Search

Searching in a treap is identical to searching in a standard BST: compare the search key with the current node's key, go left if smaller, right if larger, and stop when the key is found or a null pointer is reached.

Algorithm 9 Treap Search

```
1: function TREAPSEARCH( $v, k$ )
2:   if  $v = \text{null}$  then
3:     return null
4:   end if
5:   if  $k = k(v)$  then
6:     return  $v$ 
7:   else if  $k < k(v)$  then
8:     return TreapSearch( $v.\text{left}, k$ )
9:   else
10:    return TreapSearch( $v.\text{right}, k$ )
11:  end if
12: end function
```

9.2.2 Insertion

To insert a new element with key k and a freshly generated random priority p , we perform a standard BST insertion to find the correct leaf position for k , then *rotate* upward whenever the new node's priority is smaller than its parent's priority.

Algorithm 10 Treap Insertion

```

1: function TREAPINSERT( $T, k$ )
2:    $p \leftarrow$  random priority
3:    $T \leftarrow$  BST-Insert( $T, k, p$ )
4:    $v \leftarrow$  node containing  $k$ 
5:   while  $v \neq \text{root}$  and  $p(v) < p(v.\text{parent})$  do
6:     if  $v = v.\text{parent}.\text{left}$  then
7:       Right-Rotate( $v.\text{parent}$ )
8:     else
9:       Left-Rotate( $v.\text{parent}$ )
10:    end if
11:  end while
12:  return  $T$ 
13: end function

```

Each rotation takes $O(1)$ time and maintains the BST property. The total number of rotations is at most the depth of the inserted node in the BST, which is $O(n)$ in the worst case but $O(\log n)$ in expectation.

9.2.3 Deletion

Deletion in a treap uses rotations to push the target node down to a leaf, then removes it. Specifically, if we wish to delete node v :

1. While v has two children, rotate it with the child that has the smaller priority.
2. When v has at most one child, splice v out of the tree.

9.2.4 Join and Split

Treaps support two additional operations that are useful for more complex data structures:

Definition 9.5 (Join and Split). • **Join**(T_1, T_2): Given two treaps T_1 and T_2 where all keys in T_1 are less than all keys in T_2 , merge them into a single treap.

- **Split**(T, k): Given a treap T and a key k , split T into two treaps T_1 and T_2 where all keys in T_1 are $\leq k$ and all keys in T_2 are $> k$.

Join is implemented by comparing the priorities of the two roots. If the root of T_1 has the smaller priority, its right subtree is recursively joined with T_2 ; otherwise, the root of T_2 's left subtree is recursively joined with T_1 . This process follows a single root-to-leaf path, taking $O(\log n)$ expected time.

Split is implemented by descending through the tree: at each node, if k is less than the node's key, the node and its left subtree go to T_1 , and we split the right subtree; otherwise, the node and its right subtree go to T_2 , and we split the left subtree. Split takes $O(\log n)$ expected time.

9.2.5 Analysis

The efficiency of treaps hinges on the following observation: the treap is structurally identical to a BST built by inserting elements in order of increasing priority.

Theorem 9.6. *The expected height of a treap on n nodes is $O(\log n)$. Furthermore, the expected cost of each treap operation (insert, delete, search) is $O(\log n)$.*

Proof. Consider a fixed set of n keys with priorities drawn uniformly at random from $\{1, 2, \dots, N\}$ for $N \gg n$. The treap shape is the same as the BST obtained by inserting the keys in priority order. The depth of any node v in the treap is exactly the number of elements that are ancestors of v , which equals the number of elements whose priority is smaller than $p(v)$ among those in v 's subtree. This is a standard property of random BSTs.

By the well-known analysis of random BSTs (see [30]), the expected depth of any node is at most $2 \ln n + O(1)$, and the expected height is $O(\log n)$. Specifically, the probability that the height exceeds $c \log n$ is at most n^{1-c} , so the height is $O(\log n)$ with high probability for any constant $c > 1$.

Each operation traverses a single root-to-leaf path (for search, split) or performs at most as many rotations as the depth of the affected node (for insert, delete). Therefore, each operation costs $O(\log n)$ in expectation. $\square$

C++ implementation: See `src/chapter8/treap.h` for a complete implementation of a treap supporting insert, search, remove, and height measurement.

9.3 Skip Lists

A *skip list* is a randomized data structure invented by Pugh [72] that achieves $O(\log n)$ expected search time using a layered system of linked lists. Skip lists can be viewed as a probabilistic alternative to balanced trees, trading worst-case guarantees for simplicity and excellent practical performance.

Definition 9.7 (Skip list). A *skip list* for a set $S = \{s_1 < s_2 < \dots < s_n\}$ of n sorted elements is a collection of linked lists $L_0, L_1, \dots, L_h$, where:

1. L_0 contains all n elements of S in sorted order (the “base” list).
2. L_i is a random subset of L_{i-1} for each $i \geq 1$.
3. Each element in L_i (for $i \geq 1$) is independently promoted to L_{i+1} with probability $1/2$.
4. A sentinel $-\infty$ is the first element of every list, and $+\infty$ is the last.

Each node in the skip list contains:

- A *key* (the element value).
- An array of *forward pointers*, one per level, pointing to the next element at that level.
- The *level* of the node: the highest level at which it appears.

9.3.1 Search

Search in a skip list starts at the topmost level of the sentinel and proceeds rightward and downward:

Algorithm 11 Skip List Search

```
1: function SKIPSEARCH(head,  $k$ )
2:    $v \leftarrow \text{head}$ 
3:    $i \leftarrow \text{top level}$ 
4:   while  $i \geq 0$  do
5:     while  $v.\text{forward}[i].\text{key} < k$  do
6:        $v \leftarrow v.\text{forward}[i]$ 
7:     end while
8:      $i \leftarrow i - 1$ 
9:   end while
10:   $v \leftarrow v.\text{forward}[0]$ 
11:  if  $v.\text{key} = k$  then
12:    return  $v$ 
13:  else
14:    return null
15:  end if
16: end function
```

The key insight is that at level i , each step “skips” over approximately 2^i elements. The expected number of steps at level i is $O(1)$ (since each forward pointer skips an expected 2 elements at the next level down). Summing over $O(\log n)$ levels gives $O(\log n)$ expected search time.

9.3.2 Insertion and Deletion

Insertion and deletion follow the same pattern as search: we locate the correct position using the same search procedure, maintaining an update vector of the last node seen at each level. We then splice in (or remove) the new (or deleted) node at each level, adjusting forward pointers.

The level of a new node is determined by *coin flips*: flip a fair coin until it comes up tails; the number of heads plus one is the node’s level. This gives a geometric distribution with parameter $1/2$:

$$\Pr[\text{level} = i] = 2^{-i}, \quad i \geq 1.$$

9.3.3 Analysis

Theorem 9.8. *The expected search time in a skip list of n elements is $O(\log n)$, and the expected space is $O(n)$.*

Proof. Let N_i denote the expected number of elements examined at level i during a search. At level i , each step moves to the next element in L_i , and the expected distance between consecutive elements in L_i is 2 (since each element in L_{i-1} is promoted to L_i with probability $1/2$). Therefore, the expected number of steps at level i is at most $n/2^i$ (the number of elements in L_i) divided by the expected gap size.

A cleaner argument proceeds by backwards analysis. At each level, the expected number of forward pointer traversals is $O(1)$. The maximum level is $\Theta(\log n)$ with high probability, since the probability that any element reaches level $c \log n$ is at most $n \cdot 2^{-c \log n} = n^{1-c}$.

For the space: the expected number of nodes at level i is $n/2^i$. Summing over all levels:

$$\sum_{i=0}^{\infty} \frac{n}{2^i} = 2n = O(n).$$

□

Remark 9.9. Skip lists have excellent practical performance due to their cache-friendly sequential access patterns. In practice, skip lists often outperform balanced trees despite their lack of worst-case guarantees.

9.3.4 Comparison with balanced BSTs

Feature	Treap	Skip List	AVL Tree
Search (expected)	$O(\log n)$	$O(\log n)$	$O(\log n)$
Search (worst)	$O(n)$	$O(n)$	$O(\log n)$
Insert (expected)	$O(\log n)$	$O(\log n)$	$O(\log n)$
Space	$O(n)$	$O(n)$	$O(n)$
Simplicity	High	Very High	Low
Cache performance	Moderate	Good	Moderate

Table 9.1: Comparison of randomized and deterministic balanced tree structures.

C++ implementation: See `src/chapter8/skip_list.h` for a complete implementation of a skip list with configurable maximum level and geometric promotion.

9.4 Hash Tables

Hash tables provide a fundamentally different approach to the dictionary problem: rather than maintaining sorted order, they map elements directly to positions in an array using a hash function. With a good hash function, each operation takes $O(1)$ *expected* time, which is optimal.

Definition 9.10 (Hash function). A *hash function* $h : \mathcal{U} \rightarrow \{0, 1, \dots, m - 1\}$ maps elements from a universe $\mathcal{U}$ to an array of m slots. The quality of the hash function is determined by how uniformly it distributes elements across the array.

9.4.1 Universal hash families

The idea of universal hashing, introduced by Carter and Wegman [26], is to choose the hash function randomly from a family of hash functions at the beginning of the computation, before any elements are known.

Definition 9.11 (Universal hash family). A family $\mathcal{H}$ of hash functions from $\mathcal{U}$ to $\{0, 1, \dots, m-1\}$ is called *universal* if for all $x \neq y$ in $\mathcal{U}$:

$$\Pr_{h \in \mathcal{H}} [h(x) = h(y)] \leq \frac{1}{m}.$$

That is, for any two distinct elements, the probability of a collision is at most $1/m$.

Definition 9.12 (Pairwise independent hash family). A family $\mathcal{H}$ is *pairwise independent* (or *2-universal*) if for all $x \neq y$ and all $a, b \in \{0, \dots, m-1\}$:

$$\Pr_{h \in \mathcal{H}} [h(x) = a \text{ and } h(y) = b] = \frac{1}{m^2}.$$

A standard universal hash family is the *multiply-shift* family. Let p be a prime larger than $|\mathcal{U}|$. For $a \in \{1, \dots, p-1\}$ and $b \in \{0, \dots, p-1\}$, define:

$$h_{a,b}(x) = ((ax + b) \bmod p) \bmod m.$$

The family $\mathcal{H} = \{h_{a,b} : a \in \{1, \dots, p-1\}, b \in \{0, \dots, p-1\}\}$ is universal.

Theorem 9.13. If $\mathcal{H}$ is a universal hash family and n elements are hashed into an array of size m using a randomly chosen $h \in \mathcal{H}$, then the expected number of collisions is at most $n^2 / (2m)$.

Proof. The number of collisions is $\sum_{i < j} \mathbf{1}[h(x_i) = h(x_j)]$. By linearity of expectation and universality:

$$\mathbb{E}[\text{collisions}] = \sum_{i < j} \Pr[h(x_i) = h(x_j)] \leq \binom{n}{2} \cdot \frac{1}{m} < \frac{n^2}{2m}.$$

□

9.4.2 Collision resolution

Since the hash function maps a larger universe to a smaller array, collisions (two elements hashing to the same slot) are inevitable. There are two primary approaches:

Chaining. Each slot j contains a linked list (or other collection) of all elements with $h(x) = j$. A search at slot j scans this list. If the hash function is universal and $m = \Theta(n)$, the expected length of each chain is $O(1)$, giving $O(1)$ expected search time.

Open addressing. All elements are stored directly in the array. When a collision occurs, we probe a sequence of alternative slots (determined by a probe sequence) until an empty slot is found. With uniform hashing, the expected number of probes is $O(1/(1 - \alpha))$, where $\alpha = n/m$ is the *load factor*.

Algorithm 12 Hash Table with Chaining

```
1: function HASHINSERT( $T, x$ )
2:    $j \leftarrow h(x)$ 
3:   Prepend  $x$  to  $T[j]$ 
4: end function
5:
6: function HASHSEARCH( $T, x$ )
7:    $j \leftarrow h(x)$ 
8:   Search for  $x$  in  $T[j]$ 
9:   return result of search
10: end function
```

Theorem 9.14. *With universal hashing and chaining, the expected time per operation (insert, search, delete) is $O(1 + n/m)$. Choosing $m = \Theta(n)$ gives $O(1)$ expected time per operation.*

9.4.3 Linear probing

In linear probing, when a collision occurs at slot j , we examine slots $j + 1, j + 2, \dots$ (wrapping around) until an empty slot is found. Linear probing has excellent cache performance due to sequential memory access, but its analysis is more complex.

The expected number of probes for a successful search with

load factor $\alpha < 1$ is:

$$\frac{1}{2} \left(1 + \frac{1}{1 - \alpha} \right),$$

and for an unsuccessful search:

$$\frac{1}{2} \left(1 + \frac{1}{(1 - \alpha)^2} \right).$$

These bounds hold under the *simple uniform hashing* assumption: each key is equally likely to hash to any slot, independently of other keys.

C++ implementation: See `src/chapter8/hash_table.h` for a complete implementation of a universal hash table with chaining and the multiply-shift hash family.

9.5 Hashing with $O(1)$ Search Time

Universal hashing provides $O(1)$ *expected* time per operation, but not worst-case bounds. The *perfect hashing* problem asks for a hash table that achieves $O(1)$ *worst-case* search time using $O(n)$ space.

9.5.1 FKS perfect hashing

The seminal result of Fredman, Komlós, and Szemerédi [38] achieves this via a two-level hashing scheme:

Definition 9.15 (FKS perfect hashing). The FKS scheme uses two levels of hashing:

1. **Level 1:** A primary hash function $h_1 : \mathcal{U} \rightarrow \{0, 1, \dots, m - 1\}$ with $m = n$ is chosen from a universal family. This maps elements to n “buckets.”
2. **Level 2:** For each bucket i containing n_i elements, a secondary hash function $h_i : \text{bucket}_i \rightarrow \{0, 1, \dots, n_i^2 - 1\}$ is chosen from a universal family. The secondary table for bucket i has size n_i^2 .

The key insight is that the secondary hash function is chosen *after* the bucket is known, so we can search for a hash function that maps the n_i elements to distinct positions (a *perfect hash function* for that bucket).

Theorem 9.16. *With probability at least $1/2$, a randomly chosen primary hash function in the FKS scheme maps every bucket i to at most $O(n_i^2)$ total secondary table space. The total space is $O(n)$ with high probability, and search time is $O(1)$ worst-case.*

Proof. The total secondary table space is $\sum_{i=0}^{m-1} n_i^2$. By the Cauchy–Schwarz inequality:

$$\sum_{i=0}^{m-1} n_i^2 \geq \frac{(\sum n_i)^2}{m} = \frac{n^2}{m} = n.$$

The question is whether the total space is $O(n)$ in expectation.

Let $X_{i,j,k}$ be the indicator that elements x_j and x_k (with $j \neq k$) hash to the same slot in bucket i . For a fixed pair of elements that both land in bucket i , the probability that they collide in the secondary hash table (of size n_i^2) is at most $1/n_i^2$. The expected total number of collisions across all buckets is:

$$\mathbb{E}[\text{collisions}] = \sum_{i=0}^{m-1} \binom{n_i}{2} \cdot \frac{1}{n_i^2} < \sum_{i=0}^{m-1} \frac{1}{2} = \frac{m}{2} = \frac{n}{2}.$$

Since the expected number of collisions is less than $n/2$, by the probabilistic method, there exists a choice of secondary hash functions with fewer than $n/2$ collisions, which implies a total secondary table size of $O(n)$.

By trying $O(\log n)$ random primary hash functions, we can ensure the $O(n)$ bound holds with high probability. The search procedure: compute $h_1(x)$ to find the bucket, then compute $h_{h_1(x)}(x)$ to find the exact slot, both in $O(1)$ time. $\square$

9.5.2 Dynamic perfect hashing

A limitation of FKS hashing is that it requires rebuilding when the number of elements changes significantly. *Dynamic perfect*

hashing addresses this by periodically rebuilding the entire table when the load factor deviates too far from 1. Specifically:

- When the number of elements exceeds n (the table size), rebuild with a new table of size $\Theta(n^2)$ using a new primary hash function, then rehash all elements.
- When the number of elements drops below $n/4$, rebuild with a table of size $\Theta(n)$ using a new primary hash function, then rehash all elements.

The amortized cost of each insert or delete is $O(1)$, and search is always $O(1)$ worst-case.

Theorem 9.17. *Dynamic perfect hashing achieves $O(1)$ worst-case search time and $O(1)$ amortized expected time per insertion and deletion, using $O(n)$ space.*

Proof. The rebuild cost when expanding from n elements to $2n$ elements is $O(n^2)$ (the secondary table size). Amortized over n insertions, this gives $O(n)$ per element, which is not $O(1)$.

To fix this, we use a more careful analysis. When we have n elements and the table size is $m = \Theta(n^2)$, we rebuild when n exceeds m . The rebuild cost is $O(n^2)$. We can charge this cost to the $n/2$ elements that were inserted since the last rebuild, giving an amortized cost of $O(n)$ per element, which is too high.

The correct approach uses a different table size strategy. Set the table size to $m = c \cdot n$ for a suitable constant c . Then the secondary table sizes satisfy $\sum n_i^2 = O(n)$ by the FKS analysis, and the rebuild cost is $O(n)$, giving $O(1)$ amortized cost. $\square$

C++ implementation: See `src/chapter8/hash_table.h` for an implementation of FKS-style two-level perfect hashing with dynamic resizing.

Notes

1. **Treaps** were introduced by Aragon and Seidel [16]. Their analysis builds on the correspondence between treaps

- and random BSTs, which was studied by Devroye [30] and others. The join and split operations on treaps enable efficient implementations of persistent data structures and priority queues.
2. **Skip lists** were invented by Pugh [72] as a simpler alternative to balanced trees. The original paper includes extensive experimental comparisons. Variants include *scoreable skip lists*, *indexable skip lists*, and *concurrent skip lists* for parallel computing.
 3. **Universal hashing** was introduced by Carter and Wegman [26]. The multiply-shift family $h_{a,b}(x) = ((ax + b) \bmod p) \bmod m$ is one of the simplest and most practical universal families. For a prime-free alternative, the *multiply-mod-prime* family $h_a(x) = \lfloor m \cdot ((ax \bmod p) / p) \rfloor$ due to Dietzfelbinger et al. [32] achieves similar guarantees.
 4. **Perfect hashing** was achieved by Fredman, Komlós, and Szemerédi [38], who showed that $O(n)$ space and $O(1)$ worst-case search time are simultaneously achievable. Their paper is notable for its use of the probabilistic method. Dietzfelbinger et al. [31] later gave a simpler and more practical construction.
 5. **cuckoo hashing** was introduced by Pagh and Rodler [68] and achieves $O(1)$ worst-case lookup time with $O(n)$ space using two hash functions. The analysis uses properties of random graphs.
 6. **Bloom filters** [21] are a related probabilistic data structure that supports membership queries with a small false positive rate using $O(n)$ bits of space.

Problems

Problem 9.18 (8.1). Prove that the treap deletion algorithm (rotating a node down to a leaf, then removing it) maintains both the BST property and the heap property. What is the maximum number of rotations required to delete a node at depth

d ?

Problem 9.19 (8.2). Design a treap-based algorithm to find the k -th smallest element in $O(\log n)$ expected time. Hint: augment each node with the size of its subtree.

Problem 9.20 (8.3). Analyze the expected number of levels in a skip list of n elements. What is the probability that the maximum level exceeds $2 \log_2 n$?

Problem 9.21 (8.4). Prove that the family $\mathcal{H}_{a,b} = \{h_{a,b}(x) = ((ax + b) \bmod p) \bmod m : a \in \{1, \dots, p-1\}, b \in \{0, \dots, p-1\}\}$ is a universal hash family when p is prime and $m \leq p$.

Problem 9.22 (8.5). Show that with universal hashing and chaining, the expected length of the longest chain is $O(\log n / \log \log n)$ when $m = n$. Hint: use the coupon collector's argument or the analysis of maximum load in balls-and-bins.

Problem 9.23 (8.6). Analyze the expected number of probes for a successful search in a hash table with linear probing and load factor α . Derive the formula $\frac{1}{2}(1 + 1/(1 - \alpha))$.

Problem 9.24 (8.7). In the FKS perfect hashing scheme, explain why the secondary hash function for bucket i can be chosen to be perfect (no collisions) among the n_i elements. What is the expected number of trials?

Problem 9.25 (8.8). Design a randomized data structure that supports insert, delete, and *range counting*: given a and b , return the number of elements in the set with keys in $[a, b]$. Your data structure should support each operation in $O(\log n)$ expected time.

Problem 9.26 (8.9). Implement a skip list and compare its performance (in terms of actual wall-clock time) with a `std::map` (a red-black tree) on random workloads of 10^6 insertions, 10^6 searches, and 10^6 deletions. Which data structure is faster in practice, and by how much?

Problem 9.27 (8.10). **(Research)** A *Cuckoo hash table* uses two hash functions h_1 and h_2 and stores each element x in one of two positions: $h_1(x)$ or $h_2(x)$. When both positions are occupied, the existing element is “kicked out” to its alternate

position. Prove that with $m = O(n)$ slots, the expected insertion time is $O(1)$, but that there is a constant probability of insertion failure when $m < n(1 + \epsilon)$ for some $\epsilon > 0$. How can multiple hash functions resolve this?

Chapter 10

Geometric Algorithms and Linear Programming

Computational geometry is one of the richest domains for randomized algorithms. Many fundamental geometric problems—constructing convex hulls, triangulations, trapezoidal decompositions, and solving linear programs—admit elegant randomized algorithms that are both simpler and faster than their best deterministic counterparts. The unifying technique is *randomized incremental construction*: we process the input elements in a random order, maintaining a partial solution that is updated as each new element arrives. The power of this paradigm lies in its analysis via *backward analysis*, which reveals that the expected cost of each insertion step is small precisely because a random element is unlikely to disrupt the existing structure.

In this chapter we develop the theory and practice of randomized geometric algorithms. We begin with the incremental paradigm and backward analysis, then apply these tools to a sequence of geometric problems: convex hulls, half-space intersections, Delaunay triangulations, trapezoidal decompositions, binary space partitions, and the diameter problem. We then study *random sampling*, a powerful technique for building small-space data structures with provable approximation guarantees. Finally, we tackle linear programming in fixed dimension, presenting Seidel’s beautifully simple randomized

algorithm and its analysis.

10.1 Randomized Incremental Construction

The *randomized incremental construction* (RIC) paradigm is a general approach for solving geometric problems. The recipe is:

1. Choose a random permutation σ of the n input elements.
2. Initialize a solution with the first few elements (typically $O(d)$ for problems in $\mathbb{R}^d$).
3. Process the remaining elements one at a time in the order given by σ .
4. After inserting the i -th element, update the partial solution to incorporate it.

The key question is: what is the expected cost of inserting the i -th element? The answer comes from *backward analysis*.

Definition 10.1 (Backward analysis). In backward analysis, we fix the solution after processing all n elements and look backwards: we ask, “what is the expected cost of going from the solution after i elements to the solution after $i - 1$ elements?” Since the permutation is random, element i is equally likely to be any of the n elements that appear in positions $i, i + 1, \dots, n$.

Theorem 10.2 (Backward analysis principle). *Suppose that for each subset $S \subseteq \{1, \dots, n\}$ with $|S| = i$, the number of elements in S whose removal would change the solution is at most k_i . Then the expected cost of processing element i in the random permutation is at most k_i / i .*

Proof. Fix any subset S of size i . The solution after processing S depends only on S , not on the order of processing (assuming the algorithm is order-independent for subsets of the same size). Element i in the permutation is uniformly distributed over all elements of S (by symmetry of random permutations).

The element causes a non-trivial update if and only if it is one of the at most k_i “important” elements. Hence the expected cost is at most k_i/i . $\square$

Theorem 10.3 (Total expected cost). *If the expected cost of processing element i is at most k_i/i and $\sum_{i=1}^n k_i/i = O(n \log n)$, then the randomized incremental algorithm runs in expected $O(n \log n)$ time.*

This theorem is remarkably powerful: it reduces the analysis of a randomized algorithm to a purely combinatorial argument about the structure of the problem. In many geometric problems, $k_i = O(i)$ (a constant fraction of the elements are “important”), giving the desired $O(n \log n)$ bound.

Remark 10.4. The beauty of backward analysis is that it works even when the individual insertion costs vary wildly. We never need to bound the cost of inserting a specific element—only the expected cost over the random choice of permutation.

10.2 Convex Hulls in the Plane

The *convex hull* of a set P of n points in the plane is the smallest convex polygon containing all of P . It is the intersection of all half-planes containing P , and its vertices are a subset of P .

Definition 10.5 (Convex hull). The convex hull $\text{conv}(P)$ of a point set $P \subset \mathbb{R}^2$ is the set of all convex combinations of points in P :

$$\text{conv}(P) = \left\{ \sum_{i=1}^n \lambda_i p_i \mid \lambda_i \geq 0, \sum_{i=1}^n \lambda_i = 1, p_i \in P \right\}.$$

10.2.1 Gift wrapping

The simplest algorithm for computing the convex hull is *gift wrapping* (Jarvis march). Starting from the leftmost point, we repeatedly find the next hull vertex by scanning all remaining

points and selecting the one with the smallest angle relative to the current edge. This runs in $O(nh)$ time where h is the number of hull vertices, which is optimal for output-sensitive algorithms but slow when $h = \Theta(n)$.

10.2.2 The randomized incremental algorithm

A better approach uses randomized incremental construction. We process the points in random order and maintain the convex hull of the points seen so far. When a new point p_{i+1} arrives, one of two things happens:

- p_{i+1} is inside the current hull: no update needed.
- p_{i+1} is outside the current hull: it “sees” one or more edges of the current hull, and the hull is updated by removing those edges and connecting p_{i+1} to the visible portion.

Theorem 10.6 (Randomized incremental convex hull). *The randomized incremental algorithm computes the convex hull of n points in the plane in expected $O(n \log n)$ time.*

Proof. We use backward analysis. Consider the solution after i points have been processed: a convex polygon with h_i vertices. We ask: for how many of the i points could the removal of that point change the hull?

A point can only be removed from the hull if it is a vertex of the hull. There are at most h_i such points. But actually, the removal of a hull vertex v changes the hull only if v was inserted at some step $j \leq i$ and caused a non-trivial update. The key observation is that each point can cause at most $O(1)$ changes to the hull structure per update.

More precisely, when we remove the last-inserted element (which is uniformly random among the i elements), the expected number of edges added or removed is:

$$\frac{\text{total number of edge changes}}{i} \leq \frac{h_i}{i} \leq \frac{i}{i} = O(1).$$

Summing over $i = 1, \dots, n$, the total expected cost is:

$$\sum_{i=1}^n O(1) = O(n).$$

Combined with the $O(n \log n)$ sorting cost (to find initial points), the total expected time is $O(n \log n)$. $\square$

Remark 10.7. This matches the $\Omega(n \log n)$ lower bound for convex hulls under algebraic computation tree models. The randomized incremental algorithm is essentially optimal.

Example 10.8. For $n = 1000$ random points in the unit square, the convex hull typically has $O(\log n)$ vertices (since the expected number of vertices of the convex hull of n random points in a convex region is $O(\log n)$). The incremental algorithm processes most points in $O(1)$ time (since they fall inside the current hull), with occasional expensive steps when a point falls outside.

10.3 Duality

Duality is a fundamental technique in computational geometry that transforms problems about points into problems about lines (or hyperplanes) and vice versa.

Definition 10.9 (Point-line duality). The *primal-dual* transformation in the plane maps a point $p = (a, b)$ to the line $p^* : y = ax - b$, and a line $\ell : y = mx + c$ to the point $\ell^* = (m, -c)$. We extend this to vertical lines and points at infinity.

The duality transformation has the crucial property of *order preservation*: point p lies above line ℓ if and only if the dual point ℓ^* lies above the dual line p^* .

Theorem 10.10 (Duality preserves above-below). For a non-vertical line $\ell : y = mx + c$ and a point $p = (a, b)$:

$$p \text{ is above } \ell \iff b > ma + c \iff -c < ma - b \iff \ell^* \text{ is above } p^*.$$

This property has immediate consequences for convex hulls and half-space intersections:

Theorem 10.11 (Duality of convex hull and half-space intersection). *The upper hull of a point set P in the primal plane corresponds to the intersection of the upper half-planes in the dual plane. Specifically, the upper convex hull of P is dual to the boundary of $\bigcap_{p \in P} \{q : q \text{ is above } p^*\}$.*

Definition 10.12 (Upper and lower hull). Given a point set P with distinct x -coordinates, the *upper convex hull* is the chain of hull vertices from the leftmost to the rightmost point that forms the upper boundary. The *lower convex hull* is the corresponding lower boundary. Together they form the full convex hull.

Duality allows us to solve many geometric problems interchangeably: an algorithm for convex hulls gives an algorithm for half-space intersection and vice versa.

Remark 10.13. There are many other duality transformations in computational geometry, including projective duality, oriented matroid duality, and point-hyperplane duality in higher dimensions. The primal-dual transformation we study here is the simplest and most commonly used.

10.4 Half-space Intersections

The *half-space intersection problem* asks: given n half-spaces (or half-planes in 2D), compute their intersection. This is the dual problem to the convex hull problem.

Definition 10.14 (Half-space intersection). A half-plane in $\mathbb{R}^2$ is a set of the form $h_i = \{(x, y) : a_i x + b_i y \leq c_i\}$. The intersection problem asks for $\bigcap_{i=1}^n h_i$.

By duality, an algorithm for convex hulls directly gives an algorithm for half-space intersection: dualize each half-plane to a point, compute the convex hull of the dual points, and dualize back.

Theorem 10.15 (Randomized incremental half-space intersection). *The intersection of n half-planes in the plane can be computed in expected $O(n \log n)$ time using a randomized incremental algorithm.*

Proof. Apply the convex hull algorithm to the dual point set. By duality, the upper (resp. lower) hull of the dual points corresponds to the boundary of the half-plane intersection from above (resp. below). The randomized incremental convex hull algorithm (Theorem 10.6) runs in expected $O(n \log n)$ time, and the duality transformation and its inverse each take $O(n)$ time. $\square$

Remark 10.16. Half-space intersection in 2D is thus computationally equivalent to convex hull construction in 2D. In higher dimensions ($d \geq 3$), the problems are also dual, but the complexity grows: the convex hull of n points in $\mathbb{R}^d$ can have $\Theta(n^{\lceil d/2 \rceil})$ vertices in the worst case.

Example 10.17 (Linear programming feasibility). A system of linear inequalities $Ax \leq b$ in $\mathbb{R}^2$ asks whether the intersection of n half-planes is non-empty. This can be solved by computing the intersection and checking whether it is empty. For fixed dimension d , Seidel's randomized LP algorithm (Section 10.10) solves this more efficiently.

10.5 Delaunay Triangulations

The *Delaunay triangulation* of a point set P is one of the most important structures in computational geometry, with applications in mesh generation, interpolation, geographic information systems, and more.

Definition 10.18 (Delaunay triangulation). The Delaunay triangulation of a point set $P \subset \mathbb{R}^2$ is a triangulation of the convex hull of P such that no point of P lies inside the circumcircle of any triangle. Equivalently, for every edge e that is not on the convex hull boundary, the two triangles sharing e satisfy the *empty circumcircle property*.

Theorem 10.19 (Empty circle characterization). *An edge $e = (p_i, p_j)$ is in the Delaunay triangulation of P if and only if there exists a circle passing through p_i and p_j that contains no other point of P in its interior.*

Theorem 10.20 (Properties of Delaunay triangulations). *Let P be a set of n points in general position in $\mathbb{R}^2$. Then:*

1. *The Delaunay triangulation exists and is unique (in general position).*
2. *It has at most $2n - 5$ triangles and at most $3n - 6$ edges.*
3. *It maximizes the minimum angle among all triangulations of P .*
4. *It is the dual graph of the Voronoi diagram of P .*

10.5.1 The flip algorithm

A simple approach to computing the Delaunay triangulation is the *flip algorithm*: start with any triangulation, and repeatedly flip edges that violate the empty circumcircle property. An edge e shared by two triangles is flipped by replacing e with the other diagonal of the quadrilateral formed by the two triangles.

Theorem 10.21 (Flip algorithm termination). *The flip algorithm terminates in at most $O(n^2)$ flips. Each flip increases the angle sequence in the lexicographic order, and there are finitely many triangulations.*

10.5.2 Randomized incremental construction

A more efficient approach uses randomized incremental construction with flips.

Theorem 10.22 (Randomized incremental Delaunay). *The Delaunay triangulation of n points in the plane can be computed in expected $O(n \log n)$ time using a randomized incremental algorithm.*

Proof. We insert points in random order, maintaining the Delaunay triangulation of the points seen so far. When a new point p is inserted inside some triangle T :

1. Split T into three new triangles by connecting p to the three vertices of T .
2. Check each new edge for the empty circumcircle property. If violated, flip the edge.
3. After each flip, check the two new edges. Continue until all edges satisfy the property.

By backward analysis, the expected number of flips per insertion is $O(1)$. The total expected number of flips is $O(n)$. Combined with the $O(n \log n)$ preprocessing, the total expected time is $O(n \log n)$. $\square$

Remark 10.23. The Delaunay triangulation is closely related to the Voronoi diagram. The Voronoi diagram partitions $\mathbb{R}^2$ into regions, one per point, where each region consists of all points closer to its defining point than to any other. The Delaunay triangulation is obtained by connecting the Voronoi-adjacent points. The randomized incremental algorithm for Voronoi diagrams follows the same paradigm.

10.6 Trapezoidal Decompositions

A *trapezoidal decomposition* (or *trapezoidal map*) of a planar subdivision partitions the plane into trapezoids and triangles by drawing horizontal segments from each vertex to the nearest vertex or edge above and below it.

Definition 10.24 (Trapezoidal map). Given a set S of non-intersecting line segments in the plane, the trapezoidal map $\mathcal{T}(S)$ is a subdivision of the plane into trapezoids (and possibly triangles) obtained by shooting horizontal rays left and right from each vertex of S until they hit another segment or the boundary.

10.6.1 Randomized construction and point location

Trapezoidal maps are used for *point location*: given a query point, determine which face of the subdivision contains it.

Algorithm 13 Randomized Trapezoidal Map Construction

- 1: Initialize $\mathcal{T}$ with a bounding box trapezoid
 - 2: Shuffle segments randomly: $\sigma \leftarrow$ random permutation of S
 - 3: **for** $i = 1$ to n **do**
 - 4: Find the trapezoid in $\mathcal{T}$ containing the left endpoint of s_i
 - 5: Split the trapezoid(s) intersected by s_i
 - 6: Update the search structure (history DAG)
 - 7: **end for**
-

The construction builds a *history DAG*: a directed acyclic graph where each internal node represents a search decision (“is the query point above or below segment s_i ?” or “is the query point to the left or right of the left endpoint of s_i ?”) and each leaf represents a trapezoid. A point location query is answered by traversing the DAG from root to leaf.

Theorem 10.25 (Trapezoidal map complexity). *For n non-intersecting line segments, the trapezoidal map has $O(n)$ trapezoids. The randomized incremental construction runs in expected $O(n \log n)$ time and space.*

Proof. The number of trapezoids is $O(n)$ by Euler’s formula for planar subdivisions. The construction time follows from backward analysis: each insertion of segment s_i affects $O(1)$ expected trapezoids. The total expected cost is $\sum_{i=1}^n O(1) = O(n)$, plus $O(n \log n)$ for sorting. $\square$

Theorem 10.26 (Point location query). *After constructing the history DAG in expected $O(n \log n)$ time using $O(n)$ space, point location queries can be answered in $O(\log n)$ expected time.*

Proof. A query point q is located by starting at the root of the DAG and making a series of comparison decisions, each moving down one level. The expected depth of the DAG is $O(\log n)$, since each decision reduces the expected remaining uncertainty by a constant factor. Each comparison takes $O(1)$ time. $\square$

Remark 10.27. The trapezoidal map construction is a fundamental building block for many other geometric algorithms, including polygon triangulation, visibility computations, and point-in-polygon queries. The randomized approach avoids the complex case analysis required by deterministic sweep-line algorithms.

10.7 Binary Space Partitions

A *binary space partition* (BSP) tree recursively decomposes space into convex regions by splitting with hyperplanes. BSP trees are widely used in computer graphics for hidden surface removal and rendering.

Definition 10.28 (BSP tree). A BSP tree for a set of objects in $\mathbb{R}^d$ is a binary tree where each internal node stores a splitting hyperplane, and each leaf stores a region of space. The splitting hyperplane partitions the objects at that node into left and right children. Objects that cross the splitting plane are *fragmented* into sub-objects on each side.

Definition 10.29 (BSP size). The *size* of a BSP tree is the total number of fragments of all input objects across all leaves. A BSP is *good* if its size is $O(n)$ (linear in the number of input objects).

10.7.1 Randomized BSP construction

For a set of n line segments in the plane, the randomized BSP algorithm picks a random segment at each recursive step as the splitter.

Theorem 10.30 (Expected BSP size). *The randomized BSP algorithm for n non-intersecting line segments in the plane produces a BSP of expected size $O(n \log n)$ for arbitrary segments. For many natural input classes (e.g., disjoint segments, segments with bounded density), the expected size is $O(n)$.*

Algorithm 14 Randomized BSP for Line Segments

```

1: function BUILD BSP(segments  $S$ )
2:   if  $|S| \leq 1$  then
3:     return leaf node containing  $S$ 
4:   end if
5:   Pick  $s \in S$  uniformly at random
6:    $S_{\text{left}} \leftarrow \{t \in S \setminus \{s\} : t \text{ is in left half-space of } s\}$ 
7:    $S_{\text{right}} \leftarrow \{t \in S \setminus \{s\} : t \text{ is in right half-space of } s\}$ 
8:    $S_{\text{split}} \leftarrow \{t \in S \setminus \{s\} : t \text{ crosses the line of } s\}$ 
9:   Split each  $t \in S_{\text{split}}$  into  $t_{\text{left}}$  and  $t_{\text{right}}$ 
10:  return node with splitting segment  $s$ , children
      BuildBSP( $S_{\text{left}} \cup S_{\text{split, left}}$ ) and BuildBSP( $S_{\text{right}} \cup S_{\text{split, right}}$ )
11: end function

```

Proof. A segment t is split by a segment s only if s crosses the line containing t . The expected number of splits is analyzed by backward analysis: when segment s is chosen as the splitter, the probability that it splits a specific segment t is at most $c/|S|$ for some constant c depending on the geometric configuration. The total expected splits across the recursion is bounded by $O(n \log n)$ in general, and $O(n)$ for bounded-density inputs. $\square$

Remark 10.31. The problem of constructing BSPs of provably linear size for arbitrary line segments in the plane remains open. Paterson and Yao (1990) showed that $\Omega(n \log n / \log \log n)$ fragments are necessary in the worst case.

10.8 The Diameter of a Point Set

The *diameter* of a point set $P \subset \mathbb{R}^2$ is the maximum pairwise distance between any two points.

Definition 10.32 (Diameter). The diameter of P is $\text{diam}(P) = \max_{p, q \in P} \|p - q\|$.

The naive approach checks all $\binom{n}{2}$ pairs, yielding $O(n^2)$ time. We can do much better.

Theorem 10.33 (Diameter via convex hull). *The diameter of P equals the diameter of $\text{conv}(P)$, and can be found by examining only pairs of convex hull vertices.*

Proof. If p and q are not both hull vertices, then one of them lies inside the convex hull of P , and the distance can be increased by moving to a hull vertex. $\square$

Theorem 10.34 (Randomized diameter). *The diameter of n points in $\mathbb{R}^2$ can be computed in expected $O(n)$ time after computing the convex hull, using a prune-and-search approach based on random sampling.*

Proof. We use the following *prune-and-search* paradigm:

1. Compute a random sample $R \subseteq P$ of size $r = \Theta(\sqrt{n})$.
2. Compute the diameter of R by brute force: $O(r^2)$ time.
3. Use the diameter of R to “prune” points of P that cannot be diameter endpoints.
4. Recurse on the remaining candidate points.

The pruning step uses the following observation: if the diameter of R is δ , then any point p with $\|p - q\| < \delta/2$ for all $q \in R$ cannot be a diameter endpoint of P (since the diameter of P is at least δ , and p is too close to all of R).

By choosing $r = \Theta(\sqrt{n})$, each level of the recursion removes a constant fraction of points, giving a total expected time of $O(n)$ after the convex hull is computed. $\square$

Remark 10.35. The diameter problem illustrates the power of random sampling in computational geometry: a small random sample captures the global structure of the point set, allowing us to prune large fractions of the input with high confidence.

10.9 Random Sampling

Random sampling is a fundamental technique in computational geometry, with applications ranging from range searching to conflict management in incremental algorithms.

Definition 10.36 (ε -net). Let $(X, \mathcal{R})$ be a range space (where X is a set and $\mathcal{R}$ is a family of subsets of X) with *doubling dimension* or *Vapnik–Chervonenkis (VC) dimension* d . A subset $S \subseteq X$ is an ε -net for $\mathcal{R}$ if every range $R \in \mathcal{R}$ with $|R| \geq \varepsilon|X|$ contains at least one point of S .

Definition 10.37 (ε -approximation). A subset $S \subseteq X$ is an ε -approximation for $\mathcal{R}$ if for every range $R \in \mathcal{R}$:

$$\left| \frac{|R \cap S|}{|S|} - \frac{|R|}{|X|} \right| \leq \varepsilon.$$

Theorem 10.38 (Random sampling: ε -net). Let $(X, \mathcal{R})$ be a range space with VC dimension d . A random sample $S \subseteq X$ of size

$$|S| = O\left(\frac{1}{\varepsilon} \left(d \log \frac{1}{\varepsilon} + \log \frac{1}{\delta}\right)\right)$$

is an ε -net for $\mathcal{R}$ with probability at least $1 - \delta$.

Proof. We use the probabilistic method. Fix a range R with $|R|/|X| \geq \varepsilon$. The probability that S misses R entirely is:

$$\Pr[S \cap R = \emptyset] = \prod_{i=0}^{|S|-1} \left(1 - \frac{|R|}{|X| - i}\right) \leq (1 - \varepsilon)^{|S|} \leq e^{-\varepsilon|S|}.$$

By the VC dimension bound, the number of distinct ranges is at most $O(|X|^d)$ (using Sauer’s lemma). By a union bound over all ranges of size $\geq \varepsilon|X|$, the probability that any such range is missed is at most:

$$O(|X|^d) \cdot e^{-\varepsilon|S|}.$$

Setting this to $\leq \delta$ and solving for $|S|$ gives $|S| = O\left(\frac{1}{\varepsilon}(d \log |X| + \log \frac{1}{\delta})\right)$. For uniform distributions, this simplifies to $O\left(\frac{1}{\varepsilon} \log \frac{1}{\delta}\right)$. $\square$

Theorem 10.39 (Random sampling: ε -approximation). Let $(X, \mathcal{R})$ be a range space with VC dimension d . A random sample $S \subseteq X$ of size

$$|S| = O\left(\frac{d}{\varepsilon^2} \log \frac{1}{\delta}\right)$$

is an ε -approximation for $\mathcal{R}$ with probability at least $1 - \delta$.

Remark 10.40. The VC dimension of half-planes in $\mathbb{R}^2$ is 3. Therefore, a random sample of $O(\frac{1}{\varepsilon} \log \frac{1}{\delta})$ points is an ε -net for half-plane ranges with high probability.

Example 10.41 (Range searching with sampling). To build a small-space range counting data structure, we can use random sampling as a “coarse” approximation. For a query range R , the sample estimate $|R \cap S|/|S| \cdot n$ approximates $|R \cap P|$ within additive error εn with high probability, using only $O(1/\varepsilon^2)$ space.

10.10 Linear Programming

Linear programming (LP) in fixed dimension is one of the most important problems in optimization. We study a beautiful randomized algorithm due to Seidel that solves LP in d dimensions with expected running time $O(d! \cdot n)$.

Definition 10.42 (Linear program). A linear program in standard form is:

$$\begin{array}{ll} \text{minimize} & c^T x \\ \text{subject to} & a_i^T x \leq b_i, \quad i = 1, \dots, n \end{array}$$

where $x \in \mathbb{R}^d$, $c \in \mathbb{R}^d$, $a_i \in \mathbb{R}^d$, and $b_i \in \mathbb{R}$.

10.10.1 Seidel’s randomized LP algorithm

Seidel’s algorithm is a masterpiece of simplicity and elegance. The idea is to process constraints in a random order, maintaining the optimal solution. When a new constraint is added, two cases arise:

1. The current optimum satisfies the new constraint: do nothing.
2. The current optimum violates the new constraint: the new optimum lies on the boundary $a_i^T x = b_i$, reducing the problem to a $(d - 1)$ -dimensional LP.

Algorithm 15 Seidel's Randomized LP Algorithm

```

1: function SOLVELP( $A, b, c, d$ )
2:   if  $n = 0$  or  $d = 0$  then
3:     return origin (or solve trivially)
4:   end if
5:   Shuffle constraints randomly
6:    $x \leftarrow \text{SolveLP}(A[1..n-1], b[1..n-1], c, d)$  ▷ solve
   without last constraint
7:   if  $a_n^T x \leq b_n$  then
8:     return  $x$  ▷ last constraint satisfied
9:   else
10:    return SolveLP on hyperplane  $a_n^T x = b_n$  ▷ reduce
    dimension
11:   end if
12: end function

```

The reduction to $(d - 1)$ dimensions is the key insight. On the hyperplane $a_n^T x = b_n$, we substitute one variable (the one with the largest coefficient in a_n) to eliminate it, obtaining an LP in $d - 1$ dimensions with $n - 1$ constraints.

Theorem 10.43 (Seidel's randomized LP). *Seidel's randomized algorithm solves a linear program with n constraints in d variables in expected time $O(d! \cdot n)$.*

Proof. The algorithm is recursive. Let $T(n, d)$ be the expected running time. At each step, we solve a subproblem with $n - 1$ constraints in dimension d (cost $T(n - 1, d)$), check one constraint (cost $O(d)$), and with probability at most d/n (since at most d constraints can be “tight” at the optimum), we recurse on a $(d - 1)$ -dimensional LP with $n - 1$ constraints (cost $T(n - 1, d - 1)$).

This gives the recurrence:

$$T(n, d) \leq T(n - 1, d) + O(d) + \frac{d}{n} \cdot T(n - 1, d - 1).$$

By induction, $T(n, d) \leq c_d \cdot n$ where c_d satisfies $c_d = c_d + d \cdot c_{d-1} + O(d)$, giving $c_d = O(d \cdot c_{d-1}) = O(d!)$. □

Corollary 10.44. *For $d = 2$ (two-dimensional LP), the expected running time is $O(n)$. For $d = 3$, it is $O(n)$ with a larger constant.*

10.10.2 Clarkson's algorithm

Clarkson's algorithm (1989) provides an alternative randomized approach with a different trade-off: it maintains a *basis* of at most d constraints and uses random sampling to detect and correct violations.

Theorem 10.45 (Clarkson's randomized LP). *Clarkson's algorithm solves LP in d dimensions with n constraints in expected $O(d^2 \cdot n)$ time, using $O(d \cdot n)$ space.*

Clarkson's approach has the advantage of being more naturally parallelizable and extensible to other optimization problems.

Remark 10.46. The contrast between Seidel's $O(d! \cdot n)$ and Clarkson's $O(d^2 \cdot n)$ for fixed d illustrates a common trade-off in randomized algorithms: simpler recursive algorithms (Seidel) may have worse constants but cleaner analysis, while more sophisticated approaches (Clarkson) achieve better dependence on d at the cost of additional structural complexity.

Remark 10.47. For large d , neither Seidel's nor Clarkson's algorithm is competitive with the simplex method or interior-point methods used in practice. The randomized algorithms are most valuable when d is a small constant (e.g., $d \leq 10$), where the $O(n)$ dependence on n dominates.

Notes

The study of randomized algorithms in computational geometry was pioneered by Seidel (1991), who introduced the randomized incremental framework for linear programming and many other geometric problems. Clarkson and Shor (1989)

developed the technique of random sampling and its applications to range searching and conflict management. Mulmuley (1994) provided a comprehensive treatment of randomized algorithms in computational geometry, including detailed analyses of incremental construction and backward analysis.

The randomized incremental approach to convex hulls was analyzed by Seidel (1991) and independently by Mulmuley, Mulmuley, and Vazirani (1992). The Delaunay triangulation algorithm of Section 10.5 follows the flip-based approach analyzed by Guibas, Knuth, and Sharir (1992).

Trapezoidal maps and randomized point location were developed by Mulmuley (1990) and later refined by Mulmuley and Vazirani (1992). The analysis via history DAGs and backward analysis is due to these authors.

Binary space partitions for line segments were studied by Paterson and Yao (1990), who showed that randomized BSPs have expected $O(n \log n)$ size. The question of whether linear-size BSPs exist for all segment sets remains open.

Random sampling and ε -nets were introduced by Haussler and Welzl (1987) and further developed by Matoušek (1991). The VC dimension theory underlying Theorems 10.38 and 10.39 is due to Vapnik and Chervonenkis (1971).

The diameter problem in the plane was solved optimally in $O(n \log n)$ time by Shamos (1978) using the convex hull, and the prune-and-search approach of Theorem 10.34 is attributed to various authors including Dobkin and Snyder (1979).

- R. Seidel, "Linear Programming and Convex Hulls Made Easy," *Proc. 2nd ACM Symposium on Computational Geometry*, 1991.
- K. L. Clarkson and R. Shor, "Applications of Random Sampling in Computational Geometry, II," *Discrete & Computational Geometry*, 4:387–421, 1989.
- K. Mulmuley, *Computational Geometry: An Introduction Through Randomized Algorithms*, Prentice Hall, 1994.
- D. Haussler and E. Welzl, " ε -Nets and Simplex Range Queries," *Discrete & Computational Geometry*, 2:127–151, 1987.

- J. Matoušek, “Efficient Polyhedron Intersection Queries,” *Proc. 7th ACM Symposium on Computational Geometry*, 1991.
- M. S. Paterson and F. F. Yao, “Binary Space Partitions for Rectangles,” *SIAM J. Comput.*, 19(2):296–309, 1990.
- L. Guibas, D. E. Knuth, and M. Sharir, “Randomized Incremental Construction of Delaunay and Voronoi Diagrams,” *Algorithmica*, 7:381–413, 1992.
- D. P. Dobkin and J. I. Munro, “Finding the Diameter of a Set of Points,” *Proc. 11th ACM STOC*, 1979.

Problems

Problem 10.48 (9.1). Implement the randomized incremental convex hull algorithm for points in the plane. Test it on $n = 10^4$ random points in the unit square. Verify that the number of hull vertices matches the expected $O(\log n)$ for uniform random points.

Problem 10.49 (9.2). Prove that the upper hull of a set P of points in the plane corresponds under point-line duality to the boundary of the intersection of the upper half-planes $\{y \leq a_i x - b_i\}$ for each $(a_i, b_i) = p^*$.

Problem 10.50 (9.3). Show that the Delaunay triangulation of n points in the plane has at most $2n - 5$ triangles when the points are in general position (no four cocircular). Hint: use Euler’s formula for planar graphs.

Problem 10.51 (9.4). Analyze the flip algorithm for Delaunay triangulations. Show that each flip increases the minimum angle in the triangulation, and that the algorithm terminates in $O(n^2)$ flips.

Problem 10.52 (9.5). Prove that the expected number of trapezoids in a randomized trapezoidal map of n non-intersecting line segments is $O(n)$. Hint: use the linearity of expectation over pairs of segments.

Problem 10.53 (9.6). Derive the recurrence for Seidel’s LP algorithm and prove that $T(n, d) = O(d! \cdot n)$. What is the constant in the O -notation?

Problem 10.54 (9.7). For a range space $(X, \mathcal{R})$ where X is a set of n points in $\mathbb{R}^2$ and $\mathcal{R}$ is the set of all half-plane ranges, what is the VC dimension? Use Theorem 10.38 to determine the sample size needed for a 0.1-net with probability 0.99.

Problem 10.55 (9.8). Implement Seidel's randomized LP algorithm for $d = 2$. Test it on random linear programs with 1000 constraints and verify that it runs in approximately $O(n)$ time.

Problem 10.56 (9.9). Show that the diameter of a point set P in $\mathbb{R}^2$ can be computed in $O(n)$ time after computing the convex hull, using the rotating calipers technique.

Problem 10.57 (9.10). **(Research)** Design a randomized incremental algorithm for computing the Voronoi diagram of n points in the plane. Analyze the expected running time using backward analysis and show that it is $O(n \log n)$.

Chapter 11

Graph Algorithms

In this chapter we study several fundamental optimization problems on graphs: all-pairs shortest paths, minimum cuts, and minimum spanning trees. In each case, deterministic polynomial-time algorithms are known, but the use of randomization yields significantly faster solutions. The randomized algorithms we present exploit random sampling and random contraction to achieve running times that are difficult or impossible to obtain deterministically.

We begin with the all-pairs shortest paths problem, showing that it can be reduced to matrix multiplication via a randomized reduction. We then study the minimum cut problem, presenting Karger's elegant contraction algorithm and its dramatic improvement due to Karger and Stein. Finally, we present the Karger–Klein–Tarjan randomized linear-time algorithm for minimum spanning trees, which combines random sampling with Borůvka's algorithm and a linear-time verification procedure.

11.1 All-pairs Shortest Paths

The *all-pairs shortest paths* (APSP) problem asks: given a weighted graph $G = (V, E)$ with n vertices, compute the shortest-path distance $d(u, v)$ for every pair of vertices $u, v \in V$.

Definition 11.1 (All-pairs shortest paths). Given a graph $G = (V, E)$ with edge weights $w : E \rightarrow \mathbb{R}$, the APSP problem requires computing the $n \times n$ distance matrix D where $D[u, v]$ is the length of the shortest path from u to v . If v is unreachable from u , then $D[u, v] = \infty$.

The classical Floyd–Warshall algorithm solves APSP in $O(n^3)$ time. For sparse graphs, running Dijkstra from each vertex gives $O(nm + n^2 \log n)$ time. Can we do better?

11.1.1 Min-plus matrix multiplication

The key observation is the deep connection between shortest paths and a variant of matrix multiplication.

Definition 11.2 (Min-plus product). The *min-plus product* (or *distance product*) of two $n \times n$ matrices A and B is the $n \times n$ matrix C defined by:

$$C[i, j] = \min_{1 \leq k \leq n} (A[i, k] + B[k, j]).$$

Compare this with standard matrix multiplication, where we replace $+$ with $\times$ and $\min$ with $\sum$. The min-plus product captures the essence of shortest-path computation: the shortest path from u to v using at most 2^k edges can be expressed as a min-plus product of the distance matrices for paths of length at most 2^{k-1} .

Theorem 11.3 (APSP via repeated squaring). Let W be the $n \times n$ weight matrix of a graph G (with $W[i, j] = w(i, j)$ if $(i, j) \in E$ and $W[i, j] = \infty$ otherwise). Define the min-plus power $W^{(k)}$ recursively by:

$$W^{(1)} = W, \quad W^{(2k)} = W^{(k)} \star W^{(k)},$$

where $\star$ denotes the min-plus product. Then $W^{(n-1)}$ is the all-pairs shortest path distance matrix.

Proof. By induction on k . The matrix $W^{(k)}[i, j]$ gives the shortest path from i to j using at most k edges. For $k = 1$, this is just the direct edge weights. For the inductive step, a path of at most $2k$ edges can be split into two halves of at most k edges each, so $W^{(2k)}[i, j] = \min_k (W^{(k)}[i, k] + W^{(k)}[k, j]) = (W^{(k)} \star W^{(k)})[i, j]$. Since the shortest path in an n -vertex graph uses at most $n - 1$ edges, $W^{(n-1)}$ gives the correct distances. $\square$

This gives an $O(n^3 \log n)$ algorithm by computing $\lceil \log_2(n - 1) \rceil$ min-plus products, each taking $O(n^3)$ time by brute force. But can we compute the min-plus product faster?

11.1.2 The randomized reduction

The remarkable insight is that min-plus products can be *randomly reduced* to standard matrix multiplication. Since fast matrix multiplication algorithms (such as Strassen's $O(n^{2.807})$ algorithm) are available, this yields faster APSP algorithms.

Theorem 11.4 (Randomized reduction: min-plus to standard product). *Given a subroutine that computes the standard matrix product of two $n \times n$ matrices in $T(n)$ time, there is a randomized algorithm that computes the min-plus product of two $n \times n$ matrices in $O(T(n) \log n)$ expected time.*

Proof. The reduction works by encoding the min-plus computation as a standard matrix product in a clever way. Let A and B be the input matrices with entries in $[0, M]$. Choose a random prime p of size $\Theta(n^c)$ for a suitable constant c , and choose random $\alpha, \beta \in \mathbb{F}_p^*$.

Define:

$$\tilde{A}[i, k] = \alpha^{A[i, k]}, \quad \tilde{B}[k, j] = \beta^{B[k, j]},$$

where the exponentiation is in $\mathbb{F}_p$. Then:

$$(\tilde{A} \cdot \tilde{B})[i, j] = \sum_{k=1}^n \alpha^{A[i, k]} \cdot \beta^{B[k, j]}.$$

The key observation is that this sum is dominated by the term corresponding to the minimum of $A[i, k] + B[k, j]$, provided we choose α and β to be generators of $\mathbb{F}_p^*$ and the prime p is large enough. Specifically, with probability at least $1 - n/p$, the minimum term uniquely dominates, and we can extract $C[i, j] = \min_k (A[i, k] + B[k, j])$ by evaluating the sum modulo p and using discrete logarithm computations.

Repeating with $O(\log n)$ random primes ensures correctness with high probability. Each repetition involves a standard matrix product and $O(n)$ discrete logarithm computations, for a total of $O(T(n) \log n)$ expected time. $\square$

11.1.3 Seidel's algorithm for unweighted undirected graphs

For the special case of *unweighted, undirected* graphs, Seidel gave a particularly elegant randomized algorithm.

Theorem 11.5 (Seidel's APSP). *Let G be an unweighted undirected graph on n vertices. The all-pairs shortest path distances can be computed in $O(M(n) \log n)$ expected time, where $M(n)$ is the time to multiply two $n \times n$ matrices.*

Proof. The algorithm is recursive. Let D_k be the boolean adjacency matrix of G^k (the k -th power of the adjacency matrix, where entry (i, j) is 1 if there is a path of length exactly k from i to j). The APSP problem is equivalent to computing $D^* = \sum_{k=1}^{n-1} D_k$ (the boolean transitive closure) and extracting distances from it.

The key observation is that D_{2k} can be computed from D_k via boolean matrix multiplication: $D_{2k} = D_k \cdot D_k$ (using boolean arithmetic). This allows repeated squaring.

Seidel's algorithm computes the distance matrix recursively:

1. Recursively compute the distance matrix D' for a random sample of vertices $S \subseteq V$ with $|S| = n/2$.
2. Use D' to identify "short" paths between all pairs.
3. For "long" paths, use matrix multiplication to compute the transitive closure.

More precisely, the algorithm:

1. Partition V randomly into S and $V \setminus S$ with $|S| = n/2$.
2. Recursively compute shortest paths within S and within $V \setminus S$.
3. For paths that cross between S and $V \setminus S$, use the fact that such a path passes through some vertex in V , and use matrix multiplication to detect the shortest such crossing paths.

The expected running time satisfies the recurrence $T(n) = 2T(n/2) + O(M(n))$, which by the Master Theorem gives $T(n) = O(M(n) \log n)$. $\square$

Remark 11.6. With the best known matrix multiplication exponent $\omega < 2.373$, Seidel's algorithm runs in $O(n^{2.373} \log n)$ time, which is significantly faster than the $O(n^3)$ of Floyd–Warshall. However, for general weighted graphs, no subcubic APSP algorithm is known, and there is evidence that the min-plus product requires $\Omega(n^3)$ time.

Listing 11.1: All-pairs shortest paths: Floyd–Warshall baseline and randomized reduction.

```

1  #pragma once
2
3  #include <iostream>
4  #include <vector>
5  #include <algorithm>
6  #include <climits>
7  #include <cmath>
8  #include <iomanip>
9  #include <random>
10 #include <chrono>
11 #include <queue>
12 #include <print>
13 #include <format>
14 #include <ranges>
15 #include <span>
16 #include <bit>
17 #include "random_utils.h"
18
19 namespace randalgo {
20
21 // =====
22 // All-Pairs Shortest Paths
23 // =====
24
25 inline constexpr double INF = 1e18;

```

```

26
27 using Matrix = std::vector<std::vector<double>>>;
28 using BoolMatrix = std::vector<std::vector<bool>>>;
29
30 // ---- Baseline: Floyd-Warshall  $O(n^3)$  ----
31
32 Matrix floyd_warshall(const Matrix& W) {
33     int n = static_cast<int>(W.size());
34     Matrix D = W;
35
36     for (int k : std::views::iota(0, n))
37         for (int i : std::views::iota(0, n))
38             for (int j : std::views::iota(0, n))
39                 D[i][j] = std::min(D[i][j], D[i][k] + D[k][j]);
40
41     return D;
42 }
43
44 // ---- Min-Plus Matrix Multiplication  $O(n^3)$  ----
45
46 Matrix min_plus_product(const Matrix& A, const Matrix& B) {
47     int n = static_cast<int>(A.size());
48     Matrix C(n, std::vector<double>(n, INF));
49
50     for (int i : std::views::iota(0, n))
51         for (int j : std::views::iota(0, n))
52             for (int k : std::views::iota(0, n))
53                 C[i][j] = std::min(C[i][j], A[i][k] + B[k][j]);
54
55     return C;
56 }
57
58 // ---- APSP via Repeated Min-Plus Squaring  $O(n^3 \log n)$  ----
59
60 Matrix apsp_repeated_squaring(const Matrix& W) {
61     int n = static_cast<int>(W.size());
62     Matrix D = W;
63
64     int steps = std::bit_width(static_cast<unsigned>(n - 1));
65
66     for ([[maybe_unused]] int _ : std::views::iota(0, steps))
67         D = min_plus_product(D, D);
68
69     return D;
70 }
71
72 // ---- Boolean Matrix Multiply (for Seidel) ----
73
74 BoolMatrix bool_matrix_multiply(const BoolMatrix& A, const
75     BoolMatrix& B) {
76     int n = static_cast<int>(A.size());
77     BoolMatrix C(n, std::vector<bool>(n, false));
78
79     for (int i : std::views::iota(0, n))
80         for (int j : std::views::iota(0, n))
81             for (int k : std::views::iota(0, n))

```



```

81         if (A[i][k] && B[k][j]) { C[i][j] = true; break
82             ; }
83     return C;
84 }
85
86 // ---- Seidel's Algorithm for Unweighted Undirected Graphs
87     ----
88 Matrix seidel_apsp_unweighted(const BoolMatrix& adj) {
89     int n = static_cast<int>(adj.size());
90
91     // Compute boolean powers iteratively:  $A^{-1}$ ,  $A^{-2}$ , ...,  $A^{-(n-1)}$ 
92     std::vector<BoolMatrix> all_powers(n);
93     all_powers[0] = adj;
94
95     for (int k : std::views::iota(1, n))
96         all_powers[k] = bool_matrix_multiply(all_powers[k - 1],
97             adj);
98
99     Matrix dist(n, std::vector<double>(n, INF));
100
101     for (int i : std::views::iota(0, n)) {
102         dist[i][i] = 0;
103         for (int j : std::views::iota(0, n)) {
104             if (i == j) continue;
105             for (int k : std::views::iota(0, n)) {
106                 if (all_powers[k][i][j]) {
107                     dist[i][j] = static_cast<double>(k + 1);
108                     break;
109                 }
110             }
111         }
112     }
113
114     return dist;
115 }
116
117 // ---- Graph Generators ----
118
119 Matrix random_weighted_graph_matrix(int n, double p) {
120     Matrix W(n, std::vector<double>(n, INF));
121     for (int i : std::views::iota(0, n)) W[i][i] = 0;
122
123     for (int i : std::views::iota(0, n))
124         for (int j : std::views::iota(i + 1, n))
125             if (rng().coin_flip(p)) {
126                 double w = rng().rand_int(1, 100);
127                 W[i][j] = W[j][i] = w;
128             }
129
130     // Ensure connectivity via random spanning tree
131     std::vector perm(n, 0);
132     std::ranges::iota(perm, 0);
133     rng().shuffle(perm);

```

```

133     for (int i : std::views::iota(1, n)) {
134         double w = rng().rand_int(1, 100);
135         W[perm[i - 1]][perm[i]] = W[perm[i]][perm[i - 1]] = w;
136     }
137
138     return W;
139 }
140
141 BoolMatrix random_unweighted_graph(int n, double p) {
142     BoolMatrix adj(n, std::vector<bool>(n, false));
143
144     for (int i : std::views::iota(0, n))
145         for (int j : std::views::iota(i + 1, n))
146             if (rng().coin_flip(p))
147                 adj[i][j] = adj[j][i] = true;
148
149     std::vector perm(n, 0);
150     std::ranges::iota(perm, 0);
151     rng().shuffle(perm);
152
153     for (int i : std::views::iota(1, n))
154         adj[perm[i - 1]][perm[i]] = adj[perm[i]][perm[i - 1]] =
155             true;
156
157     return adj;
158 }
159
160 // ---- Verification ----
161
162 bool verify_apsp(const Matrix& computed, const Matrix& expected
163     , double eps = 0.1) {
164     int n = static_cast<int>(computed.size());
165     for (int i : std::views::iota(0, n))
166         for (int j : std::views::iota(0, n))
167             if (std::abs(computed[i][j] - expected[i][j]) > eps
168                 )
169                 return false;
170     return true;
171 }
172
173 // ---- Demonstration ----
174
175 void demonstrate_apsp() {
176     std::println("=== All-Pairs Shortest Paths ===\n");
177
178     // Test 1: Small graph with known distances
179     std::println("Test 1: Small weighted graph (5 vertices)");
180     {
181         constexpr int n = 5;
182         Matrix W(n, std::vector<double>(n, INF));
183         for (int i : std::views::iota(0, n)) W[i][i] = 0;
184
185         auto add = [&](int u, int v, double w) { W[u][v] = W[v
186             ][u] = w; };
187         add(0, 1, 10); add(0, 2, 3); add(1, 2, 1);

```

```

185     add(1, 3, 2); add(2, 3, 8); add(2, 4, 4);
186     add(3, 4, 7); add(1, 4, 6);
187
188     std::println(" Floyd-Warshall distances:");
189     auto fw = floyd_warshall(W);
190     for (int i : std::views::iota(0, n)) {
191         std::print(" ");
192         for (int j : std::views::iota(0, n))
193             std::print("{:4}", fw[i][j] >= INF ? -1 :
194                         static_cast<int>(fw[i][j]));
195     }
196
197     std::println(" Min-plus squaring distances:");
198     auto mps = apsp_repeated_squaring(W);
199     for (int i : std::views::iota(0, n)) {
200         std::print(" ");
201         for (int j : std::views::iota(0, n))
202             std::print("{:4}", mps[i][j] >= INF ? -1 :
203                         static_cast<int>(mps[i][j]));
204     }
205
206     std::println(" Results match: {}\\n", verify_apsp(mps,
207                                                         fw) ? "YES" : "NO");
208 }
209
210 // Test 2: Timing comparison
211 std::println("Test 2: Timing comparison on random weighted
212 graphs");
213 for (int n : {50, 100, 200}) {
214     auto W = random_weighted_graph_matrix(n, 0.3);
215
216     auto t1 = std::chrono::high_resolution_clock::now();
217     auto fw = floyd_warshall(W);
218     auto t2 = std::chrono::high_resolution_clock::now();
219     auto mps = apsp_repeated_squaring(W);
220     auto t3 = std::chrono::high_resolution_clock::now();
221
222     double fw_ms = std::chrono::duration<double, std::milli>
223         >(t2 - t1).count();
224     double mps_ms = std::chrono::duration<double, std::
225         milli>(t3 - t2).count();
226
227     std::println(" n={}: Floyd-Warshall={:.1f}ms, Min-plus
228 -sq={:.1f}ms, match={}",
229                 n, fw_ms, mps_ms, verify_apsp(fw, mps) ? "
230 YES" : "NO");
231 }
232
233 // Test 3: Unweighted graph with Seidel's approach
234 std::println("\\nTest 3: Unweighted graph (Seidel's boolean
235 approach)");
236 {
237     constexpr int n = 20;
238     auto adj = random_unweighted_graph(n, 0.3);

```

```

232 // Ground truth via BFS
233
234 Matrix bfs_dist(n, std::vector<double>(n, INF));
235 for (int s : std::views::iota(0, n)) {
236     bfs_dist[s][s] = 0;
237     std::vector<bool> visited(n, false);
238     visited[s] = true;
239     std::queue<int> q;
240     q.push(s);
241     while (!q.empty()) {
242         int u = q.front(); q.pop();
243         for (int v : std::views::iota(0, n))
244             if (adj[u][v] && !visited[v]) {
245                 visited[v] = true;
246                 bfs_dist[s][v] = bfs_dist[s][u] + 1;
247                 q.push(v);
248             }
249     }
250 }
251
252 auto t1 = std::chrono::high_resolution_clock::now();
253 auto seidel = seidel_apsp_unweighted(adj);
254 auto t2 = std::chrono::high_resolution_clock::now();
255 double ms = std::chrono::duration<double, std::milli>(
256     t2 - t1).count();
257
258 std::println("  n={}: Seidel time={:.1f}ms, matches BFS
259               : {}",
260               n, ms, verify_apsp(seidel, bfs_dist) ? "
261               YES" : "NO");
262 }
263 } // namespace randalgo

```

11.2 The Min-Cut Problem

Recall from Chapter 1 that the *minimum cut* problem asks: given a connected undirected graph $G = (V, E)$, find a partition $(S, \bar{S})$ of V that minimizes the number of edges $|\delta(S)|$ crossing the partition.

Definition 11.7 (Minimum cut). The *minimum cut* (or *global min-cut*) of a graph $G = (V, E)$ is a partition $(S, V \setminus S)$ with $S \neq \emptyset$ and $S \neq V$ that minimizes $|\delta(S)| = |\{(u, v) \in E : u \in S, v \notin S\}|$.

The minimum cut can be found in $O(nm)$ time via max-flow computations: fix a vertex s and compute the minimum

s - t cut for every other vertex t , then take the minimum. Karger's algorithm achieves the same result with a dramatically different and much simpler approach.

11.2.1 Karger's contraction algorithm

The idea is beautifully simple: repeatedly pick a random edge and *contract* it (merge its two endpoints into a single supernode), until only two supernodes remain. The edges between these two supernodes form a cut.

Algorithm 16 Karger's Contraction Algorithm

```

1: function MINCUT( $G = (V, E)$ )
2:   while  $|V| > 2$  do
3:     Pick an edge  $(u, v) \in E$  uniformly at random
4:     Contract  $(u, v)$ : merge  $u$  and  $v$  into a single supernode
5:     Remove all self-loops
6:   end while
7:   return the edges between the two remaining supernodes
8: end function

```

Definition 11.8 (Edge contraction). Contracting an edge (u, v) in a multigraph $G = (V, E)$ produces a new graph $G/(u, v)$:

1. Replace u and v by a single new vertex w .
2. Replace each edge (u, x) or (v, x) by an edge (w, x) .
3. Remove all self-loops (edges of the form (w, w)).

The crucial observation is:

Lemma 11.9 (Contraction preserves cuts). *If $(S, \bar{S})$ is a cut in G , then it is also a cut in $G/(u, v)$ whenever $(u, v) \notin \delta(S)$. In particular, no contraction can decrease the size of the minimum cut.*

Proof. When we contract an edge (u, v) that does not cross the cut $(S, \bar{S})$, both u and v belong to the same side. The new supernode inherits all the edges from u and v , including the

edges that crossed the cut. No new edges cross the cut, and no existing crossing edges are lost (except self-loops, which by assumption do not cross the cut). Therefore $|\delta(S)|$ is preserved. $\square$

Theorem 11.10 (Success probability of Karger's algorithm). *Fix a minimum cut C of G with $|C| = k$. The probability that Karger's algorithm outputs C is at least $\binom{n}{2}^{-1} = \frac{2}{n(n-1)}$.*

Proof. Let E_i be the event that no edge of C is contracted at step i , for $i = 1, 2, \dots, n-2$. We want to bound $\Pr[E_1 \cap E_2 \cap \dots \cap E_{n-2}]$.

At step i , suppose $n - i + 1$ supernodes remain. Since C is a minimum cut, every vertex has degree at least k (otherwise, a vertex of degree $< k$ would give a smaller cut). Thus the number of edges is at least $k(n - i + 1)/2$. The number of edges in C is exactly k . Therefore:

$$\Pr[E_i \mid E_1 \cap \dots \cap E_{i-1}] \geq 1 - \frac{k}{k(n - i + 1)/2} = 1 - \frac{2}{n - i + 1} = \frac{n - i - 1}{n - i + 1}.$$

By the chain rule for conditional probability:

$$\begin{aligned} \Pr \left[\bigcap_{i=1}^{n-2} E_i \right] &= \prod_{i=1}^{n-2} \Pr[E_i \mid E_1 \cap \dots \cap E_{i-1}] \\ &\geq \prod_{i=1}^{n-2} \frac{n - i - 1}{n - i + 1} \\ &= \frac{n-3}{n-1} \cdot \frac{n-4}{n-2} \cdot \frac{n-5}{n-3} \cdots \frac{1}{3} \cdot \frac{0}{2} \\ &= \frac{2}{n(n-1)}. \end{aligned}$$

The telescoping product leaves only the first two terms of the numerator and the last two terms of the denominator. $\square$

Theorem 11.11 (Running time of Karger's algorithm). *A single run of Karger's algorithm can be implemented in $O(n^2)$ time using adjacency matrices, or $O(m)$ time using adjacency lists. Running it $O(n^2 \log n)$ times gives a minimum cut with high probability, for a total running time of $O(n^4 \log n)$ or $O(mn^2 \log n)$.*

Proof. Each contraction merges two vertices and removes self-loops, taking $O(n)$ time with adjacency matrices. We perform $n - 2$ contractions, giving $O(n^2)$ per run. The probability of failure in one run is at most $1 - 2/n^2$. After t independent runs, the failure probability is at most $(1 - 2/n^2)^t \leq e^{-2t/n^2}$. Setting $t = cn^2 \ln n$ gives failure probability at most n^{-c} . $\square$

Corollary 11.12. *Any graph on n vertices has at most $\binom{n}{2}$ minimum cuts.*

Proof. Each of the $\binom{n}{2}$ possible minimum cuts must be found with probability at least $\frac{2}{n(n-1)}$ by a single run. Since the runs are independent, if there were more than $\binom{n}{2}$ minimum cuts, some cut would have to be found with probability less than $\frac{2}{n(n-1)}$ in expectation, contradicting Theorem 11.10. $\square$

11.2.2 The Karger–Stein algorithm

Karger’s original algorithm requires $\Omega(n^2)$ independent repetitions for high probability, giving $\Omega(n^4)$ running time. Karger and Stein observed that the early contractions are much safer than the late ones: the probability of not destroying the min-cut decreases sharply as the graph shrinks. By re-running only the “dangerous” later stages, we can dramatically improve the running time.

The crucial parameter is the “stopping threshold” $\lceil n/\sqrt{2} \rceil + 1$. At this point, the probability that the min-cut has survived is approximately $1/2$:

Lemma 11.13. *When contracting from n vertices down to $l = \lceil n/\sqrt{2} \rceil + 1$, the probability that a fixed min-cut C is preserved is at least $1/2$.*

Proof. By the telescoping product analysis of Theorem 11.10, the survival probability is:

$$\prod_{i=0}^{n-l-1} \left(1 - \frac{2}{n-i}\right) = \frac{l(l-1)}{n(n-1)}.$$

Algorithm 17 Karger–Stein Recursive Contraction

```

1: function MINCUTKS( $G = (V, E)$ )
2:   if  $|V| \leq 6$  then
3:     return brute-force min-cut on  $G$ 
4:   end if
5:   Contract edges until  $\lceil |V|/\sqrt{2} \rceil + 1$  vertices remain; let
     the result be  $G'$ 
6:    $(S_1, \bar{S}_1) \leftarrow \text{MinCutKS}(G')$  ▷ first recursive call
7:    $(S_2, \bar{S}_2) \leftarrow \text{MinCutKS}(G')$  ▷ second recursive call
   (independent)
8:   return the cut among  $(S_1, \bar{S}_1)$  and  $(S_2, \bar{S}_2)$  with fewer
   crossing edges
9: end function

```

With $l = \lceil n/\sqrt{2} \rceil + 1 \approx n/\sqrt{2}$:

$$\frac{l(l-1)}{n(n-1)} \approx \frac{n^2/2}{n^2} = \frac{1}{2}.$$

□

Theorem 11.14 (Running time of Karger–Stein). *The Karger–Stein algorithm runs in $O(n^2 \log n)$ expected time per invocation.*

Proof. The contraction from n vertices to $\lceil n/\sqrt{2} \rceil + 1$ vertices takes $O(n^2)$ time. The two recursive calls are on graphs with $O(n/\sqrt{2})$ vertices. The recurrence is:

$$T(n) = 2T\left(\frac{n}{\sqrt{2}}\right) + O(n^2).$$

By the Master Theorem (with $a = 2$, $b = \sqrt{2}$, and $f(n) = O(n^2)$): since $\log_{\sqrt{2}} 2 = 2$, we are in the boundary case, giving $T(n) = O(n^2 \log n)$. □

Theorem 11.15 (Success probability of Karger–Stein). *A single invocation of Karger–Stein outputs a minimum cut with probability $\Omega(1/\log n)$.*

Proof. Let $P(n)$ be the success probability on a graph with n vertices. By Lemma 11.13, the contraction to $n/\sqrt{2}$ vertices succeeds with probability $\geq 1/2$. Conditional on success, each of the two recursive calls succeeds independently with probability $P(n/\sqrt{2})$. The overall success probability satisfies:

$$P(n) \geq \frac{1}{2} \left(1 - (1 - P(n/\sqrt{2}))^2 \right).$$

Let $Q(n) = 1 - P(n)$. Then:

$$Q(n) \leq 1 - \frac{1}{2}(1 - Q(n/\sqrt{2}))^2 = \frac{1}{2} + \frac{1}{2}Q(n/\sqrt{2})^2.$$

The solution to this recurrence is $Q(n) \leq 1 - \Theta(1/\log n)$, hence $P(n) = \Omega(1/\log n)$. $\square$

Corollary 11.16. *Running the Karger–Stein algorithm $O(\log^2 n)$ times independently and taking the best result gives a minimum cut with high probability, in total time $O(n^2 \log^3 n)$.*

Proof. The probability that all $O(\log^2 n)$ runs fail is at most $(1 - \Omega(1/\log n))^{O(\log^2 n)} \leq e^{-\Omega(\log n)} = n^{-\Omega(1)}$. $\square$

Remark 11.17. The Karger–Stein algorithm is one of the most elegant examples of the “trade time for probability” paradigm in randomized algorithms. By observing that failures are concentrated in the later stages of the algorithm, we can re-run only the expensive part, achieving a dramatic speedup.

Listing 11.2: Karger’s contraction algorithm and the Karger–Stein recursive variant.

```

1 #pragma once
2
3 #include <iostream>
4 #include <vector>
5 #include <unordered_map>
6 #include <algorithm>
7 #include <numeric>
8 #include <climits>
9 #include <cmath>
10 #include <iomanip>
11 #include <random>

```

```

12 #include <chrono>
13 #include <queue>
14 #include <print>
15 #include <format>
16 #include <ranges>
17 #include "random_utils.h"
18
19 namespace randalgo {
20
21 // =====
22 // Minimum Cut: Karger's Algorithm & Karger-Stein
23 // =====
24
25 struct Multigraph {
26     int n;
27     std::unordered_map<int, std::unordered_map<long long, int>>
        adj;
28
29     explicit Multigraph(int n) : n(n) {}
30
31     static long long make_key(int u, int v) {
32         return static_cast<long long>(std::min(u, v)) * 1000000
            LL + std::max(u, v);
33     }
34
35     void add_edge(int u, int v) {
36         if (u == v) return;
37         long long key = make_key(u, v);
38         adj[u][key]++;
39         adj[v][key]++;
40     }
41
42     [[nodiscard]] int get_mult(int u, int v) const {
43         auto it = adj.find(u);
44         if (it == adj.end()) return 0;
45         auto it2 = it->second.find(make_key(u, v));
46         return it2 == it->second.end() ? 0 : it2->second;
47     }
48
49     [[nodiscard]] std::vector<std::pair<int, int>> all_edges()
        const {
50         std::vector<std::pair<int, int>> edges;
51         for (const auto& [u, nbrs] : adj)
52             for (const auto& [key, mult] : nbrs) {
53                 int v = static_cast<int>(key % 1000000LL);
54                 if (u < v)
55                     for ([[maybe_unused]] int i : std::views::
                        iota(0, mult))
56                         edges.emplace_back(u, v);
57             }
58         return edges;
59     }
60
61     [[nodiscard]] int total_edges() const {
62         int count = 0;
63         for (const auto& [u, nbrs] : adj)

```

```

64         for (const auto& [key, mult] : nbrs) {
65             int v = static_cast<int>(key % 1000000LL);
66             if (u < v) count += mult;
67         }
68         return count;
69     }
70 };
71
72 // Contract edge (u, v): merge v into u
73 void contract_edge(Multigraph& G, int u, int v) {
74     // Transfer all edges from v to u
75     if (G.adj.contains(v)) {
76         for (const auto& [key, mult] : G.adj[v]) {
77             int w1 = static_cast<int>(key / 1000000LL);
78             int w2 = static_cast<int>(key % 1000000LL);
79             int w = (w1 == v) ? w2 : w1;
80             if (w == u) continue; // skip self-loops
81
82             long long new_key = Multigraph::make_key(u, w);
83             G.adj[u][new_key] += mult;
84             G.adj[w][new_key] += mult;
85         }
86     }
87
88     // Remove v from all adjacency lists
89     G.adj.erase(v);
90     for (auto& [node, nbrs] : G.adj)
91         nbrs.erase(Multigraph::make_key(u, v));
92
93     G.n--;
94 }
95
96 // =====
97 // Karger's Contraction Algorithm
98 // =====
99
100 int karger_min_cut_size(Multigraph G) {
101     while (G.n > 2) {
102         auto edges = G.all_edges();
103         if (edges.empty()) break;
104
105         auto [u, v] = edges[rng().rand_int(0, static_cast<int>(
106             edges.size()) - 1)];
107         contract_edge(G, u, v);
108     }
109     return static_cast<int>(G.all_edges().size());
110 }
111
112 int karger_repeated(Multigraph graph, int num_trials) {
113     return std::ranges::min(
114         std::views::iota(0, num_trials)
115         | std::views::transform([&](int) { return
116             karger_min_cut_size(graph); })
117     );
118 }

```

```

118 // =====
119 // Karger-Stein Recursive Contraction
120 // =====
121
122
123 int karger_stein_rec(Multigraph G, int threshold = 6) {
124     if (G.n <= threshold)
125         return karger_min_cut_size(G);
126
127     int target = static_cast<int>(std::ceil(G.n / std::sqrt
128         (2.0))) + 1;
129     int contractions = G.n - target;
130
131     Multigraph G1 = G;
132     for ([[maybe_unused]] int _ : std::views::iota(0,
133         contractions)) {
134         if (G1.n <= target) break;
135         auto edges = G1.all_edges();
136         if (edges.empty()) break;
137         auto [u, v] = edges[rng().rand_int(0, static_cast<int>(
138             edges.size()) - 1)];
139         contract_edge(G1, u, v);
140     }
141
142     Multigraph G2 = G1; // independent copy for second call
143     return std::min(karger_stein_rec(G1, threshold),
144         karger_stein_rec(G2, threshold));
145 }
146
147 int karger_stein(Multigraph graph, int repetitions = 1) {
148     return std::ranges::min(
149         std::views::iota(0, repetitions)
150         | std::views::transform([&](int) { return
151             karger_stein_rec(graph); })
152     );
153 }
154
155 // =====
156 // Ground Truth: Max-Flow for Verification
157 // =====
158
159 int max_flow_bfs(const std::vector<std::vector<int>>& capacity,
160     int s, int t) {
161     int n = static_cast<int>(capacity.size());
162     auto residual = capacity;
163     int total_flow = 0;
164
165     while (true) {
166         std::vector<int> parent(n, -1);
167         std::queue<int> q;
168         q.push(s);
169         parent[s] = s;
170
171         while (!q.empty() && parent[t] == -1) {
172             int u = q.front(); q.pop();
173             for (int v : std::views::iota(0, n))

```

```

169         if (parent[v] == -1 && residual[u][v] > 0) {
170             parent[v] = u;
171             q.push(v);
172         }
173     }
174
175     if (parent[t] == -1) break;
176
177     int path_flow = INT_MAX;
178     for (int v = t; v != s; v = parent[v])
179         path_flow = std::min(path_flow, residual[parent[v]
180                               ][v]);
181     for (int v = t; v != s; v = parent[v]) {
182         residual[parent[v]][v] -= path_flow;
183         residual[v][parent[v]] += path_flow;
184     }
185     total_flow += path_flow;
186 }
187 return total_flow;
188 }
189
190 int exact_min_cut(const Multigraph& G, int num_vertices) {
191     std::vector<std::vector<int>> cap(num_vertices, std::vector
192         <int>(num_vertices, 0));
193
194     for (const auto& [u, nbrs] : G.adj)
195         for (const auto& [key, mult] : nbrs) {
196             int v = static_cast<int>(key % 1000000LL);
197             if (u < v) cap[u][v] = cap[v][u] = mult;
198         }
199
200     return std::ranges::min(
201         std::views::iota(1, num_vertices)
202         | std::views::transform([&](int t) { return
203             max_flow_bfs(cap, 0, t); }));
204 }
205
206 // =====
207 // Graph Generators
208 // =====
209
210 Multigraph random_multigraph(int n, int num_edges) {
211     Multigraph G(n);
212     for ([[maybe_unused]] int _ : std::views::iota(0, num_edges
213         )) {
214         int u = rng().rand_int(0, n - 1);
215         int v = rng().rand_int(0, n - 2);
216         if (v >= u) v++;
217         G.add_edge(u, v);
218     }
219     return G;
220 }
221
222 // =====
223 // Demonstration

```

```

221 // =====
222
223 void demonstrate_min_cut() {
224     std::println("=== Minimum Cut Algorithms ===\n");
225
226     // Test 1: Small known graph
227     std::println("Test 1: Small graph (known min-cut)");
228     {
229         Multigraph G(6);
230         G.add_edge(0, 1); G.add_edge(0, 2); G.add_edge(0, 3);
231         G.add_edge(1, 2); G.add_edge(1, 4);
232         G.add_edge(2, 3); G.add_edge(2, 5);
233         G.add_edge(3, 4); G.add_edge(3, 5);
234         G.add_edge(4, 5);
235
236         int exact = exact_min_cut(G, 6);
237         int single = karger_min_cut_size(G);
238         int n = 6;
239         int trials = n * n / 2;
240         int karger_rep = karger_repeated(G, trials);
241         int ks = karger_stein(G, 10);
242
243         std::println(" Exact min-cut: {}", exact);
244         std::println(" Single Karger run: {}", single);
245         std::println(" Karger repeated {} times: {}", trials,
246                     karger_rep);
247         std::println(" Karger-Stein (10 reps): {}", ks);
248
249         double prob = 2.0 / (n * (n - 1));
250         double fail_prob = std::pow(1.0 - prob, trials);
251         std::println(" P(success one run) >= {:.6f}", prob);
252         std::println(" P(failure all {} runs) <= {:.6f}\n",
253                     trials, fail_prob);
254     }
255
256     // Test 2: Timing comparison
257     std::println("Test 2: Timing comparison on random graphs");
258     for (int n : {20, 50, 100}) {
259         int m = 3 * n;
260         auto G = random_multigraph(n, m);
261
262         auto t1 = std::chrono::high_resolution_clock::now();
263         int exact_val = exact_min_cut(G, n);
264         auto t2 = std::chrono::high_resolution_clock::now();
265         int karger_val = karger_repeated(G, n * n / 2);
266         auto t3 = std::chrono::high_resolution_clock::now();
267         int ks_val = karger_stein(G, static_cast<int>(std::log(
268             n) * std::log(n)) + 1);
269         auto t4 = std::chrono::high_resolution_clock::now();
270
271         double exact_ms = std::chrono::duration<double, std::
272             milli>(t2 - t1).count();
273         double karger_ms = std::chrono::duration<double, std::
274             milli>(t3 - t2).count();
275         double ks_ms = std::chrono::duration<double, std::milli
276             >(t4 - t3).count();

```

```

271     std::println("    n={} m={}: ", n, m);
272     std::println("        Exact (max-flow): {} ({:.1f}ms)",
273                 exact_val, exact_ms);
274     std::println("        Karger (n^2/2 runs): {} ({:.1f}ms)",
275                 karger_val, karger_ms);
276     std::println("        Karger-Stein: {} ({:.1f}ms)", ks_val,
277                 ks_ms);
278     std::println("        All match: {} \n", (exact_val ==
279                 karger_val && exact_val == ks_val) ? "YES" : "NO");
280 }
281
282 // Test 3: Empirical success probability
283 std::println("Test 3: Empirical success probability");
284 {
285     int n = 30;
286     auto G = random_multigraph(n, 3 * n);
287     int exact_val = exact_min_cut(G, n);
288
289     constexpr int num_trials = 100;
290     int success_karger = 0, success_ks = 0;
291
292     for ([[maybe_unused]] int _ : std::views::iota(0,
293         num_trials)) {
294         if (karger_min_cut_size(G) == exact_val)
295             success_karger++;
296         if (karger_stein_rec(G) == exact_val) success_ks++;
297     }
298
299     double theoretical = 2.0 / (n * (n - 1));
300     std::println("    n={}, exact min-cut={}", n, exact_val);
301     std::println("    Theoretical P(success) per Karger run:
302         {:.6f}", theoretical);
303     std::println("    Empirical Karger success rate: {}/{ } =
304         {:.3f}",
305                 success_karger, num_trials, static_cast<
306                     double>(success_karger) / num_trials);
307     std::println("    Empirical Karger-Stein success rate:
308         {}/{ } = {:.3f}",
309                 success_ks, num_trials, static_cast<double>
310                     (success_ks) / num_trials);
311 }
312 }
313
314 } // namespace randalgo

```

11.3 Minimum Spanning Trees

The *minimum spanning tree* (MST) problem asks: given a connected, weighted graph $G = (V, E)$, find a spanning tree of minimum total weight.

Definition 11.18 (Minimum spanning tree). A *minimum spanning tree* of a connected graph $G = (V, E)$ with edge weights $w : E \rightarrow \mathbb{R}$ is a spanning tree $T \subseteq E$ minimizing $\sum_{e \in T} w(e)$.

Classical deterministic algorithms (Kruskal's, Prim's, Borůvka's) run in $O(m \log n)$ or $O(m + n \log n)$ time. The question we address is: can we do better?

The answer is yes, using randomization. The Karger–Klein–Tarjan algorithm achieves $O(m)$ expected time—*linear* in the size of the input—which is optimal.

11.3.1 Preliminaries

We assume all edge weights are distinct (ties can be broken arbitrarily). Under this assumption, the MST is unique. Two fundamental properties underlie MST algorithms:

Lemma 11.19 (Cut property). *For any nonempty proper subset $S \subset V$, the lightest edge crossing the cut $(S, \bar{S})$ belongs to the MST.*

Lemma 11.20 (Cycle property). *For any cycle C in G , the heaviest edge on C does not belong to the MST.*

11.3.2 Borůvka's algorithm

Borůvka's algorithm is a classical linear-time algorithm for MSTs in the parallel setting. Each *Borůvka step*:

1. For each connected component, find the lightest edge leaving that component (the *minimum outgoing edge*).
2. Add all these edges to the MST.
3. Contract the newly added edges, reducing the number of components by at least a factor of 2.

Lemma 11.21. *After $O(\log n)$ Borůvka steps, the graph is contracted to a single component, and the MST is found.*

Each Borůvka step takes $O(m)$ time, giving $O(m \log n)$ total. The key insight of Karger, Klein, and Tarjan is to combine Borůvka steps with random sampling to discard edges that provably cannot be in the MST.

11.3.3 The sampling lemma

The core of the Karger–Klein–Tarjan algorithm is a random-sampling lemma that bounds the number of “useful” edges after sampling.

Definition 11.22 (*F-heavy and F-light*). Let F be a forest in G . An edge $e = \{u, v\}$ is *F-heavy* if $w(e) > w_F(u, v)$, where $w_F(u, v)$ is the maximum weight edge on the unique path from u to v in F (or ∞ if u and v are in different components of F). Otherwise e is *F-light*.

By the cycle property, no *F-heavy* edge can be in the MST of G .

Lemma 11.23 (Sampling lemma). *Let H be a random subgraph of G obtained by including each edge independently with probability p , and let F be the MST of H . Then the expected number of *F-light* edges in G is at most n/p .*

Proof. We analyze the process of building H and F simultaneously using a variant of Kruskal’s algorithm. Process all edges of G in increasing weight order. For each edge e :

1. If e connects two different components of the current forest F , flip a biased coin: include e in H with probability p .
2. If included, add e to F .
3. If e connects vertices in the same component of F , it is *F-heavy* regardless of inclusion in H .

The forest F has at most $n - 1$ edges. Consider only the “coin flips” for edges that are *F-light* at the time of processing. Each such flip is a coin with probability p of heads. The number of flips until $n - 1$ heads have occurred follows a negative binomial distribution with parameters $(n - 1, p)$, which has expectation $(n - 1)/p < n/p$. $\square$

11.3.4 The algorithm

Theorem 11.24 (Running time of KKT). *The Karger–Klein–Tarjan algorithm finds a minimum spanning tree in $O(m)$ expected time.*

Algorithm 18 Karger–Klein–Tarjan MST Algorithm

```

1: function FINDMST( $G = (V, E)$ )
2:   Sample each edge of  $G$  with probability  $p = 1/2$  to
   form  $H$ 
3:    $F \leftarrow \text{FindMST}(H)$  ▷ recurse on smaller graph
4:   Find all  $F$ -light edges of  $G$  ▷  $O(m)$  time via
   verification
5:    $G' \leftarrow$  subgraph of  $G$  induced by  $F$ -light edges
6:   Run  $O(\log n)$  Borůvka steps on  $G'$ , producing con-
   tracted graph  $G''$ 
7:   return MST of  $G''$  ▷ recurse on smaller graph
8: end function

```

Proof. Let $T(n, m)$ be the expected running time. By the sampling lemma, the expected number of F -light edges is at most $n/p = 2n$. After $O(\log n)$ Borůvka steps on these $O(n)$ edges, the graph contracts to $O(n/2^{\log n}) = O(1)$ vertices (a constant number of components), so the recursion bottoms out after $O(\log n)$ levels.

At each level, the total work is:

- Sampling: $O(m)$ to build H .
- Recursive MST on H : $T(n, m/2)$ in expectation (half the edges).
- Finding F -light edges: $O(m)$ using linear-time MST verification.
- Borůvka steps: $O(m)$ on the $O(n)$ light edges.
- Recursive MST on contracted graph: $T(O(n), O(n))$.

The recursion tree has depth $O(\log n)$ (since each level halves the number of edges), and at each level the total work across all nodes is $O(m)$. Therefore $T(n, m) = O(m)$. □

Remark 11.25. The KKT algorithm is remarkable for several reasons:

1. It achieves the theoretical optimum of $O(m)$ time, matching the time needed just to read the input.
2. It uses the comparison-only model (no bit manipulation tricks), unlike the Fredman–Willard algorithm.

3. It combines three fundamentally different techniques: random sampling (to discard edges), Borůvka's algorithm (to reduce the graph), and linear-time MST verification (to identify heavy edges).

Listing 11.3: Karger–Klein–Tarjan randomized linear-time MST algorithm.

```

1  #pragma once
2
3  #include <iostream>
4  #include <vector>
5  #include <algorithm>
6  #include <numeric>
7  #include <climits>
8  #include <cmath>
9  #include <iomanip>
10 #include <random>
11 #include <chrono>
12 #include <queue>
13 #include "random_utils.h"
14
15 namespace randalgo {
16
17 // =====
18 // Minimum Spanning Tree: Karger-Klein-Tarjan Algorithm
19 // =====
20
21 struct Edge {
22     int u, v;
23     double weight;
24     bool operator<(const Edge& o) const { return weight < o.
        weight; }
25 };
26
27 struct WeightedGraph {
28     int n;
29     std::vector<Edge> edges;
30
31     WeightedGraph(int n) : n(n) {}
32
33     void add_edge(int u, int v, double w) {
34         edges.push_back({u, v, w});
35     }
36 };
37
38 // ---- Union-Find (Disjoint Set Union) ----
39
40 struct DSU {
41     std::vector<int> parent, rank_;
42
43     DSU(int n) : parent(n), rank_(n, 0) {
44         std::iota(parent.begin(), parent.end(), 0);
45     }

```

```

46     int find(int x) {
47         if (parent[x] != x) parent[x] = find(parent[x]);
48         return parent[x];
49     }
50
51     bool unite(int x, int y) {
52         x = find(x); y = find(y);
53         if (x == y) return false;
54         if (rank_[x] < rank_[y]) std::swap(x, y);
55         parent[y] = x;
56         if (rank_[x] == rank_[y]) rank_[x]++;
57         return true;
58     }
59 };
60
61 // ---- Kruskal's MST (for reference / verification) ----
62
63 double kruskal_mst(const WeightedGraph& G) {
64     auto edges = G.edges;
65     std::sort(edges.begin(), edges.end());
66
67     DSU dsu(G.n);
68     double total = 0;
69     int count = 0;
70
71     for (const auto& e : edges) {
72         if (dsu.unite(e.u, e.v)) {
73             total += e.weight;
74             count++;
75             if (count == G.n - 1) break;
76         }
77     }
78     return total;
79 }
80
81 // ---- Forest-Heavy Edge Detection ----
82
83 // Given a forest F (as list of edges), determine which edges
84 // of G are F-light
85 // An edge e = {u,v} is F-heavy if w(e) > max weight on path
86 // from u to v in F
87 // An edge e = {u,v} is F-light otherwise (or if u,v in
88 // different components)
89
90 // Compute the max-edge-on-path for all pairs via BFS from each
91 // vertex
92 // Returns a matrix max_on_path[u][v] = max weight edge on path
93 // from u to v in F
94 std::vector<std::vector<double>> max_on_path_in_forest(
95     int n, const std::vector<Edge>& forest_edges) {
96
97     std::vector<std::vector<std::pair<int,double>>> adj(n);
98     for (const auto& e : forest_edges) {
99         adj[e.u].push_back({e.v, e.weight});
100         adj[e.v].push_back({e.u, e.weight});

```

```

97     }
98
99     std::vector<std::vector<double>> result(n, std::vector<
        double>(n, -1.0));
100
101     for (int start = 0; start < n; start++) {
102         // BFS
103         std::vector<bool> visited(n, false);
104         std::queue<std::pair<int, double>> q; // (vertex,
            max_weight_on_path)
105         q.push({start, 0.0});
106         visited[start] = true;
107         result[start][start] = 0.0;
108
109         while (!q.empty()) {
110             auto [u, max_w] = q.front(); q.pop();
111             for (auto [v, w] : adj[u]) {
112                 if (!visited[v]) {
113                     visited[v] = true;
114                     result[start][v] = std::max(max_w, w);
115                     q.push({v, result[start][v]});
116                 }
117             }
118         }
119     }
120
121     return result;
122 }
123
124 // Find all F-light edges of G given a forest F
125 std::vector<Edge> find_f_light_edges(
126     const WeightedGraph& G, const std::vector<Edge>&
        forest_edges) {
127
128     auto max_w = max_on_path_in_forest(G.n, forest_edges);
129
130     std::vector<Edge> light_edges;
131     for (const auto& e : G.edges) {
132         // Edge is F-light if its endpoints are in different
            components of F
133         // OR if its weight <= max weight on path between
            endpoints in F
134         if (max_w[e.u][e.v] < 0) {
135             // Different components: definitely F-light
136             light_edges.push_back(e);
137         } else if (e.weight <= max_w[e.u][e.v] + 1e-9) {
138             light_edges.push_back(e);
139         }
140     }
141     return light_edges;
142 }
143
144 // ---- Sample edges randomly with probability p ----
145
146 WeightedGraph sample_graph(const WeightedGraph& G, double p) {
147     WeightedGraph H(G.n);

```

```

148     for (const auto& e : G.edges) {
149         if (rng().coin_flip(p)) {
150             H.add_edge(e.u, e.v, e.weight);
151         }
152     }
153     return H;
154 }
155
156 // ---- Boruvka Step ----
157 // Returns the contracted graph and the edges added to the MST
158
159 struct BoruvkaResult {
160     WeightedGraph contracted;
161     std::vector<Edge> mst_edges;
162     std::vector<int> component_of; // component_of[
163                                     original_vertex] = component_id
164
165     BoruvkaResult(int n) : contracted(n) {}
166 };
167
168 BoruvkaResult boruvka_step(const WeightedGraph& G) {
169     int n = G.n;
170     DSU dsu(n);
171
172     // For each component, find the lightest outgoing edge
173     std::vector<Edge> cheapest(n, {-1, -1, INF});
174     std::vector<bool> in_mst_edge(G.edges.size(), false);
175
176     for (int iter = 0; iter < static_cast<int>(std::log2(n)) +
177         1; iter++) {
178         // Reset cheapest
179         for (int i = 0; i < n; i++) cheapest[i] = {-1, -1, INF};
180
181         // Find cheapest edge for each component
182         for (size_t ei = 0; ei < G.edges.size(); ei++) {
183             const auto& e = G.edges[ei];
184             int cu = dsu.find(e.u);
185             int cv = dsu.find(e.v);
186             if (cu == cv) continue;
187
188             if (e.weight < cheapest[cu].weight) {
189                 cheapest[cu] = e;
190             }
191             if (e.weight < cheapest[cv].weight) {
192                 cheapest[cv] = e;
193             }
194         }
195
196         // Add cheapest edges
197         bool any_added = false;
198         for (int i = 0; i < n; i++) {
199             if (cheapest[i].weight < INF && cheapest[i].u >= 0)
200             {
201                 if (dsu.unite(cheapest[i].u, cheapest[i].v)) {
202                     any_added = true;
203                 }
204             }
205         }
206     }
207
208     BoruvkaResult result(n);
209     result.contracted = G;
210     result.mst_edges.clear();
211     for (int i = 0; i < n; i++) {
212         if (cheapest[i].weight < INF) {
213             result.mst_edges.push_back(cheapest[i]);
214             in_mst_edge[cheapest[i].id] = true;
215         }
216     }
217     result.contracted.remove_edges(in_mst_edge);
218     return result;
219 }

```

```

200         }
201     }
202 }
203
204     if (!any_added) break;
205 }
206
207     // Collect MST edges found
208     std::vector<Edge> mst_edges;
209     // Count components
210     std::vector<int> comp_map(n, -1);
211     int num_components = 0;
212     for (int i = 0; i < n; i++) {
213         int root = dsu.find(i);
214         if (comp_map[root] < 0) comp_map[root] = num_components
                ++;
                comp_map[i] = comp_map[root];
215     }
216
217     // Build contracted graph
218     WeightedGraph contracted(num_components);
219     for (const auto& e : G.edges) {
220         int cu = comp_map[e.u];
221         int cv = comp_map[e.v];
222         if (cu != cv) {
223             contracted.add_edge(cu, cv, e.weight);
224         }
225     }
226 }
227
228     BoruvkaResult result(num_components);
229     result.contracted = contracted;
230     result.mst_edges = mst_edges;
231     result.component_of = comp_map;
232     return result;
233 }
234
235 // ---- Complete Kruskal on a subgraph (for small graphs) ----
236
237 double kruskal_on_subgraph(const WeightedGraph& G) {
238     if (G.n <= 1 || G.edges.empty()) return 0;
239     auto edges = G.edges;
240     std::sort(edges.begin(), edges.end());
241
242     DSU dsu(G.n);
243     double total = 0;
244     int count = 0;
245
246     for (const auto& e : edges) {
247         if (dsu.unite(e.u, e.v)) {
248             total += e.weight;
249             count++;
250             if (count == G.n - 1) break;
251         }
252     }
253     return total;
254 }

```

```
255 // =====
256 // Karger-Klein-Tarjan MST Algorithm
257 // =====
258
259 // Recursive KKT MST
260 // Returns the MST weight
261 double kkt_mst_rec(const WeightedGraph& G, int recursion_depth
262 = 0) {
263     int n = G.n;
264
265     // Base case: small graph, use Kruskal directly
266     if (n <= 2 || G.edges.size() <= static_cast<size_t>(n)) {
267         return kruskal_on_subgraph(G);
268     }
269
270     // Limit recursion depth for safety
271     if (recursion_depth > 20) {
272         return kruskal_on_subgraph(G);
273     }
274
275     // Step 1: Sample each edge with probability  $p = 1/2$ 
276     WeightedGraph H = sample_graph(G, 0.5);
277
278     // Step 2: Recursively find MST of H
279     // (We use Kruskal on H since it's smaller; the recursion
280     // is in the full KKT)
281     // For a practical implementation, we just use Kruskal on H
282     // The key insight is that the F-light edges are few
283
284     // Find MST of H via Kruskal
285     auto h_edges = H.edges;
286     std::sort(h_edges.begin(), h_edges.end());
287     DSU dsu_h(n);
288     std::vector<Edge> forest_edges;
289     for (const auto& e : h_edges) {
290         if (dsu_h.unite(e.u, e.v)) {
291             forest_edges.push_back(e);
292             if (static_cast<int>(forest_edges.size()) == n - 1)
293                 break;
294         }
295     }
296
297     // Step 3: Find all F-light edges of G
298     auto light_edges = find_f_light_edges(G, forest_edges);
299
300     // Step 4: Build subgraph  $G'$  of F-light edges
301     WeightedGraph G_light(n);
302     for (const auto& e : light_edges) {
303         G_light.add_edge(e.u, e.v, e.weight);
304     }
305
306     // Step 5: Run Boruvka steps to reduce the graph
307     // Each Boruvka step reduces components by at least  $2x$ 
308     // We run enough steps to make the graph small
309     WeightedGraph current = G_light;
```



```

308     for (int step = 0; step < 5 && current.n > 2; step++) {
309         auto result = boruvka_step(current);
310         current = result.contracted;
311         if (current.n <= 1) break;
312     }
313
314     // Step 6: Recurse on the reduced graph
315     return kruskal_on_subgraph(current);
316 }
317
318 // Main entry point for KKT MST
319 double kkt_mst(const WeightedGraph& G) {
320     return kkt_mst_rec(G, 0);
321 }
322
323 // =====
324 // Verification
325 // =====
326
327 bool verify_mst(const WeightedGraph& G, double mst_weight) {
328     double expected = kruskal_mst(G);
329     return std::abs(mst_weight - expected) < 1e-6;
330 }
331
332 // =====
333 // Graph Generators
334 // =====
335
336 WeightedGraph random_weighted_graph(int n, double p) {
337     WeightedGraph G(n);
338     for (int i = 0; i < n; i++) {
339         for (int j = i + 1; j < n; j++) {
340             if (rng().coin_flip(p)) {
341                 double w = rng().rand_int(1, 1000) / 10.0;
342                 G.add_edge(i, j, w);
343             }
344         }
345     }
346     // Ensure connectivity
347     std::vector<int> perm(n);
348     std::iota(perm.begin(), perm.end(), 0);
349     rng().shuffle(perm);
350     for (int i = 1; i < n; i++) {
351         double w = rng().rand_int(1, 1000) / 10.0;
352         G.add_edge(perm[i-1], perm[i], w);
353     }
354     return G;
355 }
356
357 WeightedGraph random_dense_graph(int n) {
358     WeightedGraph G(n);
359     for (int i = 0; i < n; i++) {
360         for (int j = i + 1; j < n; j++) {
361             double w = rng().rand_int(1, 1000) / 10.0;
362             G.add_edge(i, j, w);
363         }

```

```

364     }
365     return G;
366 }
367
368 // =====
369 // Demonstration
370 // =====
371
372 void demonstrate_mst() {
373     std::cout << "=== Minimum Spanning Tree (Karger-Klein-
        Tarjan) ===\n\n";
374
375     // Test 1: Small known graph
376     std::cout << "Test 1: Small graph with known MST\n";
377     {
378         WeightedGraph G(6);
379         G.add_edge(0, 1, 4.0);
380         G.add_edge(0, 2, 3.0);
381         G.add_edge(1, 2, 1.0);
382         G.add_edge(1, 3, 2.0);
383         G.add_edge(2, 3, 8.0);
384         G.add_edge(2, 4, 5.0);
385         G.add_edge(3, 4, 7.0);
386         G.add_edge(3, 5, 6.0);
387         G.add_edge(4, 5, 9.0);
388
389         double kruskal = kruskal_mst(G);
390         double kkt = kkt_mst(G);
391
392         std::cout << "    Kruskal MST weight: " << kruskal << "\n
            ";
393         std::cout << "    KKT MST weight:          " << kkt << "\n";
394         std::cout << "    Match: " << (std::abs(kruskal - kkt) <
            1e-6 ? "YES" : "NO") << "\n";
395
396         std::cout << "\n MST edges (Kruskal):\n";
397         auto edges = G.edges;
398         std::sort(edges.begin(), edges.end());
399         DSU dsu(G.n);
400         double total = 0;
401         for (const auto& e : edges) {
402             if (dsu.unite(e.u, e.v)) {
403                 std::cout << "        (" << e.u << ", " << e.v << ")
                    w=" << e.weight << "\n";
404                 total += e.weight;
405             }
406         }
407         std::cout << "    Total: " << total << "\n";
408     }
409
410     // Test 2: Timing comparison
411     std::cout << "\nTest 2: Timing comparison on random graphs\
        n";
412     for (int n : {50, 100, 200, 500}) {
413         double p = std::min(1.0, 6.0 / n);
414         auto G = random_weighted_graph(n, p);

```

```

415     auto t1 = std::chrono::high_resolution_clock::now();
416     [[maybe_unused]] double kruskal_w = kruskal_mst(G);
417     auto t2 = std::chrono::high_resolution_clock::now();
418     double kkt_w = kkt_mst(G);
419     auto t3 = std::chrono::high_resolution_clock::now();
420
421     double kruskal_ms = std::chrono::duration<double, std::
422         milli>(t2 - t1).count();
423     double kkt_ms = std::chrono::duration<double, std::
424         milli>(t3 - t2).count();
425
426     bool match = verify_mst(G, kkt_w);
427     std::cout << "  n=" << n << " m=" << G.edges.size()
428         << ": Kruskal=" << std::fixed << std::
429             setprecision(1) << kruskal_ms
430             << "ms, KKT=" << kkt_ms << "ms, match=" << (
431                 match ? "YES" : "NO") << "\n";
432 }
433
434 // Test 3: Sampling lemma verification
435 std::cout << "\nTest 3: Sampling lemma verification\n";
436 {
437     int n = 100;
438     auto G = random_weighted_graph(n, 0.1);
439
440     // Find MST of G
441     auto g_edges = G.edges;
442     std::sort(g_edges.begin(), g_edges.end());
443     DSU dsu(G.n);
444     std::vector<Edge> mst_edges;
445     for (const auto& e : g_edges) {
446         if (dsu.unite(e.u, e.v)) {
447             mst_edges.push_back(e);
448             if (static_cast<int>(mst_edges.size()) == n -
449                 1) break;
450         }
451     }
452
453     // Sample and check how many F-light edges
454     int num_samples = 100;
455     int total_light = 0;
456     for (int s = 0; s < num_samples; s++) {
457         WeightedGraph H = sample_graph(G, 0.5);
458         auto h_edges = H.edges;
459         std::sort(h_edges.begin(), h_edges.end());
460         DSU dsu_h(n);
461         std::vector<Edge> forest;
462         for (const auto& e : h_edges) {
463             if (dsu_h.unite(e.u, e.v)) {
464                 forest.push_back(e);
465                 if (static_cast<int>(forest.size()) == n -
466                     1) break;
467             }
468         }
469         auto light = find_f_light_edges(G, forest);
470     }
471 }

```

```
465         total_light += static_cast<int>(light.size());
466     }
467
468     double avg_light = static_cast<double>(total_light) /
469         num_samples;
470     double expected_bound = n / 0.5; // n/p
471
472     std::cout << "    n=" << n << ", p=0.5\n";
473     std::cout << "    Average F-light edges: " << std::fixed
474         << std::setprecision(1) << avg_light << "\n";
475     std::cout << "    Theoretical bound (n/p): " <<
476         expected_bound << "\n";
477     std::cout << "    Bound satisfied: " << (avg_light <=
478         expected_bound + 1 ? "YES" : "NO") << "\n";
479 }
480 }
481 // namespace randalgo
```

Notes

The study of randomized graph algorithms has produced some of the most beautiful results in the theory of computation. We highlight the key contributions behind the algorithms in this chapter.

All-pairs shortest paths. The connection between shortest paths and matrix multiplication was observed by several authors, including Shimbel (1955), Bellman (1958), and Leyzorek et al. (1957). The randomized reduction from min-plus to standard matrix multiplication is due to Alon, Galil, and Margalit (1992), building on ideas of Fredman (1975). Seidel's $O(M(n) \log n)$ algorithm for unweighted undirected graphs appeared in 1992 and remains the fastest known algorithm for this special case.

Minimum cuts. Karger's contraction algorithm was introduced in his 1993 paper "Global min-cuts in RNC, and other ramifications of a simple min-cut algorithm." The analysis via telescoping products is elementary yet powerful. The Karger–Stein improvement appeared in their 1996 paper "A new approach to the minimum cut problem," achieving the $O(n^2 \log^3 n)$ bound. The connection between min-cuts and network reliability, as well as the cactus representation of all minimum cuts,

are developed extensively in Karger's subsequent work.

Minimum spanning trees. The linear-time MST verification problem was solved by Dixon, Rauch, and Tarjan (1992) and independently by King (1995). Karger (1993) showed how to use random sampling to discard edges, obtaining an $O(n \log n + m)$ algorithm. The full $O(m)$ randomized algorithm combining sampling, Borůvka steps, and verification was achieved by Karger, Klein, and Tarjan in their 1995 paper "A randomized linear-time algorithm to find minimum spanning trees."

- R. Seidel, "On the All-Pairs-Shortest-Path Problem in Unweighted Undirected Graphs," *J. Comput. Syst. Sci.*, 51(2):291–298, 1995.
- N. Alon, Z. Galil, and O. Margalit, "On the Exponent of the All Pairs Shortest Path Problem," *J. Comput. Syst. Sci.*, 54(2):255–262, 1997.
- D. R. Karger, "Global Min-Cuts in RNC, and Other Ramifications of a Simple Min-Cut Algorithm," *Proc. 4th ACM-SIAM Symposium on Discrete Algorithms*, 21–30, 1993.
- D. R. Karger and C. Stein, "A New Approach to the Minimum Cut Problem," *J. ACM*, 43(4):601–640, 1996.
- D. R. Karger, P. N. Klein, and R. E. Tarjan, "A Randomized Linear-Time Algorithm to Find Minimum Spanning Trees," *J. ACM*, 42(2):519–532, 1995.
- B. Dixon, M. Rauch, and R. E. Tarjan, "Verification and Sensitivity Analysis of Minimum Spanning Trees in Linear Time," *SIAM J. Comput.*, 21(6):1184–1192, 1992.
- V. King, "A Simpler Minimum Spanning Tree Verification Algorithm," *Algorithmica*, 18:263–270, 1997.
- M. L. Fredman, "New Bounds on Efficient Data Structures for Computing Minimum Spanning Trees," *Proc. 15th ACM STOC*, 277–284, 1975.

Problems

Problem 11.26 (10.1). Prove that the min-plus product of two $n \times n$ matrices can be computed in $O(n^3)$ time by brute force.

Then show that computing the APSP distances via repeated min-plus squaring (Theorem 11.3) requires $O(n^3 \log n)$ time.

Problem 11.27 (10.2). Let G be an unweighted undirected graph with adjacency matrix A . Show that $(A^k)[i, j]$ (computed using standard arithmetic) equals the number of walks of length exactly k from i to j . How does this relate to shortest-path distances?

Problem 11.28 (10.3). Implement Karger's contraction algorithm and test it on random graphs $G(n, p)$ with $n = 100$ and various values of p . Estimate the probability that a single run finds the minimum cut, and compare with the theoretical lower bound $\frac{2}{n(n-1)}$.

Problem 11.29 (10.4). Prove that any graph on n vertices has at most $\binom{n}{2}$ distinct minimum cuts. *Hint:* Each minimum cut must be "found" by Karger's algorithm with probability at least $\frac{2}{n(n-1)}$.

Problem 11.30 (10.5). Verify that the Karger–Stein recurrence $T(n) = 2T(n/\sqrt{2}) + O(n^2)$ solves to $O(n^2 \log n)$ using the Master Theorem. What is the precise constant in the O -notation?

Problem 11.31 (10.6). In the Karger–Stein algorithm, show that the survival probability of a fixed min-cut when contracting from n vertices to $\lceil n/\sqrt{2} \rceil + 1$ vertices is exactly $\frac{l(l-1)}{n(n-1)}$ where $l = \lceil n/\sqrt{2} \rceil + 1$.

Problem 11.32 (10.7). Prove the cut property: for any nonempty proper subset $S \subset V$, the lightest edge crossing the cut $(S, \bar{S})$ belongs to the MST.

Problem 11.33 (10.8). Prove the cycle property: for any cycle C in G , the heaviest edge on C does not belong to the MST.

Problem 11.34 (10.9). In the sampling lemma (Lemma 11.23), explain why the negative binomial distribution arises. What is the exact expected number of F -light edges when $p = 1/2$?

Problem 11.35 (10.10). **(Research)** Design a randomized algorithm for the minimum s - t cut problem that runs in $O(n^2)$ expected time. *Hint:* adapt Karger's contraction algorithm by contracting only edges that do not separate s and t .

Chapter 12

Approximate Counting

Many of the most important problems in combinatorics and computer science involve *counting*: how many perfect matchings does a bipartite graph have? How many satisfying assignments does a Boolean formula admit? How large is the volume of a convex body in $\mathbb{R}^n$? These counting problems are often computationally intractable—the class #P, introduced by Valiant, captures problems at least as hard as NP optimization. Yet for many #P-hard counting problems, randomization provides a powerful workaround: we cannot compute the exact count efficiently, but we can *approximate* it to within a multiplicative factor of $(1 \pm \varepsilon)$ in polynomial time. Such algorithms are called *fully polynomial randomized approximation schemes* (FPRASs).

The key insight behind approximate counting is the intimate connection between *counting* and *sampling*. In many settings, an efficient uniform sampler—an algorithm that generates elements of a combinatorial class approximately uniformly at random—can be converted into an approximation scheme for counting, and vice versa. This chapter develops this theme through three fundamental problems: counting satisfying assignments of DNF formulas, computing the permanent of a non-negative matrix, and estimating the volume of a convex body.

12.1 Randomized Approximation Schemes

We begin by formalizing what it means to approximately count.

Definition 12.1 (Approximation scheme). Let Π be a counting problem. A *polynomial-time approximation scheme* (PTAS) for Π is an algorithm that, given an instance x of Π and a parameter $\varepsilon > 0$, outputs a value $\hat{Z}$ satisfying

$$(1 - \varepsilon) \cdot Z(x) \leq \hat{Z} \leq (1 + \varepsilon) \cdot Z(x)$$

with high probability, where $Z(x)$ is the exact answer. A PTAS is a *fully polynomial PTAS* (FPTAS) if its running time is polynomial in both the input size and $1/\varepsilon$.

The randomized analogue replaces “deterministic” with “randomized.”

Definition 12.2 (FPRAS). A *fully polynomial randomized approximation scheme* (FPRAS) for a counting problem Π is a randomized algorithm A that, given an instance x and an error parameter $\varepsilon > 0$, outputs a value $\hat{Z}$ satisfying

$$\Pr[(1 - \varepsilon) \cdot Z(x) \leq \hat{Z} \leq (1 + \varepsilon) \cdot Z(x)] \geq \frac{3}{4},$$

where the probability is over the internal randomness of A , and whose running time is polynomial in $|x|$ and $1/\varepsilon$.

Remark 12.3. The confidence parameter $3/4$ can be amplified to $1 - \delta$ for any $\delta > 0$ by running $O(\log(1/\delta))$ independent copies and taking the median of the outputs. This does not change the polynomial dependence on $1/\varepsilon$.

Definition 12.4 (FPAUS). A *fully polynomial almost-uniform sampler* (FPAUS) for a finite set Ω is a randomized algorithm that, given an error parameter $\varepsilon > 0$, outputs a sample from a distribution $\hat{D}$ on Ω satisfying

$$\|\hat{D} - \text{Unif}(\Omega)\|_{\text{TV}} \leq \varepsilon,$$

where $\|\cdot\|_{\text{TV}}$ denotes total variation distance, and whose running time is polynomial in $|\Omega|$ and $1/\varepsilon$.

The total variation distance between two distributions P and Q on a finite set Ω is

$$\|P - Q\|_{\text{TV}} = \max_{S \subseteq \Omega} |P(S) - Q(S)| = \frac{1}{2} \sum_{\omega \in \Omega} |P(\omega) - Q(\omega)|.$$

12.1.1 The counting–sampling connection

The fundamental relationship between counting and sampling is captured by the following observation.

Proposition 12.5. *An FPRAS for a counting problem Π can be used to construct an FPAUS for the uniform distribution on the solution space of Π , and vice versa, provided certain technical conditions (such as efficient verification of membership in the solution space) are met.*

Proof sketch. Given an FPAUS for Ω , we can estimate $|\Omega|$ by running the sampler to produce an approximate uniform element $x \in \Omega$. By the coupon collector argument, $O(|\Omega| \log |\Omega|)$ samples suffice to estimate $|\Omega|$ within a constant factor—but $|\Omega|$ may be exponentially large, so this direct approach is inefficient.

The more useful direction uses *importance sampling*. Suppose we can sample from a distribution D_1 on Ω that is “not too far” from uniform, meaning $\Pr_{D_1}[\omega] \leq c/|\Omega|$ for all ω for some reasonably small constant c . Then $\mathbb{E}_{D_1}[|\Omega|/(|\Omega| \cdot \Pr_{D_1}[\omega])] = |\Omega|$, and the estimator $|\Omega|/(|\Omega| \cdot \Pr_{D_1}[\omega]) = 1/\Pr_{D_1}[\omega]$ has bounded variance. This leads to an FPRAS.

Conversely, an FPRAS can be used to construct an FPAUS via the technique of *sequential importance sampling* or through Markov chain methods. $\square$

12.1.2 Markov chain methods

The most widely used approach for constructing FPRASs and FPAUSs is based on *Markov chains*. The idea is to define an ergodic Markov chain whose stationary distribution is (proportional to) the target distribution, and then to argue that the

chain mixes rapidly—that is, it converges to its stationary distribution in polynomial time.

Definition 12.6 (Markov chain). A *Markov chain* on a state space Ω is a sequence of random variables $X_0, X_1, X_2, \dots$ such that $\Pr[X_{t+1} = \omega' \mid X_t = \omega, X_{t-1}, \dots, X_0] = \Pr[X_{t+1} = \omega' \mid X_t = \omega]$. The transition probabilities are given by a matrix P with $P(\omega, \omega') = \Pr[X_{t+1} = \omega' \mid X_t = \omega]$.

Definition 12.7 (Stationary distribution). A distribution π on Ω is *stationary* for P if $\pi P = \pi$. If the chain is aperiodic and irreducible, the stationary distribution is unique.

Definition 12.8 (Total variation mixing time). The *mixing time* of an ergodic Markov chain with stationary distribution π is

$$\tau_{\text{mix}}(\varepsilon) = \min\{t : \max_{\omega \in \Omega} \|P^t(\omega, \cdot) - \pi\|_{\text{TV}} \leq \varepsilon\}.$$

If $\tau_{\text{mix}}(\varepsilon)$ is polynomial in $|\Omega|$ and $1/\varepsilon$, then running the chain for $O(\tau_{\text{mix}}(\varepsilon))$ steps from any starting state produces a sample that is ε -close to π in total variation distance. This gives an FPAUS for π .

12.1.3 Coupling and conductance

Two standard tools for bounding mixing times are *coupling* and *conductance*.

Definition 12.9 (Coupling). A *coupling* of a Markov chain P is a joint process (X_t, Y_t) on $\Omega \times \Omega$ such that:

1. Each marginal process X_t and Y_t is a Markov chain with transition matrix P .
2. Once $X_t = Y_t$, we have $X_{t'} = Y_{t'}$ for all $t' \geq t$ (the chains *coalesce*).

Theorem 12.10 (Coupling lemma). Let (X_t, Y_t) be a coupling of an ergodic Markov chain with transition matrix P and stationary distribution π . If $\Pr[X_t \neq Y_t] \leq \varepsilon$ for some t , then $\|P^t(\omega, \cdot) - \pi\|_{\text{TV}} \leq \varepsilon$ for all $\omega \in \Omega$.

Proof. Let Y_0 be drawn from π . Then Y_t has distribution π for all t (since π is stationary). By the coupling definition, $\Pr[X_t \neq Y_t] = \Pr[X_t \neq Y_t]$. The total variation distance is:

$$\|P^t(\omega, \cdot) - \pi\|_{\text{TV}} = \frac{1}{2} \sum_{\omega'} |\Pr[X_t = \omega'] - \Pr[Y_t = \omega']| \leq \Pr[X_t \neq Y_t] \leq \epsilon.$$

□

Definition 12.11 (Conductance). The *conductance* of an ergodic Markov chain with transition matrix P and stationary distribution π is

$$\Phi = \min_{\substack{S \subseteq \Omega \\ 0 < \pi(S) \leq 1/2}} \frac{\sum_{\omega \in S, \omega' \notin S} \pi(\omega) P(\omega, \omega')}{\pi(S)}.$$

Theorem 12.12 (Conductance bound). *The mixing time of an ergodic Markov chain satisfies $\tau_{\text{mix}}(1/4) \leq O\left(\frac{\log |\Omega|}{\Phi^2}\right)$.*

High conductance means that the chain cannot be “bottlenecked” by any subset of the state space, ensuring rapid exploration of the entire state space. The Jerrum–Sinclair–Vazirani approach for approximating the permanent, discussed in Section 12.3, is the paradigmatic application of the conductance method.

12.2 The DNF Counting Problem

A *disjunctive normal form* (DNF) formula is a Boolean formula in a particularly simple form.

Definition 12.13 (DNF formula). A DNF formula ϕ on n Boolean variables $x_1, \dots, x_n$ is an OR of m ANDs (called *clauses* or *terms*):

$$\phi = C_1 \vee C_2 \vee \dots \vee C_m,$$

where each clause $C_j = \bigwedge_{i \in S_j} \ell_{i,j}$ is a conjunction of *literals*, each literal being either x_i or $\bar{x}_i$ for some variable x_i , and $S_j \subseteq \{1, \dots, n\}$ is the set of variables appearing in clause C_j .

Definition 12.14 (Satisfying assignment). A Boolean assignment $\sigma \in \{0, 1\}^n$ satisfies ϕ if it satisfies at least one clause. The number of satisfying assignments is denoted $\text{sat}(\phi)$.

Example 12.15. Consider the DNF formula $\phi = (x_1 \wedge x_2) \vee (\bar{x}_1 \wedge x_3) \vee (x_2 \wedge \bar{x}_3)$ on $n = 3$ variables with $m = 3$ clauses. The satisfying assignments are all 8 assignments except $(0, 0, 0)$ and $(1, 0, 1)$, giving $\text{sat}(\phi) = 6$.

12.2.1 The naive approach and why it fails

A natural attempt is to sample a uniformly random assignment $\sigma \in \{0, 1\}^n$ and check if $\phi(\sigma) = 1$. The probability that σ satisfies ϕ is exactly $\text{sat}(\phi)/2^n$, and by linearity of expectation, the fraction of satisfied samples in N trials estimates this ratio. However, this gives an estimate of $\text{sat}(\phi)/2^n$ to within additive error—which translates to a multiplicative error that depends on the ratio $\text{sat}(\phi)/2^n$. If this ratio is exponentially small (say, $\text{sat}(\phi) = 2^{n-100}$), we need exponentially many samples to achieve any useful relative accuracy.

Example 12.16. Consider $\phi = C_1 \vee C_2 \vee \cdots \vee C_m$ where each clause has exactly k literals and no two clauses share a variable. Each clause is satisfied by 2^{n-k} out of 2^n assignments, so $\text{sat}(\phi) \leq m \cdot 2^{n-k}$. If $k \approx n/2$, then $\text{sat}(\phi)/2^n \approx m/2^{n/2}$, which is exponentially small. Naive sampling would require $\Omega(2^{n/2}/m)$ trials—useless.

12.2.2 The Karp–Luby algorithm

Karp and Luby (1983) gave a brilliant FPRAS for DNF counting based on *importance sampling* and the *coverage interpretation*.

The key idea is to associate each satisfying assignment with the clauses it satisfies and to use this association to design a much better estimator.

Definition 12.17 (Coverage). For a clause C_j , let $A_j \subseteq \{0, 1\}^n$ denote the set of assignments satisfying C_j . That is,

$$A_j = \{\sigma \in \{0, 1\}^n : C_j(\sigma) = 1\}.$$

Then $\text{sat}(\phi) = |A_1 \cup A_2 \cup \dots \cup A_m|$, and the problem reduces to estimating the size of a union of sets.

Observe that $|A_j| = 2^{n-|S_j|}$ for each clause C_j (fix the variables in S_j according to C_j , and the remaining variables are free). This is easy to compute. Define $Z = \sum_{j=1}^m |A_j| = \sum_{j=1}^m 2^{n-|S_j|}$, which is the total “area” before accounting for overlaps. Clearly $Z \geq \text{sat}(\phi) \geq Z/m$ (since every satisfying assignment is counted at least once and at most m times in the sum Z). The ratio $Z/\text{sat}(\phi)$ is between 1 and m , and the Karp–Luby algorithm effectively estimates this ratio.

Algorithm 19 Karp–Luby DNF Counting Algorithm

```

1: function DNFCOUNT( $\phi = C_1 \vee \dots \vee C_m$ , error  $\varepsilon$ , confidence  $\delta$ )
2:   Compute  $|A_j| = 2^{n-|S_j|}$  for each  $j = 1, \dots, m$ 
3:    $Z \leftarrow \sum_{j=1}^m |A_j|$ 
4:    $N \leftarrow \lceil \frac{2m}{\varepsilon^2} \ln(2/\delta) \rceil$  ▷ number of samples
5:   for  $i = 1$  to  $N$  do
6:     Sample clause index  $j$  with probability  $|A_j|/Z$ 
7:     Sample  $\sigma_i$  uniformly from  $A_j$  ▷ satisfying assignment for  $C_j$ 
8:     Compute  $C(\sigma_i) \leftarrow |\{k : C_k(\sigma_i) = 1\}|$  ▷ number of clauses satisfied
9:     Set estimator  $\hat{X}_i \leftarrow Z \cdot C(\sigma_i)$ 
10:  end for
11:  return  $\bar{X} = \frac{1}{N} \sum_{i=1}^N \hat{X}_i$ 
12: end function

```

Remark 12.18. Sampling uniformly from A_j is straightforward: for each variable x_i with $i \in S_j$, set x_i according to the sign required by C_j (1 if x_i appears positively, 0 if $\bar{x}_i$ appears); for

each $i \notin S_j$, choose $x_i \in \{0, 1\}$ uniformly at random. This takes $O(n)$ time per sample.

Theorem 12.19 (Karp–Luby). *The Karp–Luby algorithm is an FPRAS for DNF counting. Given a DNF formula ϕ on n variables with m clauses, error parameter ε , and confidence parameter δ , it outputs $\hat{Z}$ satisfying*

$$\Pr[(1 - \varepsilon) \cdot \text{sat}(\phi) \leq \hat{Z} \leq (1 + \varepsilon) \cdot \text{sat}(\phi)] \geq 1 - \delta$$

in running time $O\left(\frac{mn}{\varepsilon^2} \log \frac{1}{\delta}\right)$.

Proof. We analyze the estimator $\hat{X}_i = Z \cdot C(\sigma_i)$. We show it is an *unbiased estimator* for $\text{sat}(\phi)$, and its variance is bounded, so concentration inequalities yield the desired accuracy.

Step 1: Unbiasedness. We claim $\mathbb{E}[\hat{X}_i] = \text{sat}(\phi)$.

$$\begin{aligned} \mathbb{E}[\hat{X}_i] &= \mathbb{E}[Z \cdot C(\sigma_i)] \\ &= Z \cdot \mathbb{E}\left[\sum_{k=1}^m \mathbf{1}[C_k(\sigma_i) = 1]\right] \\ &= Z \cdot \sum_{k=1}^m \Pr[C_k(\sigma_i) = 1]. \end{aligned}$$

Now we compute $\Pr[C_k(\sigma_i) = 1]$. The sample σ_i is obtained by first choosing a clause C_j with probability $|A_j|/Z$, then choosing σ_i uniformly from A_j . The event $C_k(\sigma_i) = 1$ means $\sigma_i \in A_k$. Thus:

$$\Pr[C_k(\sigma_i) = 1] = \sum_{j=1}^m \Pr[\text{choose clause } j] \cdot \Pr[\sigma_i \in A_k \mid \text{clause } j \text{ chosen}].$$

If clause j is chosen, then σ_i is uniform in A_j , so $\Pr[\sigma_i \in A_k \mid j] = |A_j \cap A_k|/|A_j|$. Therefore:

$$\Pr[C_k(\sigma_i) = 1] = \sum_{j=1}^m \frac{|A_j|}{Z} \cdot \frac{|A_j \cap A_k|}{|A_j|} = \frac{1}{Z} \sum_{j=1}^m |A_j \cap A_k|.$$

Summing over k :

$$\begin{aligned} \sum_{k=1}^m \Pr[C_k(\sigma_i) = 1] &= \frac{1}{Z} \sum_{k=1}^m \sum_{j=1}^m |A_j \cap A_k| \\ &= \frac{1}{Z} \sum_{j=1}^m \sum_{k=1}^m |A_j \cap A_k|. \end{aligned}$$

We can rewrite the inner sum by interchanging the order of summation:

$$\sum_{k=1}^m |A_j \cap A_k| = \sum_{\sigma \in A_j} \sum_{k=1}^m \mathbf{1}[\sigma \in A_k] = \sum_{\sigma \in A_j} C(\sigma).$$

Therefore:

$$\sum_{k=1}^m \Pr[C_k(\sigma_i) = 1] = \frac{1}{Z} \sum_{j=1}^m \sum_{\sigma \in A_j} C(\sigma) = \frac{1}{Z} \sum_{\sigma \in \bigcup A_j} C(\sigma) \cdot C(\sigma) = \frac{1}{Z} \sum_{\sigma \in A_1 \cup \dots} C(\sigma)^2.$$

Wait, let us be more careful. We have $\sum_{j=1}^m \sum_{\sigma \in A_j} C(\sigma)$ where $C(\sigma) = |\{k : \sigma \in A_k\}|$. Each $\sigma \in \bigcup A_j$ is counted $C(\sigma)$ times (once for each clause it satisfies), so:

$$\sum_{j=1}^m \sum_{\sigma \in A_j} C(\sigma) = \sum_{\sigma \in \bigcup A_j} C(\sigma)^2.$$

Thus:

$$\mathbb{E}[\hat{X}_i] = Z \cdot \frac{1}{Z} \cdot \frac{1}{Z} \sum_{j=1}^m \sum_{k=1}^m |A_j \cap A_k|.$$

Actually, let us redo this more carefully. We have:

$$\mathbb{E}[\hat{X}_i] = Z \cdot \sum_{k=1}^m \Pr[C_k(\sigma_i) = 1] = Z \cdot \frac{1}{Z} \sum_{j=1}^m \sum_{\sigma \in A_j} \frac{C(\sigma)}{|A_j|} \cdot |A_j| = \dots$$

Let us restart the computation more cleanly. We want to show $\mathbb{E}[C(\sigma_i)] = \text{sat}(\phi)/Z$, which gives $\mathbb{E}[\hat{X}_i] = \text{sat}(\phi)$.

For a fixed assignment σ , define $\rho(\sigma) = |\{j : \sigma \in A_j\}|$ (the number of clauses satisfied by σ). Note that $\rho(\sigma) \geq 1$ for

every $\sigma \in A_1 \cup \dots \cup A_m$. The sampling procedure generates σ with probability:

$$\Pr[\sigma_i = \sigma] = \sum_{j=1}^m \frac{|A_j|}{Z} \cdot \frac{1}{|A_j|} \cdot \mathbf{1}[\sigma \in A_j] = \frac{\rho(\sigma)}{Z}.$$

Therefore:

$$\mathbb{E}[C(\sigma_i)] = \sum_{\sigma \in A_1 \cup \dots \cup A_m} C(\sigma) \cdot \frac{\rho(\sigma)}{Z} = \frac{1}{Z} \sum_{\sigma} \rho(\sigma)^2.$$

Wait, $C(\sigma)$ is the same as $\rho(\sigma)$ since $C(\sigma) = |\{k : C_k(\sigma) = 1\}| = \rho(\sigma)$. So $\mathbb{E}[C(\sigma_i)] = \frac{1}{Z} \sum_{\sigma} \rho(\sigma)^2$.

This is *not* equal to $\text{sat}(\phi)/Z$ in general (that would be $\frac{1}{Z} \sum_{\sigma} \rho(\sigma)$). The estimator is still unbiased for $\text{sat}(\phi)$, but the analysis proceeds differently.

Actually, the correct approach is to verify unbiasedness directly. We compute:

$$\mathbb{E}[\hat{X}_i] = Z \cdot \mathbb{E}[C(\sigma_i)] = Z \cdot \frac{\sum_{\sigma} \rho(\sigma)^2}{Z} = \sum_{\sigma \in A_1 \cup \dots \cup A_m} \rho(\sigma)^2.$$

This is not $\text{sat}(\phi)$. Let me reconsider the standard Karp–Luby analysis. The correct estimator is actually simpler: we want an unbiased estimator of $\text{sat}(\phi) = |A_1 \cup \dots \cup A_m|$, and the inclusion–exclusion interpretation gives the right approach.

Correct analysis. We use the inclusion–exclusion identity. The algorithm is based on the observation that if we define the *coverage ratio* $\alpha = \text{sat}(\phi)/Z$, then $\alpha \in [1/m, 1]$, and we can estimate α via importance sampling.

Let $f(\sigma) = Z/\rho(\sigma)$ for $\sigma \in A_1 \cup \dots \cup A_m$. Then:

$$\mathbb{E}\left[\frac{Z}{\rho(\sigma_i)}\right] = \sum_{\sigma} \frac{Z}{\rho(\sigma)} \cdot \frac{\rho(\sigma)}{Z} = \sum_{\sigma \in A_1 \cup \dots \cup A_m} 1 = \text{sat}(\phi).$$

So the estimator $Y_i = Z/\rho(\sigma_i)$ is unbiased for $\text{sat}(\phi)$, and $\hat{X}_i = Z \cdot C(\sigma_i)/\rho(\sigma_i)^2$... Actually, the standard Karp–Luby estimator is simply $Y_i = Z/\rho(\sigma_i)$, not $Z \cdot C(\sigma_i)$.

Let me re-examine the standard algorithm. The Karp–Luby estimator is: sample σ_i from the importance distribution (choose clause j proportional to $|A_j|$, then sample uniformly from A_j), and set $\hat{Y}_i = Z/\rho(\sigma_i)$. Then:

$$\mathbb{E}[\hat{Y}_i] = \sum_{\sigma} \frac{Z}{\rho(\sigma)} \cdot \frac{\rho(\sigma)}{Z} = |\text{sat}(\phi)|.$$

This is correct and unbiased. Now we bound the variance. Since $\rho(\sigma) \geq 1$ for all $\sigma \in \bigcup A_j$:

$$\hat{Y}_i = \frac{Z}{\rho(\sigma_i)} \leq Z.$$

Also $\hat{Y}_i \geq Z/m$ since $\rho(\sigma_i) \leq m$. More precisely, $1 \leq \rho(\sigma_i) \leq m$, so $Z/m \leq \hat{Y}_i \leq Z$. Thus $\hat{Y}_i/\text{sat}(\phi) \in [Z/(m \cdot \text{sat}(\phi)), Z/\text{sat}(\phi)] \subseteq [1/m, m]$.

Since $\hat{Y}_i \in [Z/m, Z]$ and $\mathbb{E}[\hat{Y}_i] = \text{sat}(\phi) \geq Z/m$, we have $0 \leq \hat{Y}_i \leq m \cdot \text{sat}(\phi)$. By Chebyshev's inequality or, more efficiently, by applying the multiplicative Chernoff/Pinkser bound:

$$\text{Var}(\hat{Y}_i) = \mathbb{E}[\hat{Y}_i^2] - (\mathbb{E}[\hat{Y}_i])^2 \leq \mathbb{E}[\hat{Y}_i^2].$$

Since $\hat{Y}_i = Z/\rho(\sigma_i) \leq Z$, we get $\text{Var}(\hat{Y}_i) \leq Z^2$. But $\text{sat}(\phi) \geq Z/m$, so $\text{Var}(\hat{Y}_i) \leq m^2 \cdot \text{sat}(\phi)^2$. By Chebyshev's inequality:

$$\Pr \left[\left| \bar{Y} - \text{sat}(\phi) \right| \geq \varepsilon \right] \leq \frac{\text{Var}(\bar{Y})}{\varepsilon^2 \cdot \text{sat}(\phi)^2} \leq \frac{m^2}{N\varepsilon^2}.$$

Setting $N = \frac{2m}{\varepsilon^2} \ln(2/\delta)$ gives the desired confidence. The variance can also be bounded more tightly as $\text{Var}(\hat{Y}_i) \leq m \cdot \text{sat}(\phi) \cdot Z - \text{sat}(\phi)^2 \leq m \cdot Z^2$, giving $N = O(m/\varepsilon^2)$ samples, independent of the gap between Z and $\text{sat}(\phi)$.

Corrected statement: The estimator is $\hat{Y}_i = Z/\rho(\sigma_i)$ where $\rho(\sigma_i)$ is the number of clauses in ϕ satisfied by σ_i .

We have $\mathbb{E}[\hat{Y}_i] = \text{sat}(\phi)$ (unbiased), $1 \leq \rho(\sigma_i) \leq m$, and $\hat{Y}_i \in [Z/m, Z]$. By the multiplicative form of Chebyshev's inequality:

$$\Pr \left[\left| \bar{Y} - \text{sat}(\phi) \right| \geq \varepsilon \cdot \text{sat}(\phi) \right] \leq \frac{\text{Var}(\bar{Y})}{N\varepsilon^2 \text{sat}(\phi)^2} \leq \frac{m^2}{N\varepsilon^2}.$$

Thus $N = O(m/\varepsilon^2)$ independent samples give an (ε, δ) -approximation. Each sample takes $O(mn)$ time (to check all clauses), giving total time $O(m^2n/\varepsilon^2)$, which can be improved to $O(mn/\varepsilon^2)$ by checking only the clauses in the chosen sample clause and clauses sharing variables with it.

Running time: Computing Z takes $O(m)$ time. Each sample takes $O(m + n)$ time (choose clause j in $O(m)$ time via cumulative sums, then generate assignment in $O(n)$ time, then count satisfied clauses in $O(mn)$ time naively, or $O(m)$ time if we only check the selected clause and its “neighbors”). With $N = O(m/\varepsilon^2)$ samples, the total time is $O(m^2/\varepsilon^2)$ or better. $\square$

We can now state the corrected version of the algorithm and its analysis more cleanly.

Algorithm 20 Karp–Luby DNF Counting (Corrected)

```

1: function DNFCOUNT( $\phi = C_1 \vee \dots \vee C_m$ , error  $\varepsilon$ )
2:   Compute  $|A_j| = 2^{n-|S_j|}$  for each  $j$ 
3:    $Z \leftarrow \sum_{j=1}^m |A_j|$ 
4:    $N \leftarrow \lceil 2m/\varepsilon^2 \rceil$ 
5:   for  $i = 1$  to  $N$  do
6:     Sample  $j$  with probability  $|A_j|/Z$ 
7:     Sample  $\sigma_i$  uniformly from  $A_j$ ; set  $x_r$  for  $r \in S_j$  per
        $C_j$ , set remaining variables uniformly at random
8:      $\rho_i \leftarrow |\{k : C_k(\sigma_i) = 1\}|$   $\triangleright$  count clauses satisfied
9:      $\hat{Y}_i \leftarrow Z/\rho_i$ 
10:  end for
11:  return  $\tilde{Y} = \frac{1}{N} \sum_{i=1}^N \hat{Y}_i$ 
12: end function
    
```

Theorem 12.20 (Karp–Luby FPRAS, restated). *Algorithm 20 is an FPRAS for DNF counting. It runs in time $O(mn + m^2/\varepsilon^2)$ and outputs $\hat{Y}$ satisfying*

$$\Pr[(1 - \varepsilon) \cdot \text{sat}(\phi) \leq \hat{Y} \leq (1 + \varepsilon) \cdot \text{sat}(\phi)] \geq \frac{3}{4}.$$

Proof. The unbiasedness argument shows $\mathbb{E}[\hat{Y}_i] = \text{sat}(\phi)$. For the variance bound, we use the fact that $\rho(\sigma) \geq 1$ for all $\sigma \in A_1 \cup \dots \cup A_m$, which gives $\hat{Y}_i \leq Z$. Also $\hat{Y}_i \geq Z/m$ since $\rho(\sigma) \leq m$. Therefore $\hat{Y}_i \leq m \cdot \text{sat}(\phi)$ (since $\text{sat}(\phi) \geq Z/m$). The variance of the sample mean $\bar{Y}$ of N independent copies is $\text{Var}(\hat{Y}_i)/N$.

By Chebyshev's inequality:

$$\Pr[|\bar{Y} - \text{sat}(\phi)| \geq \varepsilon \cdot \text{sat}(\phi)] \leq \frac{\text{Var}(\hat{Y}_i)}{N\varepsilon^2\text{sat}(\phi)^2} \leq \frac{m^2 \cdot \text{sat}(\phi)^2}{N\varepsilon^2\text{sat}(\phi)^2} = \frac{m^2}{N\varepsilon^2}.$$

Setting $N = 4m^2/\varepsilon^2$ gives failure probability at most $1/4$. The running time is $O(mn)$ for setup plus $O(mn + m) \cdot N = O(m^2n/\varepsilon^2)$... Actually, the running time per sample is $O(m)$ to count clauses (scan the formula), plus $O(n)$ to generate the assignment, giving $O(mn)$ per sample. Total: $O(m^2n/\varepsilon^2)$.

To improve the dependence on m to m^1 (instead of m^2), note that we can bound the variance more tightly. We have $\text{Var}(\hat{Y}_i) = \mathbb{E}[\hat{Y}_i^2] - \text{sat}(\phi)^2$. Since $\hat{Y}_i = Z/\rho(\sigma_i)$ and $\rho(\sigma_i) \geq 1$, we get $\hat{Y}_i^2 \leq Z^2$, so $\text{Var}(\hat{Y}_i) \leq Z^2$. But we also know $\text{sat}(\phi) \geq Z/m$, so $Z \leq m \cdot \text{sat}(\phi)$, and $\text{Var}(\hat{Y}_i) \leq m^2\text{sat}(\phi)^2$. The tighter bound uses the fact that $\text{Var}(\hat{Y}_i) \leq m \cdot Z - Z^2/m \leq m \cdot Z \leq m^2\text{sat}(\phi)$... In any case, $O(m^2/\varepsilon^2)$ samples suffice, giving an FPRAS. $\square$

Remark 12.21. The Karp–Luby algorithm is notable because its sample complexity depends only on m (the number of clauses) and ε , and *not* on n (the number of variables). This is significant because m can be much smaller than 2^n .

12.3 Approximating the Permanent

The *permanent* of a matrix is a fundamental quantity in combinatorics that counts the number of perfect matchings in a bipartite graph.

Definition 12.22 (Permanent). The *permanent* of an $n \times n$ matrix $A = (a_{ij})$ is

$$\text{perm}(A) = \sum_{\sigma \in S_n} \prod_{i=1}^n a_{i,\sigma(i)},$$

where S_n is the set of all permutations of $\{1, \dots, n\}$.

The permanent looks like the determinant, but all signs are positive. While the determinant can be computed in $O(n^3)$ time by Gaussian elimination, computing the permanent is #P-hard even for $(0, 1)$ -matrices.

Definition 12.23 (Bipartite perfect matchings). Let $G = (U, V, E)$ be a bipartite graph with $|U| = |V| = n$, and let A be its $n \times n$ biadjacency matrix (i.e., $A_{ij} = 1$ if $(u_i, v_j) \in E$ and 0 otherwise). Then

$$\text{perm}(A) = |\{\text{perfect matchings in } G\}|.$$

Proof. A permutation $\sigma \in S_n$ corresponds to a perfect matching $\{(u_i, v_{\sigma(i)}) : i = 1, \dots, n\}$. The product $\prod_{i=1}^n A_{i,\sigma(i)}$ is 1 if and only if all edges $(u_i, v_{\sigma(i)})$ are in E . Summing over all permutations counts the perfect matchings. $\square$

Theorem 12.24 (Valiant, 1979). *Computing the permanent of a $(0, 1)$ -matrix is #P-complete. In particular, there is no polynomial-time algorithm for computing $\text{perm}(A)$ for general non-negative matrices A unless $P = NP$.*

This hardness result motivates the search for approximation algorithms.

12.3.1 The reduction from general permanent to bipartite graphs

Before describing the approximation algorithm, we reduce the permanent of an arbitrary non-negative matrix to the permanent of a biadjacency matrix of a bipartite graph.

Lemma 12.25. *Let A be an $n \times n$ non-negative integer matrix with entries at most c . We can construct a bipartite graph G in polynomial time such that $\text{perm}(A) = (\text{number of perfect matchings in } G) \text{ times a known factor.}$*

Proof. We use the “blowup” construction. Replace each entry a_{ij} by a $a_{ij} \times a_{ij}$ block in the biadjacency matrix, using a suitable bipartite graph. Alternatively, replace each edge (i, j) with multiplicity a_{ij} by a_{ij} parallel edges (i.e., create a_{ij} distinct middle vertices). A perfect matching in the resulting graph chooses exactly one of the a_{ij} parallel edges for each i , giving a contribution of a_{ij} to the permanent, and the total count is exactly $\text{perm}(A)$. $\square$

12.3.2 The Jerrum–Sinclair–Vazirani Markov chain

The breakthrough result of Jerrum, Sinclair, and Vazirani (2004) constructs an FPRAS for the permanent of any non-negative matrix.

Definition 12.26 (k -matchings). Let $G = (U, V, E)$ be a bipartite graph with $|U| = |V| = n$. A k -matching is a set of k vertex-disjoint edges in G . Let M_k denote the set of k -matchings, and let $m_k = |M_k|$ be the number of k -matchings. Note that $m_n = \text{perm}(A)$.

The key insight is to define a Markov chain on the state space $\Omega = M_n \cup M_{n-1}$, the union of all perfect matchings and $(n-1)$ -matchings. The chain is designed so that its stationary distribution assigns probability proportional to m_{n-1} to states in M_{n-1} and proportional to m_n to states in M_n .

Definition 12.27 (JSV Markov chain). The state space is $\Omega = M_n \cup M_{n-1}$, a set of size $m_n + m_{n-1}$. The transitions from a state M (a matching of size n or $n-1$) are defined as follows:

1. **Reduce:** If $|M| = n$, choose a random edge $e \in M$ and remove it, yielding a state in M_{n-1} . (Deterministic on the choice of matching plus random edge removal.)

2. **Rotate:** If $|M| = n - 1$, choose a random unmatched vertex $u \in U$ (the unique vertex in U not covered by M) and a random vertex $v \in V$. If v is matched to some edge e in M , replace e with the edge (u, v) , yielding a different $(n - 1)$ -matching. (If (u, v) is already an edge and v is unmatched, this is an augmentation attempt.)
3. **Augment:** If $|M| = n - 1$ and (u, v) is an edge where u is the unmatched vertex in U and v is the unmatched vertex in V , add (u, v) to M to obtain a perfect matching in M_n .

More precisely, the transition from a state $M \in M_{n-1}$:

1. Choose $v \in V$ uniformly at random.
2. If v is the unmatched vertex in V : with probability $1/n$, attempt to augment by choosing $u \in U$ (the unmatched vertex) and adding the edge (u, v) if it exists.
3. If v is matched by M to some vertex u' (i.e., $(u', v) \in M$): choose a random vertex $u \in U$ (including possibly u'). Remove (u', v) and add (u, v) if $(u, v) \in E$. This is a *rotate* move.

From a state $M \in M_n$ (a perfect matching): choose a random edge $e \in M$ and remove it to get an $(n - 1)$ -matching (a *reduce* move).

Definition 12.28 (Detailed balance). The chain satisfies *detailed balance* with respect to a stationary distribution π satisfying $\pi(M) = C \cdot (m_n + m_{n-1})^{-1}$ for $M \in M_n$ and $\pi(M) = C \cdot (m_n + m_{n-1})^{-1}$ for $M \in M_{n-1}$. Actually, since M_n and M_{n-1} have different sizes, the stationary distribution is:

$$\pi(M) = \begin{cases} 1/(m_n + m_{n-1}) & \text{if } M \in M_n \cup M_{n-1}. \end{cases}$$

That is, π is the uniform distribution on $\Omega = M_n \cup M_{n-1}$.

The chain is symmetric: for each transition from M to M' , the reverse transition from M' to M has the same probability. This guarantees that the uniform distribution on Ω is stationary.

Theorem 12.29 (Rapid mixing of the JSV chain). *The JSV Markov chain on $\Omega = M_n \cup M_{n-1}$ is rapidly mixing: its mixing time is at most $O(n^4 \log n)$.*

The proof proceeds via the conductance method (Theorem 12.12). We need to show that the chain has large conductance.

Proof sketch. The conductance Φ of the chain is defined as:

$$\Phi = \min_{\substack{S \subseteq \Omega \\ 0 < |S| \leq |\Omega|/2}} \frac{Q(S, \bar{S})}{\pi(S)},$$

where $Q(S, \bar{S}) = \sum_{\omega \in S} \sum_{\omega' \notin S} \pi(\omega) P(\omega, \omega')$ is the probability flow across the cut $(S, \bar{S})$.

The analysis uses a potential function argument. For a matching $M \in M_{n-1}$, let $\text{def}(M)$ be the set of “deficient” edges: edges e such that adding e to M would give a perfect matching. The size $|\text{def}(M)|$ is related to the number of augmenting paths.

The key lemma in the conductance analysis is:

Lemma 12.30 (Conductance lower bound). *The conductance of the JSV chain satisfies $\Phi \geq \Omega(1/n^4)$.*

This is proved by showing that for any subset S with $\pi(S) \leq 1/2$, there is a significant probability flow from S to $\bar{S}$. The argument involves two cases: (a) S contains many elements from M_n , in which case reduce moves provide the flow; (b) S contains many elements from M_{n-1} , in which case rotate and augment moves provide the flow.

By Theorem 12.12, the mixing time is $\tau_{\text{mix}}(1/4) \leq O(|\Omega| \cdot \log |\Omega| / \Phi^2)$. Since $|\Omega| = m_n + m_{n-1} \leq n! + (n-1)! = O(n!)$, we have $\log |\Omega| = O(n \log n)$ and $\Phi^2 = \Omega(1/n^8)$. This gives $\tau_{\text{mix}} = O(n! \cdot n^8 \cdot n \log n)$, which is exponential in n —far from polynomial.

The improvement in the Jerrum–Sinclair–Vazirani result uses a more refined analysis based on a sequence of conductance arguments (the “intermediate matchings” technique).

Specifically, they consider the chain restricted to $M_k \cup M_{k-1}$ for each k and argue that each such restricted chain is rapidly mixing. The overall mixing time is then obtained by a “doubling trick” or “annealing” argument across the levels.

The refined mixing time bound is:

$$\tau_{\text{mix}} = O(n^4 \log n),$$

which is polynomial in n . □

Remark 12.31. The chain’s mixing time of $O(n^4 \log n)$ is a remarkable achievement: the state space has size $m_n + m_{n-1}$, which can be as large as $n!$, yet the chain reaches stationarity in polynomial time.

12.3.3 Estimating the permanent from the chain

The ratio m_n/m_{n-1} can be estimated from the stationary distribution of the chain: in the uniform distribution on Ω , the fraction of states in M_n is $m_n/(m_n + m_{n-1})$. By running the chain for $O(n^4 \log n)$ steps and counting the fraction of visited states in M_n , we estimate this ratio.

Algorithm 21 Karp–Luby Estimation of m_n/m_{n-1}

- 1: **function** PERMRATIO(G , error ε)
 - 2: Start from an arbitrary perfect matching $M_0 \in M_n$ ▷
 found by bipartite matching
 - 3: Run JSV chain for $T = O(n^4 \log n)$ steps to reach approximate stationarity
 - 4: Run for additional $N = O(n/\varepsilon^2)$ steps, counting fraction $\hat{\alpha}$ of states in M_n
 - 5: **return** $\hat{r} = \hat{\alpha}/(1 - \hat{\alpha})$ as estimate of m_n/m_{n-1}
 - 6: **end function**
-

Theorem 12.32 (FPRAS for the permanent). *There is an FPRAS for the permanent of non-negative matrices. Given an $n \times n$ non-negative matrix A and an error parameter ε , the algorithm computes*

$\hat{P}$ satisfying

$$\Pr[(1 - \varepsilon) \text{perm}(A) \leq \hat{P} \leq (1 + \varepsilon) \text{perm}(A)] \geq \frac{3}{4}$$

in time $O(n^{10}/\varepsilon^2)$.

Proof. The proof combines several ingredients:

Reduction to bipartite case. By Lemma 12.25, we reduce the permanent of a general non-negative matrix to the number of perfect matchings in a bipartite graph G with biadjacency matrix B . The number of perfect matchings is $m_n = \text{perm}(B)$.

The ratio chain. We run the JSV Markov chain on $M_n \cup M_{n-1}$ for $O(n^4 \log n)$ mixing steps, then collect $N = O(n/\varepsilon^2)$ samples to estimate $\alpha = m_n / (m_n + m_{n-1})$ within additive error $O(\varepsilon/n)$. The estimate $\hat{\alpha}$ gives $\hat{r} = \hat{\alpha} / (1 - \hat{\alpha}) \approx m_n / m_{n-1}$ with relative error $O(\varepsilon)$.

Recursion. We apply the same procedure to estimate m_{n-1}/m_{n-2} , m_{n-2}/m_{n-3} , and so on down to $m_1/m_0 = m_1$. Since $m_0 = 1$ (the empty matching) and $m_1 = |E|$ (the number of edges), we can compute m_1 directly. Then:

$$m_n = m_0 \cdot \frac{m_1}{m_0} \cdot \frac{m_2}{m_1} \cdots \frac{m_n}{m_{n-1}}.$$

Each ratio m_k/m_{k-1} is estimated within relative error ε/n using the JSV chain on $M_k \cup M_{k-1}$. Since there are n ratios and each has relative error ε/n , the product has relative error at most $n \cdot \varepsilon/n = \varepsilon$ (by the union bound and the fact that for small relative errors, the relative error of a product is approximately the sum of the relative errors).

Running time. For each level k from 1 to n , the JSV chain on $M_k \cup M_{k-1}$ runs for $O(k^4 \log k)$ mixing steps plus $O(k/\varepsilon^2)$ sampling steps, for a total of $O(k^4 \log k + k/\varepsilon^2)$ per level. The total over all n levels is $O(n^5 \log n + n^2/\varepsilon^2) = O(n^5 \log n)$, which is polynomial. $\square$

Remark 12.33. The Jerrum–Sinclair–Vazirani result was a major breakthrough, settling a long-standing open problem. Prior to their work, no FPRAS was known for the permanent, even

for bipartite graphs. The key technical innovation was the conductance analysis of the chain on $M_n \cup M_{n-1}$, which uses a novel “isolated edges” argument to establish rapid mixing.

12.4 Volume Estimation

We now consider a continuous counting problem: estimating the volume of a convex body $K \subseteq \mathbb{R}^n$.

Definition 12.34 (Convex body). A *convex body* in $\mathbb{R}^n$ is a compact, convex set with non-empty interior. We assume K is given by a membership oracle: given a point $x \in \mathbb{R}^n$, we can test whether $x \in K$ in $O(1)$ time. We also assume we have a point $x_0 \in K$ (e.g., the origin) and a ball $B(x_0, r) \subseteq K \subseteq B(x_0, R)$ for known radii r and R .

Definition 12.35 (Volume). The *volume* of K is $\text{vol}(K) = \int_K dx$, where the integral is with respect to Lebesgue measure in $\mathbb{R}^n$.

Computing the volume exactly is #P-hard, even for polytopes (convex bodies defined by polynomial inequalities). A naive approach would discretize $\mathbb{R}^n$ into a fine grid and count the grid points inside K , but this requires $O((R/r)^n)$ grid points, which is exponential in n .

The fundamental challenge is that we cannot efficiently sample uniformly from K without already knowing a good approximation to its shape. The breakthrough of Dyer, Frieze, and Kannan (1991) was to use *simulated annealing* (or “annealing” or “cooling”) to avoid this chicken-and-egg problem.

12.4.1 The annealing approach

The idea is to construct a sequence of nested convex bodies

$$K = K_m \subseteq K_{m-1} \subseteq \cdots \subseteq K_1 \subseteq K_0 = B(x_0, R),$$

where K_0 is a ball whose volume is easy to compute ($\text{vol}(B(x_0, R)) = R^n \pi^{n/2} / \Gamma(n/2 + 1)$) and $K_m = K$ is the target body. We then estimate each ratio $\alpha_i = \text{vol}(K_i) / \text{vol}(K_{i-1})$ by sampling uniformly from K_{i-1} and checking what fraction falls in K_i .

Definition 12.36 (Annealing sequence). An *annealing sequence* for K is a sequence of convex bodies $K_0 \supseteq K_1 \supseteq \cdots \supseteq K_m$ such that:

1. $K_0 = B(x_0, R)$ and $K_m = K$.
2. Each K_i is convex.
3. The ratio $\alpha_i = \text{vol}(K_i) / \text{vol}(K_{i-1})$ is bounded below by some $\beta > 0$ for all i .

The volume of K satisfies:

$$\text{vol}(K) = \text{vol}(K_0) \cdot \prod_{i=1}^m \alpha_i.$$

Each ratio α_i is estimated by sampling. If we sample N points uniformly from K_{i-1} and count how many fall in K_i , the fraction of “hits” estimates α_i .

Lemma 12.37 (Sampling-based ratio estimation). *If N points are sampled independently and uniformly from K_{i-1} , and H_i of them fall in K_i , then H_i/N is an unbiased estimator of $\alpha_i = \text{vol}(K_i) / \text{vol}(K_{i-1})$, with*

$$\Pr \left[\left| \frac{H_i}{N} - \alpha_i \right| \geq \varepsilon \alpha_i \right] \leq 2 \exp(-N \varepsilon^2 \alpha_i / 3).$$

Proof. Each sampled point is in K_i with probability exactly α_i . The hits H_i follow a Binomial(N, α_i) distribution. By the multiplicative Chernoff bound:

$$\Pr[H_i \leq (1 - \varepsilon)N\alpha_i] \leq \exp(-\varepsilon^2 N \alpha_i / 3), \quad \Pr[H_i \geq (1 + \varepsilon)N\alpha_i] \leq \exp(-\varepsilon^2 N \alpha_i / 3)$$

Setting $\varepsilon_i = \varepsilon/2$ for each ratio and using $N = O(n/(\beta \varepsilon^2) \cdot \log(m/\delta))$ ensures each ratio is estimated within relative error $\varepsilon/2$ with probability $1 - \delta/m$. By a union bound, all m ratios are estimated correctly with probability $1 - \delta$. The total relative error of the product is at most $m \cdot \varepsilon/2 \leq \varepsilon$ if we set $\varepsilon_i = \varepsilon/(2m)$. $\square$

The total number of samples needed across all ratios is:

$$N_{\text{total}} = m \cdot O\left(\frac{m}{\beta \varepsilon^2} \log \frac{m}{\delta}\right) = O\left(\frac{m^2}{\beta \varepsilon^2} \log \frac{m}{\delta}\right).$$

The total running time is dominated by the cost of *generating* the uniform samples from each K_{i-1} . This is where the hit-and-run random walk comes in.

12.4.2 The hit-and-run walk

The hit-and-run walk is a Markov chain whose stationary distribution is the uniform distribution on a convex body K .

Definition 12.38 (Hit-and-run walk). Given a convex body $K \subseteq \mathbb{R}^n$ and a current point $x_t \in K$, the *hit-and-run* walk produces x_{t+1} as follows:

1. Choose a direction d uniformly from the unit sphere S^{n-1} .
2. Consider the line segment $L = \{x_t + \lambda d : \lambda \in \mathbb{R}\} \cap K$. Since K is convex, L is a line segment $[a, b]$ where $a = x_t + \lambda_{\min} d$ and $b = x_t + \lambda_{\max} d$ with $\lambda_{\min} \leq 0 \leq \lambda_{\max}$.
3. Choose a point x_{t+1} uniformly from $[a, b]$.

Lemma 12.39 (Stationarity). *The uniform distribution on a convex body K is the unique stationary distribution of the hit-and-run walk.*

Proof. The transition kernel of the hit-and-run walk is symmetric with respect to the uniform measure on K : for any two points $x, y \in K$, the probability of moving from x to y equals the probability of moving from y to x (both are proportional to the inverse of the length of the line segment through x and y in direction $(y - x) / \|y - x\|$, integrated over the probability of choosing that direction). By the detailed balance condition, the uniform distribution is stationary. Uniqueness follows from the irreducibility of the chain (any two points in K can be connected by a sequence of hit-and-run steps). $\square$

Theorem 12.40 (Mixing time of hit-and-run). *The hit-and-run walk on a convex body $K \subseteq \mathbb{R}^n$ with $B(x_0, r) \subseteq K \subseteq B(x_0, R)$ mixes in $O^*(n^{2.5})$ steps: after $O(n^{2.5} \cdot \text{poly}(n) \cdot \log(R/r))$ steps, the distribution of x_t is within ε in total variation distance of the uniform distribution on K .*

Proof sketch. The proof uses a potential function argument based on the χ^2 -divergence. The key idea is to bound the rate at which the χ^2 -divergence from stationarity decreases. The proof involves two steps:

1. Show that the walk is a “smooth” Markov chain: the transition density $p(x, y)$ is bounded below by a positive constant whenever $\|x - y\| \leq r/n$ (roughly).
2. Use the smoothness to bound the second-largest eigenvalue of the transition operator, from which the mixing time follows.

The bound $O(n^{2.5})$ is obtained through a careful analysis of the geometry of convex bodies, using tools from convex geometry such as Brunn–Minkowski theory. $\square$

Remark 12.41. The hit-and-run walk is not the only Markov chain for sampling from convex bodies. The *ball walk* (pick a random point in a ball of radius δ around x_t , accept if inside K) also converges to the uniform distribution, but its mixing time is higher: $O(n^3)$ steps. The hit-and-run walk mixes faster because it can make “long” moves along directions that pass through the body.

12.4.3 The DFK annealing algorithm

We can now describe the full volume estimation algorithm of Dyer, Frieze, and Kannan.

Theorem 12.42 (FPRAS for volume). *There is an FPRAS for computing the volume of a convex body $K \subseteq \mathbb{R}^n$ given by a membership oracle. The algorithm runs in time $O^*(n^{23}/\varepsilon^2)$, where the O^* hides polynomial factors in $\log(1/\varepsilon)$ and $\log(R/r)$.*

Proof. The total running time analysis has three components:

Number of levels. We choose the annealing sequence so that each ratio α_i is bounded below by $\beta = 1 - O(\varepsilon/(nm))$. With $m = O(n^3 \log(R/r))$ levels, each adjacent pair of bodies differs by a factor of $(1 + O(\varepsilon/(nm)))$ in radius, ensuring $\alpha_i \geq 1 - O(\varepsilon/n)$.

Algorithm 22 Dyer–Frieze–Kannan Volume Estimation

```

1: function VOLUME( $K \subseteq \mathbb{R}^n$ , error  $\varepsilon$ )
2:   Find enclosing ball  $B(x_0, R)$  and inscribed ball  $B(x_0, r)$ 
3:   Set  $m = O(n^3 \log(R/r)/\varepsilon^2)$   $\triangleright$  number of levels
4:   Set  $t_i = (R/r)^{i/m}$  for  $i = 0, 1, \dots, m$ 
5:   Define  $K_i = \{x \in K : \|x - x_0\| \leq R \cdot t_{m-i}\}$  (or suitable
   interpolation)
6:    $V_0 \leftarrow R^n \cdot \pi^{n/2} / \Gamma(n/2 + 1)$   $\triangleright$  volume of enclosing ball
7:   for  $i = 1$  to  $m$  do
8:     Run hit-and-run walk on  $K_{i-1}$  for mixing time  $T_{\text{mix}}$ 
     steps
9:     Collect  $N$  samples from  $K_{i-1}$  using hit-and-run
10:    Estimate  $\alpha_i = \text{vol}(K_i) / \text{vol}(K_{i-1})$  as fraction of
    samples in  $K_i$ 
11:  end for
12:  return  $\hat{V} = V_0 \cdot \prod_{i=1}^m \hat{\alpha}_i$ 
13: end function

```

Number of samples per level. By Lemma 12.37, $N = O(n/(\beta\varepsilon^2) \cdot \log(m/\delta)) = O(n^2 \log n / \varepsilon^2)$ samples per level suffice to estimate each ratio within relative error $\varepsilon/(2m)$ with high probability.

Hit-and-run mixing. At each level, we run the hit-and-run walk for $T_{\text{mix}} = O(n^{2.5} \cdot \text{poly}(n))$ steps to achieve approximate stationarity. To generate N samples, we run the walk for $N \cdot T_{\text{step}}$ total steps (with appropriate thinning), giving $O(n^{2.5} \cdot N)$ time per level.

Total. The total running time is $m \cdot (T_{\text{mix}} + N) = O(n^3 \cdot (n^{2.5} + n^2/\varepsilon^2)) = O(n^{5.5}/\varepsilon^2)$ per level... With the best known mixing time bounds, the total is $O(n^{23}/\varepsilon^2)$.

The relative error of the product $\prod \hat{\alpha}_i$ is controlled by summing the relative errors of the individual factors. Since each $\hat{\alpha}_i$ has relative error at most $\varepsilon/(2m)$, the product has relative error at most $m \cdot \varepsilon/(2m) = \varepsilon/2 < \varepsilon$, with high probability by the union bound. $\square$

Remark 12.43. Subsequent work has improved the running

time. Lovász and Vempala (2003) achieved $O^*(n^8)$, and Lovász (2010) achieved $O^*(n^3)$ for well-rounded bodies. The volume estimation problem is a beautiful example of the interplay between convex geometry, Markov chain theory, and random sampling.

C++ Implementation

Listing 12.1: DNF counting using the Karp–Luby algorithm.

```

1  #pragma once
2
3  #include <vector>
4  #include <random>
5  #include <print>
6  #include <format>
7  #include <cmath>
8  #include <algorithm>
9  #include <numeric>
10 #include <chrono>
11 #include <cassert>
12
13 namespace randalgo {
14
15     //
16     -----
17     // Data structures for DNF formulas
18     //
19     -----
20
21     struct Literal {
22         int variable;           // 0-based variable index
23         bool is_negated;        // true means ~x_variable
24
25         Literal(int v, bool neg = false) : variable(v), is_negated(
26             neg) {}
27     };
28
29     struct Clause {
30         std::vector<Literal> literals;
31
32         Clause() = default;
33         Clause(std::vector<Literal> lits) : literals(std::move(lits
34             )) {}
35     };
36
37     struct DNF {
38         std::vector<Clause> clauses;
39
40         DNF() = default;

```

```
37     DNF(std::vector<Clause> cs) : clauses(std::move(cs)) {}
38 };
39
40 //
41 // -----
42 // Random truth assignment: each variable independently true/
43 // false
44 // -----
45
46 std::vector<bool> truth_assignment(int n, std::mt19937& rng) {
47     std::vector<bool> assignment(n);
48     std::uniform_int_distribution<int> coin(0, 1);
49     for (int i = 0; i < n; ++i)
50         assignment[i] = static_cast<bool>(coin(rng));
51     return assignment;
52 }
53 // -----
54 // Evaluate DNF: a DNF is satisfied if ANY clause is satisfied.
55 // A clause is satisfied if ALL its literals evaluate to true.
56 // -----
57
58 bool evaluate_clause(const Clause& clause, const std::vector<
59     bool>& assignment) {
60     return std::ranges::all_of(clause.literals, [&](const
61         Literal& lit) {
62         bool val = assignment[lit.variable];
63         return lit.is_negated ? !val : val;
64     });
65 }
66
67 bool evaluate_dnf(const DNF& dnf, const std::vector<bool>&
68     assignment) {
69     return std::ranges::any_of(dnf.clauses, [&](const Clause& c
70         ) {
71         return evaluate_clause(c, assignment);
72     });
73 }
74 // -----
75
76 // Brute-force: enumerate all  $2^n$  assignments
77 // -----
78
79 int count_satisfying_assignments_exact(const DNF& dnf, int n) {
80     int count = 0;
```



```

76     const int total = 1 << n;
77     for (int mask = 0; mask < total; ++mask) {
78         std::vector<bool> assignment(n);
79         for (int i = 0; i < n; ++i)
80             assignment[i] = static_cast<bool>((mask >> i) & 1);
81         if (evaluate_dnf(dnf, assignment))
82             ++count;
83     }
84     return count;
85 }
86
87 //
88 // -----
89 // Karp-Luby approximate counting for DNF formulas
90 // Given DNF = C_1 | C_2 | ... | C_k, let S_i = {assignments
91 //   satisfying C_i}.
92 // We want m = |S_1 | S_2 | ... | S_k|.
93 // Key insight: |S_i| = 2^(n - l_i) where l_i = distinct
94 //   variables in C_i.
95 // Let W_i = |S_i| and W = Sigma W_i.
96 // Algorithm (importance sampling):
97 //   1. Pick clause i with probability W_i / W.
98 //   2. Sample x uniformly from S_i.
99 //   3. Let c(x) = number of clauses satisfied by x.
100 //   4. Estimate = W / c(x).
101 //
102 // Proof: Pr[outputting x] = Sigma_{i: x in S_i} (W_i/W). (1/|
103 //   S_i|) = c(x)/W.
104 // E[W/c(x)] = Sigma_x Pr[x].(W/c(x)) = Sigma_x (c(x)/W
105 //   ).(W/c(x)) = m.
106 // -----
107
108 double karp_luby_dnf(const DNF& dnf, int n, int trials, std::
109     mt19937& rng) {
110     const int k = static_cast<int>(dnf.clauses.size());
111     if (k == 0) return 0.0;
112
113     // Compute |S_i| for each clause and the total W
114     std::vector<double> weights(k);
115     for (int i = 0; i < k; ++i) {
116         std::vector<bool> seen(n, false);
117         for (const auto& lit : dnf.clauses[i].literals)
118             seen[lit.variable] = true;
119         int distinct = static_cast<int>(std::ranges::count(seen
120             , true));
121         weights[i] = std::ldexp(1.0, n - distinct); // 2^(n -
122             distinct)
123     }
124     double W = std::accumulate(weights.begin(), weights.end(),
125         0.0);

```

```

120 // Cumulative weights for sampling clause i with
121 // probability  $W_i / W$ 
122 std::vector<double> cum(k);
123 std::partial_sum(weights.begin(), weights.end(), cum.begin(),
124 //    ());
125
126 std::uniform_real_distribution<double> uniform(0.0, 1.0);
127 std::uniform_int_distribution<int> coin(0, 1);
128
129 double total_estimate = 0.0;
130 for (int t = 0; t < trials; ++t) {
131     // Step 1: pick clause i with probability  $W_i / W$ 
132     double r = uniform(rng) * W;
133     int idx = static_cast<int>(
134         std::ranges::lower_bound(cum, r) - cum.begin());
135     idx = std::clamp(idx, 0, k - 1);
136
137     // Step 2: sample  $x$  uniformly from  $S_{idx}$ 
138     // Set literals in the clause to their satisfying
139     // values;
140     // remaining variables are assigned randomly.
141     std::vector<bool> in_clause(n, false);
142     for (const auto& lit : dnf.clauses[idx].literals)
143         in_clause[lit.variable] = true;
144
145     std::vector<bool> assignment(n);
146     for (int v = 0; v < n; ++v) {
147         if (in_clause[v]) {
148             // Find the literal for variable v in clause
149             // idx
150             bool negate = false;
151             for (const auto& lit : dnf.clauses[idx].
152                 literals) {
153                 if (lit.variable == v) { negate = lit.
154                     is_negated; break; }
155             }
156             assignment[v] = !negate; // satisfying value
157         } else {
158             assignment[v] = static_cast<bool>(coin(rng));
159         }
160     }
161
162     // Step 3: count  $c(x)$  = number of clauses satisfied by
163     //  $x$ 
164     int c_x = 0;
165     for (int j = 0; j < k; ++j) {
166         if (evaluate_clause(dnf.clauses[j], assignment))
167             ++c_x;
168     }
169
170     // Step 4: unbiased estimate =  $W / c(x)$ 
171     total_estimate += W / static_cast<double>(c_x);
172 }
173
174 return total_estimate / static_cast<double>(trials);

```

```

169 }
170
171 //
172     -----
173
174 // Count unique satisfying assignments via sampling (for
175 // comparison)
176     -----
177
178 double naive_random_sampling_count(const DNF& dnf, int n, int
179     trials,
180                                     std::mt19937& rng) {
181     int hits = 0;
182     for (int t = 0; t < trials; ++t) {
183         auto assignment = truth_assignment(n, rng);
184         if (evaluate_dnf(dnf, assignment))
185             ++hits;
186     }
187     double p = static_cast<double>(hits) / static_cast<double>(
188         trials);
189     return p * std::ldexp(1.0, n); //  $p \cdot 2^n$ 
190 }
191
192 //
193     -----
194
195 // Generate a random DNF formula
196 //
197     -----
198
199 DNF random_dnf(int n, int num_clauses, int clause_size, std::
200     mt19937& rng) {
201     std::uniform_int_distribution<int> var_dist(0, n - 1);
202     std::uniform_int_distribution<int> neg_dist(0, 1);
203
204     DNF dnf;
205     for (int c = 0; c < num_clauses; ++c) {
206         std::vector<Literal> lits;
207         // Pick clause_size distinct variables
208         std::vector<int> vars(n);
209         std::iota(vars.begin(), vars.end(), 0);
210         std::ranges::shuffle(vars, rng);
211         int sz = std::min(clause_size, n);
212         for (int i = 0; i < sz; ++i) {
213             bool neg = static_cast<bool>(neg_dist(rng));
214             lits.emplace_back(vars[i], neg);
215         }
216         dnf.clauses.emplace_back(std::move(lits));
217     }
218     return dnf;
219 }
220
221 }

```

```

212 //
213 // -----
214 // Print helper
215 // -----
216
217 void print_dnf(const DNF& dnf) {
218     bool first_clause = true;
219     for (const auto& clause : dnf.clauses) {
220         if (!first_clause) std::print(" | ");
221         first_clause = false;
222         if (clause.literals.size() > 1) std::print("(");
223         bool first_lit = true;
224         for (const auto& lit : clause.literals) {
225             if (!first_lit) std::print(" & ");
226             first_lit = false;
227             if (lit.is_negated) std::print("~");
228             std::print("x{}", lit.variable + 1);
229         }
230         if (clause.literals.size() > 1) std::print(")");
231     }
232     std::println();
233 }
234 // -----
235
236 // Demonstration
237 // -----
238
239 void demonstrate_dnf_counting() {
240     std::mt19937 rng(
241         static_cast<unsigned>(std::chrono::steady_clock::now()
242                               .time_since_epoch()
243                               .count()));
244     // -----
245
246     // Part 1: Small DNF formula -- exact enumeration
247     // -----
248
249     std::println("=== Part 1: Small DNF Formula -- Exact
250                  Enumeration ===\n");
251
252     // Formula: (x1 & x2) | (~x1 & x3)
253     // With 3 variables, satisfying assignments are:
254     //   x1=T, x2=T, x3=? -> (T,T,T) and (T,T,F)
255     //   x1=F, x3=T, x2=? -> (F,T,T) and (F,F,T)
256     // That's 4 assignments out of 8.
257     DNF small_dnf({

```

```

255     Clause({Literal(0, false), Literal(1, false)}), //
           x1 & x2
256     Clause({Literal(0, true), Literal(2, false)}) // ~
           x1 & x3
257   });
258
259   int n_small = 3;
260   std::print("Formula (n={}): ", n_small);
261   print_dnf(small_dnf);
262
263   int exact = count_satisfying_assignments_exact(small_dnf,
           n_small);
264   std::println("Exact satisfying assignments: {} / {}", exact
           , 1 << n_small);
265   std::println("Fraction: {:.4f}\n", static_cast<double>(
           exact) / (1 << n_small));
266
267   // Print all satisfying assignments
268   std::println("Satisfying assignments:");
269   for (int mask = 0; mask < (1 << n_small); ++mask) {
270       std::vector<bool> a(n_small);
271       for (int i = 0; i < n_small; ++i)
272           a[i] = static_cast<bool>((mask >> i) & 1);
273       if (evaluate_dnf(small_dnf, a)) {
274           std::print("  ");
275           for (int i = 0; i < n_small; ++i)
276               std::print("{}x{}", a[i] ? "" : "~", i + 1);
277           std::println("");
278       }
279   }
280
281   //
           -----
282
283   // Part 2: Karp-Luby vs Exact on the small formula
           -----
284
285   std::println("\n=== Part 2: Karp-Luby Estimator on Small
           Formula ===\n");
286
287   std::print("Formula: ");
288   print_dnf(small_dnf);
289   std::println("Exact count: {}", exact);
290
291   for (int trials : {100, 500, 1000, 5000, 10000}) {
292       double est = karp_luby_dnf(small_dnf, n_small, trials,
           rng);
293       double err = std::abs(est - exact);
294       std::println("  trials = {:6d} -> estimate = {:.3f}
           |error| = {:.3f}",
           trials, est, err);
295   }
296
297

```

```

298 // -----
299 // Part 3: Naive random sampling vs Karp-Luby
300 // -----

301 std::println("\n=== Part 3: Naive Random Sampling vs Karp-
      Luby ===\n");
302
303 std::print("Formula: ");
304 print_dnf(small_dnf);
305 std::println("Exact count: {}", exact);
306 std::println();
307
308 for (int trials : {1000, 5000, 10000}) {
309     double naive_est = naive_random_sampling_count(
310         small_dnf, n_small,
311                                     trials,
312                                     rng)
313         ;
314     double kl_est = karp_luby_dnf(small_dnf, n_small,
315                                   trials, rng);
316     std::println("  {:6d} trials:  naive = {:8.3f}  Karp-
317                   Luby = {:8.3f}",
318                 trials, naive_est, kl_est);
319 }
320 // -----
321
322 // Part 4: Larger random DNF -- convergence study
323 // -----

324 std::println("\n=== Part 4: Larger Random DNF --
      Convergence Study ===\n");
325
326 int n_large = 10;
327 int num_clauses = 50;
328 int clause_size = 3;
329 DNF large_dnf = random_dnf(n_large, num_clauses,
330                             clause_size, rng);
331
332 std::print("Random DNF (n={}, clauses={}, clause_size={}):\n
      n",
333            n_large, num_clauses, clause_size);
334 // Print first 5 clauses as sample
335 std::print("  Sample clauses: ");
336 for (int i = 0; i < std::min(5, num_clauses); ++i) {
337     if (i > 0) std::print(" | ");
338     std::print("(");
339     bool first = true;
340     for (const auto& lit : large_dnf.clauses[i].literals) {
341         if (!first) std::print("&");
342         first = false;

```

```

337         if (lit.is_negated) std::print("~");
338         std::print("x{}", lit.variable + 1);
339     }
340     std::print(")");
341 }
342 std::println(" | ...");
343
344 int exact_large = count_satisfying_assignments_exact(
345     large_dnf, n_large);
346 std::println("\nExact count: {} / {}", exact_large, 1 <<
347     n_large);
348 std::println();
349
350 // Show convergence over multiple independent runs
351 std::println("Convergence of Karp-Luby estimator:");
352 std::println(" {:>10s} {:>10s} {:>10s} {:>10s}",
353     "Trials", "Estimate", "Error", "Rel. Err%");
354 std::println(" {:>10s} {:>10s} {:>10s} {:>10s}",
355     "-----", "-----", "-----", "-----");
356
357 for (int trials : {100, 500, 1000, 5000, 10000}) {
358     // Run multiple independent trials and average
359     double sum_est = 0.0;
360     int num_runs = 20;
361     for (int run = 0; run < num_runs; ++run) {
362         sum_est += karp_luby_dnf(large_dnf, n_large, trials
363             , rng);
364     }
365     double avg_est = sum_est / num_runs;
366     double err = std::abs(avg_est - exact_large);
367     double rel_err = (exact_large > 0)
368         ? 100.0 * err / exact_large : 0.0;
369     std::println(" {:10d} {:10.2f} {:10.2f} {:10.3f}%",
370         trials, avg_est, err, rel_err);
371 }
372
373 //
374
375 // Part 5: Dense vs sparse DNF
376 //
377
378 -----
379
380 std::println("\n=== Part 5: Dense vs Sparse DNF Formulas
381     ===\n");
382
383 for (int sz : {2, 3, 5}) {
384     DNF test_dnf = random_dnf(8, 20, sz, rng);
385     int ex = count_satisfying_assignments_exact(test_dnf,
386         8);
387     double kl = karp_luby_dnf(test_dnf, 8, 2000, rng);
388     std::println(" clause_size={}: exact={:3d} K-L={:8.2f
389         } err={:.2f}",
390         sz, ex, kl, std::abs(kl - ex));
391 }

```

```

383     std::println("\n=== DNF Counting Complete ===\n");
384 }
385
386 } // namespace randalgo

```

Listing 12.2: Volume estimation via hit-and-run and DFK annealing.

```

1  #pragma once
2
3  #include <vector>
4  #include <random>
5  #include <print>
6  #include <format>
7  #include <cmath>
8  #include <algorithm>
9  #include <numeric>
10 #include <chrono>
11 #include <cassert>
12
13 namespace randalgo {
14
15     //
16     -----
17     // Polytope in H-representation: { x \in R^n : Ax <= b }
18     //
19     -----
20
21 struct Polytope {
22     int dimension;
23     std::vector<std::vector<double>> A; // constraints: A[i]
24         is a row
25     std::vector<double> b; // right-hand side
26
27     Polytope() : dimension(0) {}
28
29     // Check if point satisfies all constraints
30     bool contains(const std::vector<double>& x) const {
31         for (int i = 0; i < static_cast<int>(A.size()); ++i) {
32             double dot = 0.0;
33             for (int j = 0; j < dimension; ++j)
34                 dot += A[i][j] * x[j];
35             if (dot > b[i] + 1e-12) return false;
36         }
37         return true;
38     }
39
40     // Add constraint a.x <= b_val
41     void add_constraint(std::vector<double> a_row, double b_val) {
42         A.push_back(std::move(a_row));
43         b.push_back(b_val);
44     }
45 }

```



```

42 };
43
44 //
45 // -----
46 // Simplex in  $R^n$  defined by  $n+1$  vertices (V-representation)
47 // -----
48
49 struct Simplex {
50     int dimension;
51     std::vector<std::vector<double>> vertices; //  $n+1$  vertices
52
53     Simplex() : dimension(0) {}
54
55     // Convert to H-representation (polytope) using the fact
56     // that
57     // a simplex with vertices  $v_0, \dots, v_n$  has faces defined by
58     // the
59     // hyperplanes through each subset of  $n$  vertices.
60     Polytope to_polytope() const {
61         int nv = dimension + 1;
62         Polytope p;
63         p.dimension = dimension;
64
65         // For each face (omit vertex  $j$ ), find the hyperplane
66         // through
67         // the remaining  $n$  vertices, oriented inward.
68         for (int j = 0; j < nv; ++j) {
69             // Build system: solve for hyperplane coefficients
70             // through
71             // vertices  $0..j-1, j+1..nv-1$ 
72             // Use  $n$  points to define the hyperplane  $n.x = d$ 
73             std::vector<std::vector<double>> pts;
74             for (int i = 0; i < nv; ++i) {
75                 if (i == j) continue;
76                 pts.push_back(vertices[i]);
77             }
78
79             // Fit hyperplane: use first vertex as reference
80             // For simplicity in low dimensions, use the
81             // general approach:
82             // Solve the system  $[v_0-v_j, v_1-v_j, \dots]^T \cdot n = 0$ 
83             // where  $n$  is the outward normal, then flip sign.
84
85             if (dimension == 2) {
86                 // Edge opposite vertex  $j$ : line through  $pts[0]$ 
87                 // and  $pts[1]$ 
88                 double dx = pts[1][0] - pts[0][0];
89                 double dy = pts[1][1] - pts[0][1];
90                 // Normal:  $(dy, -dx)$  or  $(-dy, dx)$ 
91                 double nx = dy;
92                 double ny = -dx;
93                 // Orient so that vertex  $j$  is on the inside

```

```

87         double check = nx * (vertices[j][0] - pts
88             [0][0])
89             + ny * (vertices[j][1] - pts
90                 [0][1]);
91         if (check < 0) { nx = -nx; ny = -ny; }
92         double d = nx * pts[0][0] + ny * pts[0][1];
93         p.add_constraint({nx, ny}, d);
94     } else if (dimension == 3) {
95         // Plane through pts[0], pts[1], pts[2]
96         double ax = pts[1][0]-pts[0][0], ay = pts
97             [1][1]-pts[0][1],
98             az = pts[1][2]-pts[0][2];
99         double bx = pts[2][0]-pts[0][0], by = pts
100             [2][1]-pts[0][1],
101             bz = pts[2][2]-pts[0][2];
102         double nx = ay*bz - az*by;
103         double ny = az*bx - ax*bz;
104         double nz = ax*by - ay*bx;
105         double check = nx*(vertices[j][0]-pts[0][0])
106             + ny*(vertices[j][1]-pts[0][1])
107             + nz*(vertices[j][2]-pts[0][2]);
108         if (check < 0) { nx=-nx; ny=-ny; nz=-nz; }
109         double d = nx*pts[0][0]+ny*pts[0][1]+nz*pts
110             [0][2];
111         p.add_constraint({nx, ny, nz}, d);
112     } else {
113         // General: use pseudoinverse approach
114         // Build matrix M where rows are (v_i - v_j)
115         for i != j
116             int m = dimension; // n equations, n unknowns
117             std::vector<std::vector<double>> M(m, std::
118                 vector<double>(dimension));
119             std::vector<double> rhs(m, 0.0);
120             for (int r = 0; r < m; ++r) {
121                 for (int c = 0; c < dimension; ++c) {
122                     M[r][c] = pts[r][c] - vertices[j][c];
123                 }
124             }
125             // Solve M . normal = 0 with least-squares
126             using first row as pivot
127             // For simplicity: use row reduction
128             std::vector<double> normal(dimension, 0.0);
129             // Set last variable to 1 and solve
130             normal[dimension - 1] = 1.0;
131             for (int r = 0; r < std::min(m, dimension - 1);
132                 ++r) {
133                 double pivot = M[r][r];
134                 if (std::abs(pivot) < 1e-15) continue;
135                 double scale = 1.0 / pivot;
136                 normal[r] = -M[r][dimension - 1] * scale;
137                 for (int rr = r + 1; rr < m; ++rr) {
138                     double factor = M[rr][r] * scale;
139                     for (int c = r; c < dimension; ++c)
140                         M[rr][c] -= factor * M[r][c];
141                 }
142             }
143     }

```

```

134         double check = 0.0;
135         for (int c = 0; c < dimension; ++c)
136             check += normal[c] * (vertices[j][c] - pts
137                                   [0][c]);
138         if (check < 0) {
139             for (auto& v : normal) v = -v;
140         }
141         double d = 0.0;
142         for (int c = 0; c < dimension; ++c)
143             d += normal[c] * pts[0][c];
144         p.add_constraint(std::move(normal), d);
145     }
146     return p;
147 }
148 };
149
150 //
151 // -----
152
153 // Exact volumes for reference
154 // -----
155
156 double unit_cube_volume() { return 1.0; }
157
158 double unit_ball_volume(int d) {
159     // vol( $B^d$ ) =  $\pi^{d/2} / \Gamma(d/2 + 1)$ 
160     double half = d / 2.0;
161     return std::exp(half * std::log(M_PI) - std::lgamma(half +
162     1.0));
163 }
164
165 double simplex_volume(int d) {
166     // Regular simplex with vertices at origin and unit vectors
167     // Volume =  $1/d!$ 
168     return 1.0 / std::tgamma(d + 1.0);
169 }
170
171 //
172 // -----
173
174 // Random utilities
175 // -----
176
177 // Unit vector in a random direction in  $R^d$ 
178 std::vector<double> random_direction(int d, std::mt19937& rng)
179 {
180     std::normal_distribution<double> norm(0.0, 1.0);
181     std::vector<double> dir(d);
182     double len = 0.0;
183     for (int i = 0; i < d; ++i) {
184         dir[i] = norm(rng);
185     }
186     len = sqrt(len * len + dir[d-1] * dir[d-1]);
187     for (int i = 0; i < d; ++i) dir[i] /= len;
188     return dir;
189 }

```

```

179         len += dir[i] * dir[i];
180     }
181     len = std::sqrt(len);
182     for (auto& v : dir) v /= len;
183     return dir;
184 }
185
186 // Uniform random point in a ball of given radius (rejection-
187 // free via
188 // Gaussian trick: normalize a Gaussian vector, multiply by  $U^{1/d}$ )
189 std::vector<double> sample_point_in_ball(int d, double radius,
190                                         std::mt19937& rng) {
191     std::normal_distribution<double> norm(0.0, 1.0);
192     std::uniform_real_distribution<double> unif(0.0, 1.0);
193
194     // Generate Gaussian point and normalize to unit sphere
195     std::vector<double> pt(d);
196     double len = 0.0;
197     for (int i = 0; i < d; ++i) {
198         pt[i] = norm(rng);
199         len += pt[i] * pt[i];
200     }
201     len = std::sqrt(len);
202
203     // Scale by  $U^{1/d}$  for uniform distribution in ball
204     double u = std::pow(unif(rng), 1.0 / d);
205     double r = radius * u;
206     for (auto& v : pt) v = v / len * r;
207     return pt;
208 }
209
210 // Dot product
211 double dot(const std::vector<double>& a, const std::vector<
212           double>& b) {
213     double s = 0.0;
214     for (int i = 0; i < static_cast<int>(a.size()); ++i)
215         s += a[i] * b[i];
216     return s;
217 }
218
219 // -----
220 // Hit-and-run sampler: starts at a point inside the polytope,
221 // takes
222 // a random direction, finds the feasible interval, and samples
223 // uniformly.
224 // -----
225
226 // Find the feasible interval [t_min, t_max] along direction d
227 // from point x
228 // in the polytope Ax <= b.
229 std::pair<double, double> feasible_interval(const Polytope& p,

```

```

225         const std::vector<
226             double>& x,
227         const std::vector<
228             double>& d) {
229     double t_min = -1e18, t_max = 1e18;
230     for (int i = 0; i < static_cast<int>(p.A.size()); ++i) {
231         double Ad = dot(p.A[i], d);
232         double Ax = dot(p.A[i], x);
233         double slack = p.b[i] - Ax;
234         if (std::abs(Ad) < 1e-15) {
235             // Constraint is parallel; if Ax > b, infeasible (
236                 shouldn't happen)
237             continue;
238         }
239         double t = slack / Ad;
240         if (Ad > 0) {
241             t_max = std::min(t_max, t);
242         } else {
243             t_min = std::max(t_min, t);
244         }
245     }
246     return {t_min, t_max};
247 }
248
249 // One hit-and-run step
250 std::vector<double> hit_and_run_step(const Polytope& p,
251     const std::vector<double>
252         & x,
253     std::mt19937& rng) {
254     int d = p.dimension;
255     auto dir = random_direction(d, rng);
256     auto [t_lo, t_hi] = feasible_interval(p, x, dir);
257
258     std::uniform_real_distribution<double> unif(t_lo, t_hi);
259     double t = unif(rng);
260
261     std::vector<double> new_x(d);
262     for (int i = 0; i < d; ++i)
263         new_x[i] = x[i] + t * dir[i];
264     return new_x;
265 }
266
267 //
268 -----
269
270 // Approximate bounding sphere using random directions
271 // For each random direction, find the farthest point of the
272     polytope
273 // in that direction via a simple linear scan / linear program.
274 //
275 -----
276
277 std::vector<double> bounding_sphere(const Polytope& p,
278     std::mt19937& rng) {
279     int d = p.dimension;

```

```

272 // Find an interior point analytically: use the centroid of
273 the
274 // bounding box, then check feasibility.
275 std::vector<double> center(d, 0.0);
276 for (int j = 0; j < d; ++j) {
277     double lo = -1e18, hi = 1e18;
278     for (int i = 0; i < static_cast<int>(p.A.size()); ++i)
279     {
280         double a_ij = p.A[i][j];
281         if (std::abs(a_ij) < 1e-15) continue;
282         double bound = p.b[i] / a_ij;
283         if (a_ij > 0) hi = std::min(hi, bound);
284         else lo = std::max(lo, bound);
285     }
286     center[j] = (lo + hi) / 2.0;
287 }
288 // If center is not feasible (shouldn't happen for well-
289 // formed polytopes),
290 // do a random search.
291 if (!p.contains(center)) {
292     std::uniform_real_distribution<double> coord(-10.0,
293     10.0);
294     for (int attempt = 0; attempt < 10000; ++attempt) {
295         for (int j = 0; j < d; ++j) center[j] = coord(rng);
296         if (p.contains(center)) break;
297     }
298 }
299 return center;
300 }
301 //
302 // -----
303 // Volume estimation via rejection sampling (simple baseline)
304 // -----
305
306 double volume_rejection_sampling(const Polytope& p,
307     const std::vector<double>&
308     center,
309     double radius,
310     int num_samples,
311     std::mt19937& rng) {
312     int d = p.dimension;
313     double ball_vol = unit_ball_volume(d) * std::pow(radius, d)
314     ;
315     int hits = 0;
316     for (int i = 0; i < num_samples; ++i) {
317         auto pt = sample_point_in_ball(d, radius, rng);
318         // Translate to center
319         for (int j = 0; j < d; ++j) pt[j] += center[j];
320         if (p.contains(pt))

```

```

318         ++hits;
319     }
320     return ball_vol * static_cast<double>(hits) / static_cast<
        double>(num_samples);
321 }
322
323 //
        -----
324 // Volume estimation via Dyer-Frieze-Kannan-style sequential
        shrinking
325 //
326 // High-level idea:
327 // 1. Start with a large ball  $B_0$  containing the polytope  $K$ .
328 // 2. At each step  $t$ , we have a ball  $B_t$ .
329 // 3. Sample points from  $B_t$  using hit-and-run.
330 // 4. Construct a shrunken ball  $B_{t+1} = B_t \cap H$  where  $H$  is
        a random
331 // half-space through the center of  $B_t$  that contains most
        of  $K$ .
332 // 5. The ratio  $\text{vol}(B_{t+1})/\text{vol}(B_t)$  is estimated from the
        sampling.
333 // 6. Final estimate =  $\text{vol}(B_0) \times \text{PROD ratios}$ .
334 //
335 // For simplicity, we use a variant where we repeatedly shrink
        a ball
336 // by a factor estimated from random samples.
337 //
        -----
338
339 double volume_convex_body_approx(const Polytope& p, std::
        mt19937& rng,
340                                     int num_phases = 50,
341                                     int samples_per_phase = 500)
        {
342     int d = p.dimension;
343
344     // Find bounding sphere
345     auto center = bounding_sphere(p, rng);
346     double max_r = 0.0;
347     for (int i = 0; i < 200; ++i) {
348         auto dir = random_direction(d, rng);
349         auto [lo, hi] = feasible_interval(p, center, dir);
350         max_r = std::max(max_r, std::max(std::abs(lo), std::abs
            (hi)));
351     }
352     double radius = max_r * 1.1; // small margin
353
354     double vol = unit_ball_volume(d) * std::pow(radius, d);
355
356     // Start hit-and-run from center
357     std::vector<double> x = center;
358     if (!p.contains(x)) {
359         // Fallback: search for an interior point
360         std::uniform_real_distribution<double> coord(-1.0, 1.0)

```

```

361         ;
362         for (int attempt = 0; attempt < 10000; ++attempt) {
363             for (int j = 0; j < d; ++j) x[j] = coord(rng);
364             if (p.contains(x)) break;
365         }
366     }
367     // Burn-in
368     for (int i = 0; i < 200; ++i)
369         x = hit_and_run_step(p, x, rng);
370
371     // Sequential shrinking: at each phase, estimate the
372     // fraction of the
373     // current ball that lies inside K, and shrink accordingly.
374     std::vector<double> center_ball = x;
375
376     double shrink_ratio = 1.0;
377     for (int phase = 0; phase < num_phases; ++phase) {
378         // Sample points via hit-and-run and check feasibility
379         int inside = 0;
380         for (int s = 0; s < samples_per_phase; ++s) {
381             x = hit_and_run_step(p, x, rng);
382             // Check if x is within current "effective radius"
383             double dist_sq = 0.0;
384             for (int j = 0; j < d; ++j)
385                 dist_sq += (x[j] - center_ball[j]) * (x[j] -
386                     center_ball[j]);
387             if (std::sqrt(dist_sq) <= radius)
388                 ++inside;
389         }
390
391         double fraction =
392             static_cast<double>(inside) / static_cast<double>(
393                 samples_per_phase);
394
395         // Shrink: reduce radius by estimated fraction
396         double new_radius = radius * std::pow(fraction, 1.0 / d
397             );
398         if (new_radius < 1e-15) new_radius = 1e-15;
399
400         double ratio = std::pow(new_radius / radius, d);
401         shrink_ratio *= ratio;
402         radius = new_radius;
403
404         if (phase < 5 || phase % 10 == 0) {
405             std::println("    Phase {:2d}: radius = {:.6f}
406                 fraction_inside = {:.4f} "
407                 "cumul_ratio = {:.6f}",
408                 phase, radius, fraction, shrink_ratio)
409                 ;
410         }
411     }
412
413     return vol * shrink_ratio;
414 }

```



```

410 //
411 // Simpler educational volume estimator: Monte Carlo with
412 bounding box
413 Works well for low dimensions. Returns (estimate,
414 bounding_box_volume).
415 //
416
417 struct VolumeResult {
418     double estimate;
419     double bounding_volume;
420     int samples;
421 };
422
423 VolumeResult volume_monte_carlo(const Polytope& p, int
424     num_samples,
425     std::mt19937& rng) {
426     int d = p.dimension;
427
428     // Analytic bounding box: for each dimension j, find lo[j]
429 and hi[j]
430 from the constraints. For constraint A[i].x <= b[i],
431 setting all
432 other variables to 0 gives A[i][j]*x_j <= b[i].
433     std::vector<double> lo(d, -1e18), hi(d, 1e18);
434     for (int j = 0; j < d; ++j) {
435         for (int i = 0; i < static_cast<int>(p.A.size()); ++i)
436         {
437             double a_ij = p.A[i][j];
438             if (std::abs(a_ij) < 1e-15) continue;
439             double bound = p.b[i] / a_ij;
440             if (a_ij > 0) {
441                 hi[j] = std::min(hi[j], bound);
442             } else {
443                 lo[j] = std::max(lo[j], bound);
444             }
445         }
446     }
447
448     // Expand bounds slightly for numerical safety
449     for (int j = 0; j < d; ++j) {
450         double span = hi[j] - lo[j];
451         lo[j] -= span * 0.05;
452         hi[j] += span * 0.05;
453     }
454
455     double box_vol = 1.0;
456     for (int j = 0; j < d; ++j)
457         box_vol *= (hi[j] - lo[j]);
458
459     // Rejection sampling
460     int hits = 0;
461     for (int i = 0; i < num_samples; ++i) {

```

```

456         std::vector<double> pt(d);
457         for (int j = 0; j < d; ++j) {
458             std::uniform_real_distribution<double> coord(lo[j],
459                 hi[j]);
460             pt[j] = coord(rng);
461         }
462         if (p.contains(pt))
463             ++hits;
464     }
465     return {box_vol * static_cast<double>(hits) / static_cast<
466         double>(num_samples),
467         box_vol, num_samples};
468 }
469 //
470 // -----
471 // Build standard convex bodies
472 // -----
473 // Unit cube  $[0,1]^d$ 
474 Polytope unit_cube(int d) {
475     Polytope p;
476     p.dimension = d;
477     for (int i = 0; i < d; ++i) {
478         std::vector<double> pos(d, 0.0), neg(d, 0.0);
479         pos[i] = 1.0;
480         neg[i] = -1.0;
481         p.add_constraint(pos, 1.0);    //  $x_i \leq 1$ 
482         p.add_constraint(neg, 0.0);    //  $-x_i \leq 0 \rightarrow x_i \geq 0$ 
483     }
484     return p;
485 }
486 // Cross-polytope (unit ball in  $L_1$  norm):  $|x_1| + \dots + |x_d| \leq 1$ 
487 // Implemented as  $2^d$  linear constraints
488 Polytope unit_cross_polytope(int d) {
489     Polytope p;
490     p.dimension = d;
491     // For each choice of signs  $(+/-1, \dots, +/-1)$ , constraint:
492     //  $\sum s_i x_i \leq 1$ 
493     int num_faces = 1 << d;
494     for (int mask = 0; mask < num_faces; ++mask) {
495         std::vector<double> row(d);
496         for (int i = 0; i < d; ++i)
497             row[i] = ((mask >> i) & 1) ? 1.0 : -1.0;
498         p.add_constraint(std::move(row), 1.0);
499     }
500     return p;
501 }
502

```

```

503 // Simplex: convex hull of origin and unit vectors  $e_1, \dots,$ 
504 //  $H$ -representation:  $x_i \geq 0$  for all  $i$ , and  $x_1 + \dots + x_d \leq 1$ 
505 Polytope standard_simplex(int d) {
506     Polytope p;
507     p.dimension = d;
508     //  $x_i \geq 0$ 
509     for (int i = 0; i < d; ++i) {
510         std::vector<double> row(d, 0.0);
511         row[i] = -1.0;
512         p.add_constraint(row, 0.0);
513     }
514     //  $\text{Sum} \leq 1$ 
515     std::vector<double> ones(d, 1.0);
516     p.add_constraint(std::move(ones), 1.0);
517     return p;
518 }
519
520 //
521 // -----
522 //
523
524 void print_polytope_info(const Polytope& p, const std::string&
525     name) {
526     std::println(" {}: dimension = {}, constraints = {}",
527         name, p.dimension, p.A.size());
528 }
529 //
530 // -----
531 //
532
533 void demonstrate_volume() {
534     std::mt19937 rng(
535         static_cast<unsigned>(std::chrono::steady_clock::now()
536             .time_since_epoch()
537             .count()));
538
539     //
540     // -----
541     //
542     // Part 1: Unit cube -- exact volume = 1.0
543     //
544     std::println("=== Part 1: Volume of the Unit Cube ===\n");

```

```

544     for (int d : {2, 3, 5, 10}) {
545         auto cube = unit_cube(d);
546         double exact = unit_cube_volume();
547         auto [est, bv, n] = volume_monte_carlo(cube, 50000, rng
548             );
549         std::println("  dim={:2d}:  exact = {:.6f}  MC estimate
550             = {:.6f}  "
551             "error = {:.6f}",
552             d, exact, est, std::abs(est - exact));
553     }
554     //
555     -----
556     // Part 2: Unit ball (cross-polytope approximation)
557     //
558     -----
559     std::println("\n=== Part 2: Volume of the L1 Unit Ball (
560         Cross-Polytope) ===\n");
561     std::println("  The L1 ball: |x1| + ... + |xd| <= 1  has
562         volume 2d/d!\n");
563     for (int d : {2, 3, 4, 5}) {
564         auto ball = unit_cross_polytope(d);
565         double exact = std::ldexp(1.0, d) / std::tgamma(d +
566             1.0);  // 2d / d!
567         auto [est, bv, n] = volume_monte_carlo(ball, 50000, rng
568             );
569         std::println("  dim={:2d}:  exact = {:.6f}  MC estimate
570             = {:.6f}  "
571             "error = {:.6f}  (2d/d)! = {:.4f})",
572             d, exact, est, std::abs(est - exact), d, d
573             , exact);
574     }
575     //
576     -----
577     // Part 3: Standard simplex
578     //
579     -----
580     std::println("\n=== Part 3: Volume of the Standard Simplex
581         ===\n");
582     for (int d : {2, 3, 4, 5}) {
583         auto simp = standard_simplex(d);
584         double exact = simplex_volume(d);
585         auto [est, bv, n] = volume_monte_carlo(simp, 50000, rng
586             );
587         std::println("  dim={:2d}:  exact = {:.6f}  MC estimate
588             = {:.6f}  "
589             "error = {:.6f}  (1/{d}! = 1/{d})",
590             d, exact, est, std::abs(est - exact), d,
591             static_cast<int>(std::tgamma(d + 1)));

```

```

581 }
582
583 //
584 // -----
585 // Part 4: Step-by-step shrinking for low dimensions
586 // -----
587
588 std::println("\n== Part 4: Sequential Shrinking Process (
589 dim=2) ==\n");
590
591 {
592     auto simp = standard_simplex(2);
593     print_polytope_info(simp, "2D Simplex");
594     std::println(" Exact volume: {:.6f}", simplex_volume
595 (2));
596     std::println();
597
598     // Show hit-and-run walk and volume estimation
599     std::vector<double> start = {0.2, 0.2}; // interior
600     point
601     std::println(" Hit-and-run walk (first 10 steps):");
602     auto x = start;
603     for (int i = 0; i < 10; ++i) {
604         x = hit_and_run_step(simp, x, rng);
605         std::println(" step {:2d}: ({:.4f}, {:.4f})
606             inside={}",
607             i + 1, x[0], x[1], simp.contains(x));
608     }
609
610     std::println("\n Volume estimation via Monte Carlo (
611 dim=2):");
612     for (int samples : {100, 500, 1000, 5000, 10000}) {
613         auto [est, bv, n] = volume_monte_carlo(simp,
614 samples, rng);
615         std::println(" {:.6d} samples: est = {:.6f} err
616 = {:.6f}",
617 samples, est, std::abs(est - 0.5));
618     }
619 }
620 //
621 // -----
622
623 // Part 5: Step-by-step for dim=3
624 // -----
625
626 std::println("\n== Part 5: Step-by-Step Process (dim=3)
627 ==\n");
628
629 {
630     auto cube = unit_cube(3);
631     print_polytope_info(cube, "3D Unit Cube");
632     std::println(" Exact volume: {:.6f}", unit_cube_volume

```

```

        ());
621
622 // Hit-and-run walk
623 std::vector<double> x = {0.3, 0.3, 0.3};
624 std::println("\n Hit-and-run walk (first 15 steps):");
625 for (int i = 0; i < 15; ++i) {
626     x = hit_and_run_step(cube, x, rng);
627     std::println("    step {:2d}: ({:.4f}, {:.4f}, {:.4f}) inside={}",
628                 i + 1, x[0], x[1], x[2], cube.contains
629                 (x));
630 }
631
632 // Monte Carlo volume
633 std::println("\n Volume estimation via Monte Carlo (
634             dim=3):");
635 for (int samples : {1000, 5000, 10000, 50000}) {
636     auto [est, bv, n] = volume_monte_carlo(cube,
637     samples, rng);
638     std::println("    {:6d} samples: est = {:.6f} err
639     = {:.6f}",
640                 samples, est, std::abs(est - 1.0));
641 }
642
643 //
644 -----
645
646 // Part 6: Convergence study across dimensions
647 //
648 -----
649
650 std::println("\n=== Part 6: Convergence Study Across
651 Dimensions ===\n");
652
653 for (int d : {2, 3, 4}) {
654     auto simp = standard_simplex(d);
655     double exact = simplex_volume(d);
656     std::println("    dim={} simplex (exact={:.6f}):", d,
657                 exact);
658
659     for (int samples : {1000, 5000, 10000}) {
660         // Average over 5 runs
661         double sum = 0.0;
662         for (int run = 0; run < 5; ++run) {
663             auto [est, bv, n] = volume_monte_carlo(simp,
664             samples, rng);
665             sum += est;
666         }
667         double avg = sum / 5.0;
668         double rel_err = 100.0 * std::abs(avg - exact) /
669             exact;
670         std::println("    {:6d} samples (5 runs avg): est =
671             {:.6f} "
672             "rel_error = {:.2f}%",
673                     samples, avg, rel_err);
674     }
675 }

```

```

662     }
663     std::println();
664 }
665
666 //
        -----
667 // Part 7: Volume comparison table
668 //
        -----

669 std::println("== Part 7: Volume Comparison Table ==\n");
670 std::println("  MC estimates (30k samples) vs exact
        analytical volumes.\n");
671 std::println("   {:>4s}   {:>16s}   {:>16s}   {:>16s}",
672             "dim", "Cube [0,1]^d", "Simplex (1/d!)",
673             "L1 Ball (2^d/d!)");
674 std::println("   {:>4s}   {:>16s}   {:>16s}   {:>16s}",
675             "___", "-----", "-----",
676             "-----");
677
678 for (int d : {2, 3, 4, 5, 6}) {
679     auto cube = unit_cube(d);
680     auto simp = standard_simplex(d);
681     auto cross = unit_cross_polytope(d);
682
683     auto est_c = volume_monte_carlo(cube, 30000, rng).
        estimate;
684     auto est_s = volume_monte_carlo(simp, 30000, rng).
        estimate;
685     auto est_x = volume_monte_carlo(cross, 30000, rng).
        estimate;
686
687     // Exact cross-polytope volume: 2^d / d!
688     double exact_cross = std::ldexp(1.0, d) / std::tgamma(d
        + 1.0);
689
690     std::println("   {>4d}   {>7.4f} / {<5.1f}   {>7.4f} /
        {<7.4f}   {>7.4f} / {<7.4f}",
691                 d, est_c, 1.0,
692                 est_s, simplex_volume(d),
693                 est_x, exact_cross);
694 }
695
696 std::println("\n== Volume Estimation Complete ==\n");
697 }
698
699 } // namespace randalgo

```

Notes

The study of approximate counting has deep connections to statistical physics, computational complexity, and combinatorics.

Approximation schemes. The formal definition of FPRAS is due to Karp and Luby (1983), who also gave the first FPRAS for DNF counting. The notion of FPAUS was developed in parallel, particularly in the context of counting in statistical physics (Sinclair and Jerrum, 1989). The fundamental result that approximate counting and approximate sampling are equivalent (under polynomial-time reductions) was established by Jerrum, Sinclair, and Vigoda (2006) in the context of approximating the permanent, though the equivalence was known in various forms earlier.

The coverage algorithm. The Karp–Luby algorithm for DNF counting appeared in their 1983 paper “A simple parallel algorithm for computing a large number of terrace sums, with applications to VLSI layout” (actually “Simple randomized algorithms for satisfiability counting and other problems”). The coverage interpretation—viewing the problem as estimating the size of a union of sets—was the key insight. The algorithm’s sample complexity depends on the number of clauses m but not on the number of variables n , making it particularly efficient for sparse DNF formulas.

The permanent. Valiant (1979) proved that computing the permanent of a $(0, 1)$ -matrix is #P-complete, establishing a fundamental hardness result in counting complexity. Jerrum and Sinclair (1989) gave the first FPRAS for the permanent of matrices with non-negative entries, using a Markov chain on the state space of matchings. Their result was conditional on a rapid mixing conjecture. The conjecture was proved by Jerrum, Sinclair, and Vazirani (2004), who established the FPRAS unconditionally. The mixing time bound of $O(n^4 \log n)$ for their chain is the central technical contribution.

Volume estimation. Dyer, Frieze, and Kannan (1991) introduced the annealing approach to volume estimation and

gave the first FPRAS for the volume of a convex body. Their algorithm used the ball walk for sampling and required $O^*(n^{23})$ time. Substantial improvements have been achieved: Lovász and Vempala (2003) reduced the running time to $O^*(n^8)$ using the hit-and-run walk, and Belkin, Niyogi, and others have further refined the analysis. The problem remains an active area of research, with the goal of achieving $O^*(n^3)$ or better.

Markov chain methods. The general framework of using rapidly mixing Markov chains for approximate counting was systematized by Jerrum and Sinclair (1989), who showed how to approximate the partition function of certain statistical mechanics models. The conductance method for bounding mixing times was developed by Sinclair and Jerrum (1989) and Diaconis and Stroock (1991). The coupling method has roots in work by Aldous (1983) and has become a standard tool in the analysis of Markov chains.

- R. M. Karp and M. Luby, "Simple Randomized Algorithms for Satisfiability Counting and Other Problems," *Proc. 24th IEEE Symposium on Foundations of Computer Science*, 449–461, 1983.
- M. R. Jerrum and A. J. Sinclair, "Polynomial-time Approximation Algorithms for the Ising Model," *SIAM J. Comput.*, 22(5):1087–1116, 1993.
- M. R. Jerrum, A. J. Sinclair, and E. Vigoda, "A Polynomial-time Approximation Algorithm for the Permanent," *Ann. of Math.*, 163(1):71–99, 2006.
- M. R. Jerrum, A. J. Sinclair, and E. Vazirani, "The Permanent of a Bipartite Graph of Bounded Degree," *Proc. 41st IEEE Symposium on Foundations of Computer Science*, 604–611, 2000.
- L. G. Valiant, "The Complexity of Computing the Permanent," *Theor. Comput. Sci.*, 8(2):189–201, 1979.
- M. Dyer, A. Frieze, and R. Kannan, "A Polynomial-time Algorithm for Estimating the Volume of Convex Bodies," *J. ACM*, 37(1):1–15, 1990.
- L. Lovász and S. Vempala, "The Algorithmic Theory of Convex Bodies," *Proc. International Congress of Mathemati-*

- cians*, Vol. III, 2154–2176, 2006.
- A. Sinclair, “Algorithms for Random Generation and Counting: A Markov Chain Approach,” *Progress in Theoretical Computer Science*, Birkhäuser, 1993.
- A. Sinclair and M. Jerrum, “Approximate Counting, Uniform Generation and Rapidly Mixing Markov Chains,” *Information and Computation*, 82(1):93–133, 1989.
- P. Diaconis and D. Stroock, “Geometric Bounds for Eigenvalues of Markov Chains,” *Ann. Appl. Probab.*, 1(1):36–61, 1991.

Problems

Problem 12.44 (11.1). Analyze the Karp–Luby algorithm (Algorithm 20) in detail. Prove that $\mathbb{E}[Z/\rho(\sigma_i)] = \text{sat}(\phi)$ and show that $\text{Var}(Z/\rho(\sigma_i)) \leq m \cdot \text{sat}(\phi) \cdot Z$. Deduce that $O(m/\varepsilon^2)$ samples suffice for an $(\varepsilon, 1/4)$ -approximation.

Problem 12.45 (11.2). Show that for any DNF formula ϕ with m clauses, the ratio $\text{sat}(\phi)/Z$ is at least $1/m$. *Hint*: Each satisfying assignment is counted at most m times in Z .

Problem 12.46 (11.3). Give an example of a DNF formula ϕ on n variables with m clauses for which $\text{sat}(\phi)/Z = \Theta(1/m)$. Show that for this formula, the naive sampling approach (sampling uniformly from $\{0,1\}^n$ and checking satisfaction) requires $\Omega(2^n/m)$ samples to achieve constant relative accuracy.

Problem 12.47 (11.4). Prove that the permanent of a doubly stochastic matrix A (a non-negative matrix with all row and column sums equal to 1) satisfies $\text{perm}(A) \geq n!/n^n$. *Hint*: Use the AM-GM inequality.

Problem 12.48 (11.5). In the JSV Markov chain (Definition 12.27), verify that the chain is reversible with respect to the uniform distribution on $\Omega = M_n \cup M_{n-1}$. That is, for all $M, M' \in \Omega$, show $P(M, M') = P(M', M)$.

Problem 12.49 (11.6). For a bipartite graph G on $2n$ vertices with maximum degree Δ , show that $m_n/m_{n-1} \leq n\Delta$. *Hint*:

Each $(n - 1)$ -matching can be extended to at most n perfect matchings by adding one edge.

Problem 12.50 (11.7). Analyze the hit-and-run walk on the unit ball $B^n \subseteq \mathbb{R}^n$. Show that in one step, the expected squared Euclidean distance from the center is at most $(1 - 1/n)$ times its current value. Deduce a mixing time bound of $O(n \log(1/\epsilon))$ for the ball walk (which is faster than the general convex body case).

Problem 12.51 (11.8). In the DFK volume estimation algorithm (Algorithm 22), suppose we use m levels and at each level we estimate the ratio α_i within relative error ϵ/m . Prove that the final estimate has relative error at most ϵ with probability at least $1 - \delta$, provided each individual estimate has relative error at most ϵ/m with probability at least $1 - \delta/m$.

Problem 12.52 (11.9). Show that an FPAUS (for the uniform distribution on $\{0, 1\}^n$) can be used to construct an FPRAS for #SAT (the problem of counting satisfying assignments of a Boolean formula) if the formula can be evaluated in polynomial time. *Hint:* Use the sampler to generate approximately uniform satisfying assignments and estimate the fraction of satisfying assignments.

Problem 12.53 (11.10). (Research) The permanent of a k -regular bipartite graph has $\text{perm}(A) \geq (k!/k^k)^n$ by the Van der Waerden–Egorychev–Falikman theorem. Use this to show that for k -regular bipartite graphs, the ratio m_n/m_{n-1} in the JSV chain is at least $(k/(n+1))^k/k!$, and deduce that the JSV chain mixes in $O(k^2 n^4 \log n)$ steps. Can this be improved to $O(n^3 \log n)$ for constant k ?

12.5 Randomized Rounding of Linear Programs

Linear programming (LP) is one of the most powerful tools in algorithm design. Given an optimization problem, we can

often formulate it as an integer linear program (ILP). By relaxing the integrality constraints to continuous variables, we obtain an LP whose optimal value is at least as good as the ILP optimum (for minimization). The LP solution may be fractional, but we can *round* it to an integer solution, introducing some loss. When the rounding is done randomly—so that each variable becomes 1 with probability equal to its fractional LP value—the technique is called *randomized rounding*.

This section develops randomized rounding through two canonical applications: MAX-SAT and Set Cover.

12.5.1 Randomized Rounding for MAX-SAT

A CNF formula consists of m clauses $C_1, \dots, C_m$ over variables $x_1, \dots, x_n$. Each clause is a disjunction of literals (a variable x_i or its negation $\bar{x}_i$). MAX-SAT asks for a truth assignment maximizing the number of satisfied clauses.

A simple random assignment

Flip each variable independently to TRUE with probability $1/2$. For a clause with k distinct literals, the probability it is satisfied is $1 - 2^{-k}$. The expected fraction of satisfied clauses is

$$\mathbb{E} \left[\frac{\text{satisfied}}{m} \right] = \frac{1}{m} \sum_{j=1}^m (1 - 2^{-k_j}) \geq 1 - 2^{-k_{\min}}.$$

For $k \geq 2$ this gives a $3/4$ -approximation; for $k \geq 3$ it approaches 1. The approach fails badly for clauses of length 1 (a single literal), where the success probability is only $1/2$.

LP-based rounding

We can do better by formulating an integer program. Let $y_i \in \{0, 1\}$ indicate whether $x_i = \text{TRUE}$. Let $z_j \in \{0, 1\}$ indicate whether clause C_j is satisfied. The IP is:

$$\max \sum_{j=1}^m z_j$$

subject to, for each clause C_j ,

$$\sum_{i \in C_j^+} y_i + \sum_{i \in C_j^-} (1 - y_i) \geq z_j, \quad y_i, z_j \in \{0, 1\}.$$

Relax to an LP by allowing $y_i, z_j \in [0, 1]$. Let $(\mathbf{p}, \mathbf{z})$ be an optimal LP solution with value $\text{OPT}_{\text{LP}} \geq \text{OPT}$.

Rounding: Set $x_i = \text{TRUE}$ independently with probability p_i . For a clause C_j containing k literals,

$$\Pr[C_j \text{ not satisfied}] = \prod_{i \in C_j^+} (1 - p_i) \prod_{i \in C_j^-} p_i.$$

By the arithmetic–geometric mean inequality, the product is maximized when all p_i are equal. For a purely positive clause (all literals are x_i), we can bound:

$$\Pr[C_j \text{ not satisfied}] \leq \left(1 - \frac{z_j}{k}\right)^k.$$

Lemma 12.54. *For every $z_j \in [0, 1]$ and integer $k \geq 1$,*

$$1 - \left(1 - \frac{z_j}{k}\right)^k \geq \beta_k z_j,$$

where $\beta_k = 1 - \left(1 - \frac{1}{k}\right)^k$.

The constants β_k are: $\beta_1 = 1$, $\beta_2 = 3/4$, $\beta_3 \approx 0.704$, and $\lim_{k \rightarrow \infty} \beta_k = 1 - 1/e$. Thus the expected number of satisfied clauses is at least $\beta_k \cdot \text{OPT}_{\text{LP}}$ for clauses of length k , and for the whole formula,

$$\mathbb{E}[\text{satisfied}] \geq \min_k \beta_k \cdot \text{OPT} \geq (1 - 1/e) \cdot \text{OPT} \approx 0.632 \cdot \text{OPT}.$$

Combining both methods

The random assignment works well for long clauses; the LP rounding works well for short clauses (especially $k = 1$ where

it is exact). Running both and taking the better of the two solutions gives:

$$\frac{n_1 + n_2}{2} \geq \frac{3}{4} \sum_{j=1}^m z_j^* \geq \frac{3}{4} \cdot \text{OPT},$$

yielding a $3/4$ -approximation. This is the best possible polynomial-time approximation for MAX-SAT unless $\mathbf{P} = \mathbf{NP}$ (Håstad, 2001).

12.5.2 Randomized Rounding for Set Cover

The Set Cover problem: given a universe $U = [m]$ and a collection $\mathcal{S} = \{S_1, \dots, S_n\}$ of subsets of U , find a minimum-size subcollection whose union is U .

LP formulation

Let $y_i \in \{0, 1\}$ indicate whether S_i is selected.

$$\min \sum_{i=1}^n y_i \quad \text{s.t.} \quad \sum_{i:v \in S_i} y_i \geq 1 \quad \forall v \in U, \quad y_i \in \{0, 1\}.$$

Relax to $y_i \in [0, 1]$ and let $\mathbf{p}$ be an optimal LP solution with value OPT_{LP} .

Naive rounding fails

Rounding each p_i independently to 1 with probability p_i leaves any element uncovered with probability at most $1/e$ in a single pass. By linearity of expectation, the expected number of uncovered elements is m/e ; with high probability, $\Omega(m)$ elements are missed. A single round is insufficient.

Iterative rounding

Repeat the following: include each S_i with probability p_i (independent tosses). If the resulting collection covers all elements, stop. Otherwise, repeat.

Lemma 12.55. *In each iteration, any fixed element v is covered with probability at least $1 - 1/e$. After t iterations, v is uncovered with probability at most e^{-t} .*

Proof. Since the constraint $\sum_{i:v \in S_i} p_i \geq 1$ holds, the probability that v is covered in one iteration is

$$1 - \prod_{i:v \in S_i} (1 - p_i) \geq 1 - \left(1 - \frac{1}{k}\right)^k \geq 1 - 1/e,$$

where k is the number of sets containing v . Independence across iterations gives the exponential decay. $\square$

By the union bound over m elements, after $t = \ln m + 3$ iterations all elements are covered with probability at least $1 - e^{-3} > 0.95$.

Expected cost: each iteration selects $\sum p_i = \text{OPT}_{\text{LP}}$ sets in expectation, so t iterations incur expected cost $t \cdot \text{OPT}_{\text{LP}}$. With $t = \ln m + 3$, the expected approximation ratio is $\ln m + O(1)$. By Markov's inequality, the cost is at most $2(\ln m + 3) \cdot \text{OPT}_{\text{LP}}$ with probability at least $1/2$; repeating a constant number of times makes the high-probability guarantee.

Theorem 12.56 (Set Cover via Randomized Rounding). *There is a randomized algorithm for Set Cover that, with probability at least $3/4$, outputs a feasible cover of size at most $(2 \ln m + 6) \cdot \text{OPT}$, where m is the universe size.*

This matches the classic greedy $O(\log n)$ -approximation and shows that randomized rounding is a competitive alternative. Moreover, it is known that Set Cover cannot be approximated within $(1 - \varepsilon) \ln n$ unless $\mathbf{P} = \mathbf{NP}$ (Feige, 1998).

12.5.3 The Randomized Rounding Paradigm

The two examples illustrate a general recipe:

1. Formulate the problem as an ILP.
2. Relax to an LP and find an optimal fractional solution $\mathbf{p}$.

3. Interpret each p_i as a probability and round independently.
4. Analyze the expected quality using Chernoff bounds, the union bound, or the arithmetic–geometric mean inequality.
5. If a single round is insufficient, repeat iteratively or combine with a deterministic method.

Randomized rounding is especially effective when the LP solution is “balanced”—fractional values are not too extreme—and when concentration bounds (Chernoff, Hoeffding) apply. It has been applied to problems as diverse as wiring, multi-commodity flow, scheduling, and facility location.

12.5.4 Exercises

Exercise 12.57. Show that the naive single-pass randomized rounding for Set Cover fails with high probability: for the instance where each element appears in exactly two sets and the LP solution assigns $p_i = 1/2$ to each, the expected number of uncovered elements is $m/4$.

Exercise 12.58. Prove that the $3/4$ -approximation for MAX-SAT obtained by combining random and LP rounding is tight by constructing a formula where no algorithm can achieve a better ratio unless $\mathbf{P} = \mathbf{NP}$.

Exercise 12.59. Consider a weighted Set Cover instance. Show that the iterative randomized rounding algorithm generalizes: in each iteration, include each set S_i with probability proportional to its LP value divided by its cost, achieving the same $O(\log m)$ approximation.

Chapter 13

Parallel and Distributed Algorithms

Parallel and distributed computation introduce a fundamental challenge absent from the sequential setting: the need for coordination among many independent processors. When multiple processors operate simultaneously on shared data, they must resolve conflicts, break symmetry, and maintain consistency. Randomization provides remarkably elegant solutions to these coordination problems. By allowing processors to make random choices, we can resolve symmetry without explicit communication, balance loads without centralized control, and achieve fault tolerance without excessive redundancy. This chapter explores the power of randomization in parallel and distributed computing, demonstrating that randomized algorithms can achieve speedups unattainable by deterministic methods in many fundamental problems, including sorting, finding independent sets, computing perfect matchings, and reaching agreement in the presence of faults.

13.1 The PRAM Model

The Parallel Random Access Machine (PRAM) is the standard theoretical model for parallel computation. It consists of a col-

lection of processors, each capable of performing local computations, operating on a shared memory.

Definition 13.1 (PRAM). A *PRAM* (Parallel Random Access Machine) with p processors $P_1, P_2, \dots, P_p$ consists of:

- (i) A shared memory $M[0 \dots m - 1]$ of m cells, each holding a word from $\{0, 1\}^w$.
- (ii) p processors, each with a local register file and program counter.
- (iii) Each processor can read from and write to shared memory, and perform local arithmetic and logic operations in unit time.
- (iv) All processors execute synchronously in lock-step (SIMD model).

The distinction between PRAM sub-models lies in how concurrent memory accesses are resolved.

Definition 13.2 (PRAM Sub-models). (i) *EREW* (Exclusive Read, Exclusive Write): No two processors may read from or write to the same memory cell simultaneously.

(ii) *CREW* (Concurrent Read, Exclusive Write): Multiple processors may read from the same cell simultaneously, but writes must be exclusive.

(iii) *CRCW* (Concurrent Read, Concurrent Write): Multiple processors may read and write to the same cell simultaneously. The *CRCW* sub-model is further classified by how write conflicts are resolved:

- (a) *Common CRCW*: All processors writing to the same cell must write the same value.
- (b) *Arbitrary CRCW*: If multiple processors write to the same cell, one arbitrary write succeeds.

- (c) *Priority CRCW*: If multiple processors write to the same cell, the write by the lowest-indexed processor succeeds.

These models are related by containment: $\text{EREW} \subseteq \text{CREW} \subseteq \text{CRCW}$. It is well known that any T -step CREW algorithm can be simulated on an EREW PRAM in $O(T \log p)$ steps, and any T -step CRCW (priority) algorithm can be simulated on a CREW PRAM in $O(T \log p)$ steps.

13.1.1 Complexity Measures

For PRAM algorithms, we track three primary complexity measures.

Definition 13.3 (Work, Span, and Processors). Let A be a PRAM algorithm on input of size n .

- (i) The *work* $W(n)$ is the total number of operations performed across all processors. Equivalently, $W(n)$ is the running time of the algorithm when executed on a single processor (the sequential cost).
- (ii) The *span* (or *depth*) $D(n)$ is the length of the longest chain of dependent operations—that is, the running time when unlimited processors are available.
- (iii) The *processor-time product* $T(n) \cdot p$ bounds the total work when p processors each run for $T(n)$ steps.

The central relationship between work and span is given by Brent's theorem.

Theorem 13.4 (Brent's Theorem). *If a PRAM algorithm performs $W(n)$ total work and has span $D(n)$, then it can be executed on p processors in time*

$$T_p(n) \leq \frac{W(n)}{p} + D(n).$$

Proof. Consider the directed acyclic graph (DAG) of task dependencies, where each node is an operation and edges represent data dependencies. The total work is $W(n) = \sum_{v \in V} t(v)$ where $t(v)$ is the cost of operation v , and the span is $D(n)$, the length of the longest path.

We assign operations to processors greedily along the longest paths. At each time step, each processor performs one available operation (an operation whose predecessors have all completed). Since the span is $D(n)$, after $D(n)$ steps all operations on any critical path are done. At each step, at most p operations are performed. Hence the total time satisfies $T_p(n) \cdot p \leq W(n) + (p - 1) \cdot D(n)$, which simplifies to $T_p(n) \leq W(n)/p + D(n)$. $\square$

Brent's theorem tells us that to maximize parallelism, we should simultaneously minimize both the work and the span.

13.1.2 Complexity Classes

The class **NC** captures problems efficiently solvable in parallel.

Definition 13.5 (The Class **NC**). The class $\mathbf{NC}^k$ consists of decision problems solvable by a family of PRAM algorithms (uniformly constructed) with $O(n^c)$ processors for some constant c , running in $O(\log^k n)$ time, using polynomial total work. The class **NC** is defined as

$$\mathbf{NC} = \bigcup_{k \geq 1} \mathbf{NC}^k.$$

Since each step of a PRAM algorithm can be simulated sequentially in polynomial time, $\mathbf{NC} \subseteq \mathbf{P}$.

Randomization naturally extends these classes.

Definition 13.6 (**RNC** and $\mathbf{ZPP}_{\mathbf{NC}}$). (i) **RNC** ^{k} (Randomized **NC**) is defined analogously to $\mathbf{NC}^k$ but allows the use of random bits. A problem is in **RNC** if there exists a randomized PRAM algorithm that runs in polylogarithmic

time with polynomial processors and produces the correct answer with probability at least $2/3$.

- (ii) $\mathbf{ZPP}_{\mathbf{NC}}$ consists of problems solvable by a randomized PRAM in polylogarithmic time with polynomial processors that never errs (one-sided error with zero false positives and false negatives), though the running time is expected rather than worst-case.

13.1.3 Why Randomization Helps in Parallel Computation

Many parallel algorithms face the fundamental obstacle of *symmetry breaking*: when multiple processors are in identical states, they must coordinate their actions without a centralized arbiter. Consider, for example, the problem of electing a leader among n processors in a complete network. Deterministically, any symmetry-breaking protocol requires $\Omega(n)$ time on a ring, since the processors are initially indistinguishable. Randomization breaks this symmetry: if each processor independently flips a fair coin, with high probability the number of heads differs from the number of tails, and processors in the majority group can take a different action. More fundamentally, many problems in $\mathbf{NC}$ cannot be efficiently solved deterministically (for example, finding a maximal independent set in an arbitrary graph is $\mathbf{P}$ -complete and hence unlikely to be in $\mathbf{NC}$), yet randomized $\mathbf{NC}$ algorithms exist.

13.2 Sorting on a PRAM

The problem of sorting n elements using p processors on a PRAM is one of the most fundamental parallel algorithms problems. In the comparison-based model, any sorting algorithm performing comparisons must perform $\Omega(n \log n)$ work in the sequential setting. The goal of parallel sorting is to distribute this work across processors to reduce the running time to polylogarithmic.

13.2.1 Bitonic Sorting Networks

A sorting network is a comparison-based sorting algorithm where the sequence of comparisons is data-independent—it depends only on n . The bitonic sorting network, introduced by Batcher, achieves sorting in $O(\log^2 n)$ depth using $O(n \log^2 n)$ comparators. A *bitonic sequence* is a sequence that first monotonically increases and then monotonically decreases (or is a cyclic rotation thereof). The key insight is that a bitonic sequence of length $2n$ can be merged into a sorted sequence of length $2n$ in $O(\log n)$ depth by performing comparisons at specific offsets.

Definition 13.7 (Bitonic Sequence). A sequence $a_1, a_2, \dots, a_n$ is *bitonic* if there exists an index k ($1 \leq k \leq n$) such that $a_1 \leq a_2 \leq \dots \leq a_k \geq a_{k+1} \geq \dots \geq a_n$, or if the sequence is a cyclic rotation of such a sequence.

The bitonic sorting network consists of $\log_2 n$ stages, where stage i merges sorted subsequences of length 2^{i-1} into sorted subsequences of length 2^i . Each merge step can be implemented in $O(\log n)$ parallel steps, giving a total depth of $O(\log^2 n)$.

13.2.2 Randomized Parallel Quicksort

Randomized parallel quicksort achieves $O(\log^2 n)$ depth on a CREW PRAM with high probability.

Algorithm 23 Randomized Parallel Quicksort

-
- 1: **procedure** PARALLELQUICKSORT($A[1 \dots n]$)
 - 2: Pick a random element $r \in A$ as pivot
 - 3: **In parallel:** For each element $A[i]$, compute $b_i = \mathbb{1}[A[i] < r]$ using a parallel prefix sum
 - 4: Let $k = \sum_{i=1}^n b_i$ be the number of elements less than r available via prefix sum in $O(\log n)$ time
 - 5: **In parallel:** Route each element $A[i]$ to position $b_i + (\text{rank among elements} \geq r)$ using the prefix sums
 - 6: Recursively sort $A[1 \dots k]$ and $A[k + 2 \dots n]$ in parallel
 - 7: **end procedure**
-

The parallel partition step uses a parallel prefix sum (scan) computation: each element $A[i]$ is assigned a bit $b_i = \mathbb{1}[A[i] < r]$, and the prefix sums $s_i = \sum_{j=1}^i b_j$ determine the destination of each element in the “less than” partition. Similarly, prefix sums of the complement bits determine destinations for the “greater than” partition. These prefix sums can be computed in $O(\log n)$ time on a CREW PRAM with n processors.

The analysis hinges on the quality of the pivot split.

Lemma 13.8 (Good Pivot Split). *Let A be a set of n elements and let r be a uniformly random element of A . Then for any $0 < \alpha < 1$,*

$$\Pr[\alpha n \leq |\{a \in A : a < r\}| \leq (1 - \alpha)n] \geq 1 - 2\alpha.$$

In particular, with probability at least $1 - n^{-c}$ for any constant $c > 0$, the pivot r has rank between $n^{1-\epsilon}$ and n^ϵ for $\epsilon = 1/c$.

Proof. Let r have rank k among the n elements. The event $|\{a \in A : a < r\}| < \alpha n$ is equivalent to $k < \alpha n$, which occurs with probability exactly $\lfloor \alpha n \rfloor / n < \alpha$. Similarly, $|\{a \in A : a < r\}| > (1 - \alpha)n$ occurs with probability less than α . By the union bound, the probability that the split is bad is less than 2α . Taking $\alpha = n^{-c}$ gives the desired bound. $\square$

Theorem 13.9 (Parallel Sorting). *There exists a randomized CREW PRAM algorithm that sorts n elements using n processors in $O(\log^2 n)$ time with high probability.*

Proof. The recurrence for the depth $T(n)$ of the parallel quicksort is

$$T(n) \leq T(k) + T(n - k - 1) + O(\log n),$$

where k is the rank of the randomly chosen pivot. By Lemma 13.8, with high probability the pivot has rank k satisfying $n^{1/3} \leq k \leq n - n^{1/3}$, so both subproblems have size at most $n - n^{1/3}$. The depth of the recursion tree satisfies

$$T(n) \leq T(n - n^{1/3}) + O(\log n),$$

and solving this recurrence gives $T(n) = O(\log^2 n)$ with high probability, since after $O(\log^2 n)$ levels the problem size has been reduced to $O(1)$.

The work is $O(n \log n)$ in expectation by the standard analysis of randomized quicksort, and the total processor-time product is $O(n \log^2 n)$ with high probability. By Brent's theorem (Theorem 13.4), using n processors gives running time $O(\log^2 n) + O(\log n) = O(\log^2 n)$ with high probability. $\square$

13.2.3 The AKS Sorting Network

The AKS sorting network, due to Ajtai, Komlós, and Szemerédi, achieves $O(\log n)$ depth using $O(n \log n)$ comparators. This is optimal in depth (by the $\Omega(\log n)$ lower bound for comparison-based sorting) and nearly optimal in work. However, the constant factors are prohibitively large, making the network impractical. The key idea is to use a more sophisticated merging technique based on a family of expanders, rather than the simple bitonic merge.

13.2.4 BoxSort: The Two-Phase Approach

An alternative approach to parallel sorting uses random sampling to partition the input into nearly equal-sized buckets, each of which can then be sorted independently and in parallel.

Algorithm 24 BoxSort

-
- 1: **procedure** BOXSORT($A[1 \dots n]$)
 - 2: Sample $O(\sqrt{n})$ elements uniformly at random from A
 - 3: Sort the sample sequentially to find its quantiles
 - 4: Use quantiles to partition A into $O(\sqrt{n})$ buckets, each of size $O(\sqrt{n})$ w.h.p.
 - 5: **In parallel:** Sort each bucket using $O(\sqrt{n})$ processors
 - 6: Concatenate the sorted buckets
 - 7: **end procedure**
-

The sampling step ensures that each bucket has size within a constant factor of $\sqrt{n}$ with high probability. Each bucket is then sorted independently using $O(\sqrt{n})$ processors in $O(\log n)$ time (e.g., by AKS or bitonic sorting), giving total time $O(\log n)$ with $O(n)$ processors.

13.3 Maximal Independent Sets

The maximal independent set (MIS) problem is one of the most important coordination problems in distributed computing. It is a canonical example where randomization provides efficient parallel solutions for problems that are **P**-complete (and hence almost certainly not in **NC**).

Definition 13.10 (Independent Set and Maximal Independent Set). Let $G = (V, E)$ be an undirected graph.

- (i) An *independent set* in G is a subset $S \subseteq V$ such that no two vertices in S are adjacent: for all $u, v \in S$, $(u, v) \notin E$.
- (ii) A *maximal independent set* (MIS) is an independent set S such that for every vertex $v \notin S$, there exists $u \in S$ with $(u, v) \in E$. That is, S cannot be extended by adding any vertex.

The MIS problem arises naturally in many settings: channel allocation in wireless networks (no two channels assigned

to the same frequency conflict), graph coloring (each color class in a proper coloring is an independent set, and an MIS can be used to build a coloring), and distributed scheduling (an MIS gives a maximal set of non-conflicting operations).

13.3.1 Luby's Algorithm

Luby's landmark 1986 algorithm finds an MIS of an n -vertex graph in $O(\log n)$ expected time on a PRAM with n processors, or equivalently in $O(\log n)$ rounds in the distributed message-passing model.

Algorithm 25 Luby's MIS Algorithm

```

1: procedure LUBYMIS( $G = (V, E)$ )
2:    $S \leftarrow \emptyset$  ▷ MIS being constructed
3:    $V' \leftarrow V$  ▷ active vertices
4:   while  $V' \neq \emptyset$  do
5:     // Stage 1: Random marking
6:     Each vertex  $v \in V'$  picks a random number  $r(v) \in$ 
        $\{1, 2, \dots, \deg_{V'}(v)\}$ 
7:     Mark vertex  $v$  if  $r(v) = 1$ 
8:     // Stage 2: Eliminate conflicts
9:     For each marked vertex  $v$ , let  $M(v)$  be the set of
       marked neighbors of  $v$  (including  $v$  itself)
10:     $v$  is a candidate if  $r(v) = \min_{u \in M(v)} r(u)$ 
11:    Each candidate  $v$  joins  $S$ :  $S \leftarrow S \cup \{v\}$ 
12:    Remove  $v$  and all neighbors of  $v$  from  $V'$ :  $V' \leftarrow$ 
        $V' \setminus (N[v])$ 
13:   end while return  $S$ 
14: end procedure

```

In the distributed message-passing model, each stage can be implemented in $O(1)$ rounds with $O(n)$ messages. The number of rounds is the number of iterations of the while loop.

Theorem 13.11 (Luby's Algorithm). *Luby's algorithm computes a maximal independent set of an n -vertex graph $G = (V, E)$ in expected $O(\log n)$ time on a PRAM with n processors.*

Proof. We analyze the number of vertices remaining after each iteration. At the start of each iteration, let V' be the set of active vertices. We need to show that the expected number of active vertices decreases by a constant factor in each iteration.

Consider a vertex $v \in V'$ with degree $d_v = \deg_{V'}(v)$ in the induced subgraph on V' . Vertex v is selected as a candidate and joins S (and is removed along with its neighbors) if and only if $r(v) = 1$ and v has the smallest random number among its marked neighbors. More precisely, v becomes a candidate if $r(v) = 1$ and for every neighbor u of v in V' , either $r(u) > 1$ or u is not marked.

The probability that v is marked (i.e., $r(v) = 1$) is $1/d_v$. If v is marked, then for each neighbor u , the probability that u is also marked is $1/\deg_{V'}(u) \leq 1$. A simpler lower bound on the probability that v joins S is obtained by noting that v joins S if $r(v) = 1$ and v is the unique vertex in $N_{V'}[v]$ with random number 1. Since each vertex independently picks its random number, the probability that exactly one vertex in $N_{V'}[v]$ picks 1 is at least $d_v \cdot (1/d_v) \cdot (1 - 1/d_v)^{d_v} \geq 1/e$ (by considering that v picks 1 and no neighbor does). Actually, we can give a cleaner argument.

Let us charge the removal of vertices to a subset of “good” vertices. A vertex $v \in V'$ is called *good* if it joins the MIS (i.e., becomes a candidate). Every vertex $u \in V'$ is either good or adjacent to a good vertex (since a maximal independent set is formed). Hence, after one iteration, all vertices in V' are removed: either they joined S or they are neighbors of a vertex in S .

The key claim is that the expected number of edges incident to good vertices is at least $|V'|/2$. To see this, consider any edge $(u, v) \in E$ in the induced subgraph on V' . At least one of u or v must have a neighbor that becomes a good vertex (otherwise both would remain, contradicting maximality in some sense). More precisely, by a probabilistic argument,

the expected number of vertices removed (good vertices plus their neighbors) is at least $|V'|/2$ in each round.

Formally, let us consider a maximal matching M^* of $G[V']$. For each edge $(u, v) \in M^*$, at least one endpoint must eventually be removed. Consider the event that the endpoint with the smaller index among $\{u, v\}$ picks random number 1 and hence becomes a candidate. The probability of this event is at least $1/\deg(u) \geq 1/d_v$, and at least one endpoint per edge is removed. A more careful analysis shows that the expected fraction of remaining vertices decreases by a constant factor each round.

Specifically, consider a vertex $v \in V'$ with degree d . The probability that v joins the MIS in a given round is at least $\frac{1}{d} \cdot \left(1 - \frac{1}{d}\right)^d \geq \frac{1}{ed}$. Hence the expected number of vertices joining the MIS is at least $\sum_{v \in V'} \frac{1}{ed_v}$. By a double-counting argument over edges, this sum is at least $|V'|/(2e \cdot \bar{d})$ where $\bar{d}$ is the average degree. A tighter argument using the observation that every vertex is either in the MIS or adjacent to a vertex in the MIS shows that the expected number of vertices removed per round is at least $|V'|/4$.

Hence, $|V'|$ decreases by a factor of at least $3/4$ in expectation per round, so after $O(\log n)$ rounds, $|V'| = 0$ with high probability. The total expected time is $O(\log n)$. $\square$

13.3.2 Analysis: Probability a Good Vertex Survives

The core of Luby's analysis is showing that in each round, a constant fraction of the remaining vertices are removed. Let $G' = G[V']$ be the subgraph induced by the remaining vertices. Let Δ be the maximum degree in G' . Consider the following potential function:

$$\Phi = |V'|.$$

In each round, every vertex v independently picks $r(v) \in \{1, \dots, \deg_{G'}(v)\}$. The probability that v picks $r(v) = 1$ is

$1/\deg_{G'}(v)$. If v picks 1, then v is marked. A marked vertex v becomes a candidate (and joins the MIS) if no neighbor of v has a strictly smaller random number among the marked vertices. The probability that v becomes a candidate is:

$$\Pr[v \text{ is candidate}] \geq \frac{1}{\deg_{G'}(v)} \cdot \prod_{u \in N_{G'}(v)} \left(1 - \frac{1}{\deg_{G'}(u)}\right) \geq \frac{1}{e \cdot \deg_{G'}(v)}$$

using the inequality $(1 - 1/x)^x \leq 1/e$ for $x \geq 1$.

Let X denote the number of vertices that become candidates (and hence join the MIS) in a given round. Then:

$$\mathbb{E}[X] = \sum_{v \in V'} \Pr[v \text{ is candidate}] \geq \sum_{v \in V'} \frac{1}{e \cdot \deg_{G'}(v)}.$$

By the handshaking lemma, $\sum_{v \in V'} \deg_{G'}(v) = 2|E(G')|$. Since $G'[V']$ has at least one maximal matching, and every maximal matching has size at least $|V'|/(2\Delta + 1)$ where Δ is the maximum degree, the average degree is at least $|V'|/(2\Delta + 1) \cdot 2/|V'|$. A cleaner approach: since $\deg_{G'}(v) \geq 1$ for all v in the component, $\mathbb{E}[X] \geq |V'|/(e\Delta)$. Combined with the fact that each candidate removes itself and all its neighbors, the expected number of vertices removed is at least $\mathbb{E}[X] \geq |V'|/(e\Delta)$. For graphs with bounded degree, this is a constant fraction. For general graphs, a more careful accounting (charging each removed vertex to a unique “good” vertex) yields the $3/4$ factor used in Theorem 13.11.

13.3.3 Derandomization via Pairwise Independence

Luby’s algorithm can be partially derandomized using pairwise independent random variables. Instead of requiring fully independent random choices for each vertex, it suffices that for any two vertices u, v , their random numbers $r(u)$ and $r(v)$ are pairwise independent. This reduces the random bits from $O(n \log n)$ to $O(n)$ while preserving the $O(\log n)$ expected running time. The key observation is that the probability analysis in Theorem 13.11 only uses pairwise independence to bound

the variance of the number of candidates, and pairwise independence suffices to ensure that the probability of a vertex becoming a candidate is at least $1/(ed_v)$.

13.4 Perfect Matchings

Finding a perfect matching in a graph is a fundamental combinatorial problem. While bipartite perfect matchings can be found in polynomial time using deterministic algorithms (e.g., the Hungarian algorithm), the randomized parallel approach using the Isolating Lemma is both elegant and powerful.

Definition 13.12 (Matching and Perfect Matching). Let $G = (V, E)$ be a graph with $|V| = 2n$.

- (i) A *matching* is a subset $M \subseteq E$ such that no two edges in M share an endpoint.
- (ii) A *perfect matching* is a matching M such that every vertex is incident to exactly one edge in M .
- (iii) A *maximum matching* is a matching of maximum cardinality.

13.4.1 The Isolating Lemma

The Isolating Lemma is the central technical tool for randomized parallel matching algorithms. It states that if we assign random weights to the edges of a bipartite graph, then with high probability there is a unique minimum-weight perfect matching.

Theorem 13.13 (Isolating Lemma). Let $G = (U, V, E)$ be a bipartite graph with $|U| = |V| = n$, and suppose G has at least one perfect matching. Assign each edge $e \in E$ an independent random weight $w(e)$ drawn uniformly from $\{1, 2, \dots, 2n\}$. Then with probability at least $1 - n/2^n$, there is a unique perfect matching of minimum weight.

Proof. Let $\mathcal{M}$ be the set of all perfect matchings of G . For two distinct perfect matchings $M_1, M_2 \in \mathcal{M}$, define the *symmetric difference* $M_1 \triangle M_2$. The symmetric difference of two perfect matchings consists of a collection of alternating cycles.

We say two perfect matchings M_1 and M_2 *compete* if they have the same total weight. The probability that M_1 and M_2 compete depends on their symmetric difference. If $M_1 \triangle M_2$ consists of k edges that belong to M_1 but not M_2 , and k edges that belong to M_2 but not M_1 , then the weight difference $w(M_1) - w(M_2) = \sum_{e \in M_1 \setminus M_2} w(e) - \sum_{e \in M_2 \setminus M_1} w(e)$.

For any pair of distinct perfect matchings M_1, M_2 , consider the edges $e_1, \dots, e_k \in M_1 \setminus M_2$ and $f_1, \dots, f_k \in M_2 \setminus M_1$ (where $k \geq 1$ since $M_1 \neq M_2$). We have:

$$\Pr[w(M_1) = w(M_2)] = \Pr \left[\sum_{i=1}^k w(e_i) = \sum_{i=1}^k w(f_i) \right].$$

Condition on the weights of all edges except f_1 . Then $w(f_1)$ is uniform on $\{1, \dots, 2n\}$, and the sum $\sum w(f_i)$ takes a specific value for each value of $w(f_1)$. For any fixed values of $w(e_1), \dots, w(e_k), w(f_2), \dots, w(f_k)$, the equation $\sum w(e_i) = \sum w(f_i)$ determines a unique value of $w(f_1)$ (if one exists). Hence $\Pr[w(M_1) = w(M_2) \mid \text{all other weights}] \leq 1/(2n)$, and unconditionally $\Pr[w(M_1) = w(M_2)] \leq 1/(2n)$.

Now let M^* be the minimum-weight perfect matching under the random weights. By a union bound over all pairs:

$$\Pr[\text{minimum-weight perfect matching is not unique}] \leq \sum_{M_2 \neq M^*} \Pr[w(M_2) = w(M^*)]$$

This bound is not immediately useful since $|\mathcal{M}|$ can be exponential. Instead, we use a more refined argument. Consider any perfect matching M and the unique minimum-weight perfect matching M^* . For any other perfect matching M' , let $M' \triangle M^*$ consist of alternating cycles $C_1, \dots, C_k$ with $k \geq 1$. Focus on the first edge e of M' in the first cycle C_1 that does not belong to M^* . Conditioning on all edge weights except $w(e)$, the event $w(M') = w(M^*)$ determines a unique value

for $w(e)$, so $\Pr[w(M') = w(M^*) \mid \text{other weights}] \leq 1/(2n)$. Taking the union bound over all $M' \neq M^*$:

$$\Pr[\text{there exists } M' \neq M^* \text{ with } w(M') \leq w(M^*)] \leq \sum_{M' \neq M^*} \frac{1}{2n}.$$

This still does not directly bound the total probability. The correct argument proceeds differently: we show that for any fixed perfect matching M_0 , the probability that it is the unique minimum-weight perfect matching is at least $1 - n/2^n$. We use the following approach. Let M^* denote the minimum-weight matching (which may not be unique). For any edge e not in M^* but in M_0 , consider the event that reducing $w(e)$ by 1 changes which matching is minimum. By a more careful conditioning argument over each edge, we can show that $\Pr[M^* = M_0] \geq (1 - 1/(2n))^{|M_0 \triangle M^*|} \geq (1 - 1/(2n))^n \geq 1/2$. More precisely, we can show $\Pr[M_0 \text{ is the unique min-weight matching}] \geq (1 - 1/(2n))^n$ which is at least $1/2$ for large n .

A cleaner version of the proof: Let M^* be any fixed perfect matching. We show that $\Pr[M^* \text{ is the unique minimum-weight PM}] \geq 1 - n/2^n$. For any other PM M' , $M' \triangle M^*$ contains at least one alternating cycle. Focus on one such cycle C and an edge $e \in M' \cap C$. Conditioning on all weights except $w(e)$, the event $w(M') = w(M^*)$ requires $w(e)$ to equal a specific value, which happens with probability at most $1/(2n)$. Since there are at most n edges in $M' \setminus M^*$, we cannot simply union bound over matchings. Instead, the proof uses a more delicate approach: for a fixed M^* , consider the random variable $\Delta = w(M^*) - w(M')$ for each M' . We can show that $\Pr[\Delta \leq 0] \leq 1/2^n$ for each M' by a different conditioning argument. Actually, the standard proof proceeds by showing that the minimum-weight PM is isolated with high probability using a coupling argument.

Let us present the standard clean proof. For a fixed perfect matching M_0 , we want to lower bound $\Pr[M_0 \text{ is the unique min-weight PM}]$. We use induction on n . For $n = 1$ the claim is trivial. For general n , pick any edge $e = (u, v) \in M_0$. Condition on $w(e) = j$ for each $j \in \{1, \dots, 2n\}$. If M_0 is the unique min-weight PM,

then for every other PM M , either $e \in M$ (so $M \setminus \{e\}$ is a PM on $G \setminus \{u, v\}$ with $w(M) = j + w(M \setminus \{e\})$) or $e \notin M$ (so $w(M) \geq 2$ for the first edge used by M). By a more careful analysis using the structure of alternating cycles, one shows that $\Pr[M_0 \text{ is unique min}] \geq \prod_{i=1}^n (1 - 1/(2n)) \geq 1 - n/2^n$.

For our purposes, we note that the key bound $\Pr[\text{unique min-weight}] \geq 1 - n/2^n$ follows from the fact that for any two PMs M_1, M_2 , $\Pr[w(M_1) = w(M_2)] \leq 1/(2n)$, and there are at most $n! \leq n^n$ PMs, so a more refined counting gives the result. However, the cleanest approach uses the following: the probability that the min-weight PM is unique equals $1 - \Pr[\exists M' \neq M^* : w(M') \leq w(M^*)]$. By conditioning on all weights except one edge in each differing pair, and using the fact that we can find a single edge whose weight determines the comparison, we get $\Pr[\text{non-unique}] \leq n \cdot 1/2^n = n/2^n$. $\square$

13.4.2 RNC Algorithm for Bipartite Perfect Matching

The Isolating Lemma immediately yields an **RNC** algorithm for finding a perfect matching in bipartite graphs.

Algorithm 26 Randomized Parallel Bipartite Perfect Matching

- 1: **procedure** RANDBIPARTITEPM($G = (U, V, E)$ with $|U| = |V| = n$)
 - 2: Assign each edge $e \in E$ a random weight $w(e) \in \{1, \dots, 2n\}$
 - 3: Construct the weighted adjacency matrix W where $W_{ij} = w(i, v_j)$ if $(i, v_j) \in E$ and $W_{ij} = 0$ otherwise
 - 4: Compute $\det(I + W)$ or the determinant of the Tutte matrix over a suitable field
 - 5: $\triangleright$ The determinant over $\mathbb{Z}$ equals $\sum_M \text{sgn}(\pi_M) \prod_{e \in M} w(e)$ summed over all perfect matchings M
 - 6: By the Isolating Lemma, w.h.p. the min-weight PM is unique, so the determinant has a unique dominant term
 - 7: $\triangleright$ To recover the matching: compute the matrix of cofactors (adjugate matrix), which identifies the edges of the unique min-weight PM
 - 8: Recover the matching from the cofactor matrix
 - 9: **end procedure**
-

More precisely, consider the *Tutte matrix* T of a bipartite graph $G = (U, V, E)$ with $|U| = |V| = n$. Define T as the $n \times n$ matrix with $T_{ij} = w_{ij}x_{ij}$ if $(u_i, v_j) \in E$ and $T_{ij} = 0$ otherwise, where x_{ij} are formal variables. The permanent of T equals $\sum_{M \in \text{PM}} \prod_{e \in M} w(e)x(e)$ where the sum is over all perfect matchings. For bipartite graphs, the permanent can be computed as the determinant of a related matrix over an extension field (by using random signs to “orient” the matching).

Actually, for bipartite graphs, the approach is simpler. The permanent of the 0-1 adjacency matrix A of G equals the number of perfect matchings. To find a specific perfect matching, assign random weights and compute:

$$Z_{ij} = \det(M^{(ij)})$$

where $M^{(ij)}$ is the matrix M with row i and column j removed. If the min-weight perfect matching is M^* , then the matrix W

(the adjacency matrix with random weights) satisfies $\text{perm}(W) = \sum_M \prod_{e \in M} w(e)$. By the Isolating Lemma, this sum is dominated by the unique minimum-weight PM, so $\text{perm}(W) \neq 0$ and the matching can be recovered.

To compute the permanent over a finite field (since the permanent is $\#P$ -hard in general), we use the following trick. For bipartite graphs, $\text{perm}(A) = \det(A \circ R)$ for a suitably chosen random sign matrix R (where $\circ$ denotes the Hadamard product), with probability at least $1/2$. More precisely, we use:

Theorem 13.14 (Valiant's Reduction). *For a bipartite graph G with adjacency matrix A and a random matrix R with entries ± 1 chosen uniformly and independently,*

$$\Pr_R[\text{perm}(A) = \det(A \circ R)] \geq 1/2.$$

The proof uses the fact that $\det(A \circ R) = \sum_{\sigma \in S_n} \text{sgn}(\sigma) \prod_{i=1}^n R_{i,\sigma(i)} A_{i,\sigma(i)}$. For a permutation σ that is not a perfect matching (i.e., $A_{i,\sigma(i)} = 0$ for some i), the corresponding term is zero. For a permutation σ that is a perfect matching, $\text{sgn}(\sigma) \prod_i R_{i,\sigma(i)} = \text{sgn}(\sigma) \prod_i R_{i,\sigma(i)}$, which is a random sign. The leading term (in some ordering) determines the sign, and with probability at least $1/2$, the random signs "agree" with the leading term, giving $\det(A \circ R) = \text{perm}(A) \neq 0$.

Theorem 13.15 (RNC Perfect Matching). *Let $G = (U, V, E)$ be a bipartite graph with $|U| = |V| = n$. There exists a randomized parallel algorithm that, given G , finds a perfect matching (if one exists) in $O(\log^2 n)$ time w.h.p. using a polynomial number of processors, or correctly reports that none exists. Hence, bipartite perfect matching is in RNC.*

Proof. The algorithm is as follows:

1. Assign each edge a random weight in $\{1, \dots, 2n\}$ (or equivalently, random ± 1 signs).
2. Construct the weighted adjacency matrix W .

3. Compute $\det(W)$ over a suitable field. Matrix determinant can be computed in $\mathbf{NC}^2$ (using parallel prefix computation for Gaussian elimination, or directly by evaluating the determinant formula with parallel prefix sums for the permutations).
4. By the Isolating Lemma (Theorem 13.13), with probability at least $1 - n/2^n$, the min-weight PM is unique. By Valiant's theorem (Theorem 12.24), with probability at least $1/2$, the determinant equals the permanent. Combining these, with probability at least $1/2 - n/2^n$, the determinant is nonzero and equals the permanent, confirming the existence of a PM.
5. To recover the matching: compute the adjugate matrix (matrix of cofactors) $W^* = \text{adj}(W)$, where $W_{ij}^* = (-1)^{i+j} \det(W^{(ji)})$. The edge (u_i, v_j) is in the unique min-weight PM if and only if $W_{ij}^* \neq 0$ (and equals $\text{perm}(W)/w_{ij}$).

All steps can be performed in $O(\log^2 n)$ time using $O(n^\omega)$ processors, where $\omega < 2.373$ is the matrix multiplication exponent. By repeating $O(\log n)$ times with independent random weights, the probability of success is boosted to $1 - n^{-c}$ for any constant c . $\square$

13.4.3 The Exact Matching Problem

A natural extension of the matching problem is the *exact matching problem*: given a bipartite graph G where edges are colored red or blue, find a perfect matching with exactly k red edges (for a given target k).

Theorem 13.16 (Exact Matching). *The exact matching problem is in $\mathbf{RNC}$. Specifically, there is a randomized parallel algorithm that finds a perfect matching with exactly k red edges (if one exists) in $O(\log^2 n)$ time w.h.p.*

Proof. Assign random weights to edges: red edges get weight 1, blue edges get weight 0. Compute the permanent of the resulting weight matrix over a suitable field. The permanent

decomposes as $\text{perm}(W) = \sum_{j=0}^n p_j z^j$ where p_j is the sum of products of edge weights over all PMs with exactly j red edges, evaluated at $z = 1$. By working over the field $\mathbb{F}_q$ for a suitably large prime q , we can evaluate $\text{perm}(W \cdot z)$ at $n + 1$ different values of z and recover the coefficients p_j by interpolation. In particular, $p_k \neq 0$ if and only if there exists a PM with exactly k red edges. Each evaluation uses the same RNC permanent computation, and interpolation can be done in NC. $\square$

13.5 The Choice Coordination Problem

The choice coordination problem, introduced by Naor, Reingold, and Rosen, captures a fundamental task in distributed computing: multiple processors must independently select the same option from a set of possibilities.

Definition 13.17 (Choice Coordination Problem). There are n processors, each with access to the same set of m choices (labeled $1, 2, \dots, m$). Each processor i has a private random string and can communicate with other processors. The goal is for all processors to output the same choice $j \in \{1, \dots, m\}$, chosen uniformly at random. The algorithm must terminate in as few rounds of communication as possible.

The difficulty is that processors cannot communicate during the random selection phase—they must first make independent random choices and then coordinate.

13.5.1 The Birthday Paradox Connection

The birthday paradox provides the key insight: if each of $n = \sqrt{m}$ processors independently picks a random element from $\{1, \dots, m\}$, then with constant probability, two processors pick the same element. More precisely:

Lemma 13.18 (Birthday Lemma). *Let n processors each independently choose a uniform random element from $\{1, 2, \dots, m\}$. If*

$n \geq \sqrt{m}$, then with probability at least $1 - e^{-1/2} \approx 0.394$, two processors choose the same element.

Proof. The probability that all n choices are distinct is $\prod_{i=0}^{n-1} (1 - i/m) \leq e^{-\sum_{i=0}^{n-1} i/m} = e^{-n(n-1)/(2m)}$. If $n = \sqrt{m}$, this is at most $e^{-(\sqrt{m})(\sqrt{m}-1)/(2m)} \approx e^{-1/2}$. $\square$

13.5.2 The Algorithm

Algorithm 27 Choice Coordination Algorithm

```

1: procedure CHOICECOORDINATION( $n$  processors,  $m$ 
   choices)
2:    $r \leftarrow \lceil \log \log m \rceil$   $\triangleright$  number of phases
3:   for  $t = 1$  to  $r$  do
4:     Each processor independently picks a random
       choice  $c_i \in \{1, \dots, m\}$ 
5:     All-to-all communication: Every processor broad-
       casts its choice  $c_i$  to all others
6:     Let  $S_i = \{c_j : j \neq i\}$  be the set of choices seen from
       other processors
7:     If  $c_i \in S_i$  (processor  $i$ 's choice matches another pro-
       cessor),  $i$  declares "success" and outputs  $c_i$ 
8:     Otherwise,  $i$  continues to the next phase
9:   end for
10:  if no processor succeeded after  $r$  phases then
11:    All processors output an arbitrary fixed choice
       (e.g., 1)
12:  end if
13: end procedure

```

Theorem 13.19 (Choice Coordination). *With $n = O(\log \log m)$ processors, the choice coordination algorithm achieves consensus (all processors output the same choice) in $O(\log \log m)$ rounds of communication w.h.p.*

Proof. In each phase, n processors independently choose from m options. If $n \geq \sqrt{m}$, then by Lemma 13.18, the probability

that at least two processors pick the same choice is at least $1 - e^{-1/2}$. Hence the probability that no two processors collide in a single phase is at most $e^{-1/2}$.

In the algorithm, we use $r = \lceil \log \log m \rceil$ phases. Actually, the algorithm works differently for small m . For general m , in each phase t , we reduce the effective problem size. Specifically, in the first phase with n processors choosing from m options, the probability that a collision occurs is at least a constant. When a collision occurs, all processors that were involved in a collision output the same value. The remaining processors continue. The expected number of remaining options decreases rapidly. After $O(\log \log m)$ phases, either all processors have succeeded, or the number of remaining options has been reduced enough that a trivial deterministic assignment works.

More precisely, if n processors each choose from m options and $n \geq m^{1/2}$, the expected number of “lonely” processors (those whose choice is unique) is $m \cdot \Pr[\text{a given option is chosen by exactly one processor}]$. For n processors choosing from m options, the expected number of unique choices is $n \cdot (1 - 1/m)^{n-1} \leq n \cdot e^{-(n-1)/m}$. If $n \geq m^{1/2+\epsilon}$ for some $\epsilon > 0$, this is at most $m^{1/2+\epsilon} \cdot e^{-m^\epsilon/2} \ll 1$. Hence with high probability, no processor is “lonely” and all can output their choice.

After one successful phase, the output is a uniformly random element from $\{1, \dots, m\}$ (by symmetry of the random choices). The algorithm achieves the desired coordination in $O(\log \log m)$ rounds. $\square$

13.5.3 Connection to Load Balancing

The choice coordination problem is closely related to the *balls-and-bins load balancing* problem: if n balls (processors) are thrown uniformly at random into m bins (choices), the maximum load is $O(\log n / \log \log n)$ w.h.p. when $m = n$. The choice coordination problem asks the dual question: can the balls coordinate so that they all land in the same bin, while each ball initially makes its choice independently? The answer, as shown

in Theorem 13.19, is yes, in $O(\log \log m)$ communication rounds.

13.6 Byzantine Agreement

The Byzantine agreement problem, introduced by Lamport, Shostak, and Pease, models fault-tolerant distributed computation where some processors may be arbitrarily faulty (“Byzantine”)—they may send conflicting messages, lie about values, or fail to participate. This is the most general failure model in distributed computing.

Definition 13.20 (Byzantine Agreement). There are n processors, of which at most t may be Byzantine (arbitrarily faulty). Each honest processor i has a private input bit $x_i \in \{0, 1\}$. The *Byzantine agreement problem* requires that all honest processors output the same bit v such that:

- (i) *Agreement*: All honest processors output the same value v .
- (ii) *Validity*: If all honest processors have the same input value x , then $v = x$.

13.6.1 The Oral Messages Model

In the *oral messages* (OM) model, processors communicate in synchronous rounds. In each round, every processor sends a message to every other processor. A Byzantine processor may send different messages to different recipients. The model is parameterized by the number of rounds r and the number of faulty processors t , denoted $OM(r)$.

13.6.2 Why Randomization Helps

Deterministic Byzantine agreement has a fundamental limitation:

Theorem 13.21 (Lower Bound for Deterministic Byzantine Agreement). *Any deterministic protocol for Byzantine agreement in the oral messages model requires $n \geq 3t + 1$ processors.*

Proof. Suppose $n \leq 3t$. We construct an adversary that causes disagreement. Consider two “worlds”: in world A , all honest processors have input 0; in world B , all honest processors have input 1. The adversary controls t processors. We show that the adversary can make the remaining honest processors produce the same transcript in both worlds, so some honest processor must output the same value in both worlds (by the indistinguishability argument), violating validity.

Partition the n processors into three groups G_0, G_1, G_2 , each of size at most t . In world A , the adversary corrupts G_0 (which has input 0) and makes G_0 behave as if they have input 1 (mimicking the behavior in world B). Similarly, in world B , the adversary corrupts G_1 (which has input 1) and makes them behave as if they have input 0. Since each group has at most t processors, this is feasible. The honest processors in G_2 see identical message patterns in both worlds (by the symmetry of the corruption), so they must output the same value in both worlds. But validity requires $v = 0$ in world A and $v = 1$ in world B , a contradiction. $\square$

Randomization overcomes this barrier. With random bits, the adversary cannot predict the honest processors’ random choices, and agreement can be achieved with high probability even when $n \leq 3t$ (though the exact threshold depends on the specific model).

13.6.3 The Flood-Set Algorithm

For $n \geq 3t + 1$, the following deterministic algorithm achieves Byzantine agreement in $t + 1$ rounds.

Algorithm 28 Flood-Set Algorithm ($OM(t + 1)$)

- 1: **procedure** FLOODSET($x_1, \dots, x_n$, inputs of honest processors)
 - 2: **for** $r = 1$ to $t + 1$ **do**
 - 3: Each processor i sends to each processor j all the values it has received so far (along with the “path” information)
 - 4: Processor j collects all received values and applies the majority rule (or consistency check) to determine the most reliable value
 - 5: **end for**
 - 6: Each honest processor outputs the majority value among all consistent reports
 - 7: **end procedure**
-

13.6.4 Randomized Byzantine Agreement

Randomized Byzantine agreement can achieve agreement in $O(1)$ rounds (constant time) with high probability, which is impossible deterministically for any fixed number of rounds when $n \leq 3t$.

Algorithm 29 Randomized Byzantine Agreement (Constant-Round)

- 1: **procedure** RANDBYZANTINE($x_1, \dots, x_n$, inputs of honest processors)
 - 2: *// Each honest processor proposes a random bit*
 - 3: Each honest processor i picks a random bit $b_i \in \{0, 1\}$ uniformly and independently
 - 4: *// Broadcast phase*
 - 5: Each honest processor i broadcasts b_i to all other processors
 - 6: Each honest processor i receives bits $b'_1, \dots, b'_n$ (where $b'_j = b_j$ for honest j , and b'_j is controlled by the adversary for faulty j)
 - 7: *// Majority voting*
 - 8: Let $M_i = \text{majority of } \{b'_1, \dots, b'_n\}$ (breaking ties consistently, e.g., in favor of 0)
 - 9: *// Repeat for $O(\log n)$ rounds and take the majority of majorities*
 - 10: **for** $k = 1$ to $c \log n$ for a suitable constant c **do**
 - 11: Repeat the above (fresh random bits, broadcast, majority) to get value $M_i^{(k)}$
 - 12: **end for**
 - 13: Output the majority of $\{M_i^{(1)}, \dots, M_i^{(c \log n)}\}$
 - 14: **end procedure**
-

Theorem 13.22 (Randomized Byzantine Agreement). *For $n \geq 3t + 1$, the randomized Byzantine agreement algorithm achieves agreement and validity with probability at least $1 - n^{-c}$ for any constant c , using $O(\log n)$ rounds of communication. If $n > 3t$ and the majority of honest processors have the same input, the algorithm achieves agreement on that input value with high probability.*

Proof. Consider a single round of the protocol. Each honest processor i picks a random bit b_i uniformly and independently. There are $n - t'$ honest processors (where $t' \leq t$ is the number of actual faulty processors), and t' faulty processors controlled by the adversary.

For validity: suppose all honest processors have input $x \in \{0,1\}$. Consider the case where the adversary tries to make the honest processors output $1 - x$. The adversary controls at most t processors and can make them broadcast any values. The honest processors broadcast their random bits. After the broadcast, each honest processor computes the majority. The adversary's t processors can contribute at most t votes. The honest processors contribute $n - t'$ votes. If $n - t' > t$ (i.e., $n > 2t$), the majority of honest processors' random bits determines the majority. Since each honest processor's bit is 0 or 1 with equal probability, the probability that the majority of honest bits is x is at least $1/2 + \Omega(\sqrt{n}/n)$ by the Chernoff bound. Actually, we need a more careful argument since the adversary sees the honest processors' random bits before choosing its values.

For the one-round case with $n \geq 3t + 1$: each honest processor i picks $b_i \in \{0,1\}$ uniformly. The adversary controls t processors. After the broadcast, each honest processor sees $n - t$ random bits from honest processors and t bits from the adversary. The adversary can set its bits to be the opposite of the majority of honest bits, but this can swing the overall majority by at most t . Let H be the number of honest processors (with $H = n - t'$ for $t' \leq t$) and $F = t'$ be the number of faulty ones. The honest bits form a binomial distribution: $\text{Bin}(H, 1/2)$ ones and $H - \text{Bin}(H, 1/2)$ zeros.

For validity (all honest inputs are x): each honest processor picks b_i uniformly. The number of honest processors broadcasting 1 is $X \sim \text{Bin}(H, 1/2)$. The adversary can add at most t ones or zeros. The final majority is 1 if $X + a_1 > (H - X) + a_0$ where $a_1 + a_0 \leq t$ and $a_0, a_1 \geq 0$. This is equivalent to $X > H/2 + (a_0 - a_1)/2$, which requires $X > H/2 - t/2$. Since $X \sim \text{Bin}(H, 1/2)$, $\Pr[X > H/2 - t/2] \geq \Pr[X > H/2 - t/2] \geq 1/2$ when $t < H/2$ (which holds since $n \geq 3t + 1$ implies $H = n - t \geq 2t + 1 > 2t$).

More carefully: the adversary wants to make the majority output $1 - x$ (say 1). It adds t ones. The majority is 1 if $X + t > H - X$, i.e., $X > (H - t)/2$. By the Chernoff bound,

$\Pr[X > (H - t)/2] \geq 1 - e^{-\Omega(H)}$ when H is large relative to t . Actually, $(H - t)/2 < H/2$ when $t > 0$, so this is even easier. $\Pr[X > (H - t)/2] \geq \Pr[X \geq H/2] \geq 1/2$ by symmetry of the binomial distribution.

For agreement: after $c \log n$ independent rounds, each producing a majority value, the honest processors compute the majority of these majorities. By Chernoff bounds, the majority of majorities is determined correctly with probability $1 - n^{-c'}$ for suitable constants c, c' . Hence agreement is achieved w.h.p. $\square$

Remark 13.23. The randomized protocol achieves $O(1)$ -round agreement when repeated a constant number of times and the outputs are compared (“check-and-abort” paradigm): if two rounds give different outputs, abort and restart. The probability of disagreement per pair of rounds is small, so the expected number of restarts is $O(1)$. This gives expected constant-round agreement.

C++ Implementation

Listing 13.1: PRAM simulation: parallel prefix sum, list ranking, connected components.

```

1 #pragma once
2 #include <vector>
3 #include <functional>
4 #include <atomic>
5 #include <thread>
6 #include <latch>
7 #include <barrier>
8 #include <random>
9 #include <print>
10 #include <format>
11 #include <numeric>
12 #include <algorithm>
13 #include <cmath>
14 #include <cassert>
15 #include <chrono>
16 #include <sstream>
17 #include <string>
18
19 namespace randalgo {
20
```

```

21 //
22 // Simplified PRAM (Parallel Random Access Machine) simulation
23 //
24
25 class PRAM {
26 public:
27     int num_processors;
28     std::vector<long long> global_memory;
29     std::vector<std::vector<long long>> local_memory;
30     mutable std::mt19937 rng{std::random_device{}()};
31
32     explicit PRAM(int p, int mem_size = 0)
33         : num_processors(p),
34           global_memory(mem_size, 0),
35           local_memory(p, std::vector<long long>(16, 0)) {}
36
37     // Execute a parallel step: every processor i runs fn(i,
38     // global_memory) and
39     // the results are gathered into a per-processor vector.
40     std::vector<long long> execute_step(
41         std::function<long long(int, std::vector<long long>&)>
42         fn) {
43
44         std::vector<long long> results(num_processors);
45         std::vector<std::jthread> threads;
46         threads.reserve(num_processors);
47
48         for (int i = 0; i < num_processors; ++i) {
49             threads.emplace_back([this, i, &fn, &results]() {
50                 results[i] = fn(i, this->global_memory);
51             });
52         }
53         // jthreads join automatically
54         return results;
55     }
56
57     // Parallel prefix sum (Hillis-Steel algorithm) using p
58     // processors.
59     // Input length n is padded to num_processors.
60     std::vector<long long> prefix_sum_parallel(const std::
61         vector<long long>& input) {
62         int n = static_cast<int>(input.size());
63         int p = num_processors;
64
65         std::vector<long long> x(p, 0);
66         for (int i = 0; i < std::min(n, p); ++i)
67             x[i] = input[i];
68
69         std::println("\n=== Parallel Prefix Sum (Hillis-Steel)
70             ===");
71         std::println("Input ({} elements):", n);
72         print_vector(input);

```

```

68     int steps = static_cast<int>(std::ceil(std::log2(p)));
69     for (int d = 0; d < steps; ++d) {
70         int stride = 1 << d;
71         std::println("\nStep {}/{ } (stride = {}):", d + 1,
72                     steps, stride);
73
74         std::vector<long long> old_x = x;
75         auto results = execute_step([&](int i, std::vector<
76             long long>&) -> long long {
77             if (i < p && i >= stride) {
78                 return old_x[i] + old_x[i - stride];
79             }
80             return old_x[i];
81         });
82         x = results;
83         print_vector(x);
84     }
85
86     std::println("Prefix sum result:");
87     print_vector(x);
88     return x;
89 }
90
91 // Parallel list ranking.
92 // Given a next-pointer array (next[i] = index of successor
93 // , -1 for tail),
94 // compute the distance from each node to the tail in
95 // parallel.
96 std::vector<long long> list_ranking_parallel(const std::
97     vector<int>& next_ptr) {
98     int n = static_cast<int>(next_ptr.size());
99
100     std::println("\n=== Parallel List Ranking ===");
101     std::println("List of { } nodes:", n);
102
103     // Distance starts at 1 for every node with a successor
104     // , 0 for tail
105     std::vector<long long> rank(n, 0);
106     std::vector<int> next_cur = next_ptr;
107
108     for (int i = 0; i < n; ++i) {
109         if (next_cur[i] != -1) rank[i] = 1;
110     }
111
112     print_list(next_cur, rank);
113
114     // Parallel pointer jumping: repeat until all ranks are
115     // final.
116     // In the PRAM model this takes O(log n) steps.
117     int max_steps = static_cast<int>(std::ceil(std::log2(n
118         + 1)));
119     for (int step = 0; step < max_steps; ++step) {
120         std::vector<long long> old_rank = rank;
121         std::vector<int> old_next = next_cur;
122         bool any_active = false;

```

```

116         auto rank_results = execute_step([&](int i, std::
117         vector<long long>&) -> long long {
118             if (i < n && old_next[i] != -1) {
119                 return old_rank[i] + old_rank[old_next[i]];
120             }
121             return old_rank[i];
122         });
123
124         auto next_results = execute_step([&](int i, std::
125         vector<long long>&) -> long long {
126             if (i < n && old_next[i] != -1) {
127                 return static_cast<long long>(old_next[
128                     old_next[i]]);
129             }
130             return static_cast<long long>(old_next[i]);
131         });
132
133         for (int i = 0; i < n; ++i) {
134             rank[i] = rank_results[i];
135             int new_next = static_cast<int>(next_results[i
136                 ]);
137             if (new_next != next_cur[i]) any_active = true;
138             next_cur[i] = new_next;
139         }
140
141         std::println("Step {}: ranks = ", step + 1);
142         print_list(next_cur, rank);
143         if (!any_active) break;
144     }
145
146     std::println("Final list ranks:");
147     for (int i = 0; i < n; ++i) {
148         std::println("  node {} -> rank {}", i, rank[i]);
149     }
150     return rank;
151 }
152
153 // Parallel connected components using label propagation.
154 // Each vertex starts with its own label; in each round
155 // every vertex
156 // adopts the minimum label among itself and its neighbours
157 .
158 std::vector<int> connected_components_parallel(
159     const std::vector<std::vector<int>>& adj) {
160
161     int n = static_cast<int>(adj.size());
162     std::println("\n=== Parallel Connected Components ===");
163     ;
164     std::println("Graph with {} vertices:", n);
165     for (int i = 0; i < n; ++i) {
166         if (!adj[i].empty())
167             std::println("  {} -> [{}]", i, fmt_join(adj[i]
168                 ));
169     }

```



```

164     std::vector<int> label(n);
165     std::iota(label.begin(), label.end(), 0);
166
167     int max_rounds = static_cast<int>(std::ceil(std::log2(n
168         + 1)));
169     for (int round = 0; round < max_rounds; ++round) {
170         std::vector<int> old_label = label;
171         bool changed = false;
172
173         auto results = execute_step([&](int i, std::vector<
174             long long>&) -> long long {
175             if (i >= n) return 0;
176             int mn = old_label[i];
177             for (int nb : adj[i]) mn = std::min(mn,
178                 old_label[nb]);
179             return static_cast<long long>(mn);
180         });
181         for (int i = 0; i < n; ++i) {
182             label[i] = static_cast<int>(results[i]);
183             if (label[i] != old_label[i]) changed = true;
184         }
185         std::println("Round {}: labels = [{}]", round + 1,
186             fmt_join_int(label));
187         if (!changed) break;
188     }
189
190     // Count components
191     std::vector<int> unique_labels = label;
192     std::sort(unique_labels.begin(), unique_labels.end());
193     unique_labels.erase(std::unique(unique_labels.begin(),
194         unique_labels.end()),
195         unique_labels.end());
196     std::println("Found {} connected component(s)",
197         unique_labels.size());
198     return label;
199 }
200
201 // Sequential prefix sum for comparison
202 static std::vector<long long> prefix_sum_sequential(const
203     std::vector<long long>& input) {
204     std::vector<long long> out = input;
205     for (size_t i = 1; i < out.size(); ++i)
206         out[i] += out[i - 1];
207     return out;
208 }
209
210 private:
211 void print_vector(const std::vector<long long>& v) const {
212     std::print("  [");
213     for (size_t i = 0; i < v.size(); ++i) {
214         if (i) std::print(", ");
215         std::print("{} ", v[i]);
216     }
217     std::println("]");

```

```

213     }
214
215     void print_list(const std::vector<int>& next, const std::
        vector<long long>& rank) const {
216         for (size_t i = 0; i < next.size(); ++i) {
217             std::print("  node {} -> next={}, rank={}", i,
218                 next[i] == -1 ? std::string("NULL") :
                    std::to_string(next[i]),
219                 rank[i]);
220             std::println();
221         }
222     }
223
224     static std::string fmt_join(const std::vector<int>& v) {
225         std::ostringstream oss;
226         for (size_t i = 0; i < v.size(); ++i) {
227             if (i) oss << ", ";
228             oss << v[i];
229         }
230         return oss.str();
231     }
232
233     static std::string fmt_join_int(const std::vector<int>& v)
        {
234         return fmt_join(v);
235     }
236 };
237
238 //
    -----
239 // Demonstration
240 //
    -----
241
242 inline void demonstrate_pram() {
243     std::println("
        +=====+
244     ");
245     std::println("|    Chapter 12: PRAM Simulation -- Parallel
        Algorithms    |");
246     std::println("
        +=====+
247     ");
248
249     // --- Parallel Prefix Sum ---
250     {
251         int n = 16;
252         std::vector<long long> input(n);
253         std::iota(input.begin(), input.end(), 1);
254
255         PRAM pram(16, n);
256         auto t0 = std::chrono::steady_clock::now();
257         auto parallel_result = pram.prefix_sum_parallel(input);
258         auto t1 = std::chrono::steady_clock::now();

```

```

257     auto t2 = std::chrono::steady_clock::now();
258     auto seq_result = PRAM::prefix_sum_sequential(input);
259     auto t3 = std::chrono::steady_clock::now();
260
261     std::println("\nSequential result:");
262     std::print("  [");
263     for (size_t i = 0; i < seq_result.size(); ++i) {
264         if (i) std::print(", ");
265         std::print("{} ", seq_result[i]);
266     }
267     std::println("]");
268
269     bool match = (parallel_result == seq_result);
270     std::println("\nResults match: {}", match ? "YES" : "NO");
271
272     std::println("Parallel time: {} us",
273         std::chrono::duration_cast<std::chrono::
274             microseconds>(t1 - t0).count());
275     std::println("Sequential time: {} us",
276         std::chrono::duration_cast<std::chrono::
277             microseconds>(t3 - t2).count());
278
279     }
280
281     // --- Parallel List Ranking ---
282     {
283         // 0 -> 1 -> 2 -> 3 -> 4 (tail)
284         std::vector<int> next_ptr = {1, 2, 3, 4, -1};
285         PRAM pram(4, 0);
286         pram.list_ranking_parallel(next_ptr);
287     }
288
289     // --- Parallel Connected Components ---
290     {
291         // Graph: 0-1-2, 3-4 (two components)
292         std::vector<std::vector<int>>> adj = {
293             {1}, {0, 2}, {1}, {4}, {3}
294         };
295         PRAM pram(8, 0);
296         auto labels = pram.connected_components_parallel(adj);
297     }
298 }
299 // namespace randalgo

```

Listing 13.2: Luby's maximal independent set algorithm and vertex coloring.

```

1 #pragma once
2 #include <vector>
3 #include <set>
4 #include <unordered_set>
5 #include <algorithm>
6 #include <random>
7 #include <functional>

```

```
8 #include <atomic>
9 #include <thread>
10 #include <latch>
11 #include <print>
12 #include <format>
13 #include <cmath>
14 #include <sstream>
15 #include <numeric>
16 #include <cassert>
17
18 namespace randalgo {
19
20 //
21 // -----
22
23 // Graph representation
24 //
25 // -----
26
27 struct Graph {
28     int n; // number of vertices
29     std::vector<std::vector<int>> adj;
30
31     explicit Graph(int n_) : n(n_), adj(n_) {}
32
33     void add_edge(int u, int v) {
34         adj[u].push_back(v);
35         adj[v].push_back(u);
36     }
37
38     int degree(int v) const { return static_cast<int>(adj[v].
39         size()); }
40 };
41
42 //
43 // -----
44
45 // Independent set result
46 //
47 // -----
48
49 struct IndependentSet {
50     std::vector<bool> in_set;
51
52     explicit IndependentSet(int n = 0) : in_set(n, false) {}
53
54     int size() const {
55         return static_cast<int>(std::count(in_set.begin(),
56             in_set.end(), true));
57     }
58
59     std::vector<int> vertices() const {
60         std::vector<int> result;
61         for (int i = 0; i < static_cast<int>(in_set.size()); ++
```

```

54         i)
55         if (in_set[i]) result.push_back(i);
56         return result;
57     }
58     void print() const {
59         auto verts = vertices();
60         std::print("  IS = {{"");
61         for (size_t i = 0; i < verts.size(); ++i) {
62             if (i) std::print(", ");
63             std::print("{}", verts[i]);
64         }
65         std::println("}} (size = {})", size());
66     }
67 };
68
69 //
70 // -----
71 // Verify that a set is independent
72 // -----
73 inline bool verify_independent_set(const Graph& g, const
74 IndependentSet& is) {
75     auto verts = is.vertices();
76     std::unordered_set<int> in_set(verts.begin(), verts.end());
77     for (int v : verts) {
78         for (int nb : g.adj[v]) {
79             if (in_set.count(nb)) return false;
80         }
81     }
82     return true;
83 }
84 //
85 // -----
86 // Verify that a set is a valid matching
87 // -----
88 struct Matching {
89     std::vector<std::pair<int,int>> edges;
90
91     int size() const { return static_cast<int>(edges.size()); }
92
93     void print() const {
94         std::print("  Matching = {{"");
95         for (size_t i = 0; i < edges.size(); ++i) {
96             if (i) std::print(", ");
97             std::print("({},{})", edges[i].first, edges[i].
98                 second);
99         }
100     }

```

```

99         std::println("{} (size = {})", size(), size());
100     }
101 };
102
103 inline bool verify_matching(const Graph& g, const Matching& m)
104 {
105     std::vector<int> used(g.n, 0);
106     for (auto [u, v] : m.edges) {
107         if (u < 0 || u >= g.n || v < 0 || v >= g.n) return
108             false;
109         if (used[u]++ || used[v]++) return false;
110         // Verify edge exists
111         bool found = false;
112         for (int nb : g.adj[u]) {
113             if (nb == v) { found = true; break; }
114         }
115         if (!found) return false;
116     }
117     return true;
118 }
119
120 // -----
121
122 // Luby's Maximal Independent Set -- parallel version
123 // -----
124
125 inline IndependentSet mis_luby_parallel(const Graph& g) {
126     std::println("\n=== Luby's MIS Algorithm (Parallel) ===");
127
128     std::mt19937 rng{std::random_device{}()};
129     std::vector<bool> alive(g.n, true);
130     IndependentSet mis(g.n);
131     int round = 0;
132
133     while (true) {
134         ++round;
135         // Count alive vertices
136         int alive_count = 0;
137         for (int i = 0; i < g.n; ++i)
138             if (alive[i]) ++alive_count;
139
140         if (alive_count == 0) break;
141
142         std::println("\n--- Round {} ({} alive vertices) ---",
143             round, alive_count);
144
145         // Phase 1: Each alive vertex picks a random number
146         std::vector<double> random_vals(g.n, -1.0);
147         for (int i = 0; i < g.n; ++i) {
148             if (alive[i]) {
149                 std::uniform_real_distribution<double> dist
150                     (0.0, 1.0);
151                 random_vals[i] = dist(rng);
152             }
153         }
154     }
155 }

```

```

147     }
148 }
149
150 // Phase 1: Keep edge if this vertex has the larger
151 // random value
152 // among alive neighbours -> candidate set
153 std::vector<bool> candidate(g.n, false);
154 for (int i = 0; i < g.n; ++i) {
155     if (!alive[i]) continue;
156     bool is_max = true;
157     for (int nb : g.adj[i]) {
158         if (alive[nb] && random_vals[nb] > random_vals[
159             i]) {
160             is_max = false;
161             break;
162         }
163     }
164     candidate[i] = is_max;
165 }
166
167 std::print(" Candidates: ");
168 for (int i = 0; i < g.n; ++i)
169     if (candidate[i]) std::print("{}({:.3f}) ", i,
170         random_vals[i]);
171 std::println();
172
173 // Phase 2: Among candidates, independently add to MIS
174 // with prob 1/(2*d)
175 std::vector<bool> in_mis_now(g.n, false);
176 for (int i = 0; i < g.n; ++i) {
177     if (!candidate[i]) continue;
178     int d = std::max(1, g.degree(i));
179     std::uniform_real_distribution<double> dist(0.0,
180         1.0);
181     double threshold = 1.0 / (2.0 * d);
182     if (dist(rng) < threshold) {
183         in_mis_now[i] = true;
184         mis.in_set[i] = true;
185     }
186 }
187
188 std::print(" Added to MIS: ");
189 for (int i = 0; i < g.n; ++i)
190     if (in_mis_now[i]) std::print("{} ", i);
191 std::println();
192
193 // Phase 3: Remove MIS vertices and their neighbours
194 // from alive set
195 for (int i = 0; i < g.n; ++i) {
196     if (!in_mis_now[i]) continue;
197     alive[i] = false;
198     for (int nb : g.adj[i])
199         alive[nb] = false;
200 }

```

```

197     std::println("\nFinal MIS:");
198     mis.print();
199     std::println("Is independent set: {}",
200         verify_independent_set(g, mis) ? "YES" : "NO");
201     return mis;
202 }
203
204 //
205 // -----
206 // Random Maximal Matching
207 // -----
208
209 inline Matching maximal_matching(const Graph& g) {
210     std::println("\n=== Random Maximal Matching ===");
211
212     std::mt19937 rng{std::random_device{}()};
213     std::vector<bool> used(g.n, false);
214     Matching matching;
215
216     // Collect all edges
217     std::vector<std::pair<int,int>> all_edges;
218     for (int u = 0; u < g.n; ++u)
219         for (int v : g.adj[u])
220             if (u < v) all_edges.emplace_back(u, v);
221
222     // Shuffle edges
223     std::shuffle(all_edges.begin(), all_edges.end(), rng);
224
225     std::println(" Processing {} edges:", all_edges.size());
226     for (auto [u, v] : all_edges) {
227         if (!used[u] && !used[v]) {
228             matching.edges.emplace_back(u, v);
229             used[u] = used[v] = true;
230             std::println(" Added edge ({} , {})", u, v);
231         }
232     }
233
234     matching.print();
235     std::println("Is valid matching: {}",
236         verify_matching(g, matching) ? "YES" : "NO");
237     return matching;
238 }
239 //
240 // -----
241 // Greedy Vertex Coloring after MIS removal
242 // -----
243
244 inline std::vector<int> vertex_coloring(const Graph& g) {
245     std::println("\n=== Greedy Vertex Coloring ===");

```



```

245     std::vector<int> color(g.n, -1);
246     int num_colors = 0;
247
248
249     for (int v = 0; v < g.n; ++v) {
250         // Find colors used by neighbours
251         std::set<int> used_colors;
252         for (int nb : g.adj[v]) {
253             if (color[nb] != -1)
254                 used_colors.insert(color[nb]);
255         }
256         // Assign smallest available color
257         int c = 0;
258         while (used_colors.count(c)) ++c;
259         color[v] = c;
260         num_colors = std::max(num_colors, c + 1);
261         std::println(" Vertex {} -> color {}", v, c);
262     }
263
264     std::println("Total colors used: {}", num_colors);
265
266     // Verify proper coloring
267     bool proper = true;
268     for (int u = 0; u < g.n; ++u) {
269         for (int v : g.adj[u]) {
270             if (color[u] == color[v]) { proper = false; break;
271             }
272         }
273     }
274     std::println("Proper coloring: {}", proper ? "YES" : "NO");
275     return color;
276 }
277 //
278 // -----
279 // Build small test graphs
280 // -----
281
282 inline Graph make_path(int n) {
283     Graph g(n);
284     for (int i = 0; i < n - 1; ++i) g.add_edge(i, i + 1);
285     return g;
286 }
287
288 inline Graph make_cycle(int n) {
289     Graph g(n);
290     for (int i = 0; i < n; ++i) g.add_edge(i, (i + 1) % n);
291     return g;
292 }
293
294 inline Graph make_star(int center, int n_leaves) {
295     int n = center + 1 + n_leaves;
296     Graph g(n);

```

```

296     for (int i = 1; i <= n_leaves; ++i) g.add_edge(center,
297           center + i);
298     return g;
299 }
300 inline Graph make_random(int n, double edge_prob, int seed =
301     42) {
302     Graph g(n);
303     std::mt19937 rng(seed);
304     std::uniform_real_distribution<double> dist(0.0, 1.0);
305     for (int i = 0; i < n; ++i)
306         for (int j = i + 1; j < n; ++j)
307             if (dist(rng) < edge_prob) g.add_edge(i, j);
308     return g;
309 }
310 //
311 // -----
312 // Demonstration
313 // -----
314 inline void demonstrate_mis() {
315     std::println("
316         +=====+
317         ");
318     std::println("|    Chapter 12: Maximal Independent Set &
319         Matching    |");
320     std::println("
321         +=====+
322         ");
323     // --- Path graph ---
324     {
325         std::println("\n* Path graph P5");
326         auto g = make_path(5);
327         auto mis = mis_luby_parallel(g);
328         auto mm = maximal_matching(g);
329     }
330     // --- Cycle graph ---
331     {
332         std::println("\n* Cycle graph C6");
333         auto g = make_cycle(6);
334         auto mis = mis_luby_parallel(g);
335         auto mm = maximal_matching(g);
336     }
337     // --- Star graph ---
338     {
339         std::println("\n* Star graph S(0, 4)");
340         auto g = make_star(0, 4);
341         auto mis = mis_luby_parallel(g);
342         auto mm = maximal_matching(g);

```

```

341     }
342
343     // --- Random graph ---
344     {
345         std::println("\n* Random graph G(12, 0.3)");
346         auto g = make_random(12, 0.3);
347         auto mis = mis_luby_parallel(g);
348         auto mm = maximal_matching(g);
349     }
350
351     // --- Vertex coloring ---
352     {
353         std::println("\n* Vertex coloring on P5");
354         auto g = make_path(5);
355         vertex_coloring(g);
356     }
357 }
358
359 } // namespace randalgo

```

Listing 13.3: Random and maximum matching algorithms.

```

1  #pragma once
2  #include "mis.h"
3  #include <vector>
4  #include <set>
5  #include <unordered_set>
6  #include <algorithm>
7  #include <random>
8  #include <functional>
9  #include <atomic>
10 #include <thread>
11 #include <latch>
12 #include <print>
13 #include <format>
14 #include <queue>
15 #include <limits>
16 #include <cassert>
17
18 namespace randalgo {
19
20     //
21     -----
22
21     // Random Maximal Matching -- each edge considered
22     // independently with
23     // probability 1/(2d), then clean up for maximality.
24     //
25     -----
26
27     inline Matching random_maximal_matching(const Graph& g) {
28         std::println("\n== Random Maximal Matching (Edge-
29             Independent) ==");
30
31         std::mt19937 rng{std::random_device{}()};

```

```

29     std::vector<bool> used(g.n, false);
30     Matching matching;
31
32     // Phase 1: each vertex independently picks an incident
33     edge
34     // with probability 1/(2d). We simulate by vertex-driven
35     randomness.
36     std::vector<int> selected_edge(g.n, -1); // which
37     neighbour each vertex selects
38     for (int v = 0; v < g.n; ++v) {
39         int d = g.degree(v);
40         if (d == 0) continue;
41         double prob = 1.0 / (2.0 * d);
42         std::uniform_real_distribution<double> dist(0.0, 1.0);
43         if (dist(rng) < prob) {
44             // Pick a random neighbour
45             std::uniform_int_distribution<int> ndist(0, d - 1);
46             int nb = g.adj[v][ndist(rng)];
47             selected_edge[v] = nb;
48         }
49     }
50
51     // Phase 2: an edge (u,v) is in the candidate matching if
52     // u selected v AND v selected u (both endpoints agree)
53     // OR at least one endpoint selected and the other didn't
54     // For simplicity: if either endpoint selected and neither
55     is used
56     for (int v = 0; v < g.n; ++v) {
57         if (selected_edge[v] == -1) continue;
58         int u = selected_edge[v];
59         if (used[u] || used[v]) continue;
60         // Check if u also selected v (bidirectional) or if we
61         can take it
62         if (selected_edge[u] == v || selected_edge[u] == -1 ||
63             u < v) {
64             matching.edges.emplace_back(std::min(u, v), std:::
65                 max(u, v));
66             used[u] = used[v] = true;
67         }
68     }
69
70     // Phase 3: greedily add remaining edges for maximality
71     std::vector<std::pair<int,int>> all_edges;
72     for (int u = 0; u < g.n; ++u)
73         for (int v : g.adj[u])
74             if (u < v) all_edges.emplace_back(u, v);
75
76     std::mt19937 rng2(std::random_device{}());
77     std::shuffle(all_edges.begin(), all_edges.end(), rng2);
78
79     for (auto [u, v] : all_edges) {
80         if (!used[u] && !used[v]) {
81             matching.edges.emplace_back(u, v);
82             used[u] = used[v] = true;
83         }
84     }
85 }

```

```

78     matching.print();
79     std::println("Is valid matching: {}",
80         verify_matching(g, matching) ? "YES" : "NO");
81     return matching;
82 }
83
84 //
85 -----
86 // Maximum matching via augmenting paths (Hopcroft-Karp style
87 // for
88 // general graphs using BFS to find shortest augmenting paths).
89 // This is a simplified implementation suitable for moderate-
90 // size graphs.
91 -----
92
93 inline Matching blossom_matching(const Graph& g) {
94     std::println("\n=== Maximum Matching (Augmenting Paths) ===");
95
96     int n = g.n;
97     std::vector<int> match(n, -1); // match[v] = partner, -1
98                                     // if free
99
100     // Find augmenting path using BFS from each free vertex
101     auto try_augment = [&](int start) -> bool {
102         std::vector<int> parent(n, -1);
103         std::vector<bool> visited(n, false);
104         std::queue<int> q;
105         q.push(start);
106         visited[start] = true;
107
108         while (!q.empty()) {
109             int u = q.front(); q.pop();
110             for (int v : g.adj[u]) {
111                 if (visited[v]) continue;
112                 visited[v] = true;
113                 if (match[v] == -1) {
114                     // Found augmenting path: trace back
115                     int cur = u;
116                     int w = v;
117                     while (w != -1) {
118                         int prev = parent[cur];
119                         match[w] = cur;
120                         match[cur] = w;
121                         cur = prev;
122                         w = (prev != -1) ? parent[prev] : -1;
123                     }
124                     return true;
125                 }
126             }
127         }
128         // v is matched; follow matched edge
129         int w = match[v];
130         if (!visited[w]) {

```

```

126         parent[w] = u;
127         visited[w] = true;
128         q.push(w);
129     }
130 }
131 }
132 return false;
133 };
134
135 // Repeatedly find augmenting paths
136 int aug_count = 0;
137 for (int v = 0; v < n; ++v) {
138     if (match[v] == -1) {
139         if (try_augment(v)) {
140             ++aug_count;
141             std::println(" Augmentation {} found",
142                         aug_count);
143         }
144     }
145 }
146
147 Matching matching;
148 std::vector<bool> counted(n, false);
149 for (int v = 0; v < n; ++v) {
150     if (match[v] != -1 && !counted[v]) {
151         matching.edges.emplace_back(std::min(v, match[v]),
152                                     std::max(v, match[v]));
153         counted[v] = counted[match[v]] = true;
154     }
155 }
156
157 matching.print();
158 std::println("Is valid matching: {}",
159             verify_matching(g, matching) ? "YES" : "NO");
160 std::println("Maximum matching size: {}", matching.size());
161 return matching;
162 }
163
164 // -----
165
166 // 1/2-approximate maximum matching via random greedy
167 // -----
168
169 inline Matching approximate_max_matching(const Graph& g) {
170     std::println("\n=== 1/2-Approximate Max Matching (Random
171                 Greedy) ===");
172
173     std::mt19937 rng{std::random_device{}()};
174     int n = g.n;
175
176     // Each vertex independently picks a random neighbour
177     // with probability 1/(2d)
178     std::vector<bool> in_matching(n, false);

```

```

175 Matching matching;
176
177 // Build candidate edges
178 std::vector<std::pair<int,int>> candidates;
179 for (int u = 0; u < n; ++u) {
180     int d = g.degree(u);
181     if (d == 0) continue;
182     double prob = 1.0 / (2.0 * d);
183     std::uniform_real_distribution<double> dist(0.0, 1.0);
184     if (dist(rng) < prob) {
185         std::uniform_int_distribution<int> ndist(0, d - 1);
186         int v = g.adj[u][ndist(rng)];
187         if (u < v)
188             candidates.emplace_back(u, v);
189         else
190             candidates.emplace_back(v, u);
191     }
192 }
193
194 // Deduplicate and shuffle
195 std::sort(candidates.begin(), candidates.end());
196 candidates.erase(std::unique(candidates.begin(), candidates
197                             .end()),
198                  candidates.end());
199
200 std::shuffle(candidates.begin(), candidates.end(), rng);
201
202 // Select non-conflicting edges
203 for (auto [u, v] : candidates) {
204     if (!in_matching[u] && !in_matching[v]) {
205         matching.edges.emplace_back(u, v);
206         in_matching[u] = in_matching[v] = true;
207     }
208 }
209
210 // Fill in greedily for maximality
211 std::vector<std::pair<int,int>> all_edges;
212 for (int u = 0; u < n; ++u)
213     for (int v : g.adj[u])
214         if (u < v) all_edges.emplace_back(u, v);
215
216 std::shuffle(all_edges.begin(), all_edges.end(), rng);
217
218 for (auto [u, v] : all_edges) {
219     if (!in_matching[u] && !in_matching[v]) {
220         matching.edges.emplace_back(u, v);
221         in_matching[u] = in_matching[v] = true;
222     }
223 }
224
225 matching.print();
226 std::println("Is valid matching: {}",
227              verify_matching(g, matching) ? "YES" : "NO");
228 return matching;
229 }
230
231 //

```

```

229 // Demonstration
230 //
231 -----
232 inline void demonstrate_matchings() {
233     std::println("
234         +=====+
235         ");
236     std::println("|    Chapter 12: Matchings -- Random & Maximum
237         |");
238     std::println("
239         +=====+
240         ");
241     // --- Bipartite graph ---
242     {
243         std::println("\n* Bipartite graph K_{{3,3}}");
244         // K_{{3,3}}: left {0,1,2}, right {3,4,5}
245         Graph g(6);
246         for (int i = 0; i < 3; ++i)
247             for (int j = 3; j < 6; ++j)
248                 g.add_edge(i, j);
249
250         auto rm = random_maximal_matching(g);
251         auto bm = blossom_matching(g);
252         auto am = approximate_max_matching(g);
253
254         std::println("\n    Comparison:");
255         std::println("        Random maximal:  {}", rm.size());
256         std::println("        Maximum (aug):   {}", bm.size());
257         std::println("        1/2-approx:      {}", am.size());
258     }
259     // --- Non-bipartite: odd cycle C5 ---
260     {
261         std::println("\n* Non-bipartite: C5");
262         auto g = make_cycle(5);
263
264         auto rm = random_maximal_matching(g);
265         auto bm = blossom_matching(g);
266         auto am = approximate_max_matching(g);
267
268         std::println("\n    Comparison (C5):");
269         std::println("        Random maximal:  {}", rm.size());
270         std::println("        Maximum:         {}", bm.size());
271         std::println("        1/2-approx:      {}", am.size());
272     }
273     // --- Petersen graph ---
274     {
275         std::println("\n* Petersen graph (10 vertices)");
276         // Petersen graph: outer cycle 0-1-2-3-4, inner
277             5-6-7-8-9
278         // cross edges: 0-5, 1-6, 2-7, 3-8, 4-9

```



```

276 // inner: 5-7, 7-9, 9-6, 6-8, 8-5 (star pattern)
277 Graph g(10);
278 for (int i = 0; i < 5; ++i) {
279     g.add_edge(i, (i + 1) % 5);           // outer
280     g.add_edge(i, i + 5);                 // cross
281     g.add_edge(i + 5, ((i + 2) % 5) + 5); // inner star
282 }
283
284 auto rm = random_maximal_matching(g);
285 auto bm = blossom_matching(g);
286 auto am = approximate_max_matching(g);
287
288 std::println("\n Comparison (Petersen):");
289 std::println(" Random maximal: {}", rm.size());
290 std::println(" Maximum: {}", bm.size());
291 std::println(" 1/2-approx: {}", am.size());
292 std::println(" (Petersen matching number = 5)");
293 }
294
295 // --- Random graph ---
296 {
297     std::println("\n* Random graph G(10, 0.4)");
298     auto g = make_random(10, 0.4);
299
300     auto rm = random_maximal_matching(g);
301     auto bm = blossom_matching(g);
302     auto am = approximate_max_matching(g);
303
304     std::println("\n Comparison (random graph):");
305     std::println(" Random maximal: {}", rm.size());
306     std::println(" Maximum: {}", bm.size());
307     std::println(" 1/2-approx: {}", am.size());
308 }
309 }
310
311 } // namespace randalgo

```

Notes

- [1] The PRAM model was introduced by Wyllie [78] and further developed by many researchers. Brent's theorem appears in [24]. The class NC was introduced by Cook [29] and studied extensively by Pippenger [71].
- [2] The bitonic sorting network is due to Batcher [18]. The AKS sorting network, achieving $O(\log n)$ depth, was a major breakthrough by Ajtai, Komlós, and Szemerédi [8].

BoxSort appears in [54].

- [3] Luby's randomized MIS algorithm is from [58]. The algorithm and its analysis are among the most celebrated results in randomized parallel computing. The $O(\log n)$ expected time was later shown to hold with high probability by Alon, Babai, and Itai [14].
- [4] The Isolating Lemma was introduced by Lovász [55] and used in the RNC matching algorithm of Karp and Sipser [47]. The exact matching problem was shown to be in **RNC** by Karp, Vazirani, and Vazirani [48].
- [5] The choice coordination problem was studied by Naor, Reingold, and Rosen [67]. The connection to the birthday paradox and its application to load balancing appear in [66].
- [6] The Byzantine agreement problem was introduced by Lamport, Shostak, and Pease [53]. The lower bound $n \geq 3t + 1$ for deterministic protocols was established in the same paper. Randomized Byzantine agreement was studied by Ben-Or [19] and Rabin [73], who showed that randomized protocols can circumvent the deterministic lower bound. The constant-round randomized protocol is due to Rabin and Ben-Or [73].
- [7] Dolev and Strong [33] characterized the conditions under which Byzantine agreement is solvable. The randomized constant-round protocol for Byzantine agreement was further refined by many authors, including Goldreich, Wigderson, and others [40].

Problems

Problem 13.24. Prove that Luby's algorithm (Algorithm 25) produces a valid maximal independent set regardless of the ran-

dom choices. That is, prove the correctness (not the time bound) of Luby's algorithm unconditionally.

Problem 13.25. Show that for any n -vertex graph G , Luby's algorithm (Algorithm 25) terminates in at most $2n$ rounds. Give an example of a graph on which the algorithm requires $\Omega(n)$ rounds in the worst case.

Problem 13.26. Using Brent's theorem (Theorem 13.4), show that any $O(n)$ work, $O(\log n)$ span PRAM algorithm can be run on $O(n/\log n)$ processors in $O(\log n)$ time.

Problem 13.27. Prove that the bitonic sorting network sorts $n = 2^k$ elements using $O(k^2) = O(\log^2 n)$ comparators in $O(\log^2 n)$ depth. [Hint: Show that the merging step for a bitonic sequence of length 2^k has depth k and uses $k \cdot 2^{k-1}$ comparators.]

Problem 13.28. Prove the lower bound $n \geq 3t + 1$ for deterministic Byzantine agreement in the oral messages model (Theorem 13.21). Be precise about the adversary's strategy and the indistinguishability argument.

Problem 13.29. Consider the single-round randomized Byzantine agreement protocol (Algorithm 29 with just one round instead of $O(\log n)$ rounds). Show that for $n \geq 5t + 1$, the single-round protocol achieves agreement and validity with probability at least $2/3$.

Problem 13.30. Let $G = (U, V, E)$ be a bipartite graph with $|U| = |V| = n$ and at least one perfect matching. Assign each edge an independent random weight from $\{1, \dots, 2n\}$. Prove that the expected number of perfect matchings that are NOT the unique minimum-weight perfect matching is at most $n/2^n \cdot |\mathcal{M}|$, and hence the probability that the minimum-weight PM is unique is at least $1 - n/2^n$.

Problem 13.31. Suppose n processors use the choice coordination algorithm (Algorithm 27) with m choices and $n = m$. Show that the expected number of phases needed for all processors to agree is $\Theta(\log \log m)$.

Problem 13.32. Modify Luby's algorithm (Algorithm 25) to work on an EREW PRAM (where concurrent reads are not allowed). What is the new running time? [Hint: Use pointer jumping or prefix computation to resolve concurrent reads.]

Problem 13.33. Let G be a non-bipartite graph on $2n$ vertices. Describe how to find a perfect matching (if one exists) in G using an **RNC** algorithm. [Hint: Reduce to the bipartite case by considering the bipartite double cover of G , or use the Tutte matrix approach directly.]

Chapter 14

Online Algorithms

An *online algorithm* must process its input piece by piece, making irrevocable decisions at each step without knowledge of future requests. This stands in sharp contrast to an *offline algorithm*, which receives the entire input in advance and may compute an optimal solution by examining all of it. The central question in the study of online algorithms is: *how much worse must an online algorithm perform compared to an optimal offline algorithm that sees the entire request sequence in advance?*

This degradation is captured by the *competitive ratio*, defined as the worst-case ratio of the cost incurred by the online algorithm to that of the optimal offline algorithm. Randomization plays a pivotal role in online algorithms because it makes the algorithm's behavior unpredictable to an adversary. Whereas a deterministic online algorithm is completely determined by the request sequence and hence can be "foiled" by a carefully chosen worst-case input, a randomized algorithm can distribute its cost over many possible actions, rendering the adversary's task of exploiting worst-case structure significantly harder.

In this chapter we study several fundamental online problems, with particular emphasis on the *online paging problem* and the *k-server problem*. We develop randomized algorithms that achieve competitive ratios strictly better than what deterministic algorithms can guarantee, and we analyze the in-

terplay between different adversary models that govern the power of an adversary against randomized algorithms.

14.1 The Online Paging Problem

Definition 14.1 (Online paging). Let $\mathcal{U}$ be a universe of n pages and let the algorithm maintain a *cache* (or *fast memory*) of k pages, where $k < n$. A *request sequence* $\sigma = \sigma_1, \sigma_2, \dots, \sigma_m$ is a sequence of pages drawn from $\mathcal{U}$. When request σ_i arrives, the algorithm must ensure that σ_i is in the cache; if it is not, a *cache miss* (or *page fault*) occurs and the algorithm must bring σ_i into the cache by evicting some page currently stored there. The cost of a request is 1 if it causes a miss and 0 if it is a hit. The total cost is the number of misses incurred over the entire sequence.

The goal is to minimize the total number of cache misses. The *optimal offline algorithm* (denoted OPT) knows the entire request sequence σ in advance and computes the minimum possible number of misses. We say an algorithm A is *c-competitive* if for every request sequence σ ,

$$\text{cost}_A(\sigma) \leq c \cdot \text{cost}_{\text{OPT}}(\sigma) + O(1).$$

14.1.1 Deterministic Algorithms and Their Limitations

Several natural deterministic strategies are immediately available:

- **LRU (Least Recently Used):** Evict the page whose last use was furthest in the past.
- **FIFO (First-In, First-Out):** Evict the page that has been in the cache the longest.
- **FAR (Furthest in the Future):** Evict the page whose next use is furthest in the future (this is the optimal offline algorithm).

While FAR achieves the optimal cost, it requires knowledge of future requests and is therefore offline. LRU and FIFO are online, but both can be forced to perform poorly.

Example 14.2. Consider a cache of size $k = 3$ with page universe $\{1, 2, 3, 4\}$. Suppose the cache initially contains $\{1, 2, 3\}$. Consider the request sequence

$$\sigma = 4, 1, 4, 2, 4, 3, 4, 1, 4, 2, 4, 3, 4, \dots$$

In the subsequence $4, 1, 4, 2, 4, 3, 4$, every third request is page 4, and pages 1, 2, 3 cycle through the remaining slots. Every request causes a miss for LRU (and similarly for FIFO), for a total cost of approximately $\frac{4}{3} \cdot |\sigma|$. However, the optimal offline algorithm, seeing the pattern in advance, can keep page 4 in the cache at all times and suffer only one miss per three requests, for a total cost of approximately $\frac{1}{3} \cdot |\sigma|$.

14.1.2 Lower Bound for Deterministic Algorithms

The example above generalizes. We now prove the classical lower bound showing that no deterministic online paging algorithm can achieve a competitive ratio better than $k - 1$.

Theorem 14.3. *Any deterministic online algorithm A for k -page caching satisfies*

$$\sup_{\sigma} \frac{\text{cost}_A(\sigma)}{\text{cost}_{\text{OPT}}(\sigma)} \geq k - 1.$$

Proof. Let the universe consist of $k + 1$ pages $\{p_0, p_1, \dots, p_k\}$. Consider the following adversarial strategy. The adversary presents requests in blocks. In each block, the adversary requests the $k + 1$ pages in some order that avoids the page currently in position k of algorithm A 's internal ordering (i.e., the page A would evict next). Specifically, the adversary maintains a set S of $k + 1$ pages. At the start of each block, A 's cache contains some k -element subset of S . The adversary requests the one page of S not in A 's cache, causing a miss. Then

it continues requesting other pages of S to rearrange A 's cache without reducing its own cost.

More precisely, suppose at the start of a block, A 's cache contains pages $q_1, q_2, \dots, q_k \in S$ and the missing page is p . The adversary presents the single request p . Algorithm A must evict some page q_j to bring in p , incurring a miss. Meanwhile, OPT can serve this request with a hit if p was already in OPT 's cache. By choosing p to be the unique page of S not in A 's cache, the adversary guarantees A misses while OPT can serve the request with cost 0 (after the first block, OPT keeps all of $S \setminus \{q\}$ for an appropriate page q in its cache, using k slots for k pages).

After each block, A 's cache has changed (one page was evicted and p was inserted), but still consists of k pages from S . The adversary repeats, choosing the new missing page. Each block costs A at least 1 miss while OPT pays 0 (after the initial cost of loading its cache). If we group the sequence into rounds of k blocks each, we can force A to miss $k - 1$ times per round while OPT misses only once. Hence

$$\frac{\text{cost}_A(\sigma)}{\text{cost}_{\text{OPT}}(\sigma)} \geq k - 1.$$

□

The above bound is tight for randomized algorithms, as we shall see.

14.2 Adversary Models

The competitive analysis of randomized algorithms requires a careful treatment of the adversary. Unlike the deterministic case, where the input is fixed and the algorithm is deterministic, the randomized algorithm involves internal randomness, and the adversary's knowledge of this randomness fundamentally affects the analysis.

Definition 14.4 (Competitive ratio). An online algorithm A (possibly randomized) is C -competitive if, for every input sequence σ ,

$$\mathbf{E}[\text{cost}_A(\sigma)] \leq C \cdot \text{cost}_{\text{OPT}}(\sigma) + O(1),$$

where the expectation is over the random coins of A .

14.2.1 The Three Adversary Models

There are three natural adversary models for randomized algorithms, ordered from weakest to strongest:

1. **Oblivious adversary:** The adversary must fix the entire input sequence σ *before* the algorithm begins execution. The adversary has no knowledge of the algorithm's random coins. This is the *weakest* adversary.
2. **Adaptive online adversary:** The adversary produces requests one at a time. When producing request σ_i , the adversary knows the algorithm's random coins and the algorithm's behavior up to step $i - 1$, but does *not* know the algorithm's future random coins (those that will be used after step i). This adversary is *intermediate* in power.
3. **Adaptive offline adversary:** The adversary sees the algorithm's entire sequence of random coins (and hence the algorithm's complete behavior) and then produces the worst-case input sequence. Equivalently, the adversary can view the algorithm as a function from random strings to behaviors, and choose the input to maximize cost against the worst-case random string. This is the *strongest* adversary.

Proposition 14.5. *For any randomized online algorithm A and problem Π , if A is C -competitive against an adaptive offline adversary, then A is also C -competitive against an adaptive online adversary, and hence against an oblivious adversary:*

$$C_{\text{oblivious}} \leq C_{\text{adaptive online}} \leq C_{\text{adaptive offline}}.$$

Proof. If A is C -competitive against an adaptive offline adversary, then for every input σ and every random string r of A ,

$\text{cost}_A(\sigma, r) \leq C \cdot \text{cost}_{\text{OPT}}(\sigma) + O(1)$. Taking expectations over r gives the bound against oblivious adversaries. The adaptive online adversary is weaker than the adaptive offline one because it must commit to each request without seeing the algorithm's future random coins, which can only reduce the adversary's ability to choose the worst-case input. $\square$

The hierarchy is strict in general: there exist problems and algorithms where the competitive ratio against an oblivious adversary is strictly better than against an adaptive offline adversary. However, as we shall see in Section 14.4, for the paging problem the situation is more nuanced.

14.2.2 The Randomized Competitive Guarantee

In the randomized setting, the competitive ratio is defined as the worst-case (over input sequences) of the ratio of the expected cost of the algorithm to the cost of OPT. Formally:

Definition 14.6 (Randomized competitive ratio). A randomized algorithm A is C -competitive against adversary $\mathcal{A}$ if for every input sequence σ (or every σ chosen by $\mathcal{A}$),

$$\mathbb{E}[\text{cost}_A(\sigma)] \leq C \cdot \text{cost}_{\text{OPT}}(\sigma) + O(1),$$

where $O(1)$ is a constant independent of $|\sigma|$.

14.3 Paging against an Oblivious Adversary

We now present a randomized algorithm for the k -page paging problem that achieves a competitive ratio of $O(k)$ against an oblivious adversary.

14.3.1 The Randomized Marking Algorithm

Definition 14.7 (Marking algorithm). The *randomized marking algorithm* for k -page caching operates as follows. Each page in the cache carries a *mark* (either *marked* or *unmarked*).

- **On a request to page p :**

1. If p is in the cache, mark p (if not already marked). This is a *hit*.
2. If p is not in the cache (a *miss*):
 - (a) If all k pages in the cache are marked, unmark all pages and then mark p . (Equivalently, start a new “phase.”)
 - (b) Otherwise (at least one page in the cache is unmarked), *evict a uniformly random unmarked page* and insert p , marking it.

The key insight is that the marking algorithm partitions the request sequence into *phases*. A new phase begins whenever all k pages in the cache are marked and a miss occurs. Within a phase, the algorithm only evicts randomly among unmarked pages.

14.3.2 Analysis against an Oblivious Adversary

We analyze the marking algorithm using a potential function argument. Consider the configuration of the algorithm’s cache A and the optimal offline algorithm’s cache O at any point during the execution.

Definition 14.8 (Potential function). At any point in the execution, define the potential

$$\Phi = |\{p \in A \setminus O\}|,$$

i.e., the number of pages in A ’s cache that are not in O ’s cache. Here A denotes the set of pages in the algorithm’s cache and O denotes the set of pages in OPT ’s cache.

Note that $0 \leq \Phi \leq k$ since $|A| = |O| = k$.

Lemma 14.9. *Within a phase, every request to a page that is in the cache is served with cost 0 (a hit) by both the marking algorithm and OPT .*

Proof. Within a phase, once a page is marked, it is never evicted (the algorithm only evicts unmarked pages). Thus all pages that have been requested at least once during the current phase remain in the cache until the phase ends. OPT may choose to evict such a page, but it need not. $\square$

Lemma 14.10. *When the marking algorithm suffers a miss on request σ_i (with at least one unmarked page in the cache), the probability that the evicted page is one that is not in OPT's cache is at least $1/k$.*

Proof. At the moment of eviction, the marking algorithm has k pages in cache, of which some subset U of size $|U| \geq 1$ are unmarked. It evicts a page chosen uniformly at random from U . Let $B = A \setminus O$ be the set of pages in A 's cache but not in O 's cache. Since $|A| = |O| = k$, we have $|B| = |A \setminus O| = |O \setminus A|$. At least $|B|$ pages in A are “bad” (not in OPT).

The unmarked pages U are a subset of A . At most $k - |U|$ pages of A are marked. In the worst case, all pages of $O \cap A$ could be marked, so $|U \cap B| \geq |U| - |O \cap A \cap U^c|$ is not directly computable. However, observe that $|A \setminus O| = |B|$ and $|U| \geq 1$. Since $|A \cap O| \leq k - |B|$ and $|U \cap A| = |U|$, we have

$$|U \cap B| = |U| - |U \cap O| \geq |U| - |A \cap O| \geq |U| - (k - |B|).$$

In the simplest case where $|U| = k$ (all pages are unmarked at the start of a phase), we get $|U \cap B| = |B| \geq 1$ (if $|B| > 0$). The probability of evicting a page in B is $|U \cap B|/|U|$.

We use a cleaner argument: when a miss occurs, the newly arriving page is not in A . Either it is in O or it is not. Either way, $|A \setminus O|$ does not decrease by more than 1 due to the insertion. For the eviction step, note that the algorithm picks uniformly at random from U . Among U , at least one page must be in $A \setminus O$ whenever $|A \setminus O| > 0$, because $|U| \leq k$ and $|A \cap O| \leq k - 1$ (since the new page is not in A , so $|A \cap O|$ could be at most k). In fact, since $|A \setminus O|$ counts pages in A not in O , and $|U| \geq 1$, by a counting argument at least a $1/k$

fraction of U lies in $A \setminus O$:

$$\frac{|U \cap (A \setminus O)|}{|U|} \geq \frac{|A \setminus O| - (k - |U|)}{|U|}.$$

When $|A \setminus O| \geq 1$, this is at least $1/k$ because in the worst case $|A \setminus O| = 1$ and $|U| = k$, giving probability $1/k$. $\square$

We now state the main theorem.

Theorem 14.11. *The randomized marking algorithm is $2k$ -competitive against an oblivious adversary for the k -page paging problem.*

Proof. We use the potential function $\Phi = |A \setminus O|$ as defined in Definition 14.8. Recall that $0 \leq \Phi \leq k$.

We partition the request sequence into *phases*. A phase ends when all k pages in the cache are marked and a new miss occurs (triggering a global unmark). Consider one phase. Let m_A be the number of misses the marking algorithm suffers in this phase, and let m_O be the number of misses OPT suffers in this phase (for the requests in this phase only, considering OPT's cache state at the start of the phase).

Claim: $m_O \geq m_A - k$.

Proof of claim: Each miss by the marking algorithm brings a new page into the cache. At the end of the phase, all k pages in the cache are marked, meaning k distinct pages were brought in during the phase (each was requested at least once while unmarked). OPT starts the phase with k pages. Each miss by OPT introduces one new page. To serve all the requests in the phase that cause misses for the marking algorithm, OPT must also miss on at least $m_A - k$ of them, because OPT's cache can contain at most k of the m_A pages that the marking algorithm brought in (OPT might already have some of them, but not all m_A). Hence $m_O \geq m_A - k$.

Now we do the potential function analysis. Over a phase, define $\Delta\Phi$ as the change in Φ from the start to the end of the phase. Since Φ measures $|A \setminus O|$ and the phase ends with a new marking configuration, we have $\Delta\Phi \leq k$ (since $\Phi \leq k$ at all times).

We relate costs and potential. Consider the amortized cost of the marking algorithm per phase:

$$m_A + \mathbf{E}[\Delta\Phi] \leq m_A + k.$$

By the claim, $m_A \leq m_O + k$. Thus the expected amortized cost is at most $m_O + 2k$. Since $\Phi \geq 0$ initially and $\Phi \geq 0$ always, summing over all phases gives

$$\mathbf{E}[\text{cost}_A(\sigma)] = \sum_{\text{phases}} m_A \leq \sum_{\text{phases}} (m_O + 2k) = \text{cost}_{\text{OPT}}(\sigma) + 2k \cdot (\text{number of phases})$$

Each phase has at least one miss by the marking algorithm (otherwise the phase wouldn't end), and OPT misses at least once per phase (since the page requested at the phase boundary is not in A 's cache and hence the adversary can arrange for OPT to miss as well, or more precisely the number of phases is at most $\text{cost}_{\text{OPT}}(\sigma)$ since each phase boundary corresponds to an OPT miss). Hence the number of phases is at most $\text{cost}_{\text{OPT}}(\sigma)$, and

$$\mathbf{E}[\text{cost}_A(\sigma)] \leq \text{cost}_{\text{OPT}}(\sigma) + 2k \cdot \text{cost}_{\text{OPT}}(\sigma) = (2k + 1) \cdot \text{cost}_{\text{OPT}}(\sigma).$$

This gives a competitive ratio of $2k + 1$, which is $O(k)$. $\square$

14.3.3 Improvement to k -Competitive

The bound above can be tightened to a k -competitive algorithm using a more refined analysis. The key observation is that the potential function Φ can be defined more carefully.

Theorem 14.12. *There exists a randomized marking algorithm that is k -competitive against an oblivious adversary.*

Proof sketch. Consider the same marking algorithm but with a refined potential function. Define $\Phi = |A \setminus O|$ as before. Analyze the algorithm on a per-request basis rather than per-phase. For a request that is a hit for the marking algorithm, the cost is 0 and Φ does not change (the page was already in A and marked). For a request that is a miss:

- The marking algorithm pays cost 1 and brings in a new page σ_i . This changes $|A \setminus O|$ by at most $+1$ (if $\sigma_i \notin O$) or -1 (if $\sigma_i \in O$).
- The eviction of a random unmarked page q changes $|A \setminus O|$ by -1 if $q \in A \setminus O$ and 0 if $q \in A \cap O$.

For a miss where $|A \setminus O| > 0$, the expected change in potential due to the eviction is at most $-1/k$ (by the argument that at least a $1/k$ fraction of unmarked pages are in $A \setminus O$). Combining the cost of the miss with the expected change in potential, we get an amortized cost of at most $1 - 1/k$ per miss. Summing over all misses and using $0 \leq \Phi \leq k$ gives

$$\mathbb{E}[\text{cost}_A(\sigma)] + 0 \leq (1 - 1/k)^{-1} \cdot \text{cost}_{\text{OPT}}(\sigma) + k = \frac{k}{k-1} \cdot \text{cost}_{\text{OPT}}(\sigma) +$$

which is $k/(k-1)$ -competitive. However, a more careful analysis that considers the interaction between the phases and the potential yields a k -competitive bound. The full details involve tracking the potential not just as $|A \setminus O|$ but weighted by a function of the number of marked pages. $\square$

14.4 Relating the Adversary Models

A natural question is whether the choice of adversary model affects the competitive ratio achievable by randomized algorithms for paging. We have seen that k -competitive algorithms exist against oblivious adversaries. What about against stronger adversaries?

Theorem 14.13. *For the k -page paging problem, any randomized algorithm that is C -competitive against an adaptive offline adversary is also C -competitive against an adaptive online adversary.*

Proof sketch. Suppose A is C -competitive against an adaptive offline adversary. We show A is also C -competitive against an adaptive online adversary $\mathcal{A}_{\text{on}}$.

The adaptive online adversary $\mathcal{A}_{\text{on}}$ produces requests one at a time. At step i , $\mathcal{A}_{\text{on}}$ chooses σ_i based on the algorithm's

coins r and the algorithm's behavior up to step $i - 1$. Consider the adaptive offline adversary $\mathcal{A}_{\text{off}}$ that, given the algorithm's entire random string r , simulates $\mathcal{A}_{\text{on}}$ by running the same adaptive strategy. Since $\mathcal{A}_{\text{off}}$ knows r in advance, it can predict exactly what $\mathcal{A}_{\text{on}}$ would do at each step. Thus for every r , the sequence produced by $\mathcal{A}_{\text{off}}$ is identical to that produced by $\mathcal{A}_{\text{on}}$ against A with coins r . Therefore

$$\mathbb{E}_r[\text{cost}_A(r, \sigma_r)] \leq C \cdot \text{cost}_{\text{OPT}}(\sigma_r) + O(1)$$

for every r , where σ_r is the sequence produced by $\mathcal{A}_{\text{on}}$ when A uses coins r . Taking expectations over r yields the C -competitive bound against $\mathcal{A}_{\text{on}}$. $\square$

Remark 14.14. The above argument shows that the adaptive online adversary is no stronger than the adaptive offline adversary for *any* online problem, not just paging. The converse is not true in general: there exist problems where the adaptive offline adversary is strictly stronger. The hierarchy collapses for paging because the adaptive online adversary can be “simulated” by the offline adversary.

Theorem 14.15. *No randomized algorithm for k -page caching can achieve a competitive ratio better than k against an adaptive offline adversary (and hence against any adversary).*

Proof. The argument is an adversary strategy. Consider a universe of $k + 1$ pages. The adversary maintains a set S of $k + 1$ candidate pages. At each step, the adversary requests a page that is not in the algorithm's cache (if possible). Since the algorithm's cache has k pages and $|S| = k + 1$, there is always at least one page in S not in the algorithm's cache.

The adversary can use the algorithm's random coins to predict which page the algorithm will evict and choose the request to force a miss. Specifically, given the algorithm's coins r , the adversary knows the algorithm's complete strategy. It requests the page of S not in the algorithm's cache. The algorithm must miss (cost 1). Meanwhile, OPT can always serve this request with cost at most $1/k$ on average (amortized), because OPT can rotate its cache contents to keep k of the $k + 1$

pages, missing only once every k requests. Thus each request costs the algorithm 1 and OPT at most $1/k$, giving a ratio of at least k .

More precisely, the adversary requests pages from S in a round-robin fashion, always picking the one not in the algorithm's cache. Each round of k requests costs the algorithm k misses while OPT misses at most once (after an initial cost of 1 to load its cache). Hence

$$\frac{\text{cost}_A(\sigma)}{\text{cost}_{\text{OPT}}(\sigma)} \geq k.$$

This holds for every random string r , so the expectation is at least k as well. $\square$

Combining with Theorem 14.12, we see that the k -competitive ratio is *optimal* for randomized paging against any adversary.

14.5 The Adaptive Online Adversary

The adaptive online adversary model provides a natural middle ground between the oblivious and fully adaptive offline models. In this section, we establish lower bounds and discuss the implications of this model for paging and related problems.

Theorem 14.16. *No randomized algorithm for k -page caching can achieve a competitive ratio better than k against an adaptive online adversary.*

Proof. The proof is essentially the same as that of Theorem 14.15, adapted to the online setting. The adaptive online adversary produces requests one at a time. At each step, the adversary knows the algorithm's coins r used so far (those that affected the algorithm's behavior up to this point) and the algorithm's current cache state.

The adversary maintains a set S of $k + 1$ pages. At each step, the adversary selects σ_i to be the page of S not in the

algorithm's current cache. Since $|S| = k + 1$ and the cache holds k pages, such a page always exists. This forces a miss (cost 1).

Meanwhile, OPT can keep k pages of S in its cache and rotate them. In the worst case, OPT misses once every k requests. Hence the cost ratio per request is at least k .

The adaptive online adversary does not need to know the algorithm's future random coins to execute this strategy—it only needs to know the current cache state, which is determined by the algorithm's past coins and requests. Therefore this strategy is valid against the adaptive online adversary. $\square$

14.5.1 Comparison of Models for Paging

For the paging problem, the competitive ratios achievable against the three adversary models are:

Adversary model	Best deterministic	Best randomized
Oblivious	$k - 1$	k
Adaptive online	$k - 1$	k
Adaptive offline	$k - 1$	k

The gap between the best deterministic ratio ($k - 1$) and the best randomized ratio (k) is small for paging. The advantage of randomization for paging is primarily in the *constant* hidden in the $O(\cdot)$ notation and in the adaptability of the algorithm.

Remark 14.17. For other problems, such as the k -server problem discussed in the next section, the gap between adversary models is much more dramatic, and randomization provides a significantly larger advantage.

14.6 The k -Server Problem

The k -server problem is a fundamental generalization of paging and one of the most important problems in the theory of

online algorithms.

Definition 14.18 (*k*-server problem). Let $(\mathcal{M}, d)$ be a metric space with n points. There are k servers located at points of $\mathcal{M}$ (initially at some fixed positions $s_1, \dots, s_k$). Requests arrive one at a time: request r_t is a point in $\mathcal{M}$. When r_t arrives, some server must be moved to r_t . If a server is already at r_t , the cost is 0. Otherwise, if server s_j at position p_j is moved to r_t , the cost is $d(p_j, r_t)$. The goal is to minimize the total cost of all movements.

Note that when $k = 1$ and $\mathcal{M}$ is a set of pages with the discrete metric, the k -server problem reduces to a variant of the paging problem. In general, the k -server problem captures many important applications, including caching, load balancing, and data migration.

14.6.1 Deterministic Lower Bound

Theorem 14.19 (Manasse, McGeoch, and Sleator). *Any deterministic online algorithm for the k -server problem on an n -point metric space is at least k -competitive. More precisely, there exist metric spaces on which no deterministic algorithm achieves a competitive ratio better than k .*

Proof sketch. Consider the metric space consisting of $k + 1$ points arranged in a line (or a complete graph with equal edge weights). The adversary places k servers at the first k points and then alternates requesting the $(k + 1)$ -th point and one of the first k points. At each step, the adversary can force the algorithm to move a server across a long distance, while OPT can move a server only a short distance. The resulting competitive ratio is at least k . $\square$

14.6.2 Randomized k -Competitive Algorithm

Theorem 14.20 (Koutsoupias and Papadimitriou). *There exists a randomized online algorithm for the k -server problem that is k -competitive on every metric space.*

The Koutsoupias–Papadimitriou (KP) algorithm is a landmark result. We describe the algorithm and its analysis.

The Algorithm

The KP algorithm maintains, in addition to the actual server configuration, a probability distribution over all possible configurations. At each step, it moves servers to minimize a potential function Φ defined over pairs of configurations.

Definition 14.21 (KP potential function). Let $S = (s_1, \dots, s_k)$ be the algorithm's server configuration and let $\mathcal{C}$ be the set of all possible configurations of k servers in $\mathcal{M}$. For configurations $C, C' \in \mathcal{C}$, define

$$d(C, C') = \min_{\pi} \sum_{i=1}^k d(s_i, s'_{\pi(i)}),$$

the minimum-cost matching between the servers of C and C' . The potential function is

$$\Phi(S, R) = \max_{C \in \mathcal{C}} [d(S, C) - \alpha \cdot d(R, C)],$$

where R is the configuration of OPT and $\alpha = k$ is the competitive ratio. Here $d(S, C)$ is the minimum-cost matching between the algorithm's servers S and the servers in configuration C .

The algorithm proceeds as follows: upon a request at point r , the algorithm moves a server to r so as to minimize Φ . It can be shown that a greedy minimization of Φ (moving the server that minimizes the maximum over C of the change in $d(S, C) - k \cdot d(R, C)$) achieves a competitive ratio of k .

Analysis

Lemma 14.22. *At each step, the expected change in the potential Φ satisfies*

$$\mathbb{E}[\Delta\Phi] \leq \alpha \cdot \text{cost}_{\text{OPT}}(r_t) - \text{cost}_A(r_t),$$

where $\alpha = k$.

Proof sketch. When request r_t arrives, OPT may move a server to r_t at cost $\text{cost}_{\text{OPT}}(r_t) \leq d(s_j, r_t)$ for some server s_j of OPT. The algorithm chooses a server s_i to move to r_t so that the decrease in $\text{cost}_A(r_t)$ is at least $1/\alpha$ times the decrease in Φ . By the definition of Φ as a maximum over configurations C , the change $\Delta\Phi$ can be bounded using the triangle inequality on the metric space. The key inequality is

$$\text{cost}_A(r_t) + \mathbf{E}[\Delta\Phi] \leq \alpha \cdot \text{cost}_{\text{OPT}}(r_t),$$

which, when summed over all steps and using $0 \leq \Phi \leq k \cdot \text{diam}(\mathcal{M})$ (which is bounded), gives the k -competitive ratio. $\square$

Proof of Theorem 14.20. By Lemma 14.22, at each step t ,

$$\text{cost}_A(r_t) + \mathbf{E}[\Delta\Phi_t] \leq k \cdot \text{cost}_{\text{OPT}}(r_t).$$

Summing over all T steps:

$$\sum_{t=1}^T \text{cost}_A(r_t) + \mathbf{E}[\Phi_T - \Phi_0] \leq k \sum_{t=1}^T \text{cost}_{\text{OPT}}(r_t).$$

Since Φ_0 is determined by the initial configurations and $\Phi_T \geq 0$, and both are bounded by k times the diameter of the metric space, we obtain

$$\text{cost}_A(\sigma) \leq k \cdot \text{cost}_{\text{OPT}}(\sigma) + \Phi_0 \leq k \cdot \text{cost}_{\text{OPT}}(\sigma) + O(k \cdot \text{diam}(\mathcal{M})).$$

This gives the k -competitive ratio. $\square$

14.6.3 Connection to Paging

Corollary 14.23. *When the metric space is the discrete metric on a set of n pages and k is the cache size, the k -server problem generalizes the k -page paging problem. The KP algorithm achieves a k -competitive ratio for paging, matching the lower bound of Theorem 14.15.*

14.6.4 The Hirschberg–Sinclair Technique

The analysis of the KP algorithm draws on ideas reminiscent of the Hirschberg–Sinclair technique for the LOAD BALANCING problem. The key idea is to consider *all possible* optimal configurations and use a potential function that is a maximum over these configurations. This “max-over-all-possibilities” approach is a powerful technique in the analysis of randomized online algorithms.

Remark 14.24. The k -competitive bound for k -server was the best known for many years. In 2021, Bubeck, Cohen, Lee, and Lee achieved a breakthrough result showing that the randomized competitive ratio for k -server on general metric spaces is at most $O(\log^2 k \log n)$, where n is the number of points in the metric space. However, the k -competitive bound remains the tight answer for many important special cases, and the KP algorithm remains one of the most elegant constructions in the field.

C++ Implementation

Listing 14.1: Online paging: LRU, FIFO, random, marking algorithm, and OPT.

```

1  #pragma once
2  // Chapter 13: Paging Algorithms
3  // Deterministic and randomized paging against an oblivious
   adversary.
4  // Competitive analysis: LRU, FIFO, Random, Marking, and OPT (
   Belady's).
5
6  #include <iostream>
7  #include <vector>
8  #include <deque>
9  #include <unordered_set>
10 #include <unordered_map>
11 #include <set>
12 #include <algorithm>
13 #include <random>
14 #include <chrono>
15 #include <iomanip>
16 #include <print>
17 #include <format>
18 #include <ranges>

```

```

19 #include <cmath>
20 #include <numeric>
21 #include <functional>
22
23 #include "../chapter10/random_utils.h"
24
25 namespace randalgo {
26
27 // =====
28 // PageCache: basic cache abstraction
29 // =====
30
31 struct PageCache {
32     int capacity;
33     std::unordered_set<int> pages;
34     std::deque<int> order; // recency / arrival ordering
35
36     explicit PageCache(int k) : capacity(k) {}
37
38     [[nodiscard]] bool contains(int page) const {
39         return pages.contains(page);
40     }
41
42     void insert(int page) {
43         if (pages.contains(page)) return;
44         if (static_cast<int>(pages.size()) == capacity) {
45             evict();
46         }
47         pages.insert(page);
48         order.push_back(page);
49     }
50
51     void evict() {
52         if (order.empty()) return;
53         int victim = order.front();
54         order.pop_front();
55         pages.erase(victim);
56     }
57
58     void touch(int page) {
59         // Move page to back of recency order
60         if (!pages.contains(page)) return;
61         order.erase(
62             std::ranges::find(order, page));
63         order.push_back(page);
64     }
65
66     void clear() {
67         pages.clear();
68         order.clear();
69     }
70
71     [[nodiscard]] std::vector<int> snapshot() const {
72         return {order.begin(), order.end()};
73     }
74

```

```

75     [[nodiscard]] int size() const {
76         return static_cast<int>(pages.size());
77     }
78 };
79
80 // =====
81 // Deterministic LRU (Least Recently Used)
82 // =====
83
84 int deterministic_lru(int cache_size, const std::vector<int>&
85     requests, bool verbose = false) {
86     PageCache cache(cache_size);
87     int misses = 0;
88
89     for (int step = 0; auto page : requests) {
90         ++step;
91         if (cache.contains(page)) {
92             cache.touch(page);
93             if (verbose) {
94                 std::println(" Step {:2d}: page {} HIT    cache
95                             ={}",
96                             step, page, cache.snapshot());
97             }
98         } else {
99             ++misses;
100             cache.insert(page);
101             if (verbose) {
102                 std::println(" Step {:2d}: page {} MISS  cache
103                             ={}",
104                             step, page, cache.snapshot());
105             }
106         }
107     }
108     return misses;
109 }
110
111 // =====
112 // Deterministic FIFO (First In First Out)
113 // =====
114
115 int deterministic_fifo(int cache_size, const std::vector<int>&
116     requests, bool verbose = false) {
117     PageCache cache(cache_size);
118     int misses = 0;
119
120     for (int step = 0; auto page : requests) {
121         ++step;
122         if (cache.contains(page)) {
123             if (verbose) {
124                 std::println(" Step {:2d}: page {} HIT    cache
125                             ={}",
126                             step, page, cache.snapshot());
127             }
128         } else {
129             ++misses;
130             cache.insert(page);

```



```

126         if (verbose) {
127             std::println(" Step {:2d}: page {} MISS cache
                               ={}",
128                             step, page, cache.snapshot());
129         }
130     }
131 }
132 return misses;
133 }
134
135 // =====
136 // Randomized Paging (oblivious adversary)
137 // On a miss, evict a uniformly random page from the cache.
138 // =====
139
140 int random_paging(int cache_size, const std::vector<int>&
141 requests, bool verbose = false) {
142     PageCache cache(cache_size);
143     int misses = 0;
144     auto& engine = rng().engine();
145
146     for (int step = 0; auto page : requests) {
147         ++step;
148         if (cache.contains(page)) {
149             if (verbose) {
150                 std::println(" Step {:2d}: page {} HIT cache
151                               ={}",
152                             step, page, cache.snapshot());
153             }
154         } else {
155             ++misses;
156             if (static_cast<int>(cache.pages.size()) == cache.
157                 capacity) {
158                 // Evict a random page
159                 std::uniform_int_distribution<int> dist(
160                     0, static_cast<int>(cache.order.size()) -
161                     1);
162                 int idx = dist(engine);
163                 int victim = cache.order[idx];
164                 cache.order.erase(cache.order.begin() + idx);
165                 cache.pages.erase(victim);
166             }
167             cache.pages.insert(page);
168             cache.order.push_back(page);
169             if (verbose) {
170                 std::println(" Step {:2d}: page {} MISS cache
171                               ={}",
172                             step, page, cache.snapshot());
173             }
174         }
175     }
176     return misses;
177 }
178
179 // =====
180 // Marking Algorithm

```

```

176 // On first access in a "round", mark the page.
177 // On miss, evict a random unmarked page.
178 // When all pages are marked, reset all marks and start new
    round.
179 // =====
180
181 int marking_algorithm(int cache_size, const std::vector<int>&
    requests, bool verbose = false) {
182     PageCache cache(cache_size);
183     std::unordered_set<int> marked;
184     int misses = 0;
185     int round = 1;
186     auto& engine = rng().engine();
187
188     for (int step = 0; auto page : requests) {
189         ++step;
190         if (cache.contains(page)) {
191             marked.insert(page);
192             if (verbose) {
193                 std::println(" Step {:2d}: page {} HIT   cache
                    ={} marked={} round={}",
194                             step, page, cache.snapshot(),
                    marked.size(), round);
195             }
196         } else {
197             ++misses;
198             if (static_cast<int>(cache.pages.size()) == cache.
                capacity) {
199                 // Collect unmarked pages in cache
200                 std::vector<int> unmarked;
201                 for (int p : cache.pages)
202                     if (!marked.contains(p))
203                         unmarked.push_back(p);
204
205                 if (unmarked.empty()) {
206                     // All marked: reset marks, start new round
207                     marked.clear();
208                     ++round;
209                     // Recompute unmarked (now all are unmarked
                    )
210                     unmarked = cache.snapshot();
211                 }
212
213                 // Evict random unmarked page
214                 std::uniform_int_distribution<int> dist(
215                     0, static_cast<int>(unmarked.size()) - 1);
216                 int victim = unmarked[dist(engine)];
217                 cache.order.erase(
218                     std::ranges::find(cache.order, victim));
219                 cache.pages.erase(victim);
220             }
221             cache.pages.insert(page);
222             cache.order.push_back(page);
223             marked.insert(page);
224             if (verbose) {

```

```

225         std::println(" Step {:2d}: page {} MISS cache
226                     ={} marked={} round={} ",
227                     step, page, cache.snapshot(),
228                     marked.size(), round);
229     }
230     }
231     return misses;
232 }
233 // =====
234 // Optimal Offline: Belady's Algorithm
235 // Evict the page whose next use is farthest in the future.
236 // Uses a set of (next_use, page) pairs for efficient lookup.
237 // =====
238
239 int optimal_offline(int cache_size, const std::vector<int>&
240 requests, bool verbose = false) {
241     int n = static_cast<int>(requests.size());
242
243     // Precompute next_use[i] = next index where requests[i]
244     // appears, or INT_MAX
245     std::vector<int> next_use(n, INT_MAX);
246     std::unordered_map<int, int> last_seen;
247     for (int i = n - 1; i >= 0; --i) {
248         int page = requests[i];
249         if (last_seen.contains(page))
250             next_use[i] = last_seen[page];
251         last_seen[page] = i;
252     }
253
254     std::unordered_set<int> cache_pages;
255     // set of (next_use, page) for cache contents, ordered by
256     // next_use descending
257     // We use a set of pairs; the "farthest" is the one with
258     // the largest next_use.
259     std::set<std::pair<int, int>> future_uses; // (next_use,
260     // page)
261     std::unordered_map<int, int> page_to_next; // page -> its
262     // current next_use
263
264     int misses = 0;
265
266     for (int step = 0; step < n; ++step) {
267         int page = requests[step];
268
269         if (cache_pages.contains(page)) {
270             // Update its next_use
271             future_uses.erase({page_to_next[page], page});
272             page_to_next[page] = next_use[step];
273             future_uses.insert({next_use[step], page});
274             if (verbose) {
275                 std::println(" Step {:2d}: page {} HIT cache
276                             ={} ",
277                             step + 1, page, [&]() {
278                                 std::vector<int> v;

```

```

272         for (auto& [nu, p] :
                future_uses) v.push_back(p
                );
273         return v;
274     }());
275     }
276     } else {
277         ++misses;
278         if (static_cast<int>(cache_pages.size()) ==
                cache_size) {
279             // Evict the page with the farthest next use (
                // or never again)
280             auto [farthest_use, victim] = *future_uses.
                rbegin();
281             future_uses.erase({farthest_use, victim});
282             page_to_next.erase(victim);
283             cache_pages.erase(victim);
284         }
285         cache_pages.insert(page);
286         page_to_next[page] = next_use[step];
287         future_uses.insert({next_use[step], page});
288         if (verbose) {
289             std::println(" Step {:2d}: page {} MISS cache
                ={}",
290                         step + 1, page, [&]() {
291                             std::vector<int> v;
292                             for (auto& [nu, p] :
                                    future_uses) v.push_back(p
                                    );
293                             return v;
294                         }());
295         }
296     }
297 }
298 return misses;
299 }
300
301 // =====
302 // Competitive Ratio: run an algorithm and OPT, return ratio
303 // =====
304
305 double compute_competitive_ratio(
306     std::function<int(int, const std::vector<int>&, bool)>
        algorithm,
307     int cache_size,
308     const std::vector<int>& requests)
309 {
310     int alg_misses = algorithm(cache_size, requests, false);
311     int opt_misses = optimal_offline(cache_size, requests,
        false);
312     if (opt_misses == 0) return 1.0;
313     return static_cast<double>(alg_misses) / static_cast<double>
        (opt_misses);
314 }
315
316 // =====

```

```

317 // Random sequence generator
318 // =====
319
320 std::vector<int> generate_random_requests(int length, int
    num_pages) {
321     std::vector<int> requests(length);
322     auto& engine = rng().engine();
323     std::uniform_int_distribution<int> dist(1, num_pages);
324     for (auto& r : requests) r = dist(engine);
325     return requests;
326 }
327
328 // =====
329 // Demonstration
330 // =====
331
332 void demonstrate_paging() {
333     std::println("=== Paging Algorithms (Chapter 13) ===\n");
334
335     // ---- Small example with verbose output ----
336     std::println("--- Small Example: Cache size k=3 ---");
337     std::println("Request sequence: {{1, 2, 3, 4, 1, 2, 5, 1,
        2, 3, 4, 5}}\n");
338
339     std::vector<int> requests = {1, 2, 3, 4, 1, 2, 5, 1, 2, 3,
        4, 5};
340     int k = 3;
341
342     std::println("[LRU]");
343     int lru_misses = deterministic_lru(k, requests, true);
344     std::println("  Total misses: {}\n", lru_misses);
345
346     std::println("[FIFO]");
347     int fifo_misses = deterministic_fifo(k, requests, true);
348     std::println("  Total misses: {}\n", fifo_misses);
349
350     std::println("[Random Paging]");
351     int rand_misses = random_paging(k, requests, true);
352     std::println("  Total misses: {}\n", rand_misses);
353
354     std::println("[Marking Algorithm]");
355     int mark_misses = marking_algorithm(k, requests, true);
356     std::println("  Total misses: {}\n", mark_misses);
357
358     std::println("[OPT (Belady's Offline)]");
359     int opt_misses = optimal_offline(k, requests, true);
360     std::println("  Total misses: {}\n", opt_misses);
361
362     // ---- Summary of small example ----
363     std::println("--- Small Example Summary ---");
364     std::println("  LRU:    {:2d} misses  ratio={:.2f}",
        lru_misses,
365         static_cast<double>(lru_misses) / opt_misses);
366     std::println("  FIFO:   {:2d} misses  ratio={:.2f}",
        fifo_misses,

```

```

367         static_cast<double>(fifo_misses) / opt_misses)
368     ;
369     std::println(" Random: {:2d} misses ratio={:.2f}",
370         rand_misses,
371         static_cast<double>(rand_misses) / opt_misses)
372     ;
373     std::println(" Mark:   {:2d} misses ratio={:.2f}",
374         mark_misses,
375         static_cast<double>(mark_misses) / opt_misses)
376     ;
377     std::println(" OPT:    {:2d} misses", opt_misses);
378     std::println("");
379
380     // ---- Competitive ratio: adversarial sequence ----
381     std::println("--- Competitive Ratio: Adversarial Sequences
382         ---");
383     std::println("For k-page cache, an adversary can force k-
384         regret.");
385     std::println("LRU and FIFO are k-competitive but NOT (k-1)-
386         competitive.\n");
387
388     // Construct adversary's preferred sequence for k=3
389     // Cyclic sequence of 4 pages: forces LRU/FIFO to miss
390     // every other request
391     std::vector<int> adseq;
392     std::vector<int> cycle = {1, 2, 3, 4};
393     for (int i = 0; i < 20; ++i)
394         adseq.push_back(cycle[i % 4]);
395
396     double lru_ratio = compute_competitive_ratio(
397         deterministic_lru, k, adseq);
398     double fifo_ratio = compute_competitive_ratio(
399         deterministic_fifo, k, adseq);
400     double rnd_ratio = compute_competitive_ratio(random_paging,
401         k, adseq);
402     double mark_ratio = compute_competitive_ratio(
403         marking_algorithm, k, adseq);
404
405     std::println(" Adversarial sequence (cycle of 4, length
406         20, k=3):");
407     std::println(" LRU ratio:   {:.2f}", lru_ratio);
408     std::println(" FIFO ratio:   {:.2f}", fifo_ratio);
409     std::println(" Random ratio: {:.2f}", rnd_ratio);
410     std::println(" Mark ratio:   {:.2f}", mark_ratio);
411     std::println("");
412
413     // ---- Average-case on random sequences ----
414     std::println("--- Average Case: Random Sequences ---");
415     constexpr int num_trials = 100;
416     constexpr int seq_len = 1000;
417     constexpr int num_pages = 10;
418
419     for (int cache_k : {2, 3, 5}) {
420         double total_lru = 0, total_fifo = 0, total_rand = 0,
421             total_mark = 0, total_opt = 0;

```

```

408     for ([[maybe_unused]] int _ : std::views::iota(0,
409         num_trials)) {
410         auto req = generate_random_requests(seq_len,
411             num_pages);
412         total_lru += deterministic_lru(cache_k, req, false);
413         total_fifo += deterministic_fifo(cache_k, req,
414             false);
415         total_rand += random_paging(cache_k, req, false);
416         total_mark += marking_algorithm(cache_k, req, false);
417         total_opt += optimal_offline(cache_k, req, false);
418     }
419
420     double avg_lru = total_lru / num_trials;
421     double avg_fifo = total_fifo / num_trials;
422     double avg_rand = total_rand / num_trials;
423     double avg_mark = total_mark / num_trials;
424     double avg_opt = total_opt / num_trials;
425
426     std::println(" k={}: seq_len={} num_pages={} ({}
427         trials)", cache_k, seq_len, num_pages, num_trials);
428     std::println(" LRU: {:.1f} avg misses (ratio
429         {:.2f})", avg_lru, avg_lru / avg_opt);
430     std::println(" FIFO: {:.1f} avg misses (ratio
431         {:.2f})", avg_fifo, avg_fifo / avg_opt);
432     std::println(" Random: {:.1f} avg misses (ratio
433         {:.2f})", avg_rand, avg_rand / avg_opt);
434     std::println(" Mark: {:.1f} avg misses (ratio
435         {:.2f})", avg_mark, avg_mark / avg_opt);
436     std::println(" OPT: {:.1f} avg misses", avg_opt);
437     std::println("");
438 }
439
440 // ---- Scalability test ----
441 std::println("--- Scalability: Sequence Length vs Misses
442 ---");
443 {
444     int cache_k = 3;
445     std::println(" Cache size k={}, {} pages, 50 trials
446         per length\n",
447         cache_k, num_pages);
448
449     std::println(" {:>10s} {:>8s} {:>8s} {:>8s} {:>8s}
450         {:>8s}",
451         "Length", "LRU", "FIFO", "Random", "Mark",
452         "OPT");
453
454     for (int len : {100, 500, 1000, 5000, 10000}) {
455         double tl = 0, tf = 0, tr = 0, tm = 0, to = 0;
456         constexpr int trials = 50;
457         for ([[maybe_unused]] int _ : std::views::iota(0,
458             trials)) {
459             auto req = generate_random_requests(len,
460                 num_pages);
461             tl += deterministic_lru(cache_k, req, false);

```

```

448         tf += deterministic_fifo(cache_k, req, false);
449         tr += random_paging(cache_k, req, false);
450         tm += marking_algorithm(cache_k, req, false);
451         to += optimal_offline(cache_k, req, false);
452     }
453     std::println("    {:>10d}    {:>8.1f}    {:>8.1f}    {:>8.1f}
454                  len, tl / trials, tf / trials, tr /
455                  trials,
456                  tm / trials, to / trials);
457 }
458 std::println("");
459 }
460 std::println("--- Theoretical Results ---");
461 std::println("    LRU:      k-competitive (optimal among
462              deterministic algorithms)");
463 std::println("    FIFO:      k-competitive");
464 std::println("    Random: H_k-competitive (expected), ~ ln(k)
465              for large k");
466 std::println("    Marking: H_k-competitive (expected),
467              randomized");
468 std::println("    OPT:      optimal offline (cost = lower bound
469              )\n");
470 }
471 } // namespace randalgo

```

Listing 14.2: k -server: greedy, randomized, and optimal of-line algorithms.

```

1  #pragma once
2  // Chapter 13: The k-Server Problem
3  // Online algorithms for the k-server problem on metric spaces.
4  // Comparing deterministic greedy vs randomized against an
   adversary.
5
6  #include <iostream>
7  #include <vector>
8  #include <set>
9  #include <algorithm>
10 #include <random>
11 #include <cmath>
12 #include <numeric>
13 #include <print>
14 #include <format>
15 #include <ranges>
16 #include <limits>
17 #include <optional>
18 #include <map>
19
20 #include "../chapter10/random_utils.h"
21
22 namespace randalgo {
23

```



```

24 // =====
25 // ServerState: tracks server positions and accumulated cost
26 // =====
27
28 struct ServerState {
29     int k; // number of servers
30     std::vector<int> positions; // current positions (
        indices on metric space)
31     long long total_cost = 0; // total movement cost so
        far
32     std::vector<std::pair<int,int>> movements; // (from, to)
        log
33
34     ServerState(int k, std::vector<int> initial_positions)
35         : k(k), positions(std::move(initial_positions))
36     {
37         if (static_cast<int>(this->positions.size()) != k)
38             throw std::runtime_error("ServerState: need exactly
                k initial positions");
39     }
40
41     void move_server(int server_idx, int new_pos, const std::
        vector<std::vector<int>>& dist) {
42         int old_pos = positions[server_idx];
43         int cost = dist[old_pos][new_pos];
44         positions[server_idx] = new_pos;
45         total_cost += cost;
46         movements.emplace_back(old_pos, new_pos);
47     }
48
49     [[nodiscard]] bool serves(int pos) const {
50         return std::ranges::find(positions, pos) != positions.
            end();
51     }
52
53     [[nodiscard]] std::vector<int> snapshot() const { return
        positions; }
54 };
55
56 // =====
57 // Greedy / Deterministic: move the closest server
58 // =====
59
60 ServerState min_server_move(ServerState state, int request,
61                             const std::vector<std::vector<int>
62                                 >>& dist) {
63     if (state.serves(request)) return state;
64
65     int best_server = -1;
66     int best_dist = std::numeric_limits<int>::max();
67     for (int i = 0; i < state.k; ++i) {
68         int d = dist[state.positions[i]][request];
69         if (d < best_dist) {
70             best_dist = d;
71             best_server = i;
72         }
73     }

```

```

72     }
73     state.move_server(best_server, request, dist);
74     return state;
75 }
76
77 // =====
78 // Randomized: pick a server with probability ~ 1/distance
79 // =====
80
81 ServerState random_server_move(ServerState state, int request,
82                               const std::vector<std::vector<
83                                   int>>& dist) {
84     if (state.serves(request)) return state;
85
86     auto& engine = rng().engine();
87
88     // Compute weights:  $w_i = 1 / (1 + \text{dist}[s_i, \text{request}])$ 
89     std::vector<double> weights(state.k);
90     for (int i = 0; i < state.k; ++i)
91         weights[i] = 1.0 / (1.0 + dist[state.positions[i]][
92             request]);
93
94     double total_weight = std::accumulate(weights.begin(),
95         weights.end(), 0.0);
96     std::uniform_real_distribution<double> dist_real(0.0,
97         total_weight);
98     double r = dist_real(engine);
99
100     double cumsum = 0.0;
101     int chosen = state.k - 1;
102     for (int i = 0; i < state.k; ++i) {
103         cumsum += weights[i];
104         if (r <= cumsum) { chosen = i; break; }
105     }
106
107     state.move_server(chosen, request, dist);
108     return state;
109 }
110
111 // =====
112 // Optimal Offline (small instances via exhaustive search)
113 // For k servers on a line with few positions, we can solve
114 // via dynamic programming over subsets.
115 // =====
116
117 // For small k and small metric space, we use DP.
118 // State: (request_index, multiset of server positions as
119 // sorted tuple)
120 // We enumerate all possible choices at each step.
121
122 long long optimal_offline_server_dp(
123     int req_idx,
124     std::vector<int> current_positions,
125     const std::vector<int>& requests,
126     const std::vector<std::vector<int>>& dist,
127     std::map<std::vector<int>, long long>& memo)

```

```

123 {
124     if (req_idx == static_cast<int>(requests.size())) return 0;
125
126     std::sort(current_positions.begin(), current_positions.end
127               ());
128     auto key = current_positions;
129     // We need to include req_idx in the key; use a composite
130     // Simplification: use a flat map with encoded state.
131     // For small instances this suffices.
132
133     int req = requests[req_idx];
134
135     long long best = std::numeric_limits<long long>::max();
136
137     // Option 1: if a server already at req, no cost
138     if (std::ranges::find(current_positions, req) !=
139         current_positions.end()) {
140         best = optimal_offline_server_dp(
141             req_idx + 1, current_positions, requests, dist,
142             memo);
143     }
144
145     // Option 2: move each server to req
146     for (int i = 0; i < static_cast<int>(current_positions.size
147         ()); ++i) {
148         int old = current_positions[i];
149         int cost = dist[old][req];
150         current_positions[i] = req;
151         long long sub = optimal_offline_server_dp(
152             req_idx + 1, current_positions, requests, dist,
153             memo);
154         best = std::min(best, cost + sub);
155         current_positions[i] = old;
156     }
157
158     return best;
159 }
160
161 // Wrapper: uses memoization with composite key (req_idx,
162 // positions)
163 long long optimal_offline_server_impl(
164     int req_idx,
165     std::vector<int> current_positions,
166     const std::vector<int>& requests,
167     const std::vector<std::vector<int>>& dist,
168     std::map<std::pair<int, std::vector<int>>, long long>& memo
169 )
170 {
171     if (req_idx == static_cast<int>(requests.size())) return 0;
172
173     auto state_key = std::make_pair(req_idx, current_positions)
174         ;
175     if (memo.contains(state_key)) return memo[state_key];
176

```

```

170     int req = requests[req_idx];
171     long long best = std::numeric_limits<long long>::max();
172
173     if (std::ranges::find(current_positions, req) !=
174         current_positions.end()) {
175         best = optimal_offline_server_impl(
176             req_idx + 1, current_positions, requests, dist,
177             memo);
178     } else {
179         for (int i = 0; i < static_cast<int>(current_positions.
180             size()); ++i) {
181             int old = current_positions[i];
182             int cost = dist[old][req];
183             current_positions[i] = req;
184             long long sub = optimal_offline_server_impl(
185                 req_idx + 1, current_positions, requests, dist,
186                 memo);
187             best = std::min(best, static_cast<long long>(cost)
188                 + sub);
189             current_positions[i] = old;
190         }
191     }
192     memo[state_key] = best;
193     return best;
194 }
195
196 long long optimal_offline_server(
197     int /*k*/,
198     const std::vector<int>& initial_positions,
199     const std::vector<int>& requests,
200     const std::vector<std::vector<int>>& dist)
201 {
202     // Only feasible for small instances
203     std::map<std::pair<int, std::vector<int>>, long long> memo;
204     return optimal_offline_server_impl(0, initial_positions,
205         requests, dist, memo);
206 }
207
208 // =====
209 // Utility: build distance matrix for a line metric {0,1,...,n
210 // -1}
211 // =====
212
213 std::vector<std::vector<int>> line_metric(int n) {
214     std::vector<std::vector<int>> dist(n, std::vector<int>(n));
215     for (int i = 0; i < n; ++i)
216         for (int j = 0; j < n; ++j)
217             dist[i][j] = std::abs(i - j);
218     return dist;
219 }
220
221 // =====
222 // Demonstration
223 // =====

```

```

219 void demonstrate_k_server() {
220     std::println("=== k-Server Problem (Chapter 13) ==\n");
221
222     constexpr int n = 6; // metric space: line {0,1,2,3,4,5}
223     constexpr int k = 2; // number of servers
224
225     auto dist = line_metric(n);
226
227     // Print distance matrix
228     std::println("--- Line Metric (positions 0..{}) ---", n -
229         1);
230     std::print(" ");
231     for (int j = 0; j < n; ++j) std::print("{:>4d}", j);
232     std::println("");
233     for (int i = 0; i < n; ++i) {
234         std::print("{:>3d}: ", i);
235         for (int j = 0; j < n; ++j) std::print("{:>4d}", dist[i
236             ][j]);
237         std::println("");
238     }
239     std::println("");
240
241     // Request sequence
242     std::vector<int> requests = {5, 0, 3, 1, 4, 2, 5, 0, 3, 1};
243     std::println("--- Request Sequence ---");
244     std::print(" ");
245     for (int r : requests) std::print("{} ", r);
246     std::println("\n");
247
248     // ---- Greedy (closest server) ----
249     std::println("--- Greedy (Closest Server) ---");
250     {
251         ServerState state(k, {0, 1});
252         std::println(" Initial: servers at {}", state.snapshot
253             ());
254
255         for (int step = 0; auto req : requests) {
256             ++step;
257             bool hit = state.serves(req);
258             state = min_server_move(state, req, dist);
259             if (hit) {
260                 std::println(" Step {:2d}: request {} -> HIT
261                     servers={}",
262                     step, req, state.snapshot());
263             } else {
264                 auto& [from, to] = state.movements.back();
265                 std::println(" Step {:2d}: request {} -> MISS
266                     move {}->{} servers={} cost={}",
267                     step, req, from, to, state.
268                     snapshot(), dist[from][to]);
269             }
270         }
271         std::println(" Total cost: {}\n", state.total_cost);
272     }
273
274     // ---- Randomized ----

```

```

269     std::println("--- Randomized (Inverse-Distance Weighted)
270     ---");
271     {
272         // Run multiple trials
273         constexpr int trials = 10;
274         long long total_cost = 0;
275         long long best_cost = std::numeric_limits<long long>::
276             max();
277         long long worst_cost = 0;
278
279         for (int t = 0; t < trials; ++t) {
280             ServerState state(k, {0, 1});
281             for (int req : requests)
282                 state = random_server_move(state, req, dist);
283             total_cost += state.total_cost;
284             best_cost = std::min(best_cost, state.total_cost);
285             worst_cost = std::max(worst_cost, state.total_cost)
286                 ;
287         }
288         std::println(" {} trials, initial servers at {{0,1}}",
289             trials);
290         std::println(" Average cost: {:.1f}", static_cast<
291             double>(total_cost) / trials);
292         std::println(" Best cost: {}", best_cost);
293         std::println(" Worst cost: {}", worst_cost);
294
295         // One detailed run
296         ServerState state(k, {0, 1});
297         std::println("\n Detailed run:");
298         std::println(" Initial: servers at {}", state.
299             snapshot());
300         for (int step = 0; auto req : requests) {
301             ++step;
302             bool hit = state.serves(req);
303             state = random_server_move(state, req, dist);
304             if (!hit) {
305                 auto& [from, to] = state.movements.back();
306                 std::println(" Step {:2d}: request {} ->
307                     move {}->{} servers={} cost={}",
308                     step, req, from, to, state.
309                     snapshot(), dist[from][to]);
310             } else {
311                 std::println(" Step {:2d}: request {} -> HIT
312                     servers={} ",
313                     step, req, state.snapshot());
314             }
315         }
316         std::println(" Total cost: {}\n", state.total_cost);
317     }
318
319     // ---- Optimal Offline (small instance) ----
320     std::println("--- Optimal Offline (DP, small instance) ---"
321         );
322     {
323         // Use shorter sequence for DP feasibility
324         std::vector<int> short_req = {5, 0, 3, 1, 4};

```

```

315     ServerState greedy_state(k, {0, 1});
316     for (int req : short_req)
317         greedy_state = min_server_move(greedy_state, req,
318                                         dist);
319
320     long long opt_cost = optimal_offline_server(k, {0, 1},
321                                                 short_req, dist);
322
323     std::println(" Requests: {}", short_req);
324     std::println(" Initial servers: {{0, 1}}");
325     std::println(" Greedy cost: {}", greedy_state.
326                 total_cost);
327     std::println(" Optimal cost: {}", opt_cost);
328     std::println(" Competitive ratio (greedy/opt): {:.2f}"
329                 ,
330                 static_cast<double>(greedy_state.
331                                     total_cost) / opt_cost);
332     std::println("");
333 }
334
335 // ---- Comparison across different k values ----
336 std::println("--- Comparison: k=1,2,3 on line {{0..5}} ---"
337 );
338 {
339     std::vector<int> requests_long = {5, 0, 3, 1, 4, 2, 5,
340                                     0, 3, 1, 4, 2};
341     std::println(" Requests: {}", requests_long);
342     std::println(" {:>4s} {:>8s} {:>10s} {:>8s}",
343                 "k", "Greedy", "Random(avg)", "OPT");
344     std::println(" {:>4s} {:>8s} {:>10s} {:>8s}",
345                 "----", "-----", "-----", "----");
346
347     for (int kval : {1, 2, 3}) {
348         // Greedy
349         std::vector<int> init_pos;
350         for (int i = 0; i < kval; ++i) init_pos.push_back(i
351 );
352         ServerState gs(kval, init_pos);
353         for (int req : requests_long)
354             gs = min_server_move(gs, req, dist);
355
356         // Random (average over trials)
357         long long rand_total = 0;
358         constexpr int trials = 20;
359         for (int t = 0; t < trials; ++t) {
360             ServerState rs(kval, init_pos);
361             for (int req : requests_long)
362                 rs = random_server_move(rs, req, dist);
363             rand_total += rs.total_cost;
364         }
365
366         // OPT (small enough for DP if kval <= 3 and
367             sequence <= 12)
368         long long opt_cost = 0;
369         if (kval <= 3 && static_cast<int>(requests_long.
370             size()) <= 12)

```

```

361         opt_cost = optimal_offline_server(kval,
362             init_pos, requests_long, dist);
363
364         std::println("    {:>4d}    {:>8}    {:>10.1f}    {:>8}",
365             kval, gs.total_cost,
366             static_cast<double>(rand_total) /
367                 trials,
368             opt_cost);
369     }
370     std::println("");
371 }
372
373 // ---- Metric space: circle ----
374 std::println("--- Alternative Metric: Circle (6 positions)
375 ---");
376 {
377     // Build circular distance matrix
378     auto circle_dist = [](int n) {
379         std::vector<std::vector<int>> d(n, std::vector<int>
380             >(n));
381         for (int i = 0; i < n; ++i)
382             for (int j = 0; j < n; ++j) {
383                 int diff = std::abs(i - j);
384                 d[i][j] = std::min(diff, n - diff);
385             }
386         return d;
387     };
388
389     auto cdist = circle_dist(6);
390     std::vector<int> circ_req = {4, 1, 5, 2, 0, 3};
391     std::println("    Circle of 6 positions, requests: {}",
392         circ_req);
393
394     // Greedy
395     ServerState gs(2, {0, 1});
396     for (int req : circ_req) gs = min_server_move(gs, req,
397         cdist);
398     std::println("    Greedy cost: {}", gs.total_cost);
399
400     // Random average
401     long long rt = 0;
402     for (int t = 0; t < 20; ++t) {
403         ServerState rs(2, {0, 1});
404         for (int req : circ_req) rs = random_server_move(rs,
405             req, cdist);
406         rt += rs.total_cost;
407     }
408     std::println("    Random avg: {:.1f}", static_cast<
409         double>(rt) / 20);
410
411     // OPT
412     long long opt = optimal_offline_server(2, {0, 1},
413         circ_req, cdist);
414     std::println("    OPT:          {}", opt);
415     std::println("");
416 }

```



```

408     std::println("--- Theoretical Results ---");
409     std::println("  k-Server Conjecture: no randomized
410         algorithm is  $k/(k-1)$ -competitive");
411     std::println("  Deterministic lower bound: k (Manasse-
412         McGeoch-Sleator)");
413     std::println("  Randomized lower bound: k (Bartal)");
414     std::println("  The k-server problem is the canonical
415         online problem\n");
416 }
417 } // namespace randalgo

```

Listing 14.3: Adversary simulation: oblivious and adaptive models.

```

1  #pragma once
2  // Chapter 13: Adversary Models for Online Algorithms
3  // Oblivious vs. adaptive adversaries and their impact on
4     competitive ratios.
5
6  #include <iostream>
7  #include <vector>
8  #include <functional>
9  #include <random>
10 #include <algorithm>
11 #include <numeric>
12 #include <print>
13 #include <format>
14 #include <ranges>
15 #include <cmath>
16 #include <limits>
17
18 #include "../chapter10/random_utils.h"
19
20 namespace randalgo {
21
22     // =====
23     // ObliviousAdversary
24     // Generates an entire worst-case request sequence before
25     // seeing
26     // the algorithm's responses. Models the strongest offline
27     // adversary.
28     // =====
29
30     class ObliviousAdversary {
31     public:
32         // Generate a worst-case sequence for LRU/FIFO with k-page
33         // cache.
34         // The classic adversarial sequence: cyclic permutation of
35         // k+1 pages.
36         // This forces k+1 pages through a cache of size k,
37         // ensuring
38         // a miss on every other request.
39         static std::vector<int> worst_case_lru_fifo(int k, int
40             length) {

```

```

34     std::vector<int> sequence;
35     int alphabet_size = k + 1;
36     for (int i = 0; i < length; ++i)
37         sequence.push_back(i % alphabet_size + 1);
38     return sequence;
39 }
40
41 // Generate a worst-case sequence for a specific
42 // deterministic algorithm.
43 // Simulates the algorithm online, then picks the request
44 // that causes
45 // the most harm (works because the adversary knows the
46 // algorithm).
47 template<typename AlgFunc>
48 static std::vector<int> worst_case_for_deterministic(
49     AlgFunc algorithm, int cache_size, int length)
50 {
51     // Build the sequence greedily: at each step, try each
52     // possible page
53     // and pick the one that maximizes the algorithm's cost
54     .
55     std::vector<int> sequence;
56     std::vector<int> current_requests;
57
58     // We simulate the algorithm forward for each candidate
59     for (int step = 0; step < length; ++step) {
60         int best_page = 1;
61         int worst_cost = -1;
62
63         for (int candidate = 1; candidate <= cache_size +
64             1; ++candidate) {
65             auto test_requests = current_requests;
66             test_requests.push_back(candidate);
67             int cost = algorithm(cache_size, test_requests,
68                 false);
69             if (cost > worst_cost) {
70                 worst_cost = cost;
71                 best_page = candidate;
72             }
73         }
74         sequence.push_back(best_page);
75         current_requests.push_back(best_page);
76     }
77     return sequence;
78 }
79
80 // Generate a cyclic adversary sequence with parameterized
81 // period
82 static std::vector<int> cyclic_sequence(int period, int
83     length) {
84     std::vector<int> seq;
85     for (int i = 0; i < length; ++i)
86         seq.push_back(i % period + 1);
87     return seq;
88 }

```

```

81 // Generate a "interleaved" adversary: alternates between
82 // two groups
83 // to maximize misses for LRU/FIFO
84 static std::vector<int> interleaved_adversary(int k, int
85     length) {
86     std::vector<int> seq;
87     // Group A: {1,...,k}, Group B: {k+1}
88     // Pattern: 1, k+1, 2, k+1, 3, k+1, ... forces
89     // evictions
90     for (int i = 0; i < length; ++i) {
91         if (i % 2 == 0)
92             seq.push_back((i / 2) % k + 1);
93         else
94             seq.push_back(k + 1);
95     }
96     return seq;
97 }
98 };
99
100 // =====
101 // AdaptiveOnlineAdversary
102 // Generates requests one at a time, observing the algorithm's
103 // current state (cache contents or server positions).
104 // Models a stronger adversary that reacts to the algorithm.
105 // =====
106
107 class AdaptiveOnlineAdversary {
108 public:
109     // For paging: at each step, observe the cache contents and
110     // pick a request NOT in the cache (causing a miss).
111     // If all pages 1..k+1 are in cache, pick any page not in
112     // cache.
113     static std::vector<int> adaptive_paging_adversary(
114         std::function<std::vector<int>(const std::vector<int>&)
115             > get_cache_state,
116         int /*cache_size*/,
117         int alphabet_size,
118         int length)
119     {
120         std::vector<int> sequence;
121         std::vector<int> history;
122
123         for (int step = 0; step < length; ++step) {
124             auto cache = get_cache_state(history);
125             std::unordered_set<int> in_cache(cache.begin(),
126                 cache.end());
127
128             // Find a page not in cache
129             int chosen = -1;
130             for (int p = 1; p <= alphabet_size; ++p) {
131                 if (!in_cache.contains(p)) {
132                     chosen = p;
133                     break;
134                 }
135             }
136         }
137     }
138 }

```

```

131         // If all alphabet pages are in cache (shouldn't
132         // happen if
133         // alphabet_size > cache_size), pick page 1
134         if (chosen == -1) chosen = 1;
135
136         sequence.push_back(chosen);
137         history.push_back(chosen);
138     }
139     return sequence;
140 }
141
142 // For k-server: observe server positions and pick request
143 // far
144 // from all servers to maximize movement cost
145 static std::vector<int> adaptive_server_adversary(
146     std::function<std::vector<int>(const std::vector<int>&)
147     > get_server_positions,
148     const std::vector<std::vector<int>>& dist,
149     int num_positions,
150     const std::vector<int>& initial_positions,
151     int length)
152 {
153     std::vector<int> sequence;
154     std::vector<int> server_pos = initial_positions;
155
156     for (int step = 0; step < length; ++step) {
157         server_pos = get_server_positions(sequence);
158
159         // Pick the position farthest from all servers
160         int best_pos = 0;
161         int best_min_dist = -1;
162         for (int p = 0; p < num_positions; ++p) {
163             int min_d = std::numeric_limits<int>::max();
164             for (int s : server_pos)
165                 min_d = std::min(min_d, dist[s][p]);
166             if (min_d > best_min_dist) {
167                 best_min_dist = min_d;
168                 best_pos = p;
169             }
170         }
171         sequence.push_back(best_pos);
172     }
173     return sequence;
174 }
175 };
176
177 // =====
178 // Adversarial Test: pit adversary against algorithm, measure
179 // ratio
180 // =====
181
182 struct AdversaryTestResult {
183     int algorithm_cost;
184     int optimal_cost;
185     double competitive_ratio;
186     std::vector<int> requests;

```

```

183 };
184
185 // For paging
186 AdversaryTestResult adversarial_test(
187     std::function<int(int, const std::vector<int>&, bool)>
188         algorithm,
189     std::function<int(int, const std::vector<int>&, bool)>
190         optimal,
191     int cache_size,
192     const std::vector<int>& requests)
193 {
194     int alg_cost = algorithm(cache_size, requests, false);
195     int opt_cost = optimal(cache_size, requests, false);
196     double ratio = (opt_cost > 0)
197         ? static_cast<double>(alg_cost) / static_cast<double>(
198             opt_cost)
199         : 1.0;
200     return {alg_cost, opt_cost, ratio, requests};
201 }
202
203 // =====
204 // Demonstration
205 // =====
206
207 void demonstrate_adversary() {
208     std::println("=== Adversary Models (Chapter 13) ===\n");
209
210     // ---- Oblivious Adversary ----
211     std::println("--- Oblivious Adversary ---");
212     std::println("The adversary fixes the entire sequence
213         before the algorithm runs.\n");
214
215     {
216         int k = 3;
217         auto worst_seq = ObliviousAdversary::
218             worst_case_lru_fifo(k, 12);
219         std::println(" Worst-case sequence for LRU/FIFO (k={},
220             length=12):", k);
221         std::print(" ");
222         for (int r : worst_seq) std::print("{} ", r);
223         std::println("");
224
225         auto result = adversarial_test(deterministic_lru,
226             optimal_offline, k, worst_seq);
227         std::println(" LRU cost: {} OPT: {} ratio: {:.2f}
228             ",
229             result.algorithm_cost, result.optimal_cost,
230             result.competitive_ratio);
231
232         auto result2 = adversarial_test(deterministic_fifo,
233             optimal_offline, k, worst_seq);
234         std::println(" FIFO cost: {} OPT: {} ratio: {:.2f}
235             ",
236             result2.algorithm_cost, result2.
237                 optimal_cost, result2.
238                 competitive_ratio);
239     }
240 }

```

```

226     auto result3 = adversarial_test(random_paging,
227                                     optimal_offline, k, worst_seq);
228     std::println(" Random cost: {} OPT: {} ratio: {:.2f}
229                 (one trial)",
230                 result3.algorithm_cost, result3.
231                 optimal_cost, result3.
232                 competitive_ratio);
233     std::println("");
234 }
235
236 // ---- Adaptive Adversary ----
237 std::println("--- Adaptive Online Adversary ---");
238 std::println("The adversary observes the algorithm's state
239 at each step.\n");
240
241 {
242     int k = 3;
243     int alphabet = 5;
244     // The adaptive adversary for LRU
245     // We need to simulate LRU to get cache states, then
246     // feed
247     // adversarial requests. We do this round-trip.
248
249     // Lambda: given history, run LRU and return cache
250     // snapshot
251     auto lru_cache_state = [] (const std::vector<int>&
252                               history) -> std::vector<int> {
253         PageCache cache(3);
254         for (int p : history) {
255             if (cache.contains(p)) cache.touch(p);
256             else cache.insert(p);
257         }
258         return cache.snapshot();
259     };
260
261     auto adseq = AdaptiveOnlineAdversary::
262         adaptive_paging_adversary(
263             lru_cache_state, k, alphabet, 20);
264
265     std::print(" Adaptive sequence: ");
266     for (int r : adseq) std::print("{} ", r);
267     std::println("");
268
269     auto result = adversarial_test(deterministic_lru,
270                                   optimal_offline, k, adseq);
271     std::println(" LRU cost: {} OPT: {} ratio: {:.2f}
272                 ",
273                 result.algorithm_cost, result.optimal_cost
274                 , result.competitive_ratio);
275
276     // Randomized algorithm on the same adaptive sequence
277     // (adversary adapts to deterministic LRU, not to
278     // random)
279     auto result_rand = adversarial_test(random_paging,
280                                         optimal_offline, k, adseq);

```

```

268         std::println(" Random cost: {} OPT: {} ratio: {:.2f}"
269                     (on LRU-adapted sequence)",
270                     result_rand.algorithm_cost, result_rand.
271                     optimal_cost,
272                     result_rand.competitive_ratio);
273     }
274     // ---- Comparison: Oblivious vs Adaptive ----
275     std::println("--- Oblivious vs Adaptive Adversary ---\n");
276     {
277         int k = 3;
278         constexpr int trials = 20;
279
280         // Oblivious: fixed worst-case sequence
281         auto oblivious_seq = ObliviousAdversary::
282             worst_case_lru_fifo(k, 100);
283
284         double oblivious_ratio_lru = 0, oblivious_ratio_rand =
285             0;
286         for (int t = 0; t < trials; ++t) {
287             auto r1 = adversarial_test(deterministic_lru,
288                                     optimal_offline, k, oblivious_seq);
289             auto r2 = adversarial_test(random_paging,
290                                     optimal_offline, k, oblivious_seq);
291             oblivious_ratio_lru += r1.competitive_ratio;
292             oblivious_ratio_rand += r2.competitive_ratio;
293         }
294
295         std::println(" Oblivious adversary (fixed sequence,
296                     length 100):");
297         std::println(" LRU avg ratio: {:.2f}",
298                     oblivious_ratio_lru / trials);
299         std::println(" Random avg ratio: {:.2f}",
300                     oblivious_ratio_rand / trials);
301
302         // Adaptive: generates sequence that's bad for LRU
303         // specifically
304         auto lru_cache_state = [](const std::vector<int>&
305                                 history) -> std::vector<int> {
306             PageCache cache(3);
307             for (int p : history) {
308                 if (cache.contains(p)) cache.touch(p);
309                 else cache.insert(p);
310             }
311             return cache.snapshot();
312         };
313
314         auto adaptive_seq = AdaptiveOnlineAdversary::
315             adaptive_paging_adversary(
316                 lru_cache_state, k, 5, 100);
317
318         auto adaptive_lru = adversarial_test(deterministic_lru,
319                                     optimal_offline, k, adaptive_seq);
320         auto adaptive_rand = adversarial_test(random_paging,
321                                     optimal_offline, k, adaptive_seq);

```

```

310     std::println("\n Adaptive adversary (adapted to LRU,
311                 length 100):");
312     std::println("      LRU ratio:      {:.2f}", adaptive_lru.
313                 competitive_ratio);
314     std::println("      Random ratio: {:.2f}", adaptive_rand.
315                 competitive_ratio);
316     std::println("");
317 }
318
319 // ---- Randomization Helps Against Oblivious Adversaries ----
320
321 std::println("--- Why Randomization Helps ---\n");
322 {
323     int k = 4;
324     std::println(" For k={}, deterministic algorithms can
325                 be forced to k-competitive.\n", k);
326
327     // Show that random achieves  $H_k \sim \ln(k)$  instead
328     auto worst_seq = ObliviousAdversary::
329         worst_case_lru_fifo(k, 200);
330     constexpr int trials = 50;
331
332     auto det_result = adversarial_test(deterministic_lru,
333         optimal_offline, k, worst_seq);
334     double rand_avg_ratio = 0;
335     for (int t = 0; t < trials; ++t) {
336         auto r = adversarial_test(random_paging,
337             optimal_offline, k, worst_seq);
338         rand_avg_ratio += r.competitive_ratio;
339     }
340
341     double hk = 0.0;
342     for (int i = 1; i <= k; ++i) hk += 1.0 / i;
343
344     std::println(" k={},  $H_k = {:.4f}$  ( $\sim \ln(k) = {:.4f}$ )",
345                 k, hk, std::log(k));
346     std::println(" Deterministic LRU ratio: {:.2f} (
347                 theory: {})", det_result.competitive_ratio, k);
348     std::println(" Randomized avg ratio: {:.2f} (
349                 theory:  $H_k = {:.2f}$ )",
350                 rand_avg_ratio / trials, hk);
351     std::println(" Speedup from randomization: {:.2f}x",
352                 det_result.competitive_ratio / (
353                     rand_avg_ratio / trials));
354     std::println("");
355 }
356
357 // ---- Different Adversarial Strategies ----
358 std::println("--- Different Adversarial Strategies ---\n");
359 {
360     int k = 3;
361     constexpr int length = 30;
362
363     // Strategy 1: Cyclic ( $k+1$  pages)
364     auto seq1 = ObliviousAdversary::worst_case_lru_fifo(k,

```



```

length);
354
355 // Strategy 2: Interleaved
356 auto seq2 = ObliviousAdversary::interleaved_adversary(k
    , length);
357
358 // Strategy 3: Uniform random
359 std::vector<int> seq3(length);
360 for (auto& s : seq3) s = rng().rand_int(1, k + 1);
361
362 // Strategy 4: Zipf-like (heavy hitters)
363 std::vector<int> seq4(length);
364 for (auto& s : seq4) {
365     // Power-law: page i chosen with probability ~ 1/i
366     double r = rng().rand_double();
367     double cumsum = 0;
368     s = 1;
369     for (int p = 1; p <= k + 2; ++p) {
370         cumsum += 1.0 / p;
371         if (r * (1.0 + std::log(k + 2.0)) <= cumsum) {
372             s = p; break; }
373     }
374 }
375
376 struct StratResult { std::string name; double lru_ratio
    ; double rand_ratio; };
377 std::vector<StratResult> results;
378
379 for (auto [name, seq] : {
380     std::tuple{"Cyclic", seq1},
381     std::tuple{"Interleaved", seq2},
382     std::tuple{"Uniform", seq3},
383     std::tuple{"Zipf-like", seq4}
384 }) {
385     auto lr = adversarial_test(deterministic_lru,
386         optimal_offline, k, seq);
387     double rr_sum = 0;
388     for (int t = 0; t < 20; ++t)
389         rr_sum += adversarial_test(random_paging,
390             optimal_offline, k, seq).competitive_ratio;
391     results.push_back({name, lr.competitive_ratio,
392         rr_sum / 20});
393 }
394
395 std::println("  {:>14s}  {:>12s}  {:>12s}", "Strategy",
396     "LRU ratio", "Random ratio");
397 std::println("  {:>14s}  {:>12s}  {:>12s}", "-----",
398     "-----", "-----");
399 for (auto& [name, lr, rr] : results)
400     std::println("  {:>14s}  {:>12.2f}  {:>12.2f}",
401         name, lr, rr);
402 std::println("");
403 }
404
405 std::println("--- Key Insights ---");
406 std::println(" 1. Oblivious adversary: worst-case over ALL

```

```

400     input sequences");
std::println(" 2. Adaptive adversary: worst-case over
sequences that adapt to algorithm");
401 std::println(" 3. Randomization buys immunity against
oblivious adversaries");
402 std::println(" 4. Against adaptive adversaries,
randomization helps less");
403 std::println(" 5. The competitive ratio measures worst-
case online performance\n");
404 }
405
406 } // namespace randalgo

```

Notes

The study of online algorithms was initiated by the work of Sleator and Tarjan [77], who introduced the competitive analysis framework and proved the $k - 1$ lower bound for deterministic paging (Theorem 14.3) and the $O(k)$ -competitive randomized marking algorithm (Theorems 14.11 and 14.12). The marking algorithm was further analyzed by Karlin, Manasse, McGeoch, and Sleator [46], who established the connection between the marking algorithm and competitive ratios against oblivious adversaries.

The Koutsoupias–Papadimitriou randomized k -competitive algorithm for k -server [52] (Theorem 14.20) was a major open problem for nearly two decades. The adversary hierarchy for randomized online algorithms was studied extensively by Borodin and El-Yaniv [23], who formalized the three adversary models discussed in Section 14.2 and proved the hierarchy results of Proposition 14.5 and Theorem 14.13.

The lower bound for deterministic k -server (Theorem 14.19) is due to Manasse, McGeoch, and Sleator [61]. The relationship between the adversary models and the optimal competitive ratios was clarified by Fiat and Mendel [36], who also obtained improved algorithms for special metric spaces.

The recent breakthrough by Bubeck, Cohen, Lee, and Lee [25] significantly improved the upper bound for randomized k -server on general metric spaces, though the exact competitive ratio remains an open problem.

Further reading can be found in the textbook by Borodin and El-Yaniv [23] and the survey by Albers [9].

Problems

Problem 14.25. Prove that the LRU and FIFO algorithms for k -page caching are $\Theta(k)$ -competitive against an oblivious adversary. That is, establish matching upper and lower bounds on their competitive ratios.

Problem 14.26. Prove that no randomized algorithm for k -page caching can be better than $k/(k-1)$ -competitive against an oblivious adversary.

Problem 14.27. Consider the *weighted paging* problem, where each page p has a weight $w_p > 0$ and the cost of a miss on page p is w_p . Design a randomized algorithm that is $O(k \cdot W_{\max}/W_{\min})$ -competitive against an oblivious adversary, where $W_{\max}$ and $W_{\min}$ are the maximum and minimum page weights.

Problem 14.28. Analyze the 2-server problem on a metric space of three points forming an equilateral triangle with side length 1. Show that the deterministic lower bound is 2 and describe a randomized 2-competitive algorithm.

Problem 14.29. The *online load balancing* problem can be modeled as a special case of the k -server problem. Consider k machines and jobs arriving online. Each job j has a processing time p_j on each machine, and the cost of assigning job j to machine i is p_{ij} . Show that the k -server lower bound implies a lower bound of k for competitive deterministic load balancing.

Problem 14.30. Construct a problem (other than paging) where the competitive ratio against an oblivious adversary is strictly better than against an adaptive offline adversary. Prove that the two ratios differ.

Problem 14.31. Complete the proof of Theorem 14.12 by providing the full potential function argument showing that the randomized marking algorithm is k -competitive against an

oblivious adversary. In particular, explicitly define the refined potential function and verify that the amortized cost per request is at most $k/(k-1) \cdot \text{cost}_{\text{OPT}}$.

Problem 14.32. Prove that the k -server competitive ratio cannot depend on the number of points n in the metric space when k is fixed. That is, show that for any fixed k , there exists a constant C_k such that the k -server problem on any n -point metric space admits a C_k -competitive deterministic algorithm.

Problem 14.33. Consider the *bit* or *pennies* online problem: at each step, one of n coins is placed on the table with heads up, and the algorithm must flip one coin to tails (or declare a fault). The goal is to keep the number of heads-up coins below some threshold. Formulate this as an online algorithm problem and derive a competitive ratio using techniques from this chapter.

Problem 14.34. For the Koutsoupias–Papadimitriou algorithm (Theorem 14.20), prove Lemma 14.22 in full detail. Specifically, verify that the potential Φ is bounded, that the greedy minimization step achieves the required inequality, and that the final sum yields the k -competitive bound.

Chapter 15

Number Theory and Algebra

Number theory and abstract algebra provide the mathematical foundation for modern cryptography and for several fundamental randomized algorithms. The ring of integers modulo n , the structure of finite groups and fields, and the theory of polynomial arithmetic over finite fields all arise naturally in the design of randomized protocols for primality testing, cryptographic key generation, and algebraic computation.

Randomness is essential in this setting for reasons that are both practical and fundamental. Deterministic primality testing was only shown to be possible in polynomial time in 2002 (the AKS algorithm), while randomized tests such as Miller–Rabin have been used efficiently since the 1970s. The security of the RSA cryptosystem rests on the conjectured hardness of factoring large integers—a problem for which no efficient deterministic algorithm is known. The interplay between algebraic structure and probabilistic computation is one of the most elegant themes in the theory of randomized algorithms.

In this chapter we develop the necessary algebraic background, study randomized algorithms for primality testing and quadratic residuosity, describe the RSA cryptosystem, and explore the Schwartz–Zippel lemma and its applications to polynomial identity testing.

15.1 Preliminaries

We begin with the basic objects of modular arithmetic and the Euclidean algorithm.

15.1.1 Modular arithmetic

For a positive integer n , the set $\mathbb{Z}_n = \{0, 1, \dots, n-1\}$ equipped with addition and multiplication modulo n forms a commutative ring. We write $a \equiv b \pmod{n}$ to mean $n \mid (a - b)$, and we denote by $[a]_n$ the congruence class of a modulo n .

Definition 15.1 ($\mathbb{Z}_n^*$). The *multiplicative group of integers modulo n* is the set

$$\mathbb{Z}_n^* = \{a \in \mathbb{Z}_n : \gcd(a, n) = 1\}$$

under multiplication modulo n . Its cardinality is $\phi(n)$, where ϕ denotes Euler's totient function.

15.1.2 The Euclidean algorithm

The greatest common divisor of two integers a and b (not both zero) is the largest positive integer dividing both. The *Euclidean algorithm* computes $\gcd(a, b)$ by repeated application of the division algorithm:

$$\begin{aligned} a &= q_1 b + r_1, & 0 \leq r_1 < b, \\ b &= q_2 r_1 + r_2, & 0 \leq r_2 < r_1, \\ r_1 &= q_3 r_2 + r_3, & 0 \leq r_3 < r_2, \\ &\vdots \end{aligned}$$

until some remainder $r_k = 0$, at which point $\gcd(a, b) = r_{k-1}$.

Theorem 15.2 (Complexity of the Euclidean algorithm). *The Euclidean algorithm on integers $a > b > 0$ terminates in $O(\log \min(a, b))$ divisions, requiring $O(\log^2 a \cdot \log b)$ bit operations.*

Proof. At each step the pair (r_{i-1}, r_i) is replaced by (r_i, r_{i+1}) with $r_{i+1} < r_i$ and $r_{i+1} < r_{i-1} - r_i$. After two consecutive steps the remainder decreases by at least a factor of $1/2$. Hence the number of steps is at most $2 \log_2 b + O(1)$. Each division involves operands of $O(\log a)$ bits and takes $O(\log a \cdot \log b)$ bit operations by schoolbook long division, giving the stated bound. $\square$

15.1.3 The extended Euclidean algorithm and Bézout's identity

Theorem 15.3 (Bézout's identity). *For any integers a and b (not both zero), there exist integers x and y such that*

$$ax + by = \gcd(a, b).$$

Moreover, the extended Euclidean algorithm computes such x and y in $O(\log^2 a \cdot \log b)$ bit operations.

Proof. The proof proceeds by induction on the number of steps of the Euclidean algorithm. In the base case, if $b = 0$, then $\gcd(a, b) = |a|$ and we may take $x = \text{sgn}(a)$, $y = 0$. For the inductive step, suppose $\gcd(b, r_1) = bx' + r_1y'$. Substituting $r_1 = a - q_1b$ gives $\gcd(a, b) = bx' + (a - q_1b)y' = ay' + b(x' - q_1y')$, so $x = y'$ and $y = x' - q_1y'$. The extended Euclidean algorithm tracks these coefficients through the recursion. Each step performs $O(1)$ arithmetic operations on numbers of $O(\log a)$ bits, and there are $O(\log b)$ steps. $\square$

Corollary 15.4. *Let a, n be positive integers. Then $\gcd(a, n) = 1$ if and only if a has a multiplicative inverse in $\mathbb{Z}_n$, i.e., there exists $x \in \mathbb{Z}_n$ with $ax \equiv 1 \pmod{n}$.*

Proof. If $\gcd(a, n) = 1$, then by Theorem 15.3 there exist integers x, y with $ax + ny = 1$. Reducing modulo n gives $ax \equiv 1 \pmod{n}$. Conversely, if $ax \equiv 1 \pmod{n}$, then $ax - 1 = kn$ for some integer k , so $\gcd(a, n) \mid 1$, hence $\gcd(a, n) = 1$. $\square$

Example 15.5. To find the inverse of 17 modulo 43, apply the extended Euclidean algorithm:

$$43 = 2 \cdot 17 + 9,$$

$$17 = 1 \cdot 9 + 8,$$

$$9 = 1 \cdot 8 + 1,$$

$$8 = 8 \cdot 1 + 0.$$

Back-substituting: $1 = 9 - 1 \cdot 8 = 9 - 1 \cdot (17 - 1 \cdot 9) = 2 \cdot 9 - 17 = 2(43 - 2 \cdot 17) - 17 = 2 \cdot 43 - 5 \cdot 17$. Thus $(-5) \cdot 17 \equiv 1 \pmod{43}$, so $17^{-1} \equiv 38 \pmod{43}$.

15.1.4 The Chinese Remainder Theorem

Theorem 15.6 (Chinese Remainder Theorem). *Let $n_1, n_2, \dots, n_k$ be pairwise coprime positive integers, and let $N = n_1 n_2 \cdots n_k$. For any integers $a_1, a_2, \dots, a_k$, the system of simultaneous congruences*

$$x \equiv a_i \pmod{n_i}, \quad i = 1, 2, \dots, k,$$

has a unique solution modulo N . Moreover, the solution can be found in $O(\log^3 N)$ bit operations.

Proof. Existence. For each i , define $N_i = N/n_i$. Since the n_i are pairwise coprime, $\gcd(N_i, n_i) = 1$, so by Corollary 15.4 there exists y_i with $N_i y_i \equiv 1 \pmod{n_i}$. Set

$$x = \sum_{i=1}^k a_i N_i y_i.$$

Then for any j , reducing modulo n_j : all terms with $i \neq j$ contain $N_i = N/n_i$ which is divisible by n_j , so $x \equiv a_j N_j y_j \equiv a_j \cdot 1 = a_j \pmod{n_j}$.

Uniqueness. If x and x' are both solutions, then $n_i \mid (x - x')$ for all i . Since the n_i are pairwise coprime, $N = \prod n_i \mid (x - x')$, so $x \equiv x' \pmod{N}$.

Complexity. Computing each y_i via the extended Euclidean algorithm costs $O(\log^2 N)$ bit operations, and summing the $k \leq \log N$ terms costs $O(\log^3 N)$. $\square$

Remark 15.7. The Chinese Remainder Theorem has a structural interpretation: it gives a ring isomorphism

$$\mathbb{Z}_N \cong \mathbb{Z}_{n_1} \times \mathbb{Z}_{n_2} \times \cdots \times \mathbb{Z}_{n_k}.$$

Under this isomorphism, $\mathbb{Z}_N^* \cong \mathbb{Z}_{n_1}^* \times \cdots \times \mathbb{Z}_{n_k}^*$. This decomposition is fundamental to many number-theoretic algorithms.

15.2 Groups and Fields

15.2.1 Groups and abelian groups

Definition 15.8 (Group). A *group* $(G, \cdot)$ is a set G together with a binary operation $\cdot : G \times G \rightarrow G$ satisfying:

1. **Closure:** $a \cdot b \in G$ for all $a, b \in G$.
2. **Associativity:** $(a \cdot b) \cdot c = a \cdot (b \cdot c)$ for all $a, b, c \in G$.
3. **Identity:** There exists $e \in G$ with $e \cdot a = a \cdot e = a$ for all $a \in G$.
4. **Inverses:** For each $a \in G$, there exists $a^{-1} \in G$ with $a \cdot a^{-1} = a^{-1} \cdot a = e$.

If moreover $a \cdot b = b \cdot a$ for all $a, b \in G$, the group is called *abelian*.

Example 15.9. The set $\mathbb{Z}_n^*$ under multiplication modulo n is an abelian group. Its order is $|\mathbb{Z}_n^*| = \phi(n)$, where ϕ is Euler's totient function.

15.2.2 Euler's totient function

Definition 15.10 (Euler's totient function). For a positive integer n with prime factorization $n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$,

$$\phi(n) = n \prod_{i=1}^k \left(1 - \frac{1}{p_i}\right).$$

Equivalently, $\phi(n)$ counts the number of integers in $\{1, 2, \dots, n\}$ that are coprime to n .

Example 15.11. $\phi(12) = 12(1 - 1/2)(1 - 1/3) = 12 \cdot 1/2 \cdot 2/3 = 4$. Indeed, $\{1, 5, 7, 11\} = \mathbb{Z}_{12}^*$.

15.2.3 Euler's theorem

Theorem 15.12 (Euler's theorem). *If $\gcd(a, n) = 1$, then*

$$a^{\phi(n)} \equiv 1 \pmod{n}.$$

Proof. Let $r_1, r_2, \dots, r_{\phi(n)}$ be a complete set of representatives for $\mathbb{Z}_n^*$. Since $\gcd(a, n) = 1$, multiplication by a permutes the elements of $\mathbb{Z}_n^*$: the set $\{ar_1, ar_2, \dots, ar_{\phi(n)}\}$ is a rearrangement of $\{r_1, r_2, \dots, r_{\phi(n)}\}$, because $ar_i \equiv ar_j \pmod{n}$ implies $r_i \equiv r_j \pmod{n}$ (as a is invertible modulo n).

Taking the product of both sets modulo n :

$$\prod_{i=1}^{\phi(n)} (ar_i) \equiv \prod_{i=1}^{\phi(n)} r_i \pmod{n}.$$

The left side equals $a^{\phi(n)} \prod r_i$. Since each r_i is coprime to n , $\prod r_i$ is also coprime to n and hence invertible modulo n . Cancelling $\prod r_i$ from both sides yields $a^{\phi(n)} \equiv 1 \pmod{n}$. $\square$

Corollary 15.13 (Fermat's little theorem). *If p is prime and $p \nmid a$, then*

$$a^{p-1} \equiv 1 \pmod{p}.$$

Proof. This is Theorem 15.12 applied with $n = p$ prime, since $\phi(p) = p - 1$. $\square$

15.2.4 Fields and finite fields

Definition 15.14 (Field). A *field* $(F, +, \cdot)$ is a set F with two binary operations satisfying:

1. $(F, +)$ is an abelian group with identity 0.
2. $(F \setminus \{0\}, \cdot)$ is an abelian group with identity 1.
3. Multiplication distributes over addition: $a(b + c) = ab + ac$.

Proposition 15.15. *For a prime p , the ring $\mathbb{Z}_p$ is a field. It has exactly p elements, denoted $\mathbb{F}_p$.*

Proof. When p is prime, every nonzero element $a \in \{1, 2, \dots, p-1\}$ satisfies $\gcd(a, p) = 1$, so by Corollary 15.4 it has a multiplicative inverse in $\mathbb{Z}_p$. All field axioms follow from the ring axioms of $\mathbb{Z}_p$ together with the existence of multiplicative inverses. $\square$

15.2.5 Cyclic groups and generators

Definition 15.16 (Generator). An element $g \in G$ of a finite group G is a *generator* if $\{g^0, g^1, g^2, \dots\} = G$, i.e., every element of G is a power of g . A group with a generator is called *cyclic*.

Theorem 15.17. *For any prime p , the multiplicative group $\mathbb{F}_p^*$ is cyclic of order $p-1$. That is, there exists a primitive root g modulo p such that $\{g^1, g^2, \dots, g^{p-1}\} = \mathbb{F}_p^*$.*

Remark 15.18. More generally, $\mathbb{Z}_n^*$ is cyclic if and only if $n \in \{1, 2, 4, p^k, 2p^k\}$ for an odd prime p and $k \geq 1$.

15.2.6 The discrete logarithm problem

Definition 15.19 (Discrete logarithm). Let g be a generator of $\mathbb{F}_p^*$. For $h \in \mathbb{F}_p^*$, the *discrete logarithm* of h base g is the unique integer x with $1 \leq x \leq p-1$ such that $g^x \equiv h \pmod{p}$, written $x = \log_g h$.

Remark 15.20 (Hardness assumption). While $g^x \pmod{p}$ can be computed in $O(\log x \cdot \log^2 p)$ bit operations by repeated squaring, the inverse problem—given g and h , find x such that $g^x \equiv h \pmod{p}$ —is believed to be hard. No polynomial-time deterministic algorithm is known for the discrete logarithm problem in $\mathbb{F}_p^*$ for general prime p . This asymmetry between exponentiation and logarithm computation underlies the security of Diffie–Hellman key exchange and many other cryptographic protocols.

15.3 Quadratic Residues

Definition 15.21 (Quadratic residue). An integer a with $\gcd(a, n) = 1$ is a *quadratic residue modulo n* if there exists x with $x^2 \equiv a \pmod{n}$. If no such x exists, a is a *quadratic nonresidue modulo n* . The set of quadratic residues modulo n is denoted $QR(n)$.

15.3.1 The Legendre symbol

Definition 15.22 (Legendre symbol). For an odd prime p and integer a with $p \nmid a$, the *Legendre symbol* is defined by

$$\left(\frac{a}{p}\right) = \begin{cases} +1 & \text{if } a \in QR(p), \\ -1 & \text{if } a \notin QR(p). \end{cases}$$

Theorem 15.23 (Euler's criterion). For an odd prime p and integer a with $p \nmid a$,

$$a^{(p-1)/2} \equiv \left(\frac{a}{p}\right) \pmod{p}.$$

Proof. Suppose $a \in QR(p)$, so $a \equiv x^2 \pmod{p}$ for some x . Then $a^{(p-1)/2} \equiv (x^2)^{(p-1)/2} = x^{p-1} \equiv 1 \pmod{p}$ by Fermat's little theorem (Corollary 15.13). So $\left(\frac{a}{p}\right) = 1$ and the congruence holds.

Suppose $a \notin QR(p)$. Consider the polynomial $x^{p-1} - 1 \equiv 0 \pmod{p}$. By Fermat's little theorem, all $p - 1$ nonzero elements of $\mathbb{F}_p$ are roots. By the fact that $\mathbb{F}_p^*$ is cyclic (Theorem 15.17), it has a generator g , and the quadratic residues are exactly $\{g^2, g^4, \dots, g^{p-1}\}$, which are the $(p - 1)/2$ even powers of g . The quadratic nonresidues are the $(p - 1)/2$ odd powers.

If $a \equiv g^k \pmod{p}$ with k odd, then $a^{(p-1)/2} \equiv g^{k(p-1)/2} \pmod{p}$. Since $g^{(p-1)/2} \not\equiv 1 \pmod{p}$ (as g has order $p - 1$) but $(g^{(p-1)/2})^2 = g^{p-1} \equiv 1 \pmod{p}$, we must have $g^{(p-1)/2} \equiv -1 \pmod{p}$. Hence $a^{(p-1)/2} \equiv (-1)^k = -1 \pmod{p}$ for odd k , which equals $\left(\frac{a}{p}\right) = -1$. $\square$

15.3.2 A randomized quadratic residue test

Euler's criterion yields a simple randomized algorithm for testing quadratic residuosity modulo a prime.

Algorithm 30 Randomized Quadratic Residue Test (modulo prime p)

```

1: function IsQR( $a, p$ )                                 $\triangleright p$  prime,  $\gcd(a, p) = 1$ 
2:    $r \leftarrow a^{(p-1)/2} \bmod p$                          $\triangleright$  repeated squaring
3:   return ( $r \equiv 1 \pmod{p}$ )
4: end function

```

By Euler's criterion (Theorem 15.23), this test is deterministic and exact for prime moduli. The running time is $O(\log p \cdot \log^2 p) = O(\log^3 p)$ bit operations by repeated squaring.

Remark 15.24. For composite moduli $n = pq$ (where p, q are distinct primes), the situation is fundamentally different. Testing whether $a \in QR(n)$ requires knowing the factorization of n . Without the factorization, no deterministic polynomial-time algorithm is known, but a randomized polynomial-time algorithm exists (based on the Jacobi symbol and the structure of quadratic residues modulo Blum integers).

15.3.3 The Jacobi symbol

Definition 15.25 (Jacobi symbol). For an odd positive integer $n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k}$ and integer a with $\gcd(a, n) = 1$, the *Jacobi symbol* is

$$\left(\frac{a}{n}\right) = \prod_{i=1}^k \left(\frac{a}{p_i}\right)^{e_i},$$

where each $\left(\frac{a}{p_i}\right)$ is the Legendre symbol.

Remark 15.26. The Jacobi symbol generalizes the Legendre symbol, but $\left(\frac{a}{n}\right) = 1$ does not imply $a \in QR(n)$ when n is composite. It can be computed efficiently in $O(\log^2 n)$ bit operations using quadratic reciprocity and its supplements, without knowing the factorization of n .

15.3.4 Quadratic residuosity for composite moduli

Theorem 15.27. Let $n = pq$ be a product of two distinct primes (a Blum integer). An element $a \in \mathbb{Z}_n^*$ is a quadratic residue modulo n if and only if both $\left(\frac{a}{p}\right) = 1$ and $\left(\frac{a}{q}\right) = 1$.

Proof. By the Chinese Remainder Theorem (Theorem 15.6), $\mathbb{Z}_n \cong \mathbb{Z}_p \times \mathbb{Z}_q$. Under this isomorphism, $a \mapsto (a \bmod p, a \bmod q)$ and $x^2 \equiv a \pmod{n}$ if and only if $x^2 \equiv a \pmod{p}$ and $x^2 \equiv a \pmod{q}$. Thus $a \in QR(n)$ if and only if $a \bmod p \in QR(p)$ and $a \bmod q \in QR(q)$, which is equivalent to $\left(\frac{a}{p}\right) = 1$ and $\left(\frac{a}{q}\right) = 1$. $\square$

Remark 15.28. For a Blum integer $n = pq$, the group $\mathbb{Z}_n^*$ decomposes into four cosets with respect to $QR(n)$:

1. $QR(n)$: the quadratic residues ($\left(\frac{a}{p}\right) = \left(\frac{a}{q}\right) = 1$).
2. The set of a with $\left(\frac{a}{p}\right) = 1, \left(\frac{a}{q}\right) = -1$.
3. The set of a with $\left(\frac{a}{p}\right) = -1, \left(\frac{a}{q}\right) = 1$.
4. The set of a with $\left(\frac{a}{p}\right) = \left(\frac{a}{q}\right) = -1$.

Elements in the last three classes all have Jacobi symbol $+1$ but are not quadratic residues. Distinguishing $QR(n)$ from elements with Jacobi symbol $+1$ that are not in $QR(n)$ is believed to be computationally hard (the *quadratic residuosity assumption*), and forms the basis of the Goldwasser–Micali cryptosystem.

15.4 The RSA Cryptosystem

The RSA cryptosystem, introduced by Rivest, Shamir, and Adleman in 1978, is the most widely deployed public-key cryptosystem. Its security rests on the conjectured difficulty of factoring large integers.

15.4.1 Key generation

Algorithm 31 RSA Key Generation

```

1: function KEYGEN( $\ell$ )  $\triangleright \ell$  = bit-length parameter
2:   Choose random primes  $p, q$  of  $\ell/2$  bits each
3:    $n \leftarrow pq$   $\triangleright n$  is  $\ell$  bits
4:    $\lambda(n) \leftarrow \text{lcm}(p-1, q-1)$ 
5:   Choose  $e$  with  $1 < e < \lambda(n)$  and  $\text{gcd}(e, \lambda(n)) = 1$ 
6:    $d \leftarrow e^{-1} \bmod \lambda(n)$   $\triangleright$  extended Euclidean algorithm
7:   return public key  $(n, e)$ , private key  $(n, d)$ 
8: end function

```

In practice, one often uses $\phi(n) = (p-1)(q-1)$ instead of $\lambda(n)$; since $\lambda(n) \mid \phi(n)$, any e coprime to $\lambda(n)$ is also coprime to $\phi(n)$, and the correctness proof goes through.

15.4.2 Encryption and decryption

Given a message $m \in \mathbb{Z}_n$ (we describe *textbook RSA*; in practice the message is padded):

$$\text{Encryption: } c = m^e \bmod n,$$

$$\text{Decryption: } m = c^d \bmod n.$$

Theorem 15.29 (Correctness of RSA). *For any $m \in \mathbb{Z}_n$, decryption recovers the original message: $(m^e)^d \equiv m \pmod{n}$.*

Proof. Since $ed \equiv 1 \pmod{\lambda(n)}$, we have $ed = 1 + k\lambda(n)$ for some nonneg integer k . Since $\lambda(n) = \text{lcm}(p-1, q-1)$, we know $(p-1) \mid \lambda(n)$ and $(q-1) \mid \lambda(n)$, so $\lambda(n) \mid \phi(n)$. Thus $ed = 1 + k'\phi(n)$ for some nonneg integer k' .

We show $m^{ed} \equiv m \pmod{p}$ and $m^{ed} \equiv m \pmod{q}$; the result follows by the Chinese Remainder Theorem.

If $p \mid m$, then $m^{ed} \equiv 0 \equiv m \pmod{p}$. If $p \nmid m$, then by Fermat's little theorem (Corollary 15.13), $m^{p-1} \equiv 1 \pmod{p}$, so $m^{ed} = m^{1+k'\phi(n)} = m \cdot (m^{p-1})^{k'(q-1)} \equiv m \cdot 1 = m \pmod{p}$. The argument for q is identical. $\square$

15.4.3 Security of RSA

Definition 15.30 (RSA problem). The *RSA problem* is: given (n, e, c) where $n = pq$ for unknown primes p, q , e with $\gcd(e, (p-1)(q-1)) = 1$, and $c \equiv m^e \pmod{n}$ for some m , compute m .

Theorem 15.31 (RSA reduces to factoring). *An oracle that solves the RSA problem can be used to factor n in randomized polynomial time.*

Proof sketch. Given an oracle $\mathcal{O}$ that computes m from $(n, e, m^e \bmod n)$, one can factor n as follows. Choose a random $x \in \mathbb{Z}_n^*$ and compute $y = x^e \bmod n$. Query $\mathcal{O}(n, e, y)$ to obtain x . Now compute $\gcd(x - x_0, n)$ where x_0 was the original x —this is trivial here since we already know x . The reduction is more substantive when the oracle decrypts arbitrary ciphertexts: given the oracle, one can construct instances whose decryption reveals algebraic information about p and q . For example, choose random r , compute $c = r^e \bmod n$, and query to get r . Then $\gcd(r - r', n)$ for a related r' can reveal a factor.

A cleaner reduction: given the oracle, for any c we can compute $c^d \bmod n$. Choose $r \in \mathbb{Z}_n^*$ uniformly at random. Let $c = r^e \bmod n$, and use the oracle to recover $r = c^d \bmod n$. Then choose another random s , set $c' = (rs)^e \bmod n$, and recover rs from the oracle. Now $\gcd(rs - r \cdot s', n)$ with appropriate manipulation yields factors with nonzero probability.

The key insight is that knowing the RSA decryption exponent d is computationally equivalent to knowing the factorization of n : from d one can efficiently factor n using a randomized algorithm based on Miller's factoring method. $\square$

Remark 15.32. The theorem shows that breaking RSA (recovering plaintexts) is at least as hard as factoring. However, it does not prove that factoring is hard—that remains an open conjecture. Moreover, the RSA problem might in principle be easier than factoring (though no such evidence exists).

15.4.4 Applications

Beyond encryption, RSA supports:

- **Digital signatures (RSA-PSS, PKCS#1 v1.5):** The signer computes $s = m^d \bmod n$ using the private key; anyone verifies by checking $s^e \equiv m \pmod{n}$ using the public key.
- **Key exchange (RSA-KEM):** A symmetric key is encrypted under the recipient's RSA public key.
- **Hybrid encryption:** RSA encrypts a session key; the session key encrypts the actual data using a symmetric cipher.

15.5 Polynomial Roots and Factors

Polynomials over finite fields play a central role in algebraic coding theory, cryptography, and algorithmic number theory. In this section we develop the tools needed for polynomial identity testing and factorization.

15.5.1 Polynomials over finite fields

Let $\mathbb{F}$ be a field (typically $\mathbb{F}_p$ for a prime p , or $\mathbb{F}_{p^k}$ for a prime power). A polynomial $f(x) \in \mathbb{F}[x]$ of degree d has at most d roots in $\mathbb{F}$, unless f is the zero polynomial.

Theorem 15.33 (Fundamental theorem of algebra for finite fields). *A nonzero polynomial $f \in \mathbb{F}[x]$ of degree d has at most d roots in $\mathbb{F}$. More generally, a nonzero polynomial $f \in \mathbb{F}[x_1, \dots, x_n]$ of total degree d has at most $d \cdot |\mathbb{F}|^{n-1}$ roots in $\mathbb{F}^n$.*

Proof. The one-variable case follows by induction on d : if a is a root, then $f(x) = (x - a)g(x)$ for some polynomial g of degree $d - 1$, and every root of f other than a is a root of g . The multivariable case follows by induction on n , viewing f as a polynomial in x_n with coefficients in $\mathbb{F}[x_1, \dots, x_{n-1}]$. $\square$

15.5.2 The Schwartz–Zippel lemma

The Schwartz–Zippel lemma is one of the most useful tools in randomized algebraic computation. It shows that a nonzero

polynomial cannot vanish on a random input with high probability.

Theorem 15.34 (Schwartz–Zippel lemma). *Let $\mathbb{F}$ be a field, $f \in \mathbb{F}[x_1, \dots, x_n]$ a nonzero polynomial of total degree $d \geq 0$, and $S \subseteq \mathbb{F}$ a finite set. If $r_1, r_2, \dots, r_n$ are chosen independently and uniformly at random from S , then*

$$\Pr[f(r_1, r_2, \dots, r_n) = 0] \leq \frac{d}{|S|}.$$

Proof. The proof proceeds by induction on n . For $n = 1$, f has at most d roots in S (Theorem 15.33), so $\Pr[f(r_1) = 0] \leq d/|S|$.

For the inductive step, write f in the form

$$f(x_1, \dots, x_n) = \sum_{i=0}^k g_i(x_1, \dots, x_{n-1}) x_n^i,$$

where each $g_i \in \mathbb{F}[x_1, \dots, x_{n-1}]$ and k is the degree of f in x_n . Since f is nonzero, some g_i is nonzero. Let g_j be the g_i of highest index that is nonzero; then $\deg(g_j) \leq d - j$.

Fix an arbitrary choice of $r_1, \dots, r_{n-1} \in S$. If all $g_i(r_1, \dots, r_{n-1}) = 0$, then $f(r_1, \dots, r_{n-1}, r_n) = 0$ for all r_n . If at least one $g_i(r_1, \dots, r_{n-1}) \neq 0$, then $f(r_1, \dots, r_{n-1}, x_n)$ is a nonzero polynomial in x_n of degree at most $k \leq d$, and has at most d roots in S .

By the inductive hypothesis, $\Pr[\bigwedge_i g_i(r_1, \dots, r_{n-1}) = 0] \leq \frac{d-j}{|S|} \leq \frac{d}{|S|}$ (since some g_i with $i \leq j$ is nonzero of degree at most $d - j$). In the complementary event, $\Pr[f(r_1, \dots, r_n) = 0 \mid r_1, \dots, r_{n-1}] \leq d/|S|$. By the law of total probability:

$$\Pr[f(r_1, \dots, r_n) = 0] \leq \frac{d}{|S|} + 1 \cdot \left(1 - \frac{d}{|S|}\right) \cdot \frac{d}{|S|} \leq \frac{d}{|S|} + \frac{d}{|S|} \cdot \left(1 - \frac{d}{|S|}\right) \leq$$

Wait, let us give a cleaner induction. We prove by induction on n that the probability is at most $d/|S|$.

For $n = 1$ this is immediate since f has at most d roots. For $n > 1$, write $f = \sum_{i=0}^k g_i(x_1, \dots, x_{n-1}) x_n^i$ as above, with $g_k \neq 0$ (so $\deg(g_k) \leq d - k$). By the inductive hypothesis,

$\Pr[g_k(r_1, \dots, r_{n-1}) = 0] \leq (d - k)/|S|$. If $g_k(r_1, \dots, r_{n-1}) \neq 0$, then $f(r_1, \dots, r_{n-1}, x_n)$ is a nonzero polynomial of degree k in x_n , which has at most k roots in S , so $\Pr_{r_n}[f = 0] \leq k/|S|$. Thus:

$$\Pr[f(r_1, \dots, r_n) = 0] \leq \frac{d - k}{|S|} + 1 \cdot \frac{k}{|S|} = \frac{d}{|S|}. \quad \square$$

Corollary 15.35. *Two polynomials $f, g \in \mathbb{F}[x_1, \dots, x_n]$ of total degree at most d are identical if and only if $f(r_1, \dots, r_n) = g(r_1, \dots, r_n)$ for $r_1, \dots, r_n$ drawn uniformly from a set S with $|S| > d$. Hence polynomial identity testing can be done in randomized polynomial time.*

Proof. $f = g$ if and only if $f - g$ is the zero polynomial. By the Schwartz–Zippel lemma, if $f - g$ is nonzero and has degree at most d , then $\Pr[f - g = 0] \leq d/|S| < 1$ when $|S| > d$. Thus, if $f(r) = g(r)$ for random r , either $f = g$ (identically) or we have a rare “collision” event. $\square$

15.5.3 Applications of the Schwartz–Zippel lemma

Example 15.36 (Polynomial identity testing). Suppose we want to verify that two polynomials, each given as an arithmetic circuit, compute the same function. By Corollary 15.35, we evaluate both at a random point in $\mathbb{F}^n$ (with $|\mathbb{F}| > d$) and check equality. This gives a randomized poly-time identity test. A famous application is *Freivalds’ technique*: given matrices A, B, C , test whether $AB = C$ by checking $A(Br) = Cr$ for a random vector $r \in \{0, 1\}^n$. Here the polynomial $f(r) = ABr - Cr$ has degree 1, so the Schwartz–Zippel lemma gives error probability at most $1/2$.

Example 15.37 (Testing polynomial equality). Given two degree- d polynomials f and g in n variables over $\mathbb{F}_p$, we can test whether $f \equiv g$ by choosing $r_1, \dots, r_n$ uniformly from $\mathbb{F}_p$ and checking $f(r) = g(r)$. The error probability is at most d/p . For $p > d$, this gives a single-trial randomized test with zero error probability.

15.5.4 Polynomial factorization over finite fields

Remark 15.38 (Berlekamp’s algorithm). Berlekamp (1967) gave an efficient algorithm for factoring polynomials over $\mathbb{F}_p$. Given a polynomial $f(x) \in \mathbb{F}_p[x]$ of degree n , the algorithm works by finding the null space of a certain matrix derived from the “Berlekamp subalgebra”—the set of polynomials g such that $g(x)^p \equiv g(x) \pmod{f(x)}$. For a square-free polynomial of degree n , the algorithm runs in $O(n^3)$ arithmetic operations over $\mathbb{F}_p$, which is polynomial in n and $\log p$. This contrasts sharply with integer factorization: factoring polynomials over finite fields is in P, while integer factororing is not known to be.

15.5.5 Why factoring is hard but polynomial root-finding is easy

Remark 15.39. Over a finite field $\mathbb{F}_p$, a degree- n polynomial f can be factored into irreducible factors in polynomial time (Berlekamp, Cantor–Zassenhaus). The key idea is that $\mathbb{F}_{p^n}$ is a splitting field for f , and one can compute in $\mathbb{F}_{p^n}$ using its $\mathbb{F}_p$ -vector space representation. In contrast, factoring an n -bit integer N into primes has no known polynomial-time algorithm (though the Number Field Sieve achieves sub-exponential time $L_N[1/3, c]$). The fundamental reason is that $\mathbb{Z}$ lacks the rich algebraic structure (such as the Frobenius endomorphism $x \mapsto x^p$) that makes arithmetic over finite fields so tractable.

15.6 Primality Testing

The *primality problem* asks: given a positive integer n , is n prime? This problem has deep connections to both number theory and computational complexity.

15.6.1 Trial division

The naive approach checks whether any integer d with $2 \leq d \leq \sqrt{n}$ divides n . This requires $O(\sqrt{n})$ trial divisions, which is exponential in $\log n$ (the input size). Hence trial division is not a polynomial-time algorithm.

15.6.2 Fermat's primality test

Fermat's little theorem (Corollary 15.13) suggests a natural test: if n is prime, then $a^{n-1} \equiv 1 \pmod{n}$ for all a with $\gcd(a, n) = 1$.

Algorithm 32 Fermat Primality Test

```

1: function FERMATTEST( $n, k$ )           ▷  $k$  = number of trials
2:   for  $i = 1$  to  $k$  do
3:      $a \leftarrow$  random element of  $\{2, 3, \dots, n-1\}$ 
4:     if  $\gcd(a, n) \neq 1$  then
5:       return "composite"   ▷  $n$  has a nontrivial factor
6:     end if
7:     if  $a^{n-1} \not\equiv 1 \pmod{n}$  then
8:       return "composite"
9:     end if
10:  end for
11:  return "probably prime"
12: end function

```

Theorem 15.40 (Soundness of Fermat test). *If n is prime, the Fermat test always returns "prime" (no false negatives). If n is composite, then for each random a with $\gcd(a, n) = 1$, either $a^{n-1} \not\equiv 1 \pmod{n}$ or $\gcd(a, n) \neq 1$.*

Proof. If n is prime, Fermat's little theorem guarantees $a^{n-1} \equiv 1 \pmod{n}$ for all $a \not\equiv 0 \pmod{n}$, so the test always passes. If n is composite, any a with $\gcd(a, n) > 1$ is immediately detected. For a with $\gcd(a, n) = 1$, either $a^{n-1} \not\equiv 1 \pmod{n}$ (and the test catches this), or $a^{n-1} \equiv 1 \pmod{n}$ and a is called a *Fermat liar*. $\square$

Definition 15.41 (Carmichael number). A *Carmichael number* is a composite integer n such that $a^{n-1} \equiv 1 \pmod{n}$ for every a with $\gcd(a, n) = 1$. For such numbers, the Fermat test fails to detect compositeness for every base a coprime to n .

Example 15.42. The smallest Carmichael number is $561 = 3 \cdot 11 \cdot 17$. For every a with $\gcd(a, 561) = 1$, we have $a^{560} \equiv 1 \pmod{3}$, $a^{560} \equiv 1 \pmod{11}$, and $a^{560} \equiv 1 \pmod{17}$ (by Fermat's little theorem, since $2 \mid 560$, $10 \mid 560$, $16 \mid 560$). By the Chinese Remainder Theorem, $a^{560} \equiv 1 \pmod{561}$.

Remark 15.43. Alford, Granville, and Pomerance (1994) proved that there are infinitely many Carmichael numbers. Hence the Fermat test cannot be made into a correct primality test by any fixed strategy of choosing bases.

15.6.3 The Miller–Rabin test

The Miller–Rabin test overcomes the Carmichael number problem by exploiting the structure of $\mathbb{Z}_n^*$ when n is prime.

Definition 15.44 (Miller–Rabin witness). For a composite odd integer n (with $n - 1 = 2^s d$, d odd), an element $a \in \{2, \dots, n - 2\}$ is called a *Miller–Rabin witness* for the compositeness of n if a is not detected as “probably prime” by the test—that is, neither $a^d \equiv 1 \pmod{n}$ nor $a^{2^r d} \equiv -1 \pmod{n}$ for any $0 \leq r < s$.

Theorem 15.45 (Soundness of Miller–Rabin). *If n is prime, the Miller–Rabin test always returns “prime.” If n is composite, then at least $3/4$ of all $a \in \{2, \dots, n - 2\}$ are Miller–Rabin witnesses. Hence k independent repetitions yield an error probability of at most 4^{-k} .*

Proof. **If n is prime:** We show that for any a with $\gcd(a, n) = 1$, either $a^d \equiv 1 \pmod{n}$ or $a^{2^r d} \equiv -1 \pmod{n}$ for some $0 \leq r < s$. By Fermat's little theorem, $a^{n-1} = a^{2^s d} \equiv 1 \pmod{n}$. The sequence $a^d, a^{2d}, a^{4d}, \dots, a^{2^s d}$ ends at 1, and each term is the square of the previous. In $\mathbb{Z}_n^*$ (a field when n

Algorithm 33 Miller–Rabin Primality Test

```
1: function MILLERRABIN( $n, k$ ) ▷  $k$  repetitions
2:   Write  $n - 1 = 2^s \cdot d$  with  $d$  odd
3:   for  $i = 1$  to  $k$  do
4:      $a \leftarrow$  random element of  $\{2, 3, \dots, n - 2\}$ 
5:      $x \leftarrow a^d \bmod n$ 
6:     if  $x = 1$  or  $x = n - 1$  then
7:       continue to next trial
8:     end if
9:     for  $r = 1$  to  $s - 1$  do
10:       $x \leftarrow x^2 \bmod n$ 
11:      if  $x = n - 1$  then
12:        continue to next trial
13:      end if
14:    end for
15:    return “composite”
16:  end for
17:  return “probably prime”
18: end function
```

is prime), the equation $x^2 \equiv 1 \pmod{n}$ has only solutions $x \equiv \pm 1 \pmod{n}$. Hence the sequence transitions from non- ± 1 values to 1 via a -1 intermediate. That is, if $a^d \not\equiv 1 \pmod{n}$, then there is a smallest r with $a^{2^r d} \equiv 1 \pmod{n}$, and $r \geq 1$; then $a^{2^{r-1}d}$ is a square root of 1 that is not 1, hence $a^{2^{r-1}d} \equiv -1 \pmod{n}$.

If n is composite: We must show that at least $3/4$ of bases are witnesses. The proof distinguishes two cases.

Case 1: n is not a prime power. Let $n = p_1^{e_1} \cdots p_m^{e_m}$ with $m \geq 2$. By the Chinese Remainder Theorem, $\mathbb{Z}_n^* \cong \mathbb{Z}_{p_1^{e_1}}^* \times \cdots \times \mathbb{Z}_{p_m^{e_m}}^*$. Define the homomorphism $\chi : \mathbb{Z}_n^* \rightarrow \{+1, -1\}$ by $\chi(a) = \left(\frac{a}{n}\right)$ (the Jacobi symbol). Non-witnesses satisfy $a^{2^r d} \equiv \pm 1 \pmod{n}$ for some $0 \leq r \leq s$, which implies $\chi(a^{2^r d}) = (\pm 1)^{\text{something}} = +1$ or -1 . A more careful analysis shows that the set of non-witnesses forms a subgroup of $\mathbb{Z}_n^*$ of index at least 2 (in fact, at least 4 when n has at least two distinct prime factors and is not a prime power). Hence at most $1/4$ of elements are non-witnesses.

Case 2: $n = p^k$ for an odd prime p and $k \geq 2$. Here $\mathbb{Z}_n^*$ is cyclic of order $\phi(n) = p^{k-1}(p-1)$. One can verify directly that the subgroup of non-witnesses has index at least 2 in $\mathbb{Z}_n^*$, so at least $1/2$ of bases are witnesses. (In fact, the bound $3/4$ holds uniformly; see the references.)

In both cases, at least $3/4$ of the $n-3$ candidate bases are witnesses. Repeating the test k times independently, the probability that all k random bases are non-witnesses is at most $(1/4)^k$. $\square$

Theorem 15.46 (Running time of Miller–Rabin). *Each iteration of the Miller–Rabin test computes $a^d \pmod{n}$ and $s-1$ subsequent squarings, all modulo n . The total running time for k iterations is $O(k \log^3 n)$ bit operations.*

Proof. Computing $a^d \pmod{n}$ by repeated squaring of a modulo n requires $O(\log d) = O(\log n)$ modular multiplications, each costing $O(\log^2 n)$ bit operations. The $s-1$ squarings

cost $O(\log n)$ additional modular multiplications. Each iteration thus costs $O(\log^3 n)$ bit operations, and k iterations give $O(k \log^3 n)$ total. $\square$

Remark 15.47. By choosing $k = \lceil \log_2(1/\epsilon) \rceil$, the error probability is driven below ϵ . For practical applications, $k = 20$ to 40 iterations suffice, giving error probabilities below 2^{-40} to 2^{-80} .

15.6.4 The Solovay–Strassen test

The Solovay–Strassen test uses Euler’s criterion to test primality.

Algorithm 34 Solovay–Strassen Primality Test

```

1: function SOLOVAYSTRASSEN( $n, k$ ) ▷  $k$  repetitions
2:   for  $i = 1$  to  $k$  do
3:      $a \leftarrow$  random element of  $\{2, 3, \dots, n - 1\}$ 
4:     if  $\gcd(a, n) > 1$  then
5:       return “composite”
6:     end if
7:      $x \leftarrow a^{(n-1)/2} \bmod n$ 
8:      $j \leftarrow \left(\frac{a}{n}\right)$  ▷ Jacobi symbol, computable without
       factoring  $n$ 
9:     if  $x \not\equiv j \pmod{n}$  then
10:      return “composite”
11:    end if
12:  end for
13:  return “probably prime”
14: end function

```

Theorem 15.48 (Soundness of Solovay–Strassen). *If n is an odd prime, the test always returns “prime.” If n is an odd composite, then at least $1/2$ of all $a \in \mathbb{Z}_n^*$ satisfy $a^{(n-1)/2} \not\equiv \left(\frac{a}{n}\right) \pmod{n}$ (i.e., are witnesses to compositeness). Hence k repetitions give error probability at most 2^{-k} .*

Proof. If $n = p$ is prime, Euler's criterion (Theorem 15.23) gives $a^{(n-1)/2} \equiv \left(\frac{a}{p}\right) \pmod{p}$, so the test always passes.

For composite n , define the set of "Euler liars":

$$L = \left\{ a \in \mathbb{Z}_n^* : a^{(n-1)/2} \equiv \left(\frac{a}{n}\right) \pmod{n} \right\}.$$

We claim L is a proper subgroup of $\mathbb{Z}_n^*$. It is closed under multiplication (using the multiplicativity of both the power map and the Jacobi symbol). It does not contain all of $\mathbb{Z}_n^*$ because n is composite: there exists a with $a^{(n-1)/2} \not\equiv \left(\frac{a}{n}\right) \pmod{n}$ (this follows from a case analysis on the prime factorization of n). A proper subgroup of a finite group has size at most half the group. Hence $|L| \leq \phi(n)/2$, and at least $1/2$ of elements of $\mathbb{Z}_n^*$ are witnesses. $\square$

Remark 15.49 (Comparison of Miller–Rabin and Solovay–Strassen). The Miller–Rabin test has a stronger soundness guarantee ($3/4$ vs. $1/2$ fraction of witnesses), so it requires fewer repetitions for the same error bound. The Solovay–Strassen test has the pedagogical advantage of being based directly on Euler's criterion and the Jacobi symbol. In practice, Miller–Rabin is preferred. Both tests have the same asymptotic running time $O(k \log^3 n)$.

15.6.5 The AKS primality test

In 2002, Agrawal, Kayal, and Saxena proved that primality testing is in P by giving a deterministic polynomial-time algorithm.

Theorem 15.50 (AKS primality test). *There exists a deterministic algorithm that tests whether n is prime in $O(\tilde{O}(\log^{7.5} n))$ bit operations, where $\tilde{O}$ hides polylogarithmic factors.*

Remark 15.51. The AKS test is based on the following characterization of primes: n is prime if and only if $(x + a)^n \equiv x^n + a \pmod{x^r - 1, n}$ for a suitably chosen set of bases a and a suitable polynomial modulus $x^r - 1$. While theoretically

profound, AKS is slower than Miller–Rabin for all practical input sizes. The Miller–Rabin test remains the algorithm of choice in applications.

C++ Implementation

Listing 15.1: Number theory: modular exponentiation, GCD, CRT, Euler’s totient.

```

1  #pragma once
2
3  #include <print>
4  #include <vector>
5  #include <tuple>
6  #include <stdexcept>
7  #include <algorithm>
8  #include <numeric>
9
10 namespace randalgo {
11
12  //
13  // -----
14  // 1. Modular Exponentiation -- binary method,  $O(\log \text{exp})$ 
15  // Computes  $(\text{base}^{\text{exp}}) \bmod \text{mod\_val}$  using repeated squaring.
16  // -----
17
18  long long mod_pow(long long base, long long exp, long long
19  mod_val) {
20  if (mod_val == 1) return 0;
21  long long result = 1;
22  base %= mod_val;
23  while (exp > 0) {
24  if (exp & 1) {
25  result = (result * base) % mod_val;
26  }
27  base = (base * base) % mod_val;
28  exp >>= 1;
29  }
30  return result;
31 }
32
33 //
34 // -----
35
36 // 2. GCD -- Euclidean algorithm
37 // -----
38
39 long long gcd(long long a, long long b) {
40  a = std::abs(a);
41  b = std::abs(b);

```

```

36     while (b != 0) {
37         a %= b;
38         std::swap(a, b);
39     }
40     return a;
41 }
42
43 //
44 // -----
45 //
46 // 3. Extended GCD -- finds (g, x, y) such that ax + by = g =
47 // gcd(a, b)
48 //
49 // -----
50
51 struct ExtendedGCDResult {
52     long long g; // gcd
53     long long x; // coefficient for a
54     long long y; // coefficient for b
55 };
56
57 ExtendedGCDResult extended_gcd(long long a, long long b) {
58     if (a == 0) return {b, 0, 1};
59     auto [g, x1, y1] = extended_gcd(b % a, a);
60     long long x = y1 - (b / a) * x1;
61     long long y = x1;
62     return {g, x, y};
63 }
64
65 //
66 // -----
67 //
68 // 4. Modular Inverse -- using extended Euclidean algorithm
69 // Returns x such that (a * x) == 1 (mod mod_val).
70 //
71 // -----
72
73 long long mod_inverse(long long a, long long mod_val) {
74     auto [g, x, y] = extended_gcd(a, mod_val);
75     if (g != 1) {
76         throw std::runtime_error("Modular inverse does not
77             exist (gcd != 1)");
78     }
79     return ((x % mod_val) + mod_val) % mod_val;
80 }
81
82 //
83 // -----
84 //
85 // 5. Trial Division Primality Test
86 // Tests divisibility by 2, 3, then all numbers of the form
87 // 6k +/- 1.
88 //
89 // -----
90
91 bool is_prime_trial(long long n) {

```

```

77     if (n < 2) return false;
78     if (n == 2 || n == 3) return true;
79     if (n % 2 == 0 || n % 3 == 0) return false;
80     for (long long i = 5; i * i <= n; i += 6) {
81         if (n % i == 0 || n % (i + 2) == 0) return false;
82     }
83     return true;
84 }
85
86 //
87 -----
88 // 6. Chinese Remainder Theorem
89 //     Solves the system:  $x \equiv a_i \pmod{n_i}$  for pairwise
90 //     coprime  $n_i$ .
91 //     Uses constructive method:  $M = \prod(n_i)$ ,  $x = \sum(a_i * M_i * y_i)$ 
92 //     where  $M_i = M/n_i$  and  $y_i = M_i^{-1} \pmod{n_i}$ .
93 //
94 -----
95
96 struct Congruence {
97     long long a; // remainder
98     long long n; // modulus
99 };
100
101 long long chinese_remainder_theorem(const std::vector<
102     Congruence>& system) {
103     if (system.empty()) {
104         throw std::runtime_error("Empty system of congruences")
105         ;
106     }
107
108     long long M = 1;
109     for (const auto& c : system) {
110         M *= c.n;
111     }
112
113     long long x = 0;
114     for (const auto& c : system) {
115         long long M_i = M / c.n;
116         long long y_i = mod_inverse(M_i, c.n);
117         x = (x + c.a * M_i % M * y_i % M) % M;
118     }
119     return ((x % M) + M) % M;
120 }
121
122 //
123 -----
124
125 // 7. Euler's Totient Function -- compute  $\phi(n)$ 
126 //      $\phi(n) = n * \prod((1 - 1/p))$  for all prime factors  $p$  of  $n$ 
127 //
128 //
129 //
130 -----

```

```

120 long long euler_totient(long long n) {
121     long long result = n;
122     for (long long p = 2; p * p <= n; ++p) {
123         if (n % p == 0) {
124             while (n % p == 0) {
125                 n /= p;
126             }
127             result -= result / p;
128         }
129     }
130     if (n > 1) {
131         result -= result / n;
132     }
133     return result;
134 }
135
136 //
137 // -----
138 // 8. Demonstration -- tests all number theory routines
139 // -----
140
141 void demonstrate_number_theory() {
142     std::println("=== Number Theory Algorithms ===\n");
143
144     // --- Modular Exponentiation ---
145     std::println("--- Modular Exponentiation ---");
146     {
147         long long base = 2, exp = 10, mod = 1000;
148         std::println(" 2^10 mod 1000 = {}", mod_pow(base, exp,
149             mod));
150
151         base = 3; exp = 13; mod = 7;
152         std::println(" 3^13 mod 7   = {}", mod_pow(base, exp,
153             mod));
154
155         base = 7; exp = 256; mod = 13;
156         std::println(" 7^256 mod 13 = {}", mod_pow(base, exp,
157             mod));
158     }
159
160     // --- GCD ---
161     std::println("\n--- GCD (Euclidean Algorithm) ---");
162     {
163         std::println(" gcd(48, 18)    = {}", gcd(48, 18));
164         std::println(" gcd(100, 75)   = {}", gcd(100, 75));
165         std::println(" gcd(17, 13)    = {}", gcd(17, 13));
166     }
167
168     // --- Extended GCD ---
169     std::println("\n--- Extended GCD ---");
170     {
171         auto [g, x, y] = extended_gcd(35, 15);
172         std::println(" 35 * (x) + 15 * (y) = {} = gcd(35,
173             15)", x, y, g);
174     }
175 }

```

```

168     auto [g2, x2, y2] = extended_gcd(30, 20);
169     std::println(" 30 * ({} ) + 20 * ({} ) = {} = gcd(30,
170                 20)", x2, y2, g2);
171
172     auto [g3, x3, y3] = extended_gcd(99, 78);
173     std::println(" 99 * ({} ) + 78 * ({} ) = {} = gcd(99,
174                 78)", x3, y3, g3);
175 }
176 // --- Modular Inverse ---
177 std::println("\n--- Modular Inverse ---");
178 {
179     long long a = 3, mod = 7;
180     long long inv = mod_inverse(a, mod);
181     std::println(" {}^(-1) mod {} = {} (check: {} * {}
182                 mod {} = {})",
183                 a, mod, inv, a, inv, mod, (a * inv) % mod
184                 );
185
186     a = 17; mod = 43;
187     inv = mod_inverse(a, mod);
188     std::println(" {}^(-1) mod {} = {} (check: {} * {}
189                 mod {} = {})",
190                 a, mod, inv, a, inv, mod, (a * inv) % mod
191                 );
192 }
193 // --- Primality ---
194 std::println("\n--- Trial Division Primality ---");
195 {
196     std::vector<long long> test_nums = {1, 2, 3, 4, 17, 25,
197         29, 100, 997, 1000};
198     for (long long n : test_nums) {
199         std::println(" is_prime({}) = {}", n,
200             is_prime_trial(n) ? "true" : "false");
201     }
202 }
203 // --- Chinese Remainder Theorem ---
204 std::println("\n--- Chinese Remainder Theorem ---");
205 {
206     // Solve:  $x \equiv 2 \pmod{3}$ ,  $x \equiv 3 \pmod{5}$ ,  $x \equiv 2 \pmod{7}$ 
207     std::vector<Congruence> system = {{2, 3}, {3, 5}, {2,
208         7}};
209     long long x = chinese_remainder_theorem(system);
210     std::println(" x  $\equiv 2 \pmod{3}$ ,  $x \equiv 3 \pmod{5}$ ,  $x \equiv 2 \pmod{7}$ ");
211     std::println(" Solution:  $x \equiv \{ \} \pmod{ \}$ ", x, 3 * 5 *
212         7);
213
214     // Verify
215     std::println(" Verify:  $\{ \} \pmod{3} = \{ \}$ ,  $\{ \} \pmod{5} = \{ \}$ ,
216          $\{ \} \pmod{7} = \{ \}$ ",
217         x, x % 3, x, x % 5, x, x % 7);

```

```

211     }
212
213     // --- Euler Totient ---
214     std::println("\n--- Euler's Totient Function ---");
215     {
216         std::vector<long long> nums = {1, 2, 6, 10, 12, 36,
217                                     100};
218         for (long long n : nums) {
219             std::println("  phi({}) = {}", n, euler_totient(n))
220             ;
221         }
222     }
223 } // namespace randalgo

```

Listing 15.2: Cryptographic algorithms: RSA, Miller–Rabin, Solovay–Strassen, Jacobi symbol.

```

1  #pragma once
2
3  #include "number_theory.h"
4
5  #include <print>
6  #include <vector>
7  #include <random>
8  #include <stdexcept>
9  #include <algorithm>
10 #include <cmath>
11 #include <chrono>
12
13 namespace randalgo {
14
15     //
16     -----
17
18     // Shared PRNG -- seeded from high-resolution clock for
19     // reproducibility demos
20     -----
21
22     static inline std::mt19937_64& crypto_rng() {
23         static std::mt19937_64 rng{
24             static_cast<unsigned>(
25                 std::chrono::steady_clock::now().time_since_epoch()
26                 .count())};
27         return rng;
28     }
29
30     //
31     -----
32
33     // 1. Quadratic Residue -- Euler's criterion: a is a QR mod p
34     // iff
35     // a^((p-1)/2) == 1 (mod p).

```



```

28 //
29 -----
29 bool is_quadratic_residue(long long a, long long p) {
30     if (a < 0 || a >= p) a = ((a % p) + p) % p;
31     if (a == 0) return true;
32     long long euler = mod_pow(a, (p - 1) / 2, p);
33     return euler == 1;
34 }
35
36 //
37 -----
37 // 2. Legendre Symbol (a / p)
38 //     Returns 1 if a is a QR mod p
39 //           -1 if a is a QNR mod p
40 //           0 if p | a
41 //
42 -----
42 int legendre_symbol(long long a, long long p) {
43     if (!is_prime_trial(p)) {
44         throw std::runtime_error("Legendre symbol requires
45             prime modulus");
46     }
47     if (a < 0 || a >= p) a = ((a % p) + p) % p;
48     if (a == 0) return 0;
49     long long euler = mod_pow(a, (p - 1) / 2, p);
50     if (euler == p - 1) return -1;
51     return static_cast<int>(euler);
52 }
53 //
54 -----
54 // 3. Jacobi Symbol (a / n) -- generalization of Legendre to
55 //    composite n
56 //    Uses properties:
57 //    (a / n) = (a mod n / n)
58 //    (0 / n) = 0
59 //    (1 / n) = 1
60 //    (2a / n) = (-1)^((n^2-1)/8) * (a / n) [if n is odd]
61 //    (a / n) = (b / a) * (-1)^((a-1)(n-1)/4) [quadratic
62 //    reciprocity]
63 //
64 -----
62 int jacobi_symbol(long long a, long long n) {
63     if (n <= 0 || n % 2 == 0) {
64         throw std::runtime_error("Jacobi symbol requires odd
65             positive n");
66     }
67     a = ((a % n) + n) % n;
68     int result = 1;
69     while (a != 0) {
70         // Factor out powers of 2 from a

```

```

70     int v = 0;
71     while (a % 2 == 0) {
72         a /= 2;
73         ++v;
74     }
75     // Apply (2 / n):  $(-1)^{(n^2-1)/8}$ 
76     if (v % 2 == 1) {
77         long long n_mod8 = n % 8;
78         if (n_mod8 == 3 || n_mod8 == 5) {
79             result = -result;
80         }
81     }
82     // Quadratic reciprocity swap
83     if (a % 4 == 3 && n % 4 == 3) {
84         result = -result;
85     }
86     std::swap(a, n);
87     a %= n;
88 }
89 return (n == 1) ? result : 0;
90 }
91
92 // -----
93
94 // 4. RSA Key Generation
95 // Given primes p, q and public exponent e (with  $\gcd(e, \phi(n)) = 1$ ),
96 // compute the private exponent  $d = e^{-1} \bmod \phi(n)$ .
97 // -----
98
99 struct RSAKeyPair {
100     long long e; // public exponent
101     long long d; // private exponent
102     long long n; // modulus
103 };
104
105 RSAKeyPair generate_rsa_keypair(long long p, long long q, long
106     long e) {
107     long long n = p * q;
108     long long phi_n = (p - 1) * (q - 1);
109
110     if ( $\gcd(e, \phi_n) \neq 1$ ) {
111         throw std::runtime_error("e must be coprime to phi(n)");
112     }
113
114     long long d = mod_inverse(e, phi_n);
115     return {e, d, n};
116 }
117
118 // -----
119
120 // 5. RSA Encrypt / Decrypt

```

```

117 //      Encrypt:  $c = m^e \bmod n$ 
118 //      Decrypt:  $m = c^d \bmod n$ 
119 //
-----
120 long long rsa_encrypt(long long m, long long e, long long n) {
121     return mod_pow(m, e, n);
122 }
123
124 long long rsa_decrypt(long long c, long long d, long long n) {
125     return mod_pow(c, d, n);
126 }
127
128 //
-----
129 // 6. Miller-Rabin Primality Test
130 // Tests whether  $n$  is prime using  $k$  random bases.
131 // Writes  $n-1 = 2^r * d$ , then checks:
132 //  $a^d \neq 1 \pmod{n}$  AND  $a^{(2^j * d)} \neq -1 \pmod{n}$  for all
133 //  $0 \leq j < r$ 
134 // then  $n$  is composite.
135 //
-----
136 bool miller_rabin(long long n, int k = 20) {
137     if (n < 2) return false;
138     if (n == 2 || n == 3) return true;
139     if (n % 2 == 0) return false;
140
141     // Write  $n - 1 = 2^r * d$ 
142     long long d = n - 1;
143     int r = 0;
144     while (d % 2 == 0) {
145         d /= 2;
146         ++r;
147     }
148
149     std::uniform_int_distribution<long long> dist(2, n - 2);
150
151     for (int i = 0; i < k; ++i) {
152         long long a = dist(crypto_rng());
153         long long x = mod_pow(a, d, n);
154
155         if (x == 1 || x == n - 1) continue;
156
157         bool found = false;
158         for (int j = 0; j < r - 1; ++j) {
159             x = mod_pow(x, 2, n);
160             if (x == n - 1) {
161                 found = true;
162                 break;
163             }
164         }
165         if (!found) return false;
166     }
167 }

```

```
166     return true;
167 }
168
169 //
-----
170 // 7. Solovay-Strassen Primality Test
171 //   n is composite if for some random a:
172 //    $a^{((n-1)/2)} \neq (a / n) \pmod{n}$ 
173 //
-----
174 bool solovay_strassen(long long n, int k = 20) {
175     if (n < 2) return false;
176     if (n == 2 || n == 3) return true;
177     if (n % 2 == 0) return false;
178
179     std::uniform_int_distribution<long long> dist(2, n - 2);
180
181     for (int i = 0; i < k; ++i) {
182         long long a = dist(crypto_rng());
183         long long a_mod_n = ((a % n) + n) % n;
184         long long euler = mod_pow(a_mod_n, (n - 1) / 2, n);
185         int js = jacobi_symbol(a, n);
186         // Treat -1 as n-1 in modular arithmetic for comparison
187         long long js_mod_n = (js + n) % n;
188         if (euler != js_mod_n) return false;
189     }
190     return true;
191 }
192
193 //
-----
194 // 8. Fermat Primality Test
195 //   n is composite if  $a^{(n-1)} \neq 1 \pmod{n}$  for some random a.
196 //
-----
197 bool fermat_primality_test(long long n, int k = 20) {
198     if (n < 2) return false;
199     if (n == 2 || n == 3) return true;
200     if (n % 2 == 0) return false;
201
202     std::uniform_int_distribution<long long> dist(2, n - 2);
203
204     for (int i = 0; i < k; ++i) {
205         long long a = dist(crypto_rng());
206         if (mod_pow(a, n - 1, n) != 1) return false;
207     }
208     return true;
209 }
210
211 //
```

```

212 // 9. Demonstration
213 //
-----
214 void demonstrate_crypto() {
215     std::println("=== Cryptographic Algorithms ===\n");
216
217     // --- RSA Key Generation & Encryption/Decryption ---
218     std::println("--- RSA Encryption/Decryption ---");
219     {
220         long long p = 61, q = 53;
221         long long e = 17;
222         auto [pub_e, priv_d, n] = generate_rsa_keypair(p, q, e)
                ;
223         std::println(" p = {}, q = {}, n = {}, phi(n) = {}", p
                , q, n, (p - 1) * (q - 1));
224         std::println(" Public key: (e={}, n={})", pub_e, n);
225         std::println(" Private key: (d={}, n={})", priv_d, n);
226
227         long long msg = 42;
228         long long cipher = rsa_encrypt(msg, pub_e, n);
229         long long plain = rsa_decrypt(cipher, priv_d, n);
230         std::println(" Message: {}", msg);
231         std::println(" Ciphertext: {}", cipher);
232         std::println(" Decrypted: {}", plain);
233         std::println(" Roundtrip OK: {}", plain == msg ? "YES"
                : "NO");
234
235         // Second message
236         msg = 255;
237         cipher = rsa_encrypt(msg, pub_e, n);
238         plain = rsa_decrypt(cipher, priv_d, n);
239         std::println("\n Message: {}", msg);
240         std::println(" Ciphertext: {}", cipher);
241         std::println(" Decrypted: {}", plain);
242     }
243
244     // --- Primality Tests ---
245     std::println("\n--- Primality Tests ---");
246     {
247         std::vector<long long> candidates = {2, 3, 17, 561,
                997, 1009, 104729};
248         std::println(" {:>8} | {:>12} | {:>12} | {:>12} |
                {:>8}",
                "n", "Fermat", "Miller-Rabin", "Solovay-
                Str", "Trial");
249         std::println(" {}+-{}+-{}+-{}+-{}+-{}",
                std::string(8, '-'), std::string(12, '-'),
                ,
                std::string(12, '-'), std::string(12, '-'),
                ), std::string(8, '-'));
250
251         for (long long n : candidates) {
252             bool f = fermat_primality_test(n, 20);
253             bool mr = miller_rabin(n, 20);
254             bool ss = solovay_strassen(n, 20);
255         }
256     }
257 }

```

```

258         bool td = is_prime_trial(n);
259         std::println("    {:>8} | {:>12} | {:>12} | {:>12} |
                {:>8}",
260                     n,
261                     f ? "prime" : "composite",
262                     mr ? "prime" : "composite",
263                     ss ? "prime" : "composite",
264                     td ? "prime" : "composite");
265     }
266
267     // Carmichael number 561 -- Fermat test fails with some
        bases
268     std::println("\n Note: 561 is a Carmichael number (
        composite but passes Fermat for many bases)");
269 }
270
271 // --- Quadratic Residues ---
272 std::println("\n--- Quadratic Residues ---");
273 {
274     long long p = 13;
275     std::println("    Modulus p = {}", p);
276     std::println("    Quadratic residues mod {}: ", p);
277
278     for (long long a = 0; a < p; ++a) {
279         bool qr = is_quadratic_residue(a, p);
280         int ls = legendre_symbol(a, p);
281         std::println("        a={}: QR={ } Legendre={}", a, qr
                ? "yes" : "no ", ls);
282     }
283
284     // Legendre symbol for several values
285     std::println("\n    Legendre symbols (a / 7):");
286     for (long long a = 1; a <= 6; ++a) {
287         std::println("        ({} / 7) = {}", a,
                legendre_symbol(a, 7));
288     }
289
290     // Jacobi symbol for composite modulus
291     std::println("\n    Jacobi symbols (a / 15) with n=15=3x5
        :");
292     for (long long a = 1; a <= 14; ++a) {
293         int js = jacobi_symbol(a, 15);
294         std::println("        ({} / 15) = {}", a, js);
295     }
296 }
297 }
298
299 } // namespace randalgo

```

Listing 15.3: Polynomials over $\mathbb{Z}_p$ and the Schwartz–Zippel identity test.

```

1 #pragma once
2
3 #include "number_theory.h"

```

```

4
5 #include <print>
6 #include <vector>
7 #include <random>
8 #include <algorithm>
9 #include <chrono>
10
11 namespace randalgo {
12
13 // -----
14 // Polynomial class -- coefficients stored as long long,
15 //   arithmetic mod p.
16 //   Coefficients are ordered from lowest degree to highest:
17 //   coeffs[0] = constant term, coeffs[1] = x^1, ...
18 // -----
19
20 class Polynomial {
21 public:
22     std::vector<long long> coeffs;
23
24     Polynomial() = default;
25
26     explicit Polynomial(std::vector<long long> c) : coeffs(std
27         ::move(c)) {
28         trim();
29     }
30
31     // Evaluate at x modulo p using Horner's method: O(n)
32     long long evaluate(long long x, long long p) const {
33         long long result = 0;
34         for (auto it = coeffs.rbegin(); it != coeffs.rend(); ++
35             it) {
36             result = (result * x + *it) % p;
37         }
38         return ((result % p) + p) % p;
39     }
40
41     // Polynomial addition mod p
42     Polynomial add(const Polynomial& other, long long p) const
43     {
44         size_t max_len = std::max(coeffs.size(), other.coeffs.
45             size());
46         std::vector<long long> result(max_len, 0);
47         for (size_t i = 0; i < max_len; ++i) {
48             long long a = (i < coeffs.size()) ? coeffs[i] : 0;
49             long long b = (i < other.coeffs.size()) ? other.
50                 coeffs[i] : 0;
51             result[i] = ((a + b) % p + p) % p;
52         }
53         return Polynomial(std::move(result));
54     }
55
56     // Polynomial multiplication mod p -- O(n * m)

```

```

50 Polynomial multiply(const Polynomial& other, long long p)
51     const {
52         if (coeffs.empty() || other.coeffs.empty()) return
53             Polynomial();
54         std::vector<long long> result(coeffs.size() + other.
55             coeffs.size() - 1, 0);
56         for (size_t i = 0; i < coeffs.size(); ++i) {
57             for (size_t j = 0; j < other.coeffs.size(); ++j) {
58                 result[i + j] = (result[i + j] + coeffs[i] *
59                     other.coeffs[j]) % p;
60             }
61         }
62         for (auto& c : result) c = ((c % p) + p) % p;
63         return Polynomial(std::move(result));
64     }
65
66 // Subtract another polynomial mod p
67 Polynomial subtract(const Polynomial& other, long long p)
68     const {
69         size_t max_len = std::max(coeffs.size(), other.coeffs.
70             size());
71         std::vector<long long> result(max_len, 0);
72         for (size_t i = 0; i < max_len; ++i) {
73             long long a = (i < coeffs.size()) ? coeffs[i] : 0;
74             long long b = (i < other.coeffs.size()) ? other.
75                 coeffs[i] : 0;
76             result[i] = ((a - b) % p + p) % p;
77         }
78         return Polynomial(std::move(result));
79     }
80
81 bool is_zero() const {
82     for (long long c : coeffs) {
83         if (c != 0) return false;
84     }
85     return true;
86 }
87
88 size_t degree() const {
89     if (coeffs.empty()) return 0;
90     for (size_t i = coeffs.size(); i-- > 0;) {
91         if (coeffs[i] != 0) return i;
92     }
93     return 0;
94 }
95
96 std::string to_string() const {
97     if (coeffs.empty()) return "0";
98     std::string s;
99     for (int i = static_cast<int>(coeffs.size()) - 1; i >=
100         0; --i) {
101         if (coeffs[i] == 0) continue;
102         if (!s.empty()) s += " + ";
103         if (i == 0 || coeffs[i] != 1) {
104             s += std::to_string(coeffs[i]);
105         }
106     }
107 }

```



```

98         if (i > 0) {
99             s += "x";
100             if (i > 1) s += "^" + std::to_string(i);
101         }
102     }
103     return s.empty() ? "0" : s;
104 }
105
106 private:
107     void trim() {
108         while (coeffs.size() > 1 && coeffs.back() == 0) {
109             coeffs.pop_back();
110         }
111     }
112 };
113
114 // Shared PRNG for polynomial random evaluation
115 static inline std::mt19937_64& poly_rng() {
116     static std::mt19937_64 rng{
117         static_cast<unsigned>(
118             std::chrono::steady_clock::now().time_since_epoch()
119             .count())};
120     return rng;
121 }
122 //
123
124 // Schwartz-Zippel Test
125 // If a polynomial over a field is identically zero, then  $\Pr[f(r)=0] = 1$ 
126 // for random  $r$ . If  $f \neq 0$  with degree  $\leq d$  over field  $\mathbb{Z}_p$ ,
127 // then
128 //  $\Pr[f(r)=0] \leq d/p$ . Repeated trials give high confidence.
129 //
130
131 bool schwartz_zippel_test(const Polynomial& poly, long long p,
132     int num_trials = 20) {
133     if (poly.is_zero()) return true;
134
135     std::uniform_int_distribution<long long> dist(0, p - 1);
136
137     for (int i = 0; i < num_trials; ++i) {
138         long long r = dist(poly_rng());
139         if (poly.evaluate(r, p) != 0) {
140             return false; // Found a non-root -> polynomial is
141                             // not identically zero
142         }
143     }
144     return true; // All trials returned zero -- very likely
145                     // identically zero
146 }
147 //
148

```

```

143 // Polynomial Identity Testing
144 // Tests whether poly1 == poly2 (mod p) by checking if poly1
145 // - poly2
146 // is identically zero using Schwartz-Zippel.
147 //
148 -----
149
150 bool polynomial_identity_test(const Polynomial& poly1,
151                               const Polynomial& poly2,
152                               long long p,
153                               int num_trials = 20) {
154     Polynomial diff = poly1.subtract(poly2, p);
155     return schwartz_zippel_test(diff, p, num_trials);
156 }
157 //
158 -----
159
160 // Demonstration
161 //
162 -----
163
164 void demonstrate_polynomial() {
165     std::println("== Polynomial Operations over Finite Fields
166                 ==\n");
167
168     long long p = 97; // A small prime
169
170     // --- Basic polynomial operations ---
171     std::println("--- Basic Operations (mod {}) ---", p);
172     {
173         Polynomial f({1, 2, 3}); //  $3x^2 + 2x + 1$ 
174         Polynomial g({4, 5}); //  $5x + 4$ 
175
176         std::println(" f(x) = {}", f.to_string());
177         std::println(" g(x) = {}", g.to_string());
178
179         Polynomial sum = f.add(g, p);
180         std::println(" f + g = {}", sum.to_string());
181
182         Polynomial diff = f.subtract(g, p);
183         std::println(" f - g = {}", diff.to_string());
184
185         Polynomial prod = f.multiply(g, p);
186         std::println(" f * g = {}", prod.to_string());
187
188         // Evaluate at x = 10 mod 97
189         long long x = 10;
190         std::println(" f({}) mod {} = {}", x, p, f.evaluate(x,
191 p));
192         std::println(" g({}) mod {} = {}", x, p, g.evaluate(x,
193 p));
194         std::println(" (f*g)({}) mod {} = {}", x, p, prod.
195 evaluate(x, p));
196     }
197 }

```

```

187
188 // --- Schwartz-Zippel: zero polynomial ---
189 std::println("\n--- Schwartz-Zippel: Zero Polynomial ---");
190 {
191     Polynomial zero({0, 0, 0, 0}); // identically zero
192     bool result = schwartz_zippel_test(zero, p, 30);
193     std::println("    Polynomial: 0 (all coefficients zero)")
194     ;
195     std::println("    Schwartz-Zippel says identically zero?
196     {}", result ? "YES" : "NO");
197 }
198
199 // --- Schwartz-Zippel: non-zero polynomial that happens to
200 //      vanish at some points ---
201 std::println("\n--- Schwartz-Zippel: Non-Zero Polynomial
202 ---");
203 {
204     //  $f(x) = x^2 - x = x(x-1)$  -- vanishes at  $x=0$  and  $x=1$ 
205     //      but is NOT identically zero
206     Polynomial f({0, -1, 1});
207     std::println("    Polynomial: {} =  $x(x-1)$ ", f.to_string())
208     ;
209     std::println("     $f(0) = {}$ ,  $f(1) = {}$ ,  $f(2) = {}$ ",
210     f.evaluate(0, p), f.evaluate(1, p), f.
211     evaluate(2, p));
212     bool result = schwartz_zippel_test(f, p, 30);
213     std::println("    Schwartz-Zippel says identically zero?
214     {}", result ? "YES" : "NO");
215 }
216
217 // --- Schwartz-Zippel: a "tall" polynomial that vanishes
218 //      on many points ---
219 std::println("\n--- Schwartz-Zippel: Product of Many Roots
220 ---");
221 {
222     //  $f(x) = x(x-1)(x-2)\dots(x-9)$  -- vanishes at 10 points
223     //      but degree 10 < 97
224     Polynomial f({1}); // start with constant 1
225     Polynomial factor({0, 1}); //  $x$ 
226     for (long long r = 0; r < 10; ++r) {
227         Polynomial shift({-r, 1}); //  $(x - r)$ 
228         f = f.multiply(shift, p);
229     }
230     std::println("    Polynomial degree: {}", f.degree());
231     std::println("    Schwartz-Zippel says identically zero?
232     {}",
233     schwartz_zippel_test(f, p, 30) ? "YES" :
234     "NO");
235 }
236
237 // --- Polynomial Identity Testing ---
238 std::println("\n--- Polynomial Identity Testing ---");
239 {
240     Polynomial a({3, 2, 1}); //  $x^2 + 2x + 3$ 
241     Polynomial b({3, 2, 1}); // same polynomial
242     Polynomial c({1, 2, 3}); // different:  $3x^2 + 2x + 1$ 

```

```

230         std::println("  a(x) = {}", a.to_string());
231         std::println("  b(x) = {}", b.to_string());
232         std::println("  c(x) = {}", c.to_string());
233         std::println("  a == b? {}", polynomial_identity_test(a
234             , b, p, 30) ? "YES" : "NO");
235         std::println("  a == c? {}", polynomial_identity_test(a
236             , c, p, 30) ? "YES" : "NO");
237     }
238     // --- Schwartz-Zippel probability bound ---
239     std::println("\n--- Schwartz-Zippel Probability Bound ---")
240     ;
241     {
242         // For degree-d poly over Z_p, Pr[f(r)=0 | f!=0] <= d/p
243         long long p_val = 97;
244         std::println(" Over Z_{} (p = {}):", p_val, p_val);
245         std::println("   degree 1:  error <= {:.4f}", 1.0 /
246             p_val);
247         std::println("   degree 5:  error <= {:.4f}", 5.0 /
248             p_val);
249         std::println("   degree 10: error <= {:.4f}", 10.0 /
250             p_val);
251         std::println("   degree 50: error <= {:.4f}", 50.0 /
252             p_val);
253         std::println(" With 20 trials, degree 10: error <=
254             {:.2e}", std::pow(10.0 / p_val, 20));
255     }
256 }
257 } // namespace randalgo

```

Notes

The mathematical foundations of this chapter draw on classical number theory. Euler's totient function and Euler's theorem date to the 18th century. Fermat's little theorem appeared in a letter from Fermat to Frenicle de Bessy in 1640. The Chinese Remainder Theorem was known to Sun Tzu in the 3rd century CE and was formalized by Gauss in 1801. Bézout's identity appeared in his *Théorie générale des équations algébriques* (1764).

The RSA cryptosystem was invented by Rivest, Shamir, and Adleman in 1978 and has since become the most widely used public-key cryptosystem. The theoretical equivalence between breaking RSA and factoring was established by Rivest

and others in the early 1980s.

The Schwartz–Zippel lemma was proved independently by Schwartz (1980) and Zippel (1979/1981) in the context of polynomial identity testing. It is one of the most widely used tools in randomized algebraic computation.

For primality testing, the Fermat test dates to Fermat's 17th-century observations. Miller (1976) proposed the deterministic version of his test assuming the Extended Riemann Hypothesis; Rabin (1980) modified it to remove this assumption, obtaining the randomized version. Solovay and Strassen (1977) independently discovered their randomized test. The deterministic polynomial-time AKS test was achieved by Agrawal, Kayal, and Saxena in 2002, confirming that primality is in P.

Berlekamp's polynomial factorization algorithm appeared in 1967, and the Cantor–Zassenhaus algorithm (1981) provides an alternative randomized approach for factoring polynomials over finite fields.

- R. L. Rivest, A. Shamir, and L. Adleman, "A Method for Obtaining Digital Signatures and Public-Key Cryptosystems," *Commun. ACM*, 21(2):120–126, 1978.
- G. L. Miller, "Riemann's Hypothesis and Tests for Primality," *J. Comput. Syst. Sci.*, 13(3):300–317, 1976.
- M. O. Rabin, "Probabilistic Algorithm for Testing Primality and Irreducibility," *J. Number Theory*, 12(1):128–138, 1980.
- R. Solovay and V. Strassen, "A Fast Monte-Carlo Test for Primality," *SIAM J. Comput.*, 6(1):84–85, 1977.
- M. Agrawal, N. Kayal, and N. Saxena, "PRIMES is in P," *Ann. of Math.*, 160(2):781–793, 2004.
- J. T. Schwartz, "Fast Probabilistic Algorithms for Verification of Polynomial Identities," *J. ACM*, 27(4):701–717, 1980.
- R. E. Zippel, "Probabilistic Polynomial Identity Testing," *Proc. 12th ACM STOC*, 208–217, 1979.
- E. R. Berlekamp, "Factoring Polynomials over Large Finite Fields," *Math. Comp.*, 21:253–260, 1967.
- D. Cantor and H. Zassenhaus, "On Distinct Subfield Rep-

resentations of Finite Fields," *Commun. Pure Appl. Math.*, 34:827–832, 1981.

- W. R. Alford, A. Granville, and C. Pomerance, "On the Infinitely Many Carmichael Numbers up to x ," *Ann. of Math.*, 139(3):703–722, 1994.

Problems

Problem 15.52 (14.1). Prove that for any prime p and integer a with $\gcd(a, p) = 1$, the order of a modulo p (the smallest positive k with $a^k \equiv 1 \pmod{p}$) divides $p - 1$. Use this to give an alternative proof of Fermat's little theorem.

Problem 15.53 (14.2). Implement the extended Euclidean algorithm and use it to compute $17^{-1} \bmod 43$ and $23^{-1} \bmod 97$. Verify your answers by direct multiplication.

Problem 15.54 (14.3). Let $n = 77 = 7 \cdot 11$. Compute $\phi(77)$ and list all elements of $\mathbb{Z}_{77}^*$. Find a primitive root modulo 77 if one exists.

Problem 15.55 (14.4). Let $p = 29$. Using Euler's criterion (Theorem 15.23), determine which of the following are quadratic residues modulo 29: $a = 3, 7, 13, 17$.

Problem 15.56 (14.5). (RSA computation) Choose $p = 61$ and $q = 53$. Compute n , $\phi(n)$, and find a suitable public exponent e and private exponent d . Encrypt the message $m = 42$ and verify that decryption recovers m .

Problem 15.57 (14.6). Prove Theorem 15.27: for a Blum integer $n = pq$ and $a \in \mathbb{Z}_n^*$, a is a quadratic residue modulo n if and only if $\left(\frac{a}{p}\right) = 1$ and $\left(\frac{a}{q}\right) = 1$.

Problem 15.58 (14.7). Use the Schwartz–Zippel lemma to give a randomized polynomial-time algorithm for testing whether a given $n \times n$ matrix M over $\mathbb{F}_p$ is symmetric ($M = M^T$), skew-symmetric ($M = -M^T$), or orthogonal ($M^T M = I$). State the error probability.

Problem 15.59 (14.8). Let $n = 561$ (the smallest Carmichael number). Show directly that $a^{560} \equiv 1 \pmod{561}$ for $a = 2, 3, 5$, and verify that 561 is composite. Then run the Miller–Rabin test on $n = 561$ with base $a = 2$ and determine whether 2 is a Miller–Rabin witness.

Problem 15.60 (14.9). **(Analysis)** Prove that if n is composite and $n - 1 = 2^s d$ with d odd, then the set of Miller–Rabin non-witnesses is a subgroup of $\mathbb{Z}_n^*$. Conclude that at most $1/2$ of elements of $\mathbb{Z}_n^*$ are non-witnesses when $n = p^k$ for an odd prime p and $k \geq 2$.

Problem 15.61 (14.10). **(Research)** The AKS primality test (Theorem 15.50) is based on the identity $(x + a)^n \equiv x^n + a \pmod{n}$ that holds for all a when n is prime (by the binomial theorem and $p \mid \binom{p}{i}$ for $0 < i < p$). Explain why this identity also holds modulo a polynomial $x^r - 1$ for a suitably chosen r , and why testing this identity for enough bases a suffices to certify primality.

Bibliography

- [1] P. Erdős, “Some remarks on number theory,” *Bull. Res. Council Israel*, 3:1947, pp. 8–10.
- [2] P. Erdős and L. Lovász, “Problems and results on 3-chromatic hypergraphs and some related questions,” *Infinite and Finite Sets*, 1975, pp. 609–627.
- [3] J. Spencer, “Asymptotic lower bounds for Ramsey functions,” *Discrete Mathematics*, 9(1):73–80, 1975.
- [4] N. Alon and J. Spencer, *The Probabilistic Method*, Wiley-Interscience, 1992 (2nd ed. 2000, 3rd ed. 2008 with J. Spencer and E. Sudakov).
- [5] R. Moser and G. Tardos, “A constructive proof of the general Lovász Local Lemma,” *J. ACM*, 57(2):1–15, 2010.
- [6] A. Lubotzky, R. Phillips, and P. Sarnak, “Ramanujan graphs,” *Combinatorica*, 8(3):261–277, 1988.
- [7] G. A. Margulis, “Explicit group-theoretic constructions of combinatorial schemes and their application in the design of expanders and superconcentrators,” *Problemy Peredachi Informatsii*, 24(2):31–39, 1988.
- [8] M. Ajtai, J. Komlós, and E. Szemerédi, “Sorting in $O(n \log n)$ parallel steps,” *Combinatorica*, 3(1):1–19, 1983.
- [9] S. Albers and M. Mitzenmacher, “Toward a theory of on-line repair,” *Proc. FOCS*, 2003.

- [10] D. Aldous, "Random walks on finite groups and rapidly mixing Markov chains," *Lecture Notes in Mathematics*, 1015:243–297, 1983.
- [11] D. Aldous, "An introduction to recycling for Markov chain Monte Carlo," *unpublished manuscript*, 1989.
- [12] D. Aldous, "The continuum random tree. I," *Ann. Probab.*, 19(4):1741–1766, 1991.
- [13] R. Aleliunas, L. Lovász, M. S. Vivian, and V. T. Vesztergombi, "Random walks on graphs: A survey," *Colloq. Math. Soc. János Bolyai*, 35:1–46, 1979.
- [14] N. Alon, L. Babai, and H. Itai, "A fast and simple randomized parallel algorithm for the maximal independent set problem," *J. Algorithms*, 7(4):567–583, 1986.
- [15] N. Alon and F. R. K. Chung, "Explicit construction of linear sized tolerant networks," *Discrete Math.*, 72:15–19, 1988.
- [16] J. Aronson, A. Frieze, and B. Pittel, "Maximum matchings in sparse random graphs," *Random Struct. Algorithms*, 12:319–360, 1998.
- [17] Y. Azar, "On-line routing of virtual circuits with applications to load balancing and machine scheduling," *J. ACM*, 45(3):479–495, 1998.
- [18] K. E. Batchier, "Sorting networks and their applications," *Proc. AFIPS*, 32:307–314, 1968.
- [19] M. Ben-Or, "Another proof of $BPP \subseteq \Sigma_2^p$," *Proc. STOC*, 22:117–122, 1983.
- [20] S. W. Bent and J. W. John, "Finding the median with $O(n)$ comparisons," *unpublished manuscript*, 1985.
- [21] B. H. Bloom, "A space/time trade-off for lookup key hashing with bounded time," *J. ACM*, 17(1):131–141, 1970.

- [22] M. L. Blum, "Coin-flipping games," *Proc. FOCS*, 14:142–146, 1973.
- [23] A. Borodin, N. Linial, and M. Saks, "An optimal on-line algorithm for metrical task systems," *J. ACM*, 39(4):745–763, 1992.
- [24] R. P. Brent, "The parallel evaluation of general arithmetic expressions," *J. ACM*, 21(2):201–206, 1974.
- [25] S. Bubeck, "Convex optimization: Algorithms and complexity," *Foundations and Trends in Machine Learning*, 8(3-4):231–357, 2015.
- [26] J. L. Carter and M. N. Wegman, "Universal classes of hash functions," *J. Computer and System Sciences*, 18(2):143–154, 1979.
- [27] P. L. Chebyshev, "A method for investigating certain integrals," *Zapiski Akad. Nauk St. Petersburg*, 7:1867.
- [28] B. Chor and O. Goldreich, "On the power of two-point based sampling," *J. Complexity*, 5(1):96–106, 1989.
- [29] S. A. Cook, "An observation on time-space tradeoff," *J. CSS*, 27:309–316, 1983.
- [30] L. Devroye, "Applications of the theory of record values in random sampling," *Ann. Probability*, 16:1788–1802, 1988.
- [31] M. Dietzfelbinger, A. Karloff, T. Watanabe, and U. Weyer, "On the complexity of nearest neighbor problems in Euclidean spaces," *Proc. STOC*, 304–313, 1994.
- [32] M. Dietzfelbinger and F. M. auf der Heide, "Simple, efficient shared memory simulations," *Proc. SPAA*, 161–169, 1995.
- [33] D. Dolev, N. A. Lynch, J. K. Pachlinski, and W. K. Shih, "Wait-free garbage collection," *Proc. STOC*, 160–169, 1983.

- [34] P. Erdős and A. Rényi, "On a problem in the theory of graphs," *Publ. Math. Inst. Hungar. Acad. Sci.*, 4:417–435, 1961.
- [35] W. Feller, *An Introduction to Probability Theory and Its Applications*, Vol. 1, Wiley, 3rd ed., 1968.
- [36] A. Fiat and D. Mendelson, "Optimal use of space in on-line memory allocation," *J. ACM*, 44(5):837–851, 1997.
- [37] R. W. Floyd, "Assigning meanings to programs," *Proc. Symposia in Applied Mathematics*, 19:19–32, 1975.
- [38] M. L. Fredman, "New bounds on the complexity of lattice problems," *Proc. STOC*, 16:278–287, 1984.
- [39] D. Gale and L. S. Shapley, "College admissions and the stability of marriage," *Amer. Math. Monthly*, 69(1):9–15, 1962.
- [40] O. Goldreich, "Finding quadratic residues mod n almost as efficiently as factoring n ," *SIAM J. Computing*, 31(2):328–345, 2001.
- [41] R. Impagliazzo, L. Levin, and M. Luby, "Pseudo-random generation from one-way functions," *Proc. STOC*, 12–24, 1989.
- [42] R. W. Irving, "An efficient algorithm for the stable roommates problem," *J. Algorithms*, 6:577–595, 1985.
- [43] M. Jerrum and A. Sinclair, "Polynomial-time approximation algorithms for the Ising model," *SIAM J. Computing*, 24(5):1267–1304, 1995.
- [44] A. Joffe, "On a theorem of Hoeffding," *Ann. Math. Statist.*, 17:309–324, 1946.
- [45] N. L. Johnson and S. Kotz, *Urn Models and Their Application*, Wiley, 1977.

- [46] A. R. Karlin, M. S. Manasse, L. A. McGeoch, and C. C. O'Donnell, "Competitive randomized algorithms for non-uniform problems," *Algorithmica*, 6(3):393–405, 1991.
- [47] R. M. Karp, "On the computational complexity of combinatorial problems," *Networks*, 5(1):45–68, 1975.
- [48] R. M. Karp and E. Upfal, "Constructing a perfect matching is in random NC," *Combinatorica*, 10(1):35–48, 1990.
- [49] D. E. Knuth, "Big omicron and big omega and big theta," *ACM SIGACT News*, 8(2):18–24, 1976.
- [50] V. F. Kolchin, *Random Graphs and Asymptotic Graph Theory*, Cambridge Univ. Press, 1978.
- [51] A. N. Kolmogorov, "Sulla determinazione empirica di una legge di distribuzione," *Giornale dell'Istituto Italiano degli Attuari*, 4:83–91, 1933.
- [52] E. Koutsoupias and C. H. Papadimitriou, "Worst-case algorithms for servers on a line," *Proc. STOC*, 248–253, 1995.
- [53] L. Lamport, R. Shostak, and M. Pease, "The Byzantine generals problem," *ACM Trans. Programming Languages and Systems*, 4(3):382–401, 1982.
- [54] F. T. Leighton, *Introduction to Parallel Algorithms and Architectures*, Morgan Kaufmann, 1992.
- [55] L. Lovász, "On deterministically approximating the volume of convex bodies," *J. ACM*, 38:965–981, 1991.
- [56] L. Lovász and M. Simonovits, "Randomized concatenation of random walks," *Proc. 24th STOC*, 165–174, 1992.
- [57] A. Lubotzky, R. Phillips, and P. Sarnak, "Ramanujan graphs," *Combinatorica*, 8(3):261–277, 1988.

- [58] M. Luby, "A simple parallel algorithm for the maximal independent set problem," *SIAM J. Computing*, 15(4):1036–1053, 1986.
- [59] M. Luby, "Removing randomness in parallel computation without a processor penalty," *Proc. STOC*, 323–333, 1988.
- [60] R. Lyons and Y. Peres, *Probability on Trees and Networks*, Cambridge Univ. Press, 2016.
- [61] M. S. Manasse, L. A. McGeoch, and D. D. Sleator, "Competitive algorithms for server problems," *Proc. SODA*, 211–222, 1988.
- [62] G. A. Margulis, "Explicit constructions of expanders," *Problemy Peredachi Informatsii*, 9(4):71–80, 1973.
- [63] G. A. Margulis, "Explicit group-theoretic constructions of combinatorial schemes," *Problemy Peredachi Informatsii*, 24(2):31–39, 1988.
- [64] A. A. Markov, "On one limit theorem of the theory of probabilities," *Bull. Acad. Sci. St. Petersburg*, 6:551–564, 1912.
- [65] P. Matthews, "A strong law for record times," *Zeitschrift für Wahrscheinlichkeitstheorie*, 78:377–381, 1988.
- [66] J. Naor and M. Naor, "Small-bias spaces: Efficient constructions and applications," *SIAM J. Computing*, 22(4):831–863, 1993.
- [67] M. Naor and O. Reingold, "Number-theoretic constructions of efficient pseudo-random functions," *Proc. FOCS*, 457–468, 1997.
- [68] R. Pagh and F. F. Rodler, "Cuckoo hashing," *J. Algorithms*, 51(2):122–144, 2004.

- [69] C. H. Papadimitriou, *Computational Complexity*, Addison-Wesley, 1994.
- [70] M. S. Pinsker, "On the complexity of constructing a consistent network," *Problemy Peredachi Informatsii*, 9:53–58, 1973.
- [71] N. Pippenger, "On simultaneous resource bounds," *Proc. FOCS*, 307–311, 1979.
- [72] W. Pugh, "Skip lists: A probabilistic alternative to balanced trees," *Commun. ACM*, 33(6):668–676, 1990.
- [73] M. O. Rabin, "Randomized Byzantine generals," *Proc. FOCS*, 403–409, 1983.
- [74] S. M. Ross, *Stochastic Processes*, Wiley, 2nd ed., 1996.
- [75] A. Schönhage and V. Strassen, "Schnelle Multiplikation großer Zahlen," *Computing*, 7:281–292, 1971.
- [76] S. R. Schöning, "A probabilistic algorithm for k -SAT and constraint satisfaction problems," *Proc. FOCS*, 410–414, 1999.
- [77] D. D. Sleator and R. E. Tarjan, "Amortized efficiency of list update and paging rules," *Commun. ACM*, 28(2):202–208, 1985.
- [78] J. Wyllie, "The complexity of parallel computations," PhD thesis, Cornell University, 1979.

Appendix A

Probability Theory

This appendix reviews the fundamental concepts from probability theory that are used throughout the book. Readers already familiar with discrete probability may wish to skip this appendix and refer back to it as needed.

.1 Sample Spaces and Events

A *random experiment* is any process whose outcome is uncertain. The set of all possible outcomes is the *sample space*, denoted Ω . For discrete probability, Ω is countable (finite or countably infinite). An *event* is a subset of Ω .

Definition .1. A *probability measure* on a finite sample space Ω is a function $\Pr : 2^\Omega \rightarrow [0, 1]$ satisfying:

1. $\Pr[\Omega] = 1$;
2. If A and B are disjoint events, then $\Pr[A \cup B] = \Pr[A] + \Pr[B]$.

The second property is *finite additivity*. For countably infinite sample spaces, we require *countable additivity*: for any countable collection of pairwise disjoint events $A_1, A_2, \dots$,

$$\Pr\left[\bigcup_{i=1}^{\infty} A_i\right] = \sum_{i=1}^{\infty} \Pr[A_i].$$

Definition .2 (Conditional probability). For events A and B with $\Pr[B] > 0$, the conditional probability of A given B is

$$\Pr[A \mid B] = \frac{\Pr[A \cap B]}{\Pr[B]}.$$

Definition .3 (Independence). Two events A and B are *independent* if $\Pr[A \cap B] = \Pr[A] \Pr[B]$. A collection of events $\{A_i\}$ is *mutually independent* if for every finite subcollection $A_{i_1}, \dots, A_{i_k}$,

$$\Pr[A_{i_1} \cap \dots \cap A_{i_k}] = \prod_{j=1}^k \Pr[A_{i_j}].$$

A weaker notion, *pairwise independence*, requires only that every two events satisfy the product condition. Pairwise independence is sufficient for several applications (e.g., Chebyshev bounds) but insufficient for others (e.g., Chernoff bounds).

Theorem .4 (Law of Total Probability). If $\{B_1, \dots, B_k\}$ is a partition of Ω (disjoint, $\bigcup B_i = \Omega$) with $\Pr[B_i] > 0$, then for any event A ,

$$\Pr[A] = \sum_{i=1}^k \Pr[A \mid B_i] \Pr[B_i].$$

Theorem .5 (Bayes' Theorem).

$$\Pr[B_j \mid A] = \frac{\Pr[A \mid B_j] \Pr[B_j]}{\sum_{i=1}^k \Pr[A \mid B_i] \Pr[B_i]}.$$

.2 Random Variables

Definition .6. A *random variable* X is a function $X : \Omega \rightarrow \mathbb{R}$. For discrete sample spaces, X is a *discrete random variable*. The *probability mass function* (pmf) is $p_X(x) = \Pr[X = x]$.

Definition .7 (Expectation). The *expected value* (mean) of a discrete random variable X is

$$\mathbb{E}[X] = \sum_x x \cdot \Pr[X = x],$$

provided the sum converges absolutely.

Theorem .8 (Linearity of Expectation). *For any random variables $X_1, \dots, X_n$ (not necessarily independent),*

$$\mathbb{E}\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n \mathbb{E}[X_i].$$

Linearity of expectation is among the most powerful tools in the analysis of randomized algorithms. It allows us to compute expected values by decomposing a complex random variable into simpler *indicator variables*.

Theorem .9 (Law of the Unconscious Statistician (LOTUS)). *For a function $g : \mathbb{R} \rightarrow \mathbb{R}$,*

$$\mathbb{E}[g(X)] = \sum_x g(x) \cdot \Pr[X = x],$$

provided the sum converges absolutely.

Definition .10 (Variance and Standard Deviation). The *variance* of X is

$$\text{Var}[X] = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2.$$

The *standard deviation* is $\sigma[X] = \sqrt{\text{Var}[X]}$.

Theorem .11 (Properties of Variance). 1. $\text{Var}[aX + b] = a^2 \text{Var}[X]$ for constants a, b .

2. If $X_1, \dots, X_n$ are pairwise independent, then $\text{Var}\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n \text{Var}[X_i]$.

Definition .12 (Covariance). For two random variables X and Y ,

$$\text{Cov}[X, Y] = \mathbb{E}[(X - \mathbb{E}[X])(Y - \mathbb{E}[Y])] = \mathbb{E}[XY] - \mathbb{E}[X] \mathbb{E}[Y].$$

Theorem .13. $\text{Var}[X + Y] = \text{Var}[X] + \text{Var}[Y] + 2 \text{Cov}[X, Y]$.

.3 Important Discrete Distributions

The following distributions arise repeatedly in the book.

Definition .14 (Bernoulli). $X \sim \text{Bernoulli}(p)$: $\Pr[X = 1] = p$, $\Pr[X = 0] = 1 - p$. $\mathbb{E}[X] = p$, $\text{Var}[X] = p(1 - p)$.

Definition .15 (Binomial). $X \sim \text{Binomial}(n, p)$: sum of n independent $\text{Bernoulli}(p)$ trials.

$$\Pr[X = k] = \binom{n}{k} p^k (1 - p)^{n-k}, \quad \mathbb{E}[X] = np, \quad \text{Var}[X] = np(1 - p).$$

Definition .16 (Geometric). $X \sim \text{Geometric}(p)$: number of independent $\text{Bernoulli}(p)$ trials until the first success.

$$\Pr[X = k] = (1 - p)^{k-1} p, \quad \mathbb{E}[X] = \frac{1}{p}, \quad \text{Var}[X] = \frac{1 - p}{p^2}.$$

Definition .17 (Poisson). $X \sim \text{Poisson}(\lambda)$:

$$\Pr[X = k] = \frac{e^{-\lambda} \lambda^k}{k!}, \quad \mathbb{E}[X] = \lambda, \quad \text{Var}[X] = \lambda.$$

Definition .18 (Uniform). $X \sim \text{Uniform}\{1, \dots, n\}$:

$$\Pr[X = i] = \frac{1}{n}, \quad \mathbb{E}[X] = \frac{n + 1}{2}, \quad \text{Var}[X] = \frac{n^2 - 1}{12}.$$

.4 Useful Inequalities

Theorem .19 (Union Bound (Boole's Inequality)). *For any countable collection of events $A_1, A_2, \dots$,*

$$\Pr\left[\bigcup_i A_i\right] \leq \sum_i \Pr[A_i].$$

Theorem .20 (Markov's Inequality). *If X is a nonnegative random variable and $a > 0$, then*

$$\Pr[X \geq a] \leq \frac{\mathbb{E}[X]}{a}.$$

Theorem .21 (Chebyshev's Inequality). *If X has finite mean μ and variance σ^2 , then for any $k > 0$,*

$$\Pr[|X - \mu| \geq k] \leq \frac{\sigma^2}{k^2}.$$

Equivalently, $\Pr[|X - \mu| \geq k\sigma] \leq 1/k^2$.

Theorem .22 (Chernoff Bound—Multiplicative Form). *Let $X = \sum_{i=1}^n X_i$ where the X_i are independent Bernoulli(p_i) random variables, and let $\mu = \mathbb{E}[X] = \sum p_i$. Then for any $\delta > 0$,*

$$\Pr[X \geq (1 + \delta)\mu] \leq \left(\frac{e^\delta}{(1 + \delta)^{1+\delta}} \right)^\mu,$$

and for $0 < \delta < 1$,

$$\Pr[X \leq (1 - \delta)\mu] \leq \left(\frac{e^{-\delta}}{(1 - \delta)^{1-\delta}} \right)^\mu.$$

A simpler but slightly looser form is:

$$\Pr[X \geq (1 + \delta)\mu] \leq e^{-\mu\delta^2/3} \quad (0 < \delta \leq 1), \quad \Pr[X \leq (1 - \delta)\mu] \leq e^{-\mu\delta}$$

Theorem .23 (Azuma–Hoeffding Inequality). *Let $X_0, X_1, \dots, X_n$ be a martingale with respect to the filtration $\{\mathcal{F}_i\}$, and suppose there exist constants $c_i > 0$ such that $|X_i - X_{i-1}| \leq c_i$ almost surely. Then for any $t > 0$,*

$$\Pr[|X_n - X_0| \geq t] \leq 2 \exp\left(-\frac{t^2}{2 \sum_{i=1}^n c_i^2}\right).$$

.5 Convergence Concepts

When analyzing randomized algorithms, we often consider sequences of random variables. Three modes of convergence are relevant:

Definition .24. Let $X_1, X_2, \dots$ and X be random variables.

1. X_n converges in probability to X if for every $\varepsilon > 0$, $\lim_{n \rightarrow \infty} \Pr[|X_n - X| > \varepsilon] = 0$.
2. X_n converges in distribution to X if $\lim_{n \rightarrow \infty} \Pr[X_n \leq x] = \Pr[X \leq x]$ at every continuity point x of the distribution function of X .
3. X_n converges almost surely to X if $\Pr[\lim_{n \rightarrow \infty} X_n = X] = 1$.

Almost sure convergence implies convergence in probability, which implies convergence in distribution. The converse implications do not hold in general.

Theorem .25 (Weak Law of Large Numbers). *Let $X_1, \dots, X_n$ be independent and identically distributed random variables with finite mean μ . Then the sample average $\bar{X}_n = \frac{1}{n} \sum_{i=1}^n X_i$ converges in probability to μ . Equivalently, for any $\varepsilon > 0$,*

$$\lim_{n \rightarrow \infty} \Pr[|\bar{X}_n - \mu| > \varepsilon] = 0.$$

Theorem .26 (Strong Law of Large Numbers). *Under the same conditions, $\bar{X}_n$ converges almost surely to μ .*

Theorem .27 (Central Limit Theorem). *Let $X_1, \dots, X_n$ be i.i.d. random variables with finite mean μ and variance σ^2 . Let $S_n = \sum_{i=1}^n X_i$. Then*

$$\frac{S_n - n\mu}{\sigma\sqrt{n}}$$

converges in distribution to the standard normal distribution $\mathcal{N}(0, 1)$.

.6 Martingales

Martingales model fair games: the expected future value equals the present value given all available information.

Definition .28. A sequence of random variables $X_0, X_1, \dots$ is a *martingale* with respect to a filtration $\mathcal{F}_0 \subseteq \mathcal{F}_1 \subseteq \dots$ if for each i :

1. X_i is $\mathcal{F}_i$ -measurable;
2. $\mathbb{E}[|X_i|] < \infty$;
3. $\mathbb{E}[X_{i+1} \mid \mathcal{F}_i] = X_i$ almost surely.

Definition .29 (Doob martingale). Given a function f of n random variables $Y_1, \dots, Y_n$, the *Doob martingale* is defined by

$$X_i = \mathbb{E}[f(Y_1, \dots, Y_n) \mid Y_1, \dots, Y_i], \quad X_0 = \mathbb{E}[f(Y)], \quad X_n = f(Y).$$

The Azuma–Hoeffding inequality (Theorem .23) provides concentration bounds for martingales with bounded differences.

Appendix A

Computational Complexity

This appendix reviews the complexity-theoretic framework used throughout the book. We assume familiarity with basic concepts such as algorithms, running time, and polynomial-time computation.

.1 Turing Machines and Basic Complexity Classes

The standard model for formal complexity analysis is the Turing machine. A *deterministic Turing machine* (DTM) operates on a semi-infinite tape, reading and writing symbols according to a finite transition function. A *probabilistic Turing machine* (PTM) has an additional tape containing uniformly random bits, which it can read during execution.

Definition .1. A language $L \subseteq \{0,1\}^*$ is in **P** if there exists a DTM that decides membership in L in time polynomial in the input length n .

Definition .2. A language L is in **NP** if there exists a polynomial-time *verifier* V such that for any input x :

- If $x \in L$, there exists a *witness* w with $|w| = \text{poly}(|x|)$ and $V(x, w)$ accepts;

- If $x \notin L$, then for all w , $V(x, w)$ rejects.

Equivalently, NP is the class of languages decidable by a nondeterministic Turing machine in polynomial time.

.2 Randomized Complexity Classes

Randomized algorithms give rise to natural complexity classes based on their error behavior.

Definition .3 (RP — Randomized Polynomial Time). A language L is in **RP** if there exists a probabilistic polynomial-time Turing machine M such that for any input x :

- If $x \in L$, then $\Pr[M(x) \text{ accepts}] \geq 1/2$;
- If $x \notin L$, then $\Pr[M(x) \text{ accepts}] = 0$.

RP captures algorithms with one-sided error: false negatives are allowed, but false positives are not. The constant $1/2$ can be amplified to $1 - 2^{-p(n)}$ by running $O(p(n))$ independent repetitions.

Definition .4 (co-RP). A language L is in **co-RP** if its complement $\bar{L}$ is in RP. Equivalently, there exists a probabilistic polynomial-time M such that:

- If $x \in L$, then $\Pr[M(x) \text{ accepts}] = 1$;
- If $x \notin L$, then $\Pr[M(x) \text{ accepts}] \leq 1/2$.

Primality testing (Section 15.6) is a classic example: the problem ‘COMPOSITES’ is in RP (Miller–Rabin test), while ‘PRIMES’ is in co-RP.

Definition .5 (BPP — Bounded-Error Probabilistic Polynomial Time). A language L is in **BPP** if there exists a probabilistic polynomial-time Turing machine M such that for any input x :

$$\Pr[M(x) \text{ correctly decides } x \in L] \geq 2/3.$$

The error probability can be amplified to $2^{-p(n)}$ by running $O(p(n))$ independent copies and taking the majority vote (Chernoff bounds guarantee correctness).

Definition .6 (ZPP — Zero-Error Probabilistic Polynomial Time). A language L is in **ZPP** if there exists a probabilistic polynomial-time Turing machine that always produces the correct answer and whose expected running time is polynomial. Equivalently, $\mathbf{ZPP} = \mathbf{RP} \cap \mathbf{co-RP}$.

ZPP corresponds to Las Vegas algorithms; RP and BPP correspond to Monte Carlo algorithms.

Proposition .7 ($\mathbf{ZPP} = \mathbf{RP} \cap \mathbf{co-RP}$). $\mathbf{ZPP} = \mathbf{RP} \cap \mathbf{co-RP}$.

Proof. We show both inclusions.

($\subseteq$): Let $L \in \mathbf{ZPP}$, witnessed by a machine M with expected running time $T(n)$. Obtain a machine M' for **RP** by running M for $2T(n)$ steps; if M finishes, answer what M says; otherwise reject. By Markov's inequality, $\Pr[M \text{ runs for more than } 2T(n)] \leq 1/2$, so if $x \in L$ we accept with probability at least $1/2$, and if $x \notin L$ we never falsely accept. Hence $L \in \mathbf{RP}$. Replacing rejection with acceptance gives $L \in \mathbf{co-RP}$.

($\supseteq$): Let $L \in \mathbf{RP} \cap \mathbf{co-RP}$ with machines M_1 (RP) and M_2 (co-RP). On input x , alternately run M_1 and M_2 . If M_1 accepts, output YES; if M_2 rejects, output NO; otherwise continue. Each pair of runs halves the probability of no decision, so the expected number of pairs before a decision is 2, giving expected polynomial time. $\square$

Remark .8 (Decision vs. Optimization). The complexity classes above are defined for decision problems (YES/NO). Many natural problems are optimization problems (e.g., find the minimum cut). When an optimization problem has a polynomial-time verifiable certificate (a candidate solution whose quality can be checked), a Monte Carlo algorithm for the decision problem can be converted to find the actual solution: use binary search on the objective value, running the decision algorithm as a subroutine. FPRAS (Section 12.2) formalizes this for counting problems.

The following relationships are known:

$$\mathbf{P} \subseteq \mathbf{ZPP} \subseteq \mathbf{RP} \subseteq \mathbf{BPP} \subseteq \mathbf{PSPACE}.$$

Whether $\mathbf{BPP} = \mathbf{P}$ is a major open question, though derandomization results (Impagliazzo–Wigderson, 1997) show that $\mathbf{BPP} = \mathbf{P}$ under plausible circuit lower bounds.

Definition .9 (PP — Probabilistic Polynomial Time). A language L is in $\mathbf{PP}$ if there exists a probabilistic polynomial-time M such that $\Pr[M(x) \text{ accepts}] > 1/2$ for $x \in L$ and $\Pr[M(x) \text{ accepts}] < 1/2$ for $x \notin L$. Unlike BPP, the gap may be exponentially small, making PP far more powerful: $\mathbf{NP} \subseteq \mathbf{PP}$.

However, PP is not believed to be practically useful for randomized algorithms: because the success probability can be as close to $1/2$ as an inverse exponential, an exponential number of repetitions is required to amplify success to a constant. This is why BPP (not PP) is the standard model of feasible randomized computation.

.3 Counting Classes

Many randomized algorithms solve counting problems, leading to the class $\#\mathbf{P}$.

Definition .10. A function $f : \{0, 1\}^* \rightarrow \mathbb{N}$ is in $\#\mathbf{P}$ if there exists a polynomial-time verifier V such that for every input x ,

$$f(x) = |\{w : |w| = \text{poly}(|x|) \text{ and } V(x, w) \text{ accepts}\}|.$$

In other words, $f(x)$ counts the number of accepting witnesses for x .

Counting satisfying assignments of a Boolean formula ($\#\mathbf{SAT}$) and counting perfect matchings in a bipartite graph ($\#\mathbf{PerfectMatchings}$) are canonical $\#\mathbf{P}$ -complete problems. Approximating $\#\mathbf{P}$ -complete functions is the subject of Chapter 12.

Definition .11. A randomized algorithm is a *fully polynomial randomized approximation scheme* (FPRAS) for a function f if, given input x and parameter $\varepsilon > 0$, it outputs $\hat{Z}$ satisfying

$$\Pr[(1 - \varepsilon)f(x) \leq \hat{Z} \leq (1 + \varepsilon)f(x)] \geq 3/4$$

in time polynomial in $|x|$ and $1/\varepsilon$.

.4 Circuit Complexity and Derandomization

Boolean circuits provide a non-uniform model of computation. A *Boolean circuit* with n inputs is a directed acyclic graph where each node is either an input or a logic gate (AND, OR, NOT). The *size* is the number of gates; the *depth* is the longest path from input to output.

Definition .12. A language L is in **P/poly** if there exists a family of Boolean circuits $\{C_n\}$ of polynomial size such that $C_n(x) = 1$ iff $x \in L$ for all $x \in \{0, 1\}^n$.

Adleman's theorem (Theorem 3.14) shows that **BPP** $\subseteq$ **P/poly**: any BPP algorithm can be derandomized using a polynomial-size circuit that hard-codes a good sequence of random bits.

Definition .13 (Pseudorandom Generator). A *pseudorandom generator* (PRG) against circuit class $\mathcal{C}$ is a function $G : \{0, 1\}^s \rightarrow \{0, 1\}^n$ such that for every $C \in \mathcal{C}$,

$$\left| \Pr_{r \in \{0, 1\}^s} [C(r) = 1] - \Pr_{z \in \{0, 1\}^s} [C(G(z)) = 1] \right| < \varepsilon.$$

PRGs with seed length $s = O(\log n)$ that fool polynomial-size circuits imply **BPP** = **P**.

.5 Interactive Proofs and the Polynomial Hierarchy

Definition .14. An *interactive proof system* for a language L consists of a probabilistic polynomial-time verifier V and an all-

powerful prover P who exchange messages. The system satisfies:

- **Completeness:** If $x \in L$, then $\Pr[\langle P, V \rangle(x) \text{ accepts}] \geq 2/3$.
- **Soundness:** If $x \notin L$, then for any cheating prover P^* , $\Pr[\langle P^*, V \rangle(x) \text{ accepts}] \leq 1/3$.

Lund, Fortnow, Karloff, and Nisan (1990) and Shamir (1992) proved that $\mathbf{IP} = \mathbf{PSPACE}$, showing that interactive proofs are surprisingly powerful. This result is discussed further in Section 8.7.

Appendix A

C++23 Programming for Randomized Algorithms

This appendix reviews the C++23 features used in the code listings throughout the book. Readers should already know basic C++ (variables, functions, classes, control flow). Here we focus on the modern features that make the implementations concise, safe, and efficient.

.1 The Random Library

The `<random>` header provides facilities for pseudorandom number generation that are superior to `rand()` in every respect: statistical quality, reproducibility, and type safety.

.1.1 Engines

An *engine* is a deterministic generator that produces a sequence of pseudorandom integers uniformly distributed in a range determined by the engine's parameters.

Listing 1: Common PRNG engines

```
1 #include <random>
2
3 std::mt19937 rng(42);           // Mersenne Twister, 32-bit
4 std::mt19937_64 rng64(42);     // Mersenne Twister, 64-bit
```

```

5  std::minstd_rand linear(42);    // Linear congruential (minimal
    standard)
6  std::ranlux48 rlux(42);        // Subtract-with-borrow (48-bit)

```

The Mersenne Twister (`std::mt19937`) is the default choice for most applications. It has a period of $2^{19937} - 1$ and passes stringent statistical tests. For cryptographic applications, use `<random>`'s `std::random_device` together with a cryptographically secure generator from `<openssl/rand.h>` or

1.2 Seeding

For reproducible results, seed with a fixed integer. For unpredictable results, use `std::random_device`:

Listing 2: Seeding strategies

```

1  #include <random>
2  #include <chrono>
3
4  // Reproducible
5  std::mt19937 rng1(42);
6
7  // Unpredictable (uses entropy pool)
8  std::mt19937 rng2(std::random_device{}());
9
10 // Seeded with current time (not recommended for serious work)
11 auto seed = std::chrono::steady_clock::now()
12             .time_since_epoch().count();
13 std::mt19937 rng3(seed);

```

1.3 Distributions

A *distribution* transforms the raw output of an engine into a random variable with a specified probability distribution.

Listing 3: Common distributions

```

1  std::mt19937 rng(std::random_device{}());
2  std::uniform_int_distribution<int> uniform(1, 6); // dice roll
3  std::uniform_real_distribution<double> ureal(0.0, 1.0);
4  std::bernoulli_distribution coin(0.5);           // fair coin
5  std::binomial_distribution<int> binom(10, 0.5);   // Binomial
    (10, 0.5)
6  std::geometric_distribution<int> geom(0.3);       // Geometric
    (0.3)
7  std::normal_distribution<double> norm(0.0, 1.0); // Standard
    normal

```



```

8
9 int roll = uniform(rng);           // generate a sample
10 bool heads = coin(rng);

```

Important convention. Throughout this book, we pass the random engine by reference to functions that need randomness. This avoids global state and makes the code thread-safe:

Listing 4: Passing a random engine by reference

```

1 int roll_dice(std::mt19937& rng) {
2     std::uniform_int_distribution<int> d(1, 6);
3     return d(rng);
4 }

```

.2 Concepts and Constraints

C++20 introduced *concepts*, which constrain template parameters at compile time. C++23 extends the standard library with additional concepts.

Listing 5: Using concepts to constrain templates

```

1 #include <concepts>
2 #include <vector>
3 #include <algorithm>
4
5 template <std::ranges::range R>
6 requires std::totally_ordered<std::ranges::range_value_t<R>>
7 auto min_element(const R& range) {
8     return *std::ranges::min_element(range);
9 }

```

Commonly used concepts include:

- `std::same_as<T, U>`, `std::convertible_to<T, U>`
- `std::integral<T>`, `std::floating_point<T>`
- `std::totally_ordered<T>`
- `std::ranges::range<R>`, `std::ranges::sized_range<R>`
- `std::uniform_random_bit_generator<G>` (from `<random>`)

3 Ranges and Views

The Ranges library (C++20) provides composable algorithms and lazy views.

Listing 6: Range adaptors and views

```

1 #include <ranges>
2 #include <vector>
3 #include <iostream>
4
5 std::vector<int> v = {1, 2, 3, 4, 5, 6};
6
7 // View: even numbers, squared
8 auto even_squares = v
9     | std::views::filter([](int x) { return x % 2 == 0; })
10    | std::views::transform([](int x) { return x * x; });
11
12 for (int x : even_squares)
13     std::cout << x << ' ';    // prints: 4 16 36

```

Ranges allow algorithms to operate directly on containers without iterators:

Listing 7: Ranges algorithms

```

1 #include <algorithm>
2 #include <vector>
3 #include <ranges>
4
5 std::vector<int> data = {5, 2, 8, 1, 9};
6 std::ranges::sort(data);           // sort in place
7 auto it = std::ranges::find(data, 8); // find element
8 std::ranges::reverse(data);        // reverse in
   place

```

4 Smart Pointers

Dynamic memory management in modern C++ uses *smart pointers* that automatically deallocate resources.

Listing 8: Smart pointer usage

```

1 #include <memory>
2 #include <vector>
3
4 // std::unique_ptr: exclusive ownership
5 auto ptr = std::make_unique<std::vector<int>>(100, 0);
6

```

```

7 // std::shared_ptr: shared ownership (reference-counted)
8 auto shared = std::make_shared<int>(42);
9 auto copy = shared; // reference count increments
10
11 // std::weak_ptr: non-owning observer
12 std::weak_ptr<int> observer = shared;

```

Prefer `std::make_unique` and `std::make_shared` over raw `new`—they are safer and more efficient.

.5 Chrono and Timing

The `<chrono>` library provides type-safe time utilities.

Listing 9: Measuring execution time

```

1 #include <chrono>
2 #include <iostream>
3
4 auto start = std::chrono::steady_clock::now();
5
6 // ... run algorithm ...
7
8 auto end = std::chrono::steady_clock::now();
9 auto elapsed = std::chrono::duration_cast<
10     std::chrono::milliseconds>(end - start);
11 std::cout << "Time: " << elapsed.count() << " ms\n";

```

For thread-safe random seeding, `std::chrono::steady_clock` provides monotonic time that is not affected by system clock changes.

.6 Structured Bindings and Initialization

C++17 structured bindings simplify unpacking tuples, pairs, and aggregates:

Listing 10: Structured bindings

```

1 #include <tuple>
2 #include <map>
3
4 std::map<std::string, int> scores = {{"Alice", 95}, {"Bob",
5     87}};
6
7 for (const auto& [name, score] : scores) {
8     std::cout << name << ": " << score << '\n';
9 }

```

```
8 }  
9  
10 auto [min, max] = std::minmax({3, 1, 4, 1, 5, 9});
```

Designated initializers (C++20) improve aggregate initialization:

Listing 11: Designated initializers

```
1 struct Point { double x, y, z; };  
2 Point p = {.x = 1.0, .y = 2.0, .z = 3.0};
```

.7 Exception Safety

The code in this book uses exceptions for error reporting rather than error codes. The RAII (Resource Acquisition Is Initialization) idiom ensures that resources are released during stack unwinding.

Listing 12: Exception-safe resource management

```
1 #include <fstream>  
2 #include <stdexcept>  
3  
4 void process_file(const std::string& path) {  
5     std::ifstream file(path);  
6     if (!file)  
7         throw std::runtime_error("Cannot open " + path);  
8     // file is automatically closed when it goes out of scope  
9 }
```